

The Hydra-16

A multitasking 65C02 computer and its operating system: all of the documentation (OS 1.8C_0.6)

The documents

1. **The Hydra-16: the whole system in one place** [docs/hydra-16.md](#)

Contents · 1. The machine in one page · 2. Getting started · 3. Hardware · 4. The ROMs · 5. The kernel: tasks, interrupts, memory · 6. Input and output · 7. Namespaces · 8. Storage: disks and HydraFS · 9. The shell: HyForth · 10. WOZMON, POST and the self tests · 11. Programs: assembly and C · 12. Sound · 13. Tools: the build and the emulator · 14. The source tree · 15. Plans and status · 16. All the documents

2. **First steps with the Hydra-16** [docs/tutorial.md](#)

1. Switch it on · 2. HyForth · 3. Files · 4. A card · 5. A program in C · 6. A program in assembly · 7. An LED and a button · 8. Where next

3. **Getting started** [docs/getting-started.md](#)

What you need · The repository · Building · Programming the chips · Connecting a terminal · First boot · Without the hardware: the emulator · Testing a change · Where to go next

4. **Hydra-16 Hardware Reference** [docs/hardware.md](#)

Contents · Overview · The CPU view: memory map · The pseudo-registers T, U, V, W · Task RAM and the bank registers · The paged RAM window · The paged ROM · The BIOS ROM · I/O space · Interrupts · Clocks · Reset, power and bus control · On-board devices · Expansion slots · Companion cards · Connectors and jumpers · V1 errata · Parts by function

5. **HyForth: the Hydra's shell** [docs/using/hyforth.md](#)

Contents · The basics · Numbers · Strings · Defining words · Control flow: the training scripts · Word reference · The base and its libraries · The shell: directories, files and programs · Files and devices · Pipelines · Tasks and the console · Background tasks and exit statuses · Errors and keys · How HyForth uses memory

6. **WOZMON, the disassembler, POST and the self tests** [docs/using/wozmon.md](#)

Getting to WOZMON · Commands · Self tests · POST: the power-on self test · The hardware test

7. **Hydra-16 Programmer's Guide** [docs/programming/README.md](#)

The system in one page · Where things are in the source

8. **ROM layout, calling conventions and the API index** [docs/programming/rom-layout.md](#)

The two ROM images · BIOS ROM pages · Calling across ROM pages · Calling conventions · Zero page · Error codes · API index: the thunks · Adding code

9. **Tasks and the scheduler** [docs/programming/tasks.md](#)

What a task is · Task status · Scheduling · Starting tasks · Ending tasks · Exit statuses · Waiting and sleeping · Semaphores · Running code in another task: TASK_CALL · Signals: break and kill · The console's foreground task · Drivers: resident tasks

10. **Interrupts** [docs/programming/interrupts.md](#)

IRQ lines · How an interrupt is handled · The fast handlers: the tick, the serial port and the sound clock · Registering a handler · Writing a handler · Software interrupts · NMI · Timing notes

11. **Memory: the MMU, shared memory, far pointers** [docs/programming/memory.md](#)

The MMU: a task's own memory · Shared memory · Far pointers · References: far pointers as handles · Copying

12. **Input and output** [docs/programming/io.md](#)

Files and fds · stdio · Devices · Pipes · Namespaces · The current directory · The files on a card · Stat

13. **Writing drivers and file servers** [docs/programming/servers.md](#)
 The model · Registering a device · The serve routine · The request block and the transfer area · Waiting: when there's no data yet · Names and ctl files · Checklist for a new device · The storage layer (for the filesystem server) · The HydraFS server
14. **Programs: Hydra executables (.hyx)** [docs/programming/programs.md](#)
 The file · What a program gets · Building one · C programs · Putting it on a card · How run does it (os_rom/shell/run.s, BIOS ROM page 7)
15. **The C Programmer's Guide** [docs/programming/c.md](#)
 Contents · 1. Quick start · 2. The machine, as C sees it · 3. Building · 4. A program's life: arguments, environment, exit status · 5. Files and devices · 6. The console: stdio and conio · 7. Running other programs · 8. Time · 9. Tasks working together: semaphores · 10. Errors · 11. Memory · 12. Assembly in a C program · 13. Performance · 14. Debugging and testing · 15. Limits and gotchas · 16. Reference: hydra.h · 17. Working on the library
16. **The emulator and tools** [docs/tools/emulator.md](#)
 Using the Hydra from your terminal · Usage · Regression tests · HydraFS card images · Hydra executables · The ROM disk · Songs: the score compiler · What it models · Inside the emulator
17. **Next steps: making the Hydra fun and useful** [docs/plans/NEXT_STEPS.md](#)
 Contents · Where it stands · What a newcomer runs into · The gaps, by theme · Milestones
18. **Hydra 16: Ideas and future work** [docs/plans/IDEAS.md](#)
 Next features, in order · Slow devices on a faster CPU clock (board V2) · Serial output errors at 115200 (open)
19. **Namespaces, the Plan 9 way: one tree put together from many** [docs/plans/NAMESPACES.md](#)
 Contents · Where it stands · What Plan 9 does · Unions · The namespace a task starts with · Finding programs and libraries · Room: a bigger namespace · Hiding · Calls and words · Costs · Tests · Steps
20. **/proc: tasks as files, their memory included** [docs/plans/PROC.md](#)
 Contents · Where it stands · What Plan 9 does · The files · /proc/N/mem · /proc/N/ram · Who can use it · How it works · What it gives · Tests · Steps
21. **RAM and ROM disks: /ram and /rom as HydraFS volumes** [docs/plans/DISKS.md](#)
 Contents · What it looks like · Disks in memory · The ROM disk · Pointers and banks: what can't go wrong · RAM disks · Who can use which area · How the RAM disks work · Booting: finding the disks · What the disks simplify · The program caches · Tests · Steps
22. **Video: the Vera X card in slot 0** [docs/plans/VIDEO.md](#)
 Contents · Why the VERA · The card · The registers on the Hydra · Sharing one chip between 16 tasks · The software, in steps · The emulator · Risks and open questions · Order of work
23. **Hydra 16: Sound (the YM2151)** [docs/plans/SOUND.md](#)
 The chip on the Hydra · Phase 1: the library (done) · Phase 2: the song player (done) · Phase 3: a test song that uses the whole chip (done) · Phase 4: the chip's own clock (done, then set aside) · Importing music from other machines (investigation: nothing built yet) · Order of work
24. **HydraFS: the Hydra's SD card filesystem** [docs/plans/HYDRAFS.md](#)
 Goals and trade-offs · Names · On the card · Stat records (48 bytes) · Reading a directory · Calls and requests · The storage task's state · Formatting and tools · Build order · The check · Partitions · Time stamps · Sparse files · Later
25. **Hydra IO Plan** [docs/plans/IO_PLAN.md](#)
 Goals · Plan 9 ideas, and what they become on the Hydra · Architecture · File descriptors · Names · H9P: the request block · IO transfer areas · Blocking and the scheduler · Clock speed and wait states · Servers · API · Build order · Decisions
26. **Hydra MMU Plan** [docs/plans/MMU_PLAN.md](#)
 Goals · Tiers of allocation (per task) · Where the MMU area lives: top of Task RAM (\$7E00-\$7FFF) · Per-task layout (Task RAM, starting at \$7E00) · Shared memory (256 banks: \$F0-\$FF × U 0-F) · Messaging and buffered transfers · HyForth integration · Task completion (MM_TASK_RESET) · API (all: C = 0 success, C = 1 with error in .A) · Zero-page convention · Build order · IO subsystem (IRQ dispatch and drivers: Phase 2; the rest later) · Decisions
27. **HyForth as a shell** [docs/plans/SHELL.md](#)
 1. The current directory, in the IO layer · 2. Boot · 3. The prompt · 4. Shell commands · 5. Running programs · Build order · Decisions
28. **OS ROM reorganisation plan** [docs/plans/REORG_PLAN.md](#)
 1. Remove dead code (done) · 2. Page 1's gates (done) · 3. Page roles and fixed offsets (done) · 4. Source tree and build output (done) · 5. Shared helpers (done) · 6. RAM (done) · 7. HyForth's core from ROM (done)

29. **Code review: the Hydra-16 software** [docs/plans/CODE_REVIEW.md](#)

[Contents](#) · [Overall](#) · [Bugs](#) · [ROM space](#) · [Zero page and the namespace](#) · [The kernel and the IO layer](#) · [The source tree](#) · [The build](#) · [The emulator](#) · [The tests](#) · [HyForth](#) · [The C library](#) · [The documentation](#) · [The repository](#) · [Recommendations, in order](#) · [What was done](#)

30. **Hydra-16 documentation** [docs/README.md](#)

[Start here](#) · [Using the system](#) · [Programming it](#) · [The hardware](#) · [Tools](#) · [Plans and design notes](#)

31. **Hydra 16** [README.md](#)

[Quick start](#) · [Documentation](#) · [Repository](#)

Made from the Markdown in the repository by [sim/tools/mkpdf.js](#). The documents are also on GitHub: <https://github.com/danstruthers/hydra-16/>

The Hydra-16: the whole system in one place

The Hydra-16 is a multitasking 65C02 computer: a W65C02S whose address decoding gives each of 16 tasks its own RAM, zero page, stack and bank selections, running a small preemptive OS with Plan 9-style IO, and HyForth as its shell.

This page covers every part of it, hardware and software, with a short account of each and links to the documents that cover it in full. Read it top to bottom for the whole picture, or jump to a part from the contents. [All the documents](#) are listed at the end.

Contents

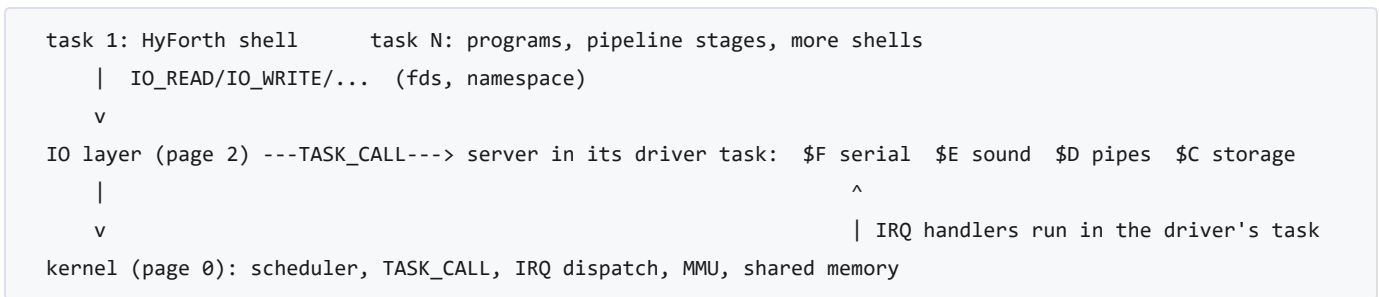
1. [The machine in one page](#)
2. [Getting started](#)
3. [Hardware](#)
4. [The ROMs](#)
5. [The kernel: tasks, interrupts, memory](#)
6. [Input and output](#)
7. [Namespaces](#)
8. [Storage: disks and HydraFS](#)
9. [The shell: HyForth](#)
10. [WOZMON, POST and the self tests](#)
11. [Programs: assembly and C](#)
12. [Sound](#)
13. [Tools: the build and the emulator](#)
14. [The source tree](#)
15. [Plans and status](#)
16. [All the documents](#)

1. The machine in one page

CPU	WDC W65C02S, 3.58 MHz as built
Tasks	16, each with its own 32K of RAM (\$0000-\$7FFF , zero page and stack included) and its own RAM and ROM bank selections: a task switch is one write to T
Memory	512K task RAM; 2 MB shared RAM and up to 15 x 2 MB task RAM modules, through an 8K window at \$8000 ; 4 MB paged ROM in 16K banks at \$A000 ; an 8K-paged BIOS ROM at \$E000
Interrupts	16 prioritised IRQ lines, each with its own vector
Devices	65C22 VIA (the tick, SPI, I2C, GPIO), 65C51 ACIA (the serial console), YM2151 (stereo FM sound), SD cards on SPI, an optional DS1747 clock
Expansion	6 slots, 8 SPI device headers, a GPIO/I2C header
OS	A preemptive scheduler (200 Hz), IRQ handlers that run in their driver's task, a memory manager per task, shared memory, semaphores, exit statuses
IO	Plan 9 style: devices are file servers; each task has 12 fds, a namespace of union mounts and binds, and pipes
Storage	HydraFS on SD cards (/sd/N), the paged ROM as a read-only disk (/rom), RAM disks (/ram , /sram)
Shell	HyForth: a Forth that's also a shell, with a current directory, pipelines, redirection, and scripts and programs run by name
Programs	.hyx executables in assembly or C (cc65, with a library for the Hydra's calls)

Tools	An emulator that runs the real ROMs, regression tests, a hardware test, card and ROM disk image tools, a score compiler for songs
--------------	---

How the software fits together:



More: [the system in one page](#) (the Programmer's Guide), [the hardware overview](#).

2. Getting started

- **First steps:** a tutorial for the first hour, in the emulator: [switching it on](#), [HyForth](#), [files](#), [a card](#), [a program in C](#), [one in assembly](#), and [an LED and a button](#).
- **Getting started:** [what you need](#), [building](#) (node `build.js`, with cc65 and Node.js), [programming the chips](#), [connecting a terminal](#) (9600 8N1, a straight-through cable), [the first boot](#), [the emulator](#), [testing a change](#).

node build.js test	build everything and run the regression tests
node sim/hydrasim.js -i	use the Hydra in your terminal, in the emulator (Ctrl-A x quits)

3. Hardware

The [Hardware Reference](#) describes the main board from its schematics, sheet by sheet, and the companion cards. The KiCad 9 projects are in `board/`.

The memory system

Addresses	What	Selected by
\$0000-\$7FFF	Task RAM: each task's own 32K (\$00 and \$01 are its RAM and ROM bank registers: written there, read back from RAM)	T
\$8000-\$9FFF	Paged RAM: a task bank (\$00-\$EF) or a shared bank (\$F0-\$FF)	\$00; T or U
\$A000-\$DFFF	Paged ROM, 16K banks	\$01
\$E000-\$FEFF	BIOS ROM, 8K pages	W
\$FF00-\$FFEF	I/O ports 0-14, 16 bytes each	
\$FFF0-\$FFF3	The pseudo-registers T, U, V, W	
\$FFFA-\$FFFD	NMI and RESET vectors (BIOS ROM)	W
\$FFFE-\$FFFF	IRQ/BRK vector RAM, 16 entries	the active IRQ, or V

- [The CPU view: memory map](#).
- [The pseudo-registers T, U, V, W](#): the task (T), the shared macro-page (U), the vector select (V), the BIOS ROM page (W). They have no reset: the ROM sets them.

- **Task RAM and the bank registers:** one HM628512 (or a DS1747, which adds a clock), and the per-task bank registers in 74LS219s.
- **The paged RAM window:** task RAM modules on memory daughter cards, and the 2 MB of shared RAM on the board.
- **The paged ROM:** eight SST39SF040, 256 banks of 16K, with the halves of each bank swapped.
- **The BIOS ROM:** an SST39SF010/020/040 paged by `W`; changing `W` changes the code being run.

I/O, interrupts, clocks and reset

- **I/O space:** 15 device ports of 16 bytes, and the system port.
- **Interrupts:** 16 lines through two 74LS148s into the vector RAM; the index is the line number XOR 7; line 15 is for software interrupts.
- **Clocks:** a 14.318 MHz crystal divided down; the CPU clock is set by a jumper (J7, 3.58 MHz, as built).
- **Reset, power and bus control:** ATX power, the front panel, the reset supervisor, `RDY`, DMA.

The devices

Device	Port	IRQ line	Used for
VIA, 65C22 (U2)	0, <code>\$FF00</code>	0	Timer 1: the scheduler's tick; port A: GPIO and I2C (header J27: <code>/dev/gpio</code>); port B: the SPI bus, with 8 device headers (J18-J25), each device a file: <code>/dev/spi/N</code>
ACIA, 65C51 (U3)	1, <code>\$FF10</code>	1	The serial console, on a DE-9 (DCE wiring)
YM2151 (U38)	4, <code>\$FF40</code>	4	8-voice FM sound, through a YM3012 DAC and a mixer to a stereo jack
Expansion slots 0-5	2-3, 5-14	2-3, 5-14	Cards on the 62-pin bus: video in slot 0 (planned), and anything else

Cards, connectors and the board as built

- **Companion cards:** the [memory daughter card](#) (a 2 MB task RAM module: `board/MemoryDaughterCard`) and the [bus breakout card](#) (`board/HydraBusBreakoutCard`).
- **Connectors and jumpers:** fit J4 (`RDY`) and one CPU clock jumper (J7).
- **V1 errata:** bank register bits 2/3 and 6/7 are crossed (the software and tools allow for it), no wait states, the audio jack's channels, the ACIA's clock.
- [Parts by function](#).

The schematics

Sheet (board/)	What	In the reference
hydra-16.kicad_sch	The root sheet: the CPU, the BIOS ROM, task RAM, the VIA, the ACIA	Overview
AddressDecode	Address decoding, the I/O ports	I/O space
FFF_Registers	<code>T</code> , <code>U</code> , <code>V</code> , <code>W</code>	The pseudo-registers
ZPMirrorRAM	The bank registers <code>\$00</code> and <code>\$01</code>	Task RAM and the bank registers
SharedMem	Shared RAM, the module selects	The paged RAM window
BankedROM	The paged ROM	The paged ROM
IRQ_P_E	The IRQ priority encoder and the vector RAM	Interrupts
Clocks	The oscillator and its dividers	Clocks
Buffers	The address, data and R/W buffers	Reset, power and bus control
Sound , Mixer	The YM2151, its DAC, the mixer	YM2151

Sheet (board/)	What	In the reference
Connectors	The slots, headers, power	Expansion slots, connectors

4. The ROMs

Everything the Hydra runs at boot is in two ROM images, from one build ([the two ROM images](#)):

- **The BIOS ROM** (`os_rom/bin/os_rom_C02.bin`, 128K): 16 pages of 8K, selected by `W`.
- **The paged ROM** (`os_rom/bin/paged_rom_C02.bin`): the whole paged ROM is one disk, the ROM disk, whose files are `/rom`; its banks 0 and 1 also hold HyForth's start-up data and the hardware test.

Page	Contents
0	The kernel: reset, tasks, the scheduler, IRQ dispatch, the MMU, shared memory; the serial and sound drivers; the thunks
1	HyForth's interpreter, and its words' headers
2	The IO layer: <code>fds</code> , namespaces, <code>/dev/cons</code> , <code>/dev/ser</code>
3	Storage: SPI, the disks' block layer (cards, the ROM and RAM disks), <code>/dev/sd</code> , HydraFS's format and check
4	POST, the self tests, WOZMON
5	Far pointers, semaphores, exit statuses
6	The HydraFS server
7	The shell: boot, the prompt, the file commands, running programs, redirection
8	The text editor (<code>edit</code>)
9	Servers: <code>/proc</code> , <code>/env</code> , <code>/dev/time</code> , <code>/dev/ram</code> , the pipes, the serial port's settings
A	HyForth's far words, its error messages, the disassembler
B	Sound: the YM2151 library, <code>/dev/snd</code> , the patches
C	The song player
D	<code>/dev/spi</code> , the SPI devices as files; <code>/dev/gpio</code> , port A's pins
E-F	Free

- [BIOS ROM pages](#): the full table, with the sources, and the fixed addresses on every page (the reset entry at `$E000`, the thunks at `$F800`, the COMMON block at `$FD00`).
- [Calling across ROM pages](#): gates and far calls.
- [Calling conventions](#): pointers in `.A.Y`; errors as `C = 1` with the code in `.A`.
- [Zero page](#): the OS's, a task's own, a program's (`$E0-$FF`).
- [Error codes](#).
- [API index: the thunks](#): every public entry point.
- [Adding code](#): where new code goes, and the space left on each page.
- The plan that set the pages' roles: [REORG_PLAN.md](#).

5. The kernel: tasks, interrupts, memory

Tasks

Task	Use
<code>\$0</code>	The system task: boot, then the idle task
<code>\$1</code>	The boot shell

Task	Use
\$2-\$B	Free: programs, pipeline stages, more shells
\$C	The storage driver: <code>/dev/sd</code> , HydraFS, <code>/dev/spi</code>
\$D	The pipe server
\$E	The sound driver
\$F	The serial driver

Tasks and the scheduler:

- [What a task is](#): its hardware, its memory map, the task numbers.
- [Scheduling](#): preemptive, round robin at 200 Hz.
- [Starting tasks and ending them](#).
- [Exit statuses](#): Plan 9's `exits`, a code and a message.
- [Waiting and sleeping, semaphores](#).
- [Running code in another task](#): `TASK_CALL`: how a client's request runs in a server's task.
- [Signals](#): `break` and `kill`, the console's foreground task.
- [Drivers](#): resident tasks, and the boot sequence.

Interrupts

[Interrupts](#): every IRQ runs the handler registered for its line, in the task that registered it.

- [IRQ lines, how an interrupt is handled](#).
- [The fast handlers](#): the tick, the serial port and the sound clock.
- [Registering a handler, writing one](#).
- [Software interrupts, NMI, timing notes](#).

Memory

[Memory](#):

- [The MMU](#): each task's allocations, as handles, from its own RAM and banks.
- [Shared memory](#): the 256 shared banks, between tasks.
- [Far pointers, references, copying](#).
- The design: [MMU_PLAN.md](#).

6. Input and output

Everything is a file. A task opens names and reads and writes fds; the IO layer passes each request to the device's server, in the server's task (`TASK_CALL`), with the data in the task's **IO transfer area** in shared RAM (1.5K a task: the request, the data, the namespace).

[Input and output](#):

- [Files and fds](#): 12 fds a task, the calls, inheritance.
- [stdio](#): fds 0-2, buffered.
- [Devices](#):

Name	What
<code>/dev/cons</code> , <code>/dev/cons/ctl</code>	The console , raw or cooked
<code>/dev/ser</code> , <code>/dev/ser/ctl</code>	The serial port, and its settings

Name	What
/dev/snd	The YM2151
/dev/sd/N/data, /dev/sd/N/ctl	A disk as bytes, and its control file
/dev/spi/N, /dev/spi/N/ctl	An SPI device (0-f): a transaction a write, its answer read after; its mode
/dev/gpio/N, port, ctl, ca1	Port A's pins on J27, CA1's edges, CA2
/sd/N/...	The files on a card (HydraFS)
/proc/N/...	The tasks: status, ctl, ns, cmd (send), and their memory as files, mem and ram
/env/NAME	The environment
/dev/time	The clock
/dev/ram	The RAM itself (task 0's only)
/rom/...	The ROM's files
/ram/..., /sram/...	The RAM disks
/dev/null, /dev/zero, /dev/pipe	The usual two, and pipes

- The current directory, stat.
- **Writing a device:** drivers and file servers: the model, registering a device, the serve routine, the request block and the transfer area, waiting, names and ctl files, a checklist.
- The design: IO_PLAN.md.

7. Namespaces

Names are put together the Plan 9 way: binds and mounts build one tree, and entries at the same path make a **union**, whose members are looked in, in order. There are no search paths: programs are found in `.` then `/bin`, libraries in `/lib`, and `/bin` and `/lib` are unions of the caches, the card and the ROM.

- **Two tables:** each task's own (32 entries, inherited by the tasks it starts) and the **system namespace** (32 entries every task sees, under its own). A name resolves to the longest matching prefix in either; on a tie, the task's own wins.
- **Mounts with a spec**, as Plan 9's `mount` takes one: `mount -s hfs /rom x` serves the ROM disk at `/rom`, and each shell's `mount hfs /ram r/N` gives it its own area of the RAM disk at `/ram`.
- **The default namespace** comes from `/rom/lib/namespace`, then a card's `lib/namespace`; `newns [file]` starts a task afresh from it (its own entries go but its `/ram`, then the file's lines); `ns` prints the current one as the lines that would make it:

```
mount -s hfs /sd
mount -s env /env
mount -s proc /proc
mount -s hfs /rom x
mount -s hfs /sram s
bind -cs /ram/bin /bin
bind -as /sram/bin /bin
bind -as /sd/0/bin /bin
bind -as /rom/bin /bin
...
mount hfs /ram r/1
```

- In full: namespaces (the calls, the flags, how a name resolves, the namespace at boot).
- In the shell: HyForth's files and devices (mount, bind, unmount, hide, newns, ns).

- In C: [files and devices](#) (`hy_mount`, `hy_bind`, `hy_unmount`, `hy_hide`, `hy_newns`).
- The design, and what's left: [NAMESPACES.md](#).

8. Storage: disks and HydraFS

Disk	Where	What
0-7	<code>/sd/N</code> , <code>/dev/sd/N</code>	An SD card on SPI device N (its number as one hex digit)
x	<code>/rom</code> , <code>/dev/sd/x</code>	The paged ROM, read-only: programs, libraries, songs, <code>boot.hys</code>
r	<code>/ram</code> , <code>/dev/sd/r</code>	The RAM disk: each shell's own area (<code>r/N</code>), kept until a reset
s	<code>/sram</code> , <code>/dev/sd/s</code>	The shared RAM disk, the same for every task

- **HydraFS**, the filesystem on all of them: directories, extents, sparse files, partitions, a checker. The spec: [HYDRAFS.md](#). On the Hydra: [the files on a card](#), [the HydraFS server](#), [the storage layer](#).
- **The ROM disk**, `/rom`: [io.md](#), [its design](#); built from `os_rom/romfs.txt` by `sim/tools/mkromdisk.js` (the ROM disk tool).
- **The RAM disks**, `/ram` and `/sram`: [io.md](#), [their design](#), [who can use which area](#), [the program caches](#), [booting with or without a card](#).
- **Cards from a PC**: [HydraFS card images](#) (`sim/tools/hydrafs.js`), [putting a program on a card](#).

9. The shell: HyForth

HyForth is the Hydra's shell and its language: a Forth whose words include a current directory, file commands (`cd`, `ls`, `cp` ...), | pipelines, redirection, scripts (`.hys`), programs (`.hyx`) run by name, more shells in other tasks (`shell`), background tasks, and `send N line` to type a line into another shell. `boot.hys` runs at boot.

HyForth: the Hydra's shell:

- [The basics](#), [numbers](#), [strings](#), [defining words](#), [control flow](#).
- [Word reference](#), the base and its libraries.
- [The shell](#): [directories](#), [files and programs](#).
- [Files and devices](#), [pipelines](#).
- [Tasks and the console](#), [background tasks and exit statuses](#).
- [Errors and keys](#), how HyForth uses memory.
- The design: [SHELL.md](#).

10. WOZMON, POST and the self tests

WOZMON, the disassembler, POST and the self tests:

- [Getting to WOZMON](#) (`bye` from HyForth), and [its commands](#), the disassembler among them.
- [Self tests](#): the MMU, the scheduler, IO.
- [POST: the power-on self test](#): the two lines at boot, and what they mean.
- [The hardware test](#): the registers, the memory, the ROMs' checksums and the devices, run from paged ROM bank 1.

11. Programs: assembly and C

A program is a `.hyx` file: a header and code. It's loaded into a task of its own, with the shell's `fds`, namespace, current directory and environment, and its arguments.

- **Programs:** the file, what a program gets, building one (`programs/asm/`), C programs, putting it on a card, and how `run` loads it.
- **The C Programmer's Guide** (cc65, `programs/c/`): quick start, the machine as C sees it, building, arguments, environment, exit status, files and devices, the console, running other programs, time, semaphores, errors, memory, assembly in C, performance, debugging, limits, `hydra.h`, working on the library.
- **ROM programs:** the text editor (`edit`, page 8) and the song player (`play`, page C) run in tasks of their own, like any program.
- **Hydra executables:** `sim/tools/mkhyx.js`, the header tool.

12. Sound

A YM2151 (8 FM voices of 4 operators each) on port 4, mixed with the slots' audio to a stereo jack.

- The chip and its audio path: [YM2151](#).
- `/dev/snd`, the device: [io.md](#). In C: [snd.h](#). In HyForth: `patch`, `note`, `noteoff`, `play`, `sndtest` ([the word reference](#)).
- The song player (ZSM files), and the score language that `os_rom/songs/test.mml` and `programs/songs/` are written in: [songs: the score compiler](#).
- The design, phase by phase, and importing music from other machines: [SOUND.md](#).

13. Tools: the build and the emulator

- **The build** (`build.js`): the ROMs, the programs, the ROM disk, the cross-page call checker and the ROM space report ([building](#)).
- **The emulator and tools:**
 - [Using the Hydra from your terminal](#), and [its options](#), the profiler (`--profile`) among them.
 - [Regression tests](#) (`sim/regress.js`, `sim/tests/`), which boot the real ROMs.
 - [HydraFS card images](#), [Hydra executables](#), [the ROM disk](#), [songs](#).
 - [What it models](#), [inside the emulator](#).
- **The documentation as one PDF** (`sim/tools/mkpdf.js`): every document in `docs/` and the top README, made into [hydra-16.pdf](#) by a headless Microsoft Edge or Chrome, with the links between them kept and bookmarks for each document and section. `node sim/tools/mkpdf.js` remakes it after a change.

14. The source tree

Folder	What
<code>os_rom/all.s</code> , <code>os_rom/os_rom_C02.cfg</code>	Includes everything, page by page; the linker config
<code>os_rom/include/</code>	Constants and macros: <code>hw.inc</code> , <code>kernel.inc</code> , <code>io.inc</code> , <code>zero.s</code> , <code>macros.inc</code> ...
<code>os_rom/kernel/</code>	Reset and boot, tasks, IRQs, the MMU, shared memory, far pointers, semaphores, exits, COMMON, gates, thunks
<code>os_rom/io/</code>	The IO layer, namespaces, pipes
<code>os_rom/servers/</code>	The console and serial port, <code>/dev/sd</code> , the RAM disks, <code>/proc</code> , <code>/env</code> , <code>/dev/time</code> , <code>/dev/ram</code>
<code>os_rom/drivers/</code>	Serial, sound, the VIA, SPI, SD cards and disks, the clock chip, the storage task
<code>os_rom/fs/</code>	HydraFS: the server, writing, sparse files, format, check
<code>os_rom/shell/</code>	Boot, the prompt, the file commands, running programs, redirection, the editor

Folder	What
os_rom/hyforth/	HyForth
os_rom/sound/	The YM2151 library, /dev/snd, the patches, the song player
os_rom/monitor/	WOZMON and the disassembler
os_rom/tests/, os_rom/hwtest/	POST and the self tests; the hardware test
os_rom/romfs/, romfs.txt, songs/	The ROM disk's files, and the test song
os_rom/tools/	The cross-page call checker, the ROM space report, the ROM checksums
programs/asm/, programs/c/, programs/songs/	Programs in assembly; the C library and samples; songs
sim/	The emulator (hydrasim.js, lib/), the regression tests, the tools
board/	KiCad 9: the main board, the memory daughter card, the bus breakout card
docs/	This documentation

More: [where things are in the source](#), [the repository](#).

15. Plans and status

The plans record the design's reasoning and what's to come; parts of them are history, marked done.

Plan	What	Status
NEXT_STEPS.md	What's missing to make the Hydra fun and useful, and the milestones	The roadmap
IDEAS.md	The next features in order, wait states for board V2, open issues	Ongoing
NAMESPACES.md	Unions, the default namespace, /bin and /lib, newns	Done
PROC.md	/proc, and a task's memory as files	Done but regs , fd and the debugger's commands
DISKS.md	The ROM and RAM disks, the program caches	Done
VIDEO.md	The Vera X card in slot 0	Planned
SOUND.md	The YM2151 library, the player, the test song	Done; importing music to do
HYDRAFS.md	The filesystem's spec	Done
IO_PLAN.md	The IO subsystem, the scheduler, the filesystem's steps	Done
MMU_PLAN.md	The memory manager, IRQ dispatch, drivers	Done
SHELL.md	HyForth as a shell	Done
REORG_PLAN.md	The ROM's reorganisation	Done
CODE_REVIEW.md	A review of the whole software tree	Its recommendations done

16. All the documents

Document	Covers
README.md	The documentation's index
tutorial.md	First steps, in the emulator
getting-started.md	Building, the chips, a terminal, the first boot
hardware.md	The Hardware Reference
using/hyforth.md	HyForth, the shell

Document	Covers
using/wozmon.md	WOZMON, POST, the self tests, the hardware test
programming/README.md	The Programmer's Guide
programming/rom-layout.md	ROM pages, gates, conventions, zero page, error codes, the API
programming/tasks.md	Tasks and the scheduler
programming/interrupts.md	Interrupts
programming/memory.md	The MMU, shared memory, far pointers
programming/io.md	fds, devices, pipes, namespaces
programming/servers.md	Drivers and file servers
programming/programs.md	.hyx programs
programming/c.md	The C Programmer's Guide
tools/emulator.md	The emulator and tools
The plans	Above

First steps with the Hydra-16

A tutorial for a first hour: the Hydra in the emulator, HyForth at its prompt, files on a card, then a program of your own in C and in assembly. It needs no board; everything here works the same on one. What it shows is explained fully in the [HyForth guide](#) and the [Programmer's Guide](#).

You need [Node.js](#) and a clone of the repository. For the C and assembly parts, also [cc65](#) ([getting started](#) says where the build looks for it).

1. Switch it on

From the top of the repository:

```
node sim/hydrasim.js -i
```

The Hydra boots in your terminal: its power-on test, the welcome, and HyForth's prompt, `/ram>` (the current directory: with no card, the shell's own area on the RAM disk, where files can be saved until a reset). Your terminal is now the Hydra's serial console. Ctrl-A then x quits the emulator, and Ctrl-A then h lists its other keys.

2. HyForth

HyForth is the Hydra's shell and its language. Type a line, press Enter, and it runs each word in turn. Numbers go on a stack, and words take them from it:

```
/ram> 1 2 + .
0003
```

`.` prints the top number in hex, 4 digits. For decimal, say so:

```
/ram> decimal 12 12 * .
144
/ram> hex
```

Make a word of your own with `:` and `;`. It works like the built-in ones:

```
/ram> : sq dup * ;
/ram> 7 sq .
0031
```

The Up arrow brings back the lines you typed (Down goes forward again), and Left, Right, Home and End move along the line, so a mistake is quick to fix. `words` lists every word there is. The Hydra multitasks: Ctrl-] then a task's number switches the console to it, and `ps` lists the tasks.

3. Files

Even with no card, the Hydra has files: the ones built into its ROM, under `/rom`.

```
/ram> ls /rom/bin
code.hyx 2156
hello.hyx 8616
...
scom.zsm 838
/ram> cat /rom/README
```

A program runs by its name: `hello` runs `/rom/bin/hello.hyx` (a C program), and `scom` plays the song `scom.zsm` on the YM2151 (the emulator times the music but makes no sound; `--ym-vgm FILE` saves it to listen to). The shell looks for it in the current directory, then in `/bin`, which isn't one directory but several put together, as in Plan 9: `ns` shows how (the caches on the RAM disk, a card's `/bin`, then `/rom/bin`), and `ls /bin` lists them all. Words that fail say why:

```
/ram> cat /rom/nope

!IO ERR! not found
```

4. A card

Real work goes on an SD card. The emulator's card is an image file, which `sim/tools/hydrafs.js` makes and fills from the PC (Ctrl-A x first):

```
node sim/tools/hydrafs.js mkfs card.img 16 WORK          a new 16 MB card, labelled WORK
node sim/tools/hydrafs.js mkdir card.img bin
node sim/tools/hydrafs.js put card.img programs/c/bin/hello.hyx bin
node sim/hydrasim.js -i --sd card.img
```

The Hydra boots into the card: the prompt is `0:/>` (volume 0, its root directory). `ls`, `cd`, `cat`, `cp`, `rm` and `mkdir` work on it, and `edit notes.txt` is a small editor (`h` in it for help). A program in the card's `/bin` runs by name too, and gets the rest of the line as its arguments:

```
0:/> hello world
Hello from C on the Hydra-16!
1 arguments: [world]
```

Put the lines you want at every start in `boot.hys` at the card's root (a script: lines as you'd type them). On a real Hydra, the card is the same format: copy an image to it with a disk imager, or write files with the same tool.

5. A program in C

Save this as `greet.c`, anywhere:

```
#include <stdio.h>

int main (int argc, char* argv[])
{
    int i;
    for (i = 1; i < argc; ++i) {
        printf ("Hello, %s!\n", argv[i]);
    }
    return argc > 1 ? 0 : 1;
}
```

Build it, put it on the card, and run it:

```
node build.js c                the C library, once (and after pulling changes to it)
node build.js prog greet.c     your program: programs/c/bin/greet.hyx
node sim/tools/hydrafs.js put card.img programs/c/bin/greet.hyx bin
node sim/hydrasim.js -i --sd card.img
```

```
0: /> greet Hydra world
Hello, Hydra!
Hello, world!
0: /> status .
0000
```

main's value is the program's exit status: run `greet` with no names and `status .` gives 1. The C library has the standard calls (stdio, files and directories, the environment, `malloc`, `time`), and the Hydra's own (tasks, semaphores, sound): the [C Programmer's Guide](#) has them all, with a section on making programs small and fast.

6. A program in assembly

`programs/asm/samples/hello.s` is a complete program: a header (`HYX_HEADER`, from `programs/asm/hyx.inc`), then code that calls the ROM's `WRITE_CHAR` thunk (`$F803`) to print. Copy it to a name of your own in the same folder, change the message, and build every sample there:

```
node build.js asm                programs/asm/samples/*.s -> programs/asm/bin/*.hyx
node sim/tools/hydrafs.js put card.img programs/asm/bin/hello.hyx bin
```

The calls a program can make are the thunks at `$F800` and up ([the API index](#)): the console, files, memory, tasks, the clock. A program has its own task: 32K of RAM, its own zero page and stack, and its own fds, so a crash in it doesn't take the shell down (Ctrl-C ends it).

7. An LED and a button

The VIA's port A comes out on header J27 ([hardware](#)): pins PA0-PA7 and CA1, with +5 V and ground. `/dev/gpio` makes each pin a file ([io.md](#)).

- **An LED:** an LED and a 330 Ω resistor in series, from PA2 (J27 pin 5) to ground (pin 1), the LED's short leg to ground. `echo 1 > /dev/gpio/2` lights it (the pin becomes an output, high), and `echo 0 > /dev/gpio/2` puts it out.

- **A button:** a push button from PA3 (pin 6) to ground, and a 10 kΩ resistor from PA3 to +5 V (pin 2). `cat /dev/gpio/3` says 1, or 0 while it's pressed. `cat /dev/gpio/ctl` shows every pin.
- **Waiting for a press:** wire the button to CA1 (pin 11) instead. `"/dev/gpio/ca1" 1 open .` gives an fd (say 3); then `3 here @ 8 read .` waits for the next press. (A button bounces, so one press can count as several edges: `cat /dev/gpio/ctl's ca1` line shows the count.)
- **In the emulator:** `--gpio-in F7` makes PA3 read 0 (pressed), `--ca1 20000000` presses CA1 at cycle 20 million, and the report's `--- VIA port A` line shows what the pins drive at the end.

8. Where next

- **Try the rest of the system:** pipelines (`words | wc . . .`), background programs (`play /rom/bin/scom.zsm 0 &`), the environment (`echo /sd/0/bin > /env/PATH`), `/dev/proc` (`cat /dev/proc`): the [HyForth guide](#).
- **See how it works:** tasks and the scheduler, interrupts, memory, the Plan 9-style IO, writing a device server: the [Programmer's Guide](#).
- **Build the ROM itself:** `node build.js test` builds everything and runs the regression tests ([getting started](#)); the emulator has a debugger's tools (`--trace`, `--watch`, `--pc`, `--profile`).
- **The board:** the [hardware reference](#); what's planned: [plans/NEXT_STEPS.md](#).

Getting started

How to build the ROMs, program the chips, connect a terminal and boot the Hydra-16, or run it in the emulator without the hardware.

What you need

For	Tool
Building the ROMs	cc65 (ca65 , ld65)
The page checker, emulator, tests and card tool	Node.js (no packages needed)
The board	KiCad 9 to view or change the schematics and PCB (board/)
Real hardware	A chip programmer that handles SST39SF010/040 flash; a serial terminal (9600 8N1) with an RS-232 port or USB adapter

The repository

Folder	What
os_rom/	The OS ROM: sources, build script, linker config, and the built images in os_rom/bin/
sim/	The emulator (hydrasim.js), the regression tests (regress.js), tools (tools/hydrasfs.js , tools/mkhyx.js)
programs/	Programs for the Hydra (.hyx): C (programs/c/ : the library and samples) and assembly (programs/asm/), and what to build others with
board/	KiCad schematics and PCBs: the main board, the memory daughter card, the bus breakout card
docs/	This documentation

Building

The build is [build.js](#), at the top of the repository, on Windows, Linux or macOS. It needs Node.js and cc65, which it finds in CC65_HOME (cc65's folder), or on the PATH, or in C:\source\cc65\win64_snapshot :

```
node build.js          everything: the C library and samples, the assembly sample, the ROM images
node build.js rom      just the ROM images
node build.js test     everything, then the regression tests
node build.js rom test the ROM images, then the tests
```

[os_rom/makeC02.bat](#) ([makeC02](#) , [makeC02 test](#)) runs [build.js rom](#) too. The ROM's steps, stopping at the first that fails:

- [sim/tools/hysong.js](#) compiles the test song ([songs/test.mml](#)), and the version ([os_rom/VERSION](#) , shown at boot) goes into an include file, both in [obj/](#).
- [ca65](#) assembles [all.s](#) (which includes every source file) for the 65C02, and [ld65](#) links with [os_rom_C02.cfg](#).
- [sim/tools/mkromdisk.js](#) writes the ROM disk (/rom's files, as a HydraFS volume) into the paged ROM image and reads every file back to check it, and [tools/romsum.js](#) adds the checksums the hardware test checks.
- [tools/check_pages.js](#) checks for calls between BIOS ROM pages that bypass a gate (No cross-page references); one is an error.
- [tools/rom_space.js](#) prints the space left on each BIOS ROM page, and warns when page 0 or COMMON is nearly full (node [tools/rom_space.js --table](#) in [os_rom](#) shows where).

GitHub runs `node build.js test` on every push (`.github/workflows/build.yml`).

Output	What
<code>os_rom/bin/os_rom_C02.bin</code>	The BIOS ROM image (128K): burn it into U6
<code>os_rom/bin/paged_rom_C02.bin</code>	The paged ROM image (192K: banks 0-11, with <code>/rom</code>): burn it at offset 0 of U31 (paged ROM bank 0)
<code>os_rom/obj/</code>	The object file, listing (<code>all_C02.txt</code>), labels (<code>os_rom_C02.lbl</code>), map (<code>os_rom_C02.map</code>), debug info (<code>os_rom_C02.dbg</code>), and the generated includes (<code>version.inc</code> , <code>test_rom.s</code>); not in source control

Build options (`os_rom/include/hw.inc`):

Setting	Values
<code>CPU_CLOCK_MULT</code>	1 = 3.58 MHz (default), 2 = 7.16 MHz: must match the CPU clock jumper
<code>SER_ACIA</code>	<code>SER_ACIA_ROCKWELL</code> (default) or <code>SER_ACIA_WDC</code> , for a WDC W65C51N ACIA
<code>SR_SELECT</code> , <code>SERIAL_RATE</code> , <code>SER_RATE_BOOT</code>	The serial rate at boot (default 9600; change all three together). After boot, HyForth's <code>stty</code> changes it

Programming the chips

- **Burn both images** after most changes: HyForth's variables (in the paged ROM) are where its code in the BIOS ROM expects them, and its sample binary words call BIOS ROM addresses.
- The BIOS ROM image goes into **U6** (SST39SF010, or a larger '020/'040).
- The paged ROM image goes at offset 0 of **U31**, the chip holding paged ROM banks `$00-$1F`. The image is already in chip order: the board swaps the 8K halves of each 16K bank.

Connecting a terminal

- **Cable:** use a **straight-through** RS-232 cable with two female ends (the Hydra's DE-9 is wired like a modem), including pins 7 and 8: the ACIA sends only while CTS is asserted.
- **Terminal settings:** 9600 baud, 8 data bits, no parity, 1 stop bit. Once booted, the Hydra can switch to another rate or format (HyForth: `"b19200" stty`, then switch the terminal); it comes back up at 9600 after a reset.
- **The terminal should:**
 - understand ANSI escape sequences;
 - send CR for Enter;
 - send either BS or DEL for Backspace (both work).

See the [Hardware Reference](#) for the pinout.

First boot

Fit the CPU clock jumper J7 (3.58 MHz) and the RDY jumper J4, then power up. You should see:

```

POST ZP:T ST:T L0:T 7D:T SH:S P1:4C
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000

Welcome to the HYDRA-16! OS 1.8C_0.6

HyForth 0.91
No card: /ram keeps your files until a reset. ls /rom/bin for the programs.

/ram>

```

- **The first two lines are POST**, the power-on self test ([what they mean](#)).
- **With no card**, the prompt is `/ram>`: the shell's own area on the RAM disk, where files last until a reset ([the RAM disks](#)).
With a HydraFS card in, `hydrafs 0` comes before HyForth's banner, and the prompt is `0:/>`: you're at card 0's root ([the shell](#)).
- **A driver that fails to start** prints `NAME FAIL ee` (ee = the [error code](#)).
- **Try it:** `1 2 + .` prints `0003`.

Then try:

- `words` (every word);
- `ps` (the tasks);
- `words | wc . . .` (a pipeline);
- `sndtest` (the test tune: `sndstop` stops it);
- `bye` (WOZMON).

The [HyForth guide](#) has the rest.

Without the hardware: the emulator

`sim/hydrasim.js` emulates the whole board, cycle-counted, and boots the same ROM images:

```

node sim/hydrasim.js -i          use the Hydra from this terminal, in real time
node sim/hydrasim.js -i --sd card.img  with an SD card (an image file)

```

Ctrl-A x quits, and Ctrl-A r presses the reset button. To make a card image with files on it:

```

node sim/tools/hydrafs.js mkfs card.img 64
node sim/tools/hydrafs.js import card.img myfiles

```

The Hydra sees them at `/sd/0`, and starts there: the prompt is `0:/>`, and `ls` lists the card's root. A `boot.hys` in it runs at boot.

Without `-i`, the emulator runs a fixed number of cycles with scripted input, then prints a report: the serial output, the last instructions, the hottest code, and each task's stack depth. That's the mode for debugging and the tests. See [the emulator](#).

Testing a change

```

cd os_rom
node build.js rom test

```

This builds, then boots the new images in the emulator for each regression test (`node sim/regress.js --list` lists them): POST, the self tests, HyForth and its libraries, line editing, pipes, tasks, sound, SD cards and their files, the shell and running programs, sleeping, C programs, exit statuses, the sound library and songs, `/rom`, fault injection. It prints `N of N tests passed`, or the failing test's output and the command that reproduces it. Then try it on the board.

Where to go next

To	Read
Use the system	HyForth , WOZMON
Program it	Programmer's Guide
Understand or change the board	Hardware Reference
Debug with the emulator, write tests	Emulator and tools
See what's planned	Plans

Hydra-16 Hardware Reference

This is the Hydra-16 main board (V1) as its schematic describes it (`board/hydra-16.kicad_sch` and its sheets), with the two companion cards in `board/`. Reference designators (U25, J18, ...) are the schematic's. For how the software uses the hardware, see the [Programmer's Guide](#).

To test a board, run the [hardware test](#) (HyForth's `hwtest`, or `T` typed during POST).

Contents

1. Overview
2. The CPU view: memory map
3. The pseudo-registers T, U, V, W
4. Task RAM and the bank registers (\$00, \$01)
5. The paged RAM window (\$8000-\$9FFF)
6. The paged ROM (\$A000-\$DFFF)
7. The BIOS ROM (\$E000-\$FFFF)
8. I/O space (\$FF00-\$FFFF)
9. Interrupts
10. Clocks
11. Reset, power and bus control
12. On-board devices: VIA, ACIA (serial), YM2151 (sound), SPI
13. Expansion slots
14. Companion cards: the memory daughter card, the bus breakout card
15. Connectors and jumpers
16. V1 errata
17. Parts by function

Overview

The Hydra-16 is a W65C02S computer built for multitasking. Its address decoding gives each of **16 tasks** its own 32K of RAM, including zero page and the stack, and its own RAM and ROM bank selections. So switching tasks is a single write to one register, T.

CPU	WDC W65C02S (U1), 3.58 MHz as built (jumper: 0.89, 1.79, 3.58 or 7.16 MHz)
Task RAM	512K (U7, HM628512): 32K at \$0000-\$7FFF for each of 16 tasks
Paged RAM	Seen through an 8K window at \$8000-\$9FFF. Task banks: up to 15 modules of 2 MB on memory daughter cards; each task sees 16 banks of 8K per module. Shared banks: 2 MB on the board (U25, U27-U29), the same for all tasks: 256 banks of 8K, 16 at a time
Paged ROM	4 MB (U30-U37, 8 x SST39SF040), seen in 16K banks at \$A000-\$DFFF
BIOS ROM	U6, SST39SF010/020/040 (128-512K): 8K pages at \$E000-\$FFFF, selected by W
I/O	15 ports of 16 bytes at \$FF00-\$FFEF, plus the system port at \$FFF0-\$FFFF
Interrupts	16 prioritised IRQ lines, each with its own vector (a 16-entry vector RAM at \$FFFE)
Devices	65C22 VIA (timers, SPI, I2C, GPIO), 65C51 ACIA (RS-232 on a DE-9), YM2151 + YM3012 (stereo FM sound)

Expansion	6 slots (62-pin edge connectors, 8-bit bus), 8 SPI device headers, a GPIO/I2C header, memory card connectors
Power	ATX-24 supply (+5 V, +3.3 V, +12 V, -12 V)

The CPU view: memory map

What the CPU sees at each address, and what selects it:

Addresses	What	Selected by
\$0000	RAM bank register (writes); task RAM (reads)	per task (see below)
\$0001	ROM bank register (writes); task RAM (reads)	per task
\$0002-\$7FFF	Task RAM (zero page, stack, program RAM)	T
\$8000-\$9FFF	Paged RAM: a task bank (\$00-\$EF) or a shared bank (\$F0-\$FF)	\$00 ; T for task banks, U for shared banks
\$A000-\$DFFF	Paged ROM, 16K banks	\$01
\$E000-\$FFFF	BIOS ROM	W
\$FF00-\$FFEF	I/O ports 0-14, 16 bytes each	
\$FFF0-\$FFF3	Pseudo-registers T, U, V, W	
\$FFF4-\$FFF9	Not decoded (reads float)	
\$FFFA-\$FFFD	BIOS ROM: the NMI and RESET vectors	W
\$FFFE-\$FFFF	IRQ/BRK vector RAM (16 entries)	the highest-priority active IRQ, or V

Every task has its own \$0000-\$7FFF and its own \$00 / \$01 values. All tasks share the I/O space, the shared RAM banks, the ROMs, and the pseudo-registers themselves.

The pseudo-registers T, U, V, W

(Sheet FFF_Registers)

Four 8-bit registers in the system port. Each is a 74F573 latch, written with the CPU's write strobe, and read back through a 74F541 buffer. So a read gives the whole byte last written, all 8 bits.

Address	Register	Latch / read-back	Bits used by the hardware
FFFF0	T, task	U48 / U49	T0-T3: which task's RAM (task RAM A15-A18; the modules' task lines)
FFF1	U, shared macro-page	U50 / U51	U0-U3: shared RAM A13-A16
FFF2	V, vector select	U52 / U53	V0-V3: the vector RAM index when no IRQ is active. V4-V7 aren't wired: the software keeps a software interrupt number there
FFF3	W, BIOS ROM page	U54 / U55	W0-W5: BIOS ROM A13-A18 (64 pages with an SST39SF040; 16 with the '010)

The latches have no reset, so they power up random; the reset code sets them. Nothing on the board resets them when the reset button is pressed. Bits 4-7 of T and U, and 6-7 of W, are stored but go nowhere.

Task RAM and the bank registers

Task RAM is one HM628512 (U7, 512K) on the main board. CPU A0-A14 go straight to it, CPU A15 is its chip enable (low for \$0000-\$7FFF), and T0-T3 drive its A15-A18. So each value of T selects a different 32K, and a task switch is a single write to

\$FFF0. Zero page and the stack page are part of it, so every task has its own zero page and stack.

A DS1747 in U7 gives the Hydra a clock that keeps the time while it's off. The DS1747 (the 5 V part; the DS1747W is 3.3 V) is a 512K battery-backed RAM with a clock, pin compatible with the HM628512, in a 600-mil module. Its clock registers are the chip's top 8 bytes, \$7FFF8-\$7FFFF, so they're **task F's** \$7FF8-\$7FFF (T0-T3 reach U7's A15-A18 through U48 and U21 in order). The ROM keeps off those bytes in every task, finds the chip at boot, and sets its clock from it ([HyForth](#)). The rest of the chip is task RAM as before, kept while the power's off. A new DS1747 comes with its battery disconnected until it first gets power, and its oscillator may be stopped: setting the time starts it.

The bank registers \$00 and \$01 (*sheet* ZPMirrorRAM) are four 74LS219 (16 x 4-bit RAM) chips, addressed by T0-T3:

Register	Chips	Drives
\$00: RAM bank	IC1 (bits 0-3), IC2 (bits 4-7)	RAMB0-7: which bank is at \$8000-\$9FFF
\$01: ROM bank	IC3 (bits 0-3), IC4 (bits 4-7)	ROMB0-7: which paged ROM bank is at \$A000-\$DFFF

- A write to \$0000 or \$0001 stores the byte in the current task's entry. The address decoding (U8-U10 on sheet *AddressDecode*) spots a write to \$000x, and U16 picks the register with A0.
- The write also goes to task RAM as usual, and a **read** of \$00/\$01 comes from task RAM: the registers themselves are write-only, and task RAM mirrors them.
- The outputs are always enabled, so the bank lines follow T at once. A task switch selects the new task's RAM bank and ROM bank without further writes.
- On V1, bits 2 and 3 of both registers are crossed on their way to the bank lines (see [V1 errata](#)).

The paged RAM window

(*\$8000-\$9FFF; sheets* ZPMirrorRAM, SharedMem, AddressDecode)

\$8000-\$9FFF shows one 8K bank. The **bank ID** is the current task's \$00 value:

Bank ID	What
\$m0-\$mF (m = \$0-\$E)	Task RAM module m: 16 banks for each task (the module's chips also get T0-T3)
\$F0-\$FF	Shared RAM: 16 banks, the same for every task, from shared macro-page U

- **Module select.** The bank ID's upper nibble (RAMB4-7) goes to a 74LS154 (U23), qualified by the window's select (nBRA_S, \$8000-\$9FFF in PHI2). Its outputs nBRC0-nBRC14 go to the memory card connector (J1), one per module; nBRC15 selects the shared RAM on the board.
- **Memory card connector.** The lower nibble and T are buffered to it by U21 (74F541), as RAMB_M0-3 and T_M0-3, along with the read/write strobes (RWB_M, !RWB_M). J2 carries D0-D7, A0-A12 and +5 V.
- **Task RAM modules** are *memory daughter cards*: 4 x HM628512 per card, set to a module number with a jumper. In a module:
 - T_M0-3 go to the chips' A13-A16, so each task has its own 16 banks.
 - RAMB_M0-1 go to A17-A18.
 - RAMB_M2-3 pick the chip.
 - A module is 2 MB: 16 tasks x 16 banks x 8K.
- **Shared RAM** is four HM628512 on the main board: U25, U27, U28 and U29.
 - RAMB_M2-3 pick the chip, through a 74F139 (U26) enabled by nBRC15.
 - RAMB_M0-1 go to the chips' A17-A18, and U0-U3 to A13-A16.
 - So the 16 bank IDs \$F0-\$FF are seen in 16 macro-pages (U = 0-F): 256 shared banks of 8K, 2 MB.
 - The software numbers them as **shared bank ID** = U << 4 | (bank & \$0F).

Bank IDs (\$00)	Shared RAM chip (V1)	Shared bank IDs
\$F0-\$F3	U25	\$x0-\$x3 for each U = x
\$F4-\$F7	U28 (V1: bits 2/3 crossed)	\$x4-\$x7
\$F8-\$FB	U27 (V1)	\$x8-\$xB
\$FC-\$FF	U29	\$xC-\$xF

A bank with no chip behind it (a missing module) reads whatever floats on the bus; the ROM's POST and module probe detect this.

HM628512 pins, for tracing a bad line (the POST prints bad lines by number):

Signal	Pin	Signal	Pin
A0-A7	12, 11, 10, 9, 8, 7, 6, 5	D0-D2	13, 14, 15
A8, A9, A10, A11, A12	27, 26, 23, 25, 4	D3-D7	17, 18, 19, 20, 21
A13-A16 (U0-U3 on shared RAM)	28, 3, 31, 2	~CE, ~OE, ~WE	22, 24, 29
A17, A18 (RAMB_M0 , RAMB_M1)	30, 1	VCC, GND	32, 16

The paged ROM

(\$A000-\$DFFF ; sheets BankedROM, ZPMirrorRAM)

Eight SST39SF040 (512K each, U30-U37) give 4 MB, seen as 256 banks of 16K. The bank is the current task's \$01 value:

- ROMB0-4 go to the chips' A14-A18 (32 banks per chip).
- ROMB5-7 pick the chip through a 74F138 (U24), qualified by the window's select (nBRO_S : \$A000-\$DFFF in PHI2):

\$01 bits 5-7	Chip	Banks
0	U31	\$00-\$1F
1	U32	\$20-\$3F
2	U34	\$40-\$5F
3	U36	\$60-\$7F
4	U30	\$80-\$9F
5	U33	\$A0-\$BF
6	U35	\$C0-\$DF
7	U37	\$E0-\$FF

- **The halves are swapped.** CPU A13 goes to the chips' A13 unchanged, but in the window \$A000-\$BFFF has A13 = 1 and \$C000-\$DFFF has A13 = 0. So CPU \$A000 reads chip offset \$2000 of the bank, and \$C000 reads offset \$0000. The build writes paged_rom_C02.bin in chip order for this, so burn it at offset 0 of U31.
- **V1: bits 2 and 3, and 6 and 7, trade places.** On the V1 board ROMB2/ROMB3 and ROMB6/ROMB7 are swapped on their way to the chips (as are the RAM bank bits; the schematic shows the board as built), so the bank the CPU selects as b is the chips' bank swap(b). Banks whose two bits match (\$00-\$03, \$0C-\$0F, ...) aren't affected. The build writes the image in the chips' order (sim/tools/mkromdisk.js), and the emulator reads it that way.
- nBROMD (a slot pin, pulled up) disables the whole paged ROM when a card pulls it low, so the card can answer \$A000-\$DFFF itself.
- The chips' ~OE is the inverted R/W; there's no write path in circuit (program the chips in a programmer).

The BIOS ROM

(U6, root sheet)

An SST39SF0x0 in a 32-pin socket: the '010 (128K, 16 pages), '020 (256K, 32 pages) or '040 (512K, 64 pages).

- CPU A0-A12 go to the chip's A0-A12, and W0-W5 to its A13-A18: each W value selects an 8K page.
- It's selected for \$E000-\$FEFF and for \$FFFA-\$FFFD (the NMI and RESET vectors). I/O space and the vector RAM take the rest of \$FF00-\$FFFF.
- The build produces `os_rom_C02.bin`, 128K, for the '010.

Changing W changes the code being run. The next instruction is fetched from the new page, at the same address. The software handles this by keeping identical code at the same address on every page: the COMMON block at \$FD00, and the reset entry at \$E000 (see [ROM layout](#)). Because W isn't reset, **every page must start with the reset code**: the RESET vector on every page points to \$E000, which sets W to 0.

I/O space

(Sheet AddressDecode)

\$FF00-\$FFFF is decoded when A8-A15 are all 1 (U14, a 74F30), qualified by PHI2 (`nIO_S`). A 74LS154 (U19) splits it by A4-A7 into 16 ports of 16 bytes, `nIOP0_S`-`nIOP15_S`. Ports 0-14 are for devices; port 15 is the system port.

Port	Addresses	Use
0	\$FF00-\$FF0F	VIA (65C22, U2)
1	\$FF10-\$FF1F	ACIA (65C51, U3): 4 registers, repeated every 4 bytes
2	\$FF20-\$FF2F	Slot 0, select A (intended for video)
3	\$FF30-\$FF3F	Slot 0, select B (intended for video)
4	\$FF40-\$FF4F	YM2151 (U38): \$FF40 address/status, \$FF41 data (repeated)
5-9	\$FF50-\$FF9F	Slots 1-5, select A
10-14	\$FFA0-\$FFEF	Slots 1-5, select B
15	\$FFF0-\$FFFF	System port (below)

The **system port** is decoded by a 74F138 (U20) on A0-A2, enabled by `nIOP15_S` and by A3 differing from A2 (U12):

Address	Select	Use
\$FFF0-\$FFF3	<code>nIOB0-3</code>	T, U, V, W
\$FFF4-\$FFF9	none	Not decoded (reads float; reserved)
\$FFFA-\$FFFD	(BIOS ROM)	NMI and RESET vectors, from the current BIOS ROM page
\$FFFE, \$FFFF	<code>nIOB14</code> , <code>nIOB15</code>	The vector RAM, low and high bytes

Slot port selects (`nIOA_S`, `nIOB_S` on each slot) and each slot's two IRQ lines follow the same numbering: see [Expansion slots](#). A card can also decode its own addresses from the whole I/O space with `nIO_S` and A0-A7.

Interrupts

(Sheet IRQ_Priority_Encoder)

Sixteen active-low IRQ lines, `nIRQ0`-`nIRQ15`, each pulled up (RN2, RN3). Line 0 has the highest priority.

Line	Source	Line	Source
0	VIA	8	Slot 4, A
1	ACIA	9	Slot 5, A
2	Slot 0, A	10	Slot 1, B
3	Slot 0, B	11	Slot 2, B
4	YM2151	12	Slot 3, B
5	Slot 1, A	13	Slot 4, B
6	Slot 2, A	14	Slot 5, B
7	Slot 3, A	15	Not connected: used for software interrupts

How it works:

- Two cascaded 74LS148 priority encoders (U17 for lines 0-7, U18 for 8-15) find the highest-priority active line.
- Any active line asserts the CPU's $\sim$ IRQ (U44, then U22). The IRQ input is level-triggered: a device holds its line until the handler clears it.
- A 74F157 (U45) forms a 4-bit index q_0 - q_3 :
 - while any IRQ line is active, the index is the encoder's output;
 - otherwise it's v_0 - v_3 .
- The index addresses the **vector RAM**, four 74LS219 (IC5-IC8), a 16-entry table of 16-bit vectors. The CPU reads its IRQ/BRK vector from $\$FFFE/\$FFFF$, so it gets the entry for the active line, and each line has its own handler.

The index is the line number XOR 7. The '148s encode active-low inputs with the highest-priority input as 7, so line n gives index $n \wedge 7$: line 0 is entry 7, line 7 entry 0, line 8 entry 15, line 15 entry 8. The ROM's `IRQ_NUMBER(n)` macro is $n \wedge 7$ for this reason.

Writing vectors. A write to $\$FFFE / \$FFFF$ stores the low / high byte of entry q . That's v_0 - v_3 when no IRQ line is active. So set v to the entry wanted, then write the vector, with no IRQ pending. The ROM does this once at boot with interrupts off.

BRK and software interrupts. BRK also reads $\$FFFE/\$FFFF$. With no IRQ line active, that's entry v_0 - v_3 . So a software interrupt is: set v to `IRQ_NUMBER(15)` (line 15 has no hardware), then `brk`. The ROM keeps the software interrupt number in v_4 - v_7 .

NMI (NMIB, pulled up, on every slot) uses the ROM's $\$FFFA$ vector. Nothing on the board drives it.

Clocks

(Sheet Clocks)

A 14.31818 MHz crystal (Y1) with a 74ACT14 (U39) oscillator, divided by a 74F191 counter (U40):

Clock	Frequency	Goes to
HS_CLK	14.318 MHz	Slots (e.g. for video)
Q0	7.159 MHz	CPU clock jumper J8
SND_CLK (Q1)	3.580 MHz	YM2151 master clock; slots; CPU clock jumper J7
SER_CLK (Q2)	1.790 MHz	ACIA clock (XTAL1); slots; CPU clock jumper J6
Q3	0.895 MHz	CPU clock jumper J5

The CPU clock (PHI2) is whichever of J5-J8 is fitted: fit exactly one. As built, it's **J7, 3.58 MHz**. The CPU's PHI0 input is PHI2 gated by DMAB (U11), so a card asserting DMAB stops the CPU clock (the W65C02S is fully static). PHI1 (inverted PHI2) qualifies the address decoding.

The ROM's timing (the scheduler's tick, sound note lengths, serial timeouts) is built for one clock: `CPU_CLOCK_MULT` in `os_rom/include/hw.inc` (1 = 3.58 MHz, 2 = 7.16 MHz). At 7.16 MHz the YM2151 is too slow for the bus: there are no wait states on V1, so don't use the sound chip at that speed. The 0.89 and 1.79 MHz settings aren't supported by the ROM's timing.

Reset, power and bus control

Power comes from an ATX-24 connector (J11):

- +5 V for the logic, +3.3 V, and ± 12 V for the audio op-amps and the slots.
- +5 V standby is brought out as +5VA.
- The slots also have a -5 V pin, which the board doesn't supply: ATX no longer has -5 V.

Front panel (J10, 2x6):

Pin	Signal
7	RESB : a push button from here to ground (pins 5, 6, 9 or 10) resets the machine
8	+5 V
11	ATX PS_ON# : connect to ground (a latching switch) to turn the supply on
12	Power LED: from the supply's PWR_OK , through R2 (470 Ω)
5, 6, 9, 10	Ground
1-4	Not connected

Reset (RESB):

- A DM1813 supervisor (U43) holds reset low at power-up and during brown-outs.
- The front panel button and the slots (open-collector) can pull it low too.
- It resets the CPU, VIA, ACIA, YM2151 and YM3012. It doesn't reset the pseudo-registers, the bank registers or RAM.

Bus control:

- RDY : pulled up through jumper **J4**, which must be fitted. The W65C02S drives RDY low itself during WAI , so RDY must never be driven high directly.
- BE (bus enable): pulled up, and on the slots.
- **DMA**: a card pulls DMAB low to take the bus.
 - It stops the CPU clock, and turns off the address buffers (U58, U60, 74F541), the data transceiver (U59, 74F245) and the R/W buffer (U61, 74F125).
 - The card then drives the address and data buses itself.
- SYNC from the CPU is on the slots, for single-stepping or debugging hardware.
- MLB , VPB are unused. SOB is tied high.

On-board devices

VIA (65C22, U2): port 0, IRQ line 0

(Registers at \$FF00-\$FF0F.)

Port A is general purpose I/O, on header **J27** (2x6), with ESD protection (J28, SP720):

J27 pin	Signal	J27 pin	Signal
1	GND	2	+5 V
3	PA0: I2C_SCL	4	PA1: I2C_SDA

J27 pin	Signal	J27 pin	Signal
5	PA2	6	PA3
7	PA4	8	PA5
9	PA6	10	PA7
11	CA1	12	CA2

- PA0/PA1 are the I2C bus (bit-banged; the SDA and SCL pull-ups are in RN1), which also goes to every slot. No I2C driver exists yet.
- **The pins, CA1 and CA2 are files:** `/dev/gpio` ([io.md](#)); CA1 can interrupt (IRQ line 0, `/dev/gpio/ca1`).
- **Port B is the SPI bus** (below).
- **Timer 1** is the scheduler's tick: free-running, 200 interrupts a second.
- **Timer 2** paces serial output when the ROM is built for a WDC ACIA.
- CB1/CB2 aren't brought out.

ACIA (65C51, U3): port 1, IRQ line 1

(Registers at \$FF10-\$FF13.)

Chip and clock:

- The board has a Rockwell R65C51 socket. A WDC W65C51N works if the ROM is built for it: its transmitter status and interrupt don't work, so the ROM paces sending with VIA timer 2 (`SER_ACIA` in `hw.inc`).
- **The ACIA's clock is** `SER_CLK`, **1.790 MHz**, not the 1.8432 MHz its baud rate table is made for. So every rate is 2.9% slow (9600 gives about 9,320 baud), which terminals and USB serial adapters accept.
- DCD and DSR are tied active.

Serial settings: 9600 baud, 8 data bits, no parity, 1 stop bit at boot, with RTS/CTS. The ROM can change the rate (300 to 19200, and 115200: the ACIA clock / 16), the data bits (5-8), the parity and the stop bits afterwards (`/dev/ser/ctl`, [io.md](#)).

115200 is paced. Sent back to back at 115200, long output (a WOZMON dump) loses and garbles characters on the board: the rate is 2.9% slow and the MAX232 is near its limit, so the receiver has little margin. A second stop bit helps but isn't enough. So at 115200 the ROM sends each byte from VIA timer 2 rather than the Rockwell ACIA's TDRE interrupt, with at least `SER_PACE_GAP` (2) idle bits after each character, about 4.5 with the interrupt's own time (with 1, a few characters in a whole-memory WOZMON dump were lost). That's about 7,000 characters a second. If long output still loses characters, raise `SER_PACE_GAP` in `os_rom/include/hw.inc` (each bit costs about 7%). The other rates are sent back to back, as before.

The DE-9 connector (J3, male) is driven by a MAX232 (U5), and wired like a modem (DCE):

DE-9 pin	Signal	Direction
2	Hydra transmits (ACIA TxD)	out
3	Hydra receives (ACIA RxD)	in
5	Ground	
7	ACIA ~CTS (from the terminal's RTS)	in
8	ACIA ~RTS (to the terminal's CTS)	out

Cable: a PC or USB serial adapter connects with a **straight-through** cable with two female ends, not a null-modem cable. The ACIA only transmits while its ~CTS is asserted, so the cable must carry pin 7. Terminal programs assert RTS by default.

YM2151 sound (U38): port 4, IRQ line 4

(Sheet [Sound](#).)

The chip:

- A Yamaha YM2151 (OPM, 8 FM channels), clocked by `SND_CLK` (3.58 MHz).
- `$FF40` selects a register (and reads the status: bit 7 = busy), and `$FF41` writes it.
- After a data write the chip is busy for 64 of its clocks (about 18 μ s); writes while it's busy are lost.
- **CT1/CT2**, its two general-purpose outputs, are on header J9.
- **Timer B on the board**: re-armed every song tick (its flag reset through `$14`, which on the board seems to restart it), it didn't keep its period: songs timed by it ran up to twice as fast, by a different amount each run. So the song player times songs by the system's tick (the VIA's timer 1) instead; the hardware test's timer checks (one period from a clean start) pass.

The audio path:

- A YM3012 DAC (U41) converts its serial output, buffered and filtered by a TL074 (U42).
- A mixer (IC9, LF353, ± 12 V) adds, for each channel:
 - the YM3012's output;
 - each slot's audio lines (`SND_CL0-5`, `SND_CR0-5`), so a sound card in a slot is mixed in;
 - a line input on header J29.
- **Output**: stereo, on the 3.5 mm jack J26. The schematic has the tip on the right channel and the ring on the left, the reverse of the usual convention (see [V1 errata](#)).

SPI bus (VIA port B)

SPI is bit-banged on the VIA's port B (the VIA's shift register uses CB1/CB2, which aren't brought out, so it can't drive these lines); a 74F138 (U4) decodes the device select:

Port B bit	Signal
PB0	<code>SPI_CLK</code> (SCLK)
PB1	<code>nSPI_CS</code> : low selects the device given by PB3-PB6
PB2	<code>SPI_MOSI</code>
PB3-PB5	<code>SPI_A0-A2</code> : device 0-7
PB6	<code>SPI_A3</code> : 0 = the board's devices 0-7 (U4); 1 = devices 8-15, for cards to decode (the slots carry <code>nSPI_CS</code> and <code>SPI_A0-A3</code>)
PB7	<code>SPI_MISO</code>

The eight SPI device headers are J18-J25, for devices 0-7. Each is a 1x6 header:

Pin	Signal
1	$\sim$ CS
2	SCLK
3	MOSI
4	MISO
5	+5 V
6	GND

The ROM runs SPI in mode 0 for SD cards, and its storage server serves an SD card on any device (`/dev/sd/N`); device 0 (J18) is the usual place for an SD card adapter. Any other device is a file, `/dev/spi/N` (modes 0 and 3: [io.md](#)). The headers supply +5 V, so the adapter must regulate and level-shift to 3.3 V for the card (common SD card modules do).

Expansion slots

(Sheet Connectors: J12-J17, "Hydra bus 8-bit", 62-pin card edge.)

Six slots, all carrying the same bus except for each slot's two port selects, two IRQ lines and its audio pair:

Slot	Connector	nIOA_S (pin 51)	nIOB_S (pin 49)	IRQA (pin 55)	IRQB (pin 53)	Audio (SND_CL/CR)
0	J12	port 2 \$FF20	port 3 \$FF30	line 2	line 3	pair 0
1	J13	port 5 \$FF50	port 10 \$FFA0	line 5	line 10	pair 1
2	J14	port 6 \$FF60	port 11 \$FFB0	line 6	line 11	pair 2
3	J15	port 7 \$FF70	port 12 \$FFC0	line 7	line 12	pair 3
4	J16	port 8 \$FF80	port 13 \$FFD0	line 8	line 13	pair 4
5	J17	port 9 \$FF90	port 14 \$FFE0	line 9	line 14	pair 5

Slot 0's A lines (port 2, IRQ 2) have the highest priority of the slots; the B lines of slots 1-5 the lowest.

Pinout (the same on every slot; ~ = active low):

Pin	Signal	Pin	Signal
1	GND	2	~nBROMD (disable the paged ROM)
3	~RESB	4	D7
5	+5 V	6	D6
7	~NMIB	8	D5
9	-5 V (not supplied)	10	D4
11	BE	12	D3
13	-12 V	14	D2
15	SER_CLK (1.79 MHz)	16	D1
17	+12 V	18	D0
19	GND	20	RDY
21	R/~W	22	SND_CLK (3.58 MHz)
23	~DMAB	24	SND_CR (audio in, right)
25	~nIO_S (any I/O port)	26	SND_CL (audio in, left)
27	SYNC	28	I2C_SDA
29	SPI_SCLK	30	I2C_SCL
31	SPI_MOSI	32	A15
33	SPI_MISO	34	A14
35	~nSPI_CS	36	A13
37	PHI1	38	A12
39	PHI2	40	A11
41	SPI_A3	42	A10
43	SPI_A2	44	A9
45	SPI_A1	46	A8
47	SPI_A0	48	A7
49	~nIOB_S	50	A6
51	~nIOA_S	52	A5
53	~IRQB	54	A4
55	~IRQA	56	A3
57	+5 V	58	A2
59	HS_CLK (14.318 MHz)	60	A1
61	GND	62	A0

A card's IRQ outputs should be open-collector: the board pulls each line up. The address and data buses are buffered on the board; a DMA card drives them after asserting $\sim$ DMAB.

Companion cards

Memory daughter card

(board/MemoryDaughterCard : *through-hole and surface-mount versions.*)

One task RAM module: 4 x HM628512, 2 MB.

- **Connectors:** it plugs into the main board's memory connectors J1 (1x32) and J2 (1x24).
- **Module number:** a 2x15 jumper block (J3) connects the card's select to one of nBRC0-nBRC14. That's its module number m, so it answers bank IDs \$m0-\$mF.
- **Addressing:**
 - a 74LS139 (U1) picks one of the four chips with RAMB_M2-3;
 - RAMB_M0-1 go to the chips' A17-A18, and T_M0-3 to A13-A16;
 - A0-A12 come from J2.

Main board J1 (memory card connector):

Pins	Signals
1-3	A13, A14, A15 (the card doesn't use them)
5-8	T_M0-3
9-12	RAMB_M0-3
13, 14	RWB_M, !RWB_M
15-17	GND
18-32	nBRC0_S-nBRC14_S (module selects)

J2: D0-D7 (pins 1-8), +5 V (9-11), A0-A12 (12-24).

The ROM finds the installed modules at boot (MMU_PROBE_MODULES), tests each one's first bank in POST, and only allocates banks on modules that are present.

Bus breakout card

(board/HydraBusBreakoutCard.) A slot card that brings every bus signal out to headers, for a logic analyzer, a scope or prototyping:

Header	Signals
J2	Data bus
J3	Address bus
J4	SPI (A0-A3, $\sim$ CS, MISO, MOSI, SCLK)
J5	NMI, nIO_S, the slot's port selects and IRQs
J6	Clocks
J7	Bus control (nBROMD, BE, RDY, R/W, SYNC)
J8	Reset
J9	Audio
J10	I2C
J11	Power

Connectors and jumpers

Ref	What	Notes
J1, J2	Memory daughter card	Task RAM modules
J3	DE-9 male: serial console	DCE wiring; straight-through F-F cable
J4	RDY pull-up jumper	Fit
J5-J8	CPU clock jumpers: 0.89 / 1.79 / 3.58 / 7.16 MHz	Fit one : J7 (3.58 MHz) for the standard ROM
J9	YM2151 CT1/CT2 outputs	
J10	Front panel	Reset, power switch, power LED
J11	ATX-24 power	
J12-J17	Expansion slots 0-5	
J18-J25	SPI devices 0-7	SD card adapter on J18 (device 0)
J26	Audio out, 3.5 mm stereo	
J27	VIA port A: GPIO, I2C, CA1/CA2	
J28	SP720 ESD protection for J27 and the SPI lines	(a part, not a connector)
J29	Line input to the audio mixer	

V1 errata

- **Bank register bits 2 and 3 are crossed.**
 - In the bank registers (sheet `ZPMirrorRAM`), data bit 2 drives `RAMB3`, and bit 3 drives `RAMB2`: IC1 for the RAM bank, IC3 for the ROM bank.
 - So a bank ID's bits 2 and 3 trade places before they reach the hardware. For example, shared bank IDs `$F4-$F7` are on U28 and `$F8-$FB` on U27, module 4 and module 8 trade places, and so do paged ROM banks `$04-$07` and `$08-$0B`.
 - IDs whose bits 2 and 3 are equal aren't affected, and the software never needs to care: an ID always reaches the same memory. It matters when you map a bank ID to a chip, for example to act on a POST report.
 - **Bits 6 and 7** are crossed too, on the V1 board as built (`RAMB6` / `RAMB7`, `ROMB6` / `ROMB7`). The ROM's tools and the emulator assume both swaps for the paged ROM (`sim/tools/mkromdisk.js`, `os_rom/tools/romsum.js`).
- **No wait states.** RDY only has a pull-up, so slow devices can't stretch a bus cycle. This is why the YM2151 can't be used above 3.58 MHz. Board V2 is planned to have programmable RDY wait states (see [plans/IDEAS.md](#)).
- **Audio jack channels.** J26 has the right channel on the tip and the left on the ring, per the schematic; the usual convention is the reverse, so left and right may come out swapped.
- **ACIA clock.** The ACIA runs from 1.790 MHz instead of 1.8432 MHz, so its baud rates are 2.9% slow (see [ACIA](#)).

Parts by function

Function	Parts
CPU	U1 W65C02S
Bus buffers	U58, U60 74F541 (address); U59 74F245 (data); U61 74F125 (R/W)
Address decoding	U8, U9 74LS260; U10 74F20; U11 74F08; U12 74F86; U13, U16 74F32; U14 74F30; U15 74F00; U19 74LS154 (I/O ports); U20 74F138 (system port); U21 74F541 (memory card buffer); U22 74F240
Pseudo-registers	U48, U50, U52, U54 74F573; U49, U51, U53, U55 74F541; U56 74F32; U57 74F02

Function	Parts
Bank registers	IC1-IC4 74LS219; U23 74LS154 (module select); U24 74F138 (paged ROM chip select)
Interrupts	U17, U18 74LS148; U44 74F00; U45 74F157; IC5-IC8 74LS219 (vector RAM); RN2, RN3 pull-ups
Task RAM	U7 HM628512
Shared RAM	U25, U27, U28, U29 HM628512; U26 74F139
Paged ROM	U30-U37 SST39SF040
BIOS ROM	U6 SST39SF010/020/040
Clocks	Y1 14.31818 MHz; U39 74ACT14; U40 74F191
Reset	U43 DM1813
VIA, SPI	U2 R65C22; U4 74F138 (SPI device select)
Serial	U3 R65C51; U5 MAX232
Sound	U38 YM2151; U41 YM3012; U42 TL074; IC9 LF353

HyForth: the Hydra's shell

HyForth is the Hydra-16's shell and programming language: a small Forth that starts in task 1 at boot, and prompts with the card and directory it's in (`0: />`). It began as AGSB's Forth engine, adapted by Patrick Struthers for the Hydra. Sources: `os_rom/hyforth/` . See also [WOZMON](#) and the [Programmer's Guide](#).

Contents

1. [The basics](#)
2. [Numbers](#)
3. [Strings](#)
4. [Defining words](#)
5. [Control flow: the training scripts](#)
6. [Word reference](#)
7. [The base and its libraries](#)
8. [The shell: directories, files and programs](#)
9. [Files and devices](#)
10. [Pipelines](#)
11. [Tasks and the console](#)
12. [Background tasks and exit statuses](#)
13. [Errors and keys](#)
14. [How HyForth uses memory](#)

The basics

HyForth reads a line, then runs each word in it, left to right. Numbers go on the **data stack**; words take their arguments from it and leave results on it.

```
0: /> 1 2 + .
0003
0: /> words
```

- **Words are separated by spaces.** Names are case-sensitive.
- **Lines** can be up to 255 characters.
- `.` prints and drops the top of the stack, and `.S` shows the whole stack. The format is `S<address> <depth> <items, top first>`, and `xS` clears it.
- `words` lists every word: the ones you've defined, then the base's, then each loaded [library's](#).
- `bye` leaves HyForth for [WOZMON](#) (in a script that `run` started, it ends the script). `Ctrl-\` starts a fresh HyForth.
- **The prompt** shows the current card and directory: `0:/games>` is `/games` on card 0 ([the shell](#)).

Stack effects are written (`before -- after`), with the top of the stack on the right.

Numbers

- **Input** is decimal: `11`, `-2`. A `$` prefix gives hex (`$B`, `$FFFF`), and `%` binary (`%101`).

- **Output** is 4 hex digits at first: `11 .` prints `000B`. After `decimal`, `.` prints in decimal, signed (`-264 .` prints `-264`), and `u.` unsigned (`$FFFF u.` prints `65535`); hex goes back. Input isn't affected: it's decimal, with `$` for hex, either way.
- **Size:** numbers are 16 bits. Decimal input is signed, `-32768` to `32767` (a bigger one isn't a number: `!UNK WORD!`); hex input covers `$0000-$FFFF`.
- **Inside a definition**, a number is compiled into the word, as in any Forth: `: x 65 . ;` prints `0041` each time `x` runs. (`lit [ 65 , ]` does the same by hand.)

Strings

`"text"` makes a string (in HyForth's memory records) and pushes its reference. `q^text^` is the same, for text with a `"` in it:

- `"hello world" .sz` prints it: spaces and all, up to the closing `"`. `""` is an empty string.
- Words that take a name take a string like this: `"/dev/zero" 1 open.`
- A string starts at the start of a word: `"` or `q^` after a space. There are no escapes: a `"` can't go in a `"..."` string (use `q^...^`), nor a `^` in a `q^...^` one.
- Like numbers, a string is made as it's read, even while compiling, so it isn't compiled into a definition.

Defining words

```
0:/> : sq dup * swap drop ;
0:/> 5 sq .
0019
0:/> var x 7 x ! x @ .
0007
0:/> 9 cons nine nine .
0009
```

Word	Stack	Does
<code>: name ... ;</code>		Define a word
<code>I</code>		After <code>;</code> : make the last word <i>immediate</i> (runs while compiling; used by control words)
<code>[,]</code>		Stop / resume compiling, inside a definition
<code>,</code>	<code>(n --)</code>	Compile <code>n</code> into the definition
<code>lit</code>		Compile the next cell as a number (<code>lit [65 ,]</code>)
<code>var name</code>		A variable: <code>name</code> pushes its address
<code>n cons name</code>	<code>(n --)</code>	A constant: <code>name</code> pushes <code>n</code>
<code>exit</code>		Return from the current word
<code>exec</code>	<code>(a --)</code>	Run the word at address <code>a</code>

Control flow: the training scripts

The control words aren't built in: they're defined in HyForth itself, by the **training scripts** in the paged ROM. Load them first:

```

0: /> ftrain autoloader
0: /> : t 3 0 do i . loop ;   t
      0000 0001 0002
0: /> : s 3 = if 1 . else 2 . then ;   3 s 4 s
      0001 0002
0: /> : c 0 begin 1 + dup 5 = until . ;   c
      0005

```

ftrain autoloader defines:

- 2*;
- ?branch and branch, which the others compile;
- if / else / then;
- begin / until, begin / again, begin / while / repeat;
- do / loop with i, j, k (loop indexes);
- mdump, decsz!, 256/.

autoloader (addr --) loads any such list: zero-terminated lines, then an empty one. cloader (addr --) loads one script from memory, and bloader (addr --) loads native code; bltest bloader loads the built-in sample.

Word reference

Stack:

Word	Stack
dup	(a -- a a)
drop	(a --)
swap	(a b -- b a)
over	(a b -- a b a)
rot	(a b c -- b c a)
nip	(a b -- b)
tuck	(a b -- b a b)
2dup, 2drop, 2over	Pairs
pick, snip	Reach into / remove from the stack
>r, r>	To / from the return stack
xS, xR	Clear the data / return stack
.S, .R	Show the data / return stack
?S, ?R	Is the data / return stack empty?

Arithmetic and logic:

Word	Stack	Notes
+, -	(a b -- a+b), (a b -- a-b)	
*	(a b -- hi lo)	The 32-bit product: lo on top (10 3 * . prints 001E; 0000 stays below)
/	(a b -- rem quot)	Unsigned: the quotient on top, the remainder below
min	(a b -- max min)	Both stay: the smaller on top

Word	Stack	Notes
max		The larger on top
and, or, xor, nand, not, neg		Bitwise; neg is the one's complement
<<, >>	(w n -- w')	Shift left / right by n
tbit, sbit, cbit	(n b -- ...)	Test, set, clear bit b
=, <>, <, >, <=, >=, 0=	(a b -- flag)	Flags are \$FFFF (true) or 0
bool	(n -- flag)	0 stays 0; anything else becomes \$FFFF
TRUE, FALSE	(-- \$FFFF), (-- 0)	
rand, rand32, rseed		Random numbers (rseed (s s --) seeds)

Memory:

Word	Stack	Does
@, !	(a -- w), (w a --)	Fetch / store a 16-bit cell
c@, c!	(a -- b), (b a --)	Fetch / store a byte
cells	(-- 2)	
here, last, back, sp, rp, memptr, >in	(-- a)	Addresses of HyForth's variables: here @ is the next free dictionary byte
memcpy	(start end dest --)	Copy start..end to dest
dump	(addr n --)	Hex and ASCII dump of n × 256 bytes; waits for a key after each 256
free	(-- n)	Bytes free between the dictionary and the MMU's lowest page
malloc	(bytes type -- addr)	A HyForth memory record (strings use these)
mten	(addr -- len)	A record's length
mktemp, purge0		Mark a record temporary; free the temporary ones
halloc	(bytes flags -- h)	An MMU allocation: flags 0 = task RAM, 1 = 8K RAM banks
hfree	(h --)	Free it
hlock	(h -- addr)	Its address (a bank allocation: selects the bank at \$8000). One at a time
hunlock	(h --)	Undo hlock

Input, output, terminal:

Word	Stack	Does
key	(-- c)	Wait for a key (stdin); -1 at end of file
emit	(c --)	Write a character (stdout)
cr, spc		Newline; spc pushes 32
., .C	(n --)	Print as a number (hex, or decimal after decimal: numbers) / as two characters
u.	(u --)	Print unsigned (hex, or decimal after decimal)
decimal, hex	(--)	The base . and u. print in (hex at first)

Word	Stack	Does
.sz	(sz --)	Print a string ("... " or q^...^)
AcIs, AscR (c r --), AcOl (c --)		ANSI: clear the screen, move the cursor, set attributes
in>, reset		Read the input buffer; reset it

System:

Word	Stack	Does
syscall	(addr a y -- x)	Call machine code at addr with .A and .Y set, and BIOS ROM page 0 selected (as a program runs), so any thunk works (API index); pushes .X
sys	(addr a x y - - a x y p)	The same with every register, in and out, and the flags after it (p : C is bit 0, which the OS's calls set when they fail, with the error in .A). E.g. \$F8EA 0 0 0 sys gives the tick count in a and y
disasm	(addr n --)	Disassemble n instructions
mmtest		Run the MMU self test
hwtest		Run the hardware test: it takes the machine over, and ends with a reset
debug, s@		Toggle debug; the status word's address
bye		Leave for WOZMON
abort		Abort to the prompt

The base and its libraries

HyForth is a base language, plus libraries of words for the rest of the system. The base is the Forth itself: the interpreter and compiler, the stacks, arithmetic and logic, memory and memory records, numbers and strings, key and emit, and loading scripts from memory. Each library is a set of words with a dictionary chain of its own: in ROM, or read from a file on a card (below). To find a word, the interpreter searches the words you've defined, then the libraries loaded from files, then the base, then each ROM library that's loaded.

At startup HyForth loads the base and every ROM library, which is everything the shell uses, and then runs the shell. So the prompt works as it always has. A new shell (shell) starts the same way. The copy of the shell that runs a pipeline's left side, or a script (run), has the libraries of the shell that started it.

A bare Forth (forth) is a new HyForth task with only the base loaded: a plain > prompt, no shell and no boot.hys. It's there for Forth on its own; lib loads what it needs, and lib all makes it a full shell.

Library	Words	Loads too
io	open, close, read, write, seek, ioctl, fdup2, pipe, create, mount, bind, unmount, hide (and their stack forms, (mount) ...), ns, stty, stty?, ctl, ioerr	
files	cd, pwd, ls, rm, rmdir, mkdir, cp, mv (and their stack forms, (cd) ...), cat, wc, vols, mkfs, mkfs-full, mkfs-size, mkfs-part, relabel, fsck, fsfix	io
shell	prompt, include, run (and (include), (run)), args, edit, echo. Also the shell's part of reading a line: the prompt's format, pipelines (), redirection (>, >>, <), and running a program for a word HyForth doesn't know	io, files
tasks	shell, forth, fg, kill, sleep, ps, wait, sem, mutex, acquire, acquire?, release, -sem	io
sound	sndinit, sndtest, sndstop, ywrite, patch, note, noteoff, play	io
mem	halloc, hfree, hlock, hunlock (MMU memory)	
tools	dump, disasm, syscall, sys, mmtest, hwtest	

Library	Words	Loads too
term	Ac1s , Ascrl , Acol (the ANSI terminal)	

Word	Does
libs	List the libraries: forth (the base), each ROM library, then the ones loaded from files. A library in parentheses isn't searched
lib name	Load a library, and the ones it needs. A name no ROM library has is a library file's (below)
-lib name	Unload a library: its words aren't found any more. Words already compiled into definitions still run
lib all, -lib all	Every library: the ROM libraries, and the ones loaded from files

```

0: /> -lib sound

0: /> libs
forth io files shell tasks (sound) mem tools term
0: /> sndtest

!UNK WORD!

0: /> -lib shell

> 1 2 + .
0003
> lib shell

0: />

```

Without the `shell` library, the prompt is a plain `>`, a line's `|`, `>` and `<` are words like any other (unknown ones), and a word HyForth doesn't know is an error, not a program to run.

Libraries from files. `lib name`, for a name that isn't a ROM library's, reads `name.hy1`: HyForth source, like a script. It's looked for the way a program is: in the current directory, then (for a name with no `/`) in `/lib` (a union, as `/bin` is: the caches' `lib` directories, the boot card's `/lib`, then `/rom/lib`), then in the directories of `$LIBPATH`. The words the file defines become the library's, and it's searched from then on.

- **While it loads**, the words you've defined yourself aren't searched: a library uses the base, the ROM libraries and other libraries. A library file can load the libraries it needs with `lib` lines of its own.
- **An error** while it loads (an unknown word, say) stops it, and the library is dropped. A file that isn't there is `!IO ERR!`.
- `-lib name` stops searching it, but leaves it in memory: `lib name` then searches it again, without reading the file again. To read a changed file, start a new HyForth (`shell`, or `forth`).
- **Up to 4** can be loaded in a task, with names up to 11 characters; a fifth is `!LOW MEM!`. A new HyForth starts with none.

```

0: /> cat /lib/greet.hyl
: greet 7 . ;
: twice dup + ;
0: /> lib greet

0: /> 3 twice .
0006
0: /> libs
forth io files shell tasks sound mem tools term greet
0: /> -lib greet

0: /> libs
forth io files shell tasks sound mem tools term (greet)

```

The shell: directories, files and programs

At boot the shell looks for HydraFS volumes on the SD cards (`/sd/0` to `/sd/7`), lists the ones it finds (`hydrafs 0 2`), and makes the lowest one's root the **current directory**. Then it runs `boot.hys` from there, if there is one, before the first prompt: a script for your own words and settings. **With no card**, the current directory is the shell's own area on the RAM disk, `/ram` (the prompt `/ram>`), where files can be saved until a reset, and the ROM's `/rom/boot.hys` runs instead.

Names are relative to the current directory unless they start with `/`, and `.` and `..` work anywhere in them: `games/star.frt`, `../notes`, `/sd/1/log`. The current directory is a directory on a card, or `/` (`cd /`); each task has one of its own, and the tasks it starts get a copy.

Commands take their arguments from the rest of the line, like a Unix shell's: `cd games`, `cp notes notes.bak`, and `cd "my games"` for a name with spaces in it. Each also has a **stack form**, in parentheses, which takes strings, for definitions and scripts' Forth code: `"games" (cd)`. (Inside a definition only the stack form works: the parsing form's argument would be read while compiling.)

Command	Stack form	Does
<code>cd [dir]</code>	<code>(cd) (sz --)</code>	Change directory. <code>cd</code> alone: <code>\$HOME</code> , or the current card's root
<code>pwd</code>		Show the current directory, as a whole path: <code>/sd/0/games</code>
<code>ls [dir]</code>	<code>(ls) (sz --)</code>	List a directory, a line per entry: <code>name size</code> , or <code>name/</code> for a directory (<code>ls</code> alone: the current one). A file: its text (<code>ls /dev/sd/0/ctl</code>)
<code>ls -l [dir]</code>		The same, with each entry's date and time (its last change, by the Hydra's clock: <code>name size 2026-09-29 18:05:30</code>); <code>ls -l file</code> shows one file's line
<code>cat [file]</code>		Show a file; with no name, copy stdin to stdout until end of file
<code>cp from to</code>	<code>(cp) (sz-from sz-to --)</code>	Copy a file: to a new name (a file that's there is replaced), or into a directory, with the same name
<code>mv from to</code>	<code>(mv) (sz-from sz-to --)</code>	Rename a file or directory (<code>to</code> a plain name: in the same directory); or move a file (<code>to</code> a path, or a directory to move it into: a copy, then the original removed)
<code>rm file</code>	<code>(rm) (sz --)</code>	Remove a file (not a directory)
<code>mkdir dir</code>	<code>(mkdir) (sz --)</code>	Make a directory
<code>rmdir dir</code>	<code>(rmdir) (sz --)</code>	Remove an empty directory
<code>include file</code>	<code>(include) (sz --)</code>	Read a HyForth script into this shell (below)

Command	Stack form	Does
run file [args]	(run) (sz --)	Run a program in a task of its own, and wait for it (below)
edit [file]		Edit a text file (below)
echo text		Print the text, and a new line
# text		A comment: the rest of the line is nothing (scripts; the namespace files)
send N line		Shell N (one this shell started, e.g. with shell) runs the line, as if typed at its prompt: now, if it's waiting there, or when it next is (/proc/N/cmd)
prompt	(sz --)	Set the prompt's format (below)

```
0: /> ls
hello.txt 13
games/
0: /> cd games
0:/games> cp star.frt /sd/1
0:/games> cd ..
0: /> mv hello.txt hi.txt
0: /> pwd
/sd/0
```

The prompt is a format, set with `prompt: %v` is the volume (0:, or nothing off the cards), `%d` the directory on the card (off the cards, the whole path: `cd /` gives `/>`), `%p` the whole path, `%l` the card's HydraFS label (nothing off the cards), `%t` the task (0-F), and `%%` a `%`. The default is `"%v%d> " prompt`; `"%t %p$ " prompt` gives `1 /sd/0/games$`, and `"[%l] %v%d> " prompt` gives `[GAMES] 0:/games>`. Up to 31 characters. The label is read from the card's `ctl` file when the prompt moves to another card, and again after `mkfs` or `relabel` (a label changed by hand through the `ctl` file shows once you're on another card and back).

Scripts (`.hys` files) are lines of HyForth, as you'd type them. `include file` reads one into this shell, as if it were typed (no prompts, no echo), so its definitions stay. An error, or Ctrl-C, stops it and the scripts that include it, and says which line (line 0002, in hex). Scripts nest up to 4 deep; a script's lines may end with CR LF, CR or LF.

Programs run in a task of their own, and the shell waits for them: `run file` (or, with `&` at the line's end, doesn't: [below](#)).

- A **Hydra executable** (`.hyx`: a file that starts with an `HYX1` header; [writing one](#)) is loaded into its new task's RAM and run until it returns.
- A **song** (`.zsm`: a file that starts with `zm`) is played by the ROM's song player, in a task of its own (see [play](#), [below](#)).
- **Anything else is a HyForth script**, read by a copy of the shell, as a pipeline's stage is: it starts with this shell's dictionary and stack, and what it defines or leaves on the stack goes away with it. `bye` in it ends it.
- A program has the console while it runs (if the shell has it), so **Ctrl-C stops it**, and gets copies of the shell's `fds`, namespace and current directory: it can be a pipeline's stage (`run hello.hyx | wc`).

A program by its name: a word HyForth doesn't know is looked for as a program, `name.hyx`, then `name.hys`, then `name.zsm` (a song, played: [below](#)): in the current directory, then (for a name with no `/`) in `/bin`, as Plan 9's shell looks (`path=(. /bin)`), then in the directories of `$PATH` ([below](#)). `/bin` is a union in the namespace ([namespaces](#)): the program caches on the RAM disks (the shell's own, `/ram/bin`, then the shared `/sram/bin`: [/ram](#)), the boot card's `/bin`, and the ROM's own, `/rom/bin` ([/rom](#)); `ls /bin` lists them all. So `cp game.hyx /bin/game.hyx` puts a copy in the shell's cache, and `game` loads from RAM; `bind -a /sd/0/tools /bin` adds a directory. So `hello` runs `hello.hyx`, and `theme` plays `theme.zsm`; with no card, `ls /rom/bin` shows what runs.

The environment: variables, `NAME=value`, as files under `/env`; each task has its own, and the tasks it starts (programs, scripts, shells) get a copy. The shell uses three, and sets two:

Variable	Does
PATH	Where programs are also found by name, after /bin: directories, : between them (/sd/1/bin:/sd/1/tools)
LIBPATH	Where lib also finds library files (name.hyl), after /lib, the same way
HOME	Where cd alone goes (without it: the current card's root)
status	Set by the shell: the last program's exit status, as Plan 9's \$status (the message, or the code if there's none, or empty for success; below)
apid	Set by the shell: the task of the last program started with & (Plan 9's \$apid)

```
0: /> echo /sd/0/bin:/sd/0/tools > /env/PATH
0: /> echo /sd/0/work > /env/HOME
0: /> ls /env
PATH=/sd/0/bin:/sd/0/tools
HOME=/sd/0/work
0: /> cat /env/HOME
/sd/0/work
0: /> rm /env/HOME
```

Put them in boot.hys to have them at every boot. A program reads one as a file (/env/NAME); /dev/proc/N/env shows task N's.

Arguments: the rest of the line after a program's name (run prog a b, or prog a b) is the program's, as it was typed (63 characters at most): an executable gets it in .A.Y (programs.md), and a script with args (-- sz), a string. A script also starts with a copy of the shell's stack. ((run) gives no arguments.)

```
0: /> hello
Hello from task B
0: /> 3 4 add \ add.hys: + .
0007
0: /> greet Ann \ greet.hys: "Hello, " .sz args .sz cr
Hello, Ann
0: /> nosuch
!UNK WORD!
```

Redirection: a command's output can go to a file, and its input come from one:

	Does
command > file	stdout to the file: made, or emptied first
command >> file	stdout added to the file's end (made, if it isn't there)
command < file	stdin from the file

>, >> and < are words of their own (spaces around them), outside strings; the name can be "in quotes". They apply to the whole line, or to a pipeline's last command, and stdin and stdout go back when the line's done. In a script, a line's < doesn't lose the script's place.

```
0: /> words > words.txt
0: /> words | wc . . . >> counts.txt
0: /> wc < notes.txt . . .
0: /> echo run hello > /sd/0/boot.hys
```

`echo text` prints the rest of the line, and a new line (without its "s): with `>`, a quick way to make a small file.

`edit [file]` edits a text file: a line editor in the manner of Unix's `ed`, run in a task of its own. It reads the file (or starts a new one), then takes commands at its `*` prompt, with line numbers before or after them (`n`, `$` for the last, `a,b` for a range):

Command	Does
<code>p [a[,b]]</code>	Print lines, numbered (<code>p</code> alone: all of them)
<code>a [n]</code>	Add lines after line <code>n</code> (the last, if none), typed until a line of just <code>.</code>
<code>i [n]</code>	Insert lines before line <code>n</code> (the first, if none), until <code>.</code>
<code>c a[,b]</code>	Change lines: they go, and the lines typed until <code>.</code> take their place
<code>d a[,b]</code>	Delete lines
<code>w [file]</code>	Write the file (or another: then that's the file)
<code>q, Q</code>	Quit (with changes not written, <code>q</code> asks for a second <code>q</code>); quit at once
<code>h</code>	Help

Ctrl-C comes back to its prompt, keeping the text. Lines end with CR LF in the file; the text can be up to 29 KB. Its commands can come from a file too: `edit notes.txt < changes.txt`.

```
0: /> edit hello.hys
hello.hys: new file
*a
"Hello, world" .sz cr
.
*w
23 bytes
*q
0: /> hello
Hello, world
```

Files and devices

Word	Stack	Does
<code>open</code>	<code>(sz mode -- fd)</code>	Open a file: mode 1 = read, 2 = write, 3 = both; + <code>\$80</code> = don't wait (reads give <code>ioerr</code> <code>\$73</code> instead); + 4 = a directory as stat records; + 8 (with 2) = empty the file first
<code>close</code>	<code>(fd --)</code>	
<code>read</code>	<code>(fd addr n - - n')</code>	Read up to <code>n</code> bytes into <code>addr</code> (task RAM); <code>n'</code> = bytes read, 0 = end of file
<code>write</code>	<code>(fd addr n - - n')</code>	Write <code>n</code> bytes from <code>addr</code>
<code>seek</code>	<code>(fd lo hi --)</code>	Set the offset for the next read or write: <code>hi * 65536 + lo</code>
<code>ioctl</code>	<code>(fd code arg --)</code>	Device control, e.g. <code>1 1 N ioctl</code> on the console makes task <code>N</code> the foreground task
<code>fdup2</code>	<code>(fd newfd --)</code>	Make <code>newfd</code> refer to <code>fd</code> 's file: <code>fd 1 fdup2</code> sends <code>emit</code> 's output there
<code>pipe</code>	<code>(-- rfd wfd)</code>	Make a pipe
<code>ioerr</code>	<code>(-- n)</code>	The last IO error code

Word	Stack	Does
cat		Copy stdin to stdout until end of file (<code>cat file</code> shows a file: the shell)
wc	(<code>-- lines words chars</code>)	Count stdin until end of file
mount [- abcs] device path [spec]	(mount) (<code>sz-dev sz-path -</code> <code>-</code>)	Mount a device at a path: <code>mount zero /z</code> , then <code>"/z" 1 open</code> . A spec, as Plan 9's, picks what the server serves there: <code>mount hfs /rom x</code> (the ROM disk), <code>mount hfs /a r</code> (the whole RAM disk, every task's area). The flags as <code>bind</code> 's
bind [-abc] new old	(bind) (<code>sz-new sz-old --</code> <code>)</code>)	Make a path stand for another: <code>bind /dev/cons /tty</code> . With no flags it replaces old's entries; <code>-b</code> and <code>-a</code> add it to old's union , before or after its members, which are looked in, in order, for a name (<code>bind -a /rom/bin /bin</code>); <code>-c</code> : a file made in the union is made in it; <code>-s</code> : in the system namespace , which every task sees (the boot shell's lines, and <code>/rom/lib/namespace</code> 's: the boot shell and task 0 only)
unmount [new] old	(unmount) (<code>sz-old --</code>)	Remove old's mounts and binds; or just one member of its union: <code>unmount /rom/bin /bin</code>
hide path		Nothing under the path is found, in this task and the tasks it starts
newns [file]		A fresh namespace, as Plan 9's: this shell's own entries go (its binds, mounts and hides, and its copies of the system's unions), but its <code>/ram</code> , so it sees the system namespace, as a new shell does; then the file's lines, as <code>include</code> reads them: <code>newns /sd/0/lib/work</code>
ns		List the namespace, as the lines that would make it: <code>bind -a /rom /u</code>
stty	(<code>sz --</code>)	Change the serial port's settings: <code>"b19200" stty</code> , <code>"17 pe s1" stty</code> (b = baud rate, l = data bits, p = parity n/o/e/m/s, s = stop bits). Output so far goes out first; then switch the terminal
stty?		Show the serial port's settings, e.g. <code>b9600 18 pn s1</code>
ctl	(<code>sz-file sz-text --</code>)	Write a command to a ctl file: <code>"/dev/sd/0/ctl" "check" ctl</code> , <code>"/proc/3/ctl" "kill" ctl</code> (a task this shell started)
create	(<code>sz mode --</code> <code>fd</code>)	Make a file (mode 0; 64 = append-only, 1 = read-only) and open it for reading and writing; a file that's there is emptied. Mode 128 makes a directory
vols		The cards: for each of 0-7, what it is (or <code>none</code>), and its HydraFS label, free space and last check
mkfs	(<code>n sz-label</code> <code>--</code>)	Make an empty HydraFS on card n (everything on it is lost): <code>0 "GAMES" mkfs</code> . A quick format: a moment, whatever the card's size
mkfs-size	(<code>n sz-label</code> <code>mb --</code>)	The same, <code>mb</code> megabytes big (up to 65535, \$FFFF), if the card is bigger: <code>0 "SMALL" 4096 mkfs-size</code>
mkfs-part	(<code>n sz-label</code> <code>--</code>)	The same, in a HydraFS partition: one is made after the card's other partitions (a FAT one a PC made, say), or with a new partition table: <code>0 "GAMES" mkfs-part</code> . (<code>mkfs</code> on a card that has one formats the partition)
mkfs-full	(<code>n sz-label</code> <code>--</code>)	A full format: the whole free map written now (a version 1 HydraFS), with its progress shown: minutes on a big card
relabel	(<code>n sz-label</code> <code>--</code>)	Give card n's HydraFS a new label: <code>0 "TOYS" relabel</code>
fsck	(<code>n --</code>)	Check card n's HydraFS, and show what it found
fsfix	(<code>n --</code>)	Check it, and repair its free map

Buffers: a failed call prints `!IO ERR!`, and `ioerr` gives the code ([error codes](#)). here `@` is a handy scratch buffer.

```
0: /> "/dev/zero" 1 open . \ fd 3
0003
0: /> 3 here @ 16 read .
0010
0: /> "/dev/sd/0/ctl" 1 open 0 fdup2 cat | cat
sdhc 7580 MB 15523840 blocks
0: /> "/dev/sd/0/data" 3 open . \ the SD card as bytes
0: /> 3 512 0 seek 3 here @ 16 read . \ block 1's first 16 bytes
```

(\ isn't a comment word: the examples just annotate.)

The files on a card are at `/sd/N` ([io.md](#)), and [the shell's commands](#) cover the everyday work. Underneath, reading a directory gives a line per entry (so `cat` lists it), and a file reads like any other `fd`:

```
0: /> "/sd/0" 1 open 0 fdup2 cat | cat
hello.txt 13
games/
0: /> "games/star.frt" 1 open 0 fdup2 cat | cat
0: /> "hello.txt" 1 open .
0003
0: /> 3 here @ 128 read .
000D
```

Writing works as on any `fd`, with `create` to make a file:

```
0: /> mkdir games
0: /> "games/hi" 0 create . \ fd 3, open for writing
0003
0: /> 72 here @ c! 105 here @ 1 + c!
0: /> 3 here @ 2 write . 3 close \ "Hi"
0002
0: /> ls games
hi 2
0: /> mv games/hi hello
0: /> rm games/hello
```

Close what you write before taking the card out: a file's new size goes to the card when it's closed.

The cards themselves (HydraFS volumes) have words of their own. They work through each card's `ctl` file (`/dev/sd/N/ctl`), so `ctl` and `ls` can do the same by hand:

```

0: /> vols                                \ what's in the sockets
0: sdhc 7580 MB 15523840 blocks
hydrafs label=GAMES
free 6246400 KB of 7761920 KB
1: none
...
0: /> 1 "WORK" mkfs                        \ a new HydraFS on card 1: everything on it is lost
0: /> 0 fsck                               \ check card 0
sdhc 7580 MB 15523840 blocks
hydrafs label=GAMES
free 6246400 KB of 7761920 KB
check: lost 0, unmarked 0, twice 0
0: /> 0 fsfix                             \ ... and repair its free map
0: /> 0 "TOYS" relabel                     \ a new label

```

`fsck` counts clusters lost (marked in use, but nothing uses them: wasted space), unmarked (in use, but marked free: a new file could be given them) and used twice (two files share them: one is damaged). `fsfix` frees the lost ones and marks the unmarked ones; a cluster used twice is only shown, as a person has to decide which file keeps it. A card takes a pass for each 256 MB, so checking a big one takes a while; from 4 GB up, 10% 20% ... 100% shows how far it's got. Empty space checks quickly: an empty 244 GB card takes about a minute and a half (quick-formatted), or 7 minutes (full-formatted: its whole free map is read).

`mkfs` is a **quick format**: it writes just the superblock, and the free map is written as the card fills, so a 244 GB card is ready in a moment. `mkfs-full` writes the whole map first (about 13 minutes for 244 GB, with its progress shown), for a card an older ROM will read. `mkfs-size` makes a HydraFS smaller than the card. `mkfs-part` puts it in a partition, so the card can also hold a FAT partition for a PC: partition the card on the PC first, leaving room after the FAT partition, then `mkfs-part` on the Hydra. By hand, the `ctl` command is `format [-f] [-p] [-s size] [label]` (size in megabytes, or gigabytes with a G: `"/dev/sd/0/ctl"` `"format -s 8G WORK" ctl`, or `echo format -p WORK > /dev/sd/0/ctl`).

Pins: `/dev/gpio`. The VIA's port A on header J27 is a file a pin: `echo 1 > /dev/gpio/2` makes PA2 an output and drives it high (an LED), `cat /dev/gpio/3` reads PA3 (1 or 0: a button), and `cat /dev/gpio/ctl` shows every pin's direction and level. `/dev/gpio/ca1` waits for a press on CA1 ([io.md](#); the [tutorial](#) wires them).

The date and time: `cat /dev/time` shows the Hydra's clock, and `echo 2026-09-29 18:05 > /dev/time` sets it. With a **DS1747** in U7 (a task RAM with a clock that runs while the Hydra's off: [hardware](#)), the boot finds it and sets the clock from it, and says so after the volumes: `clock 2026-09-30 14:05:00` (with `battery low` when its battery is flat), or `clock stopped`: set the time. Setting the time sets the DS1747 too, and a DS1747 that wasn't set (the boot said `no clock`) is found then. Without one (`no clock`), the clock starts at 2000-01-01 at power-up, so set it after a boot for the files you write to have the right dates (`ls -l` shows them); `boot.hys` can't know the time, but a line you type can.

Sparse files: a write that starts past a file's end (after `seek`) fills the gap with zeros, and whole 4 KB clusters of the gap take no space on the card: a file can have holes. `"f" 0 create . then 3 0 $400 seek 3 here @ 4 write .` makes a 64 MB file that takes 4 KB.

Pipelines

A line with `|` (with spaces around it, outside `"..."` and `q^...^` strings) is a **pipeline** (and its last command's output can go to a file: [redirection](#)):

```
0: /> words | wc . . .
0DB4 021C 002C
0: /> words | cat | wc .S
```

- **Each stage but the last** runs in a copy of the shell's task (TASK_CLONE : same dictionary, same stack), with its stdout into a pipe.
- **The last stage** runs in the shell, with its stdin from the pipe.
- When the line is done, stdin is put back.

Tasks and the console

Word	Stack	Does
shell	(-- n)	Start another HyForth in task n; it waits until brought to the front
forth	(-- n)	The same, but a bare Forth: only the base loaded (see the base and its libraries)
fg	(n --)	Bring task n to the front: it gets the keyboard, and the others wait to print
kill	(n --)	Kill task n and the tasks it started
ps		List the tasks (from /proc : its files also give each task's directory, environment, memory and namespace: cat /proc/1/pages , cat /proc/1/ns ; and a task this shell started has its memory there as files: cp /proc/3/mem core3 , or "/proc/3/mem" 3 open , seek to an address, read and write)
sleep	(n --)	Sleep n ticks (200 a second; 200 sleep is 1 s); Ctrl-C ends it
sem	(n -- s)	A semaphore of n: n takes before a task has to wait; s = its number (1-16), which every task can use
mutex	(-- s)	A mutex: a semaphore of 1 that only the task that took it can release (released if that task ends)
acquire	(s --)	Take one of semaphore s, waiting (using no CPU) until there is one; Ctrl-C ends the wait
acquire?	(s -- f)	Take one if there is one (true), or false at once
release	(s --)	Give one back (a mutex: only its holder can): a task waiting for it goes on
-sem	(s --)	Free semaphore s: the tasks waiting for it get !IO ERR! (ioerr 60)

```
0: /> shell .
000B          \ the new shell is task B: Ctrl-] B switches to it, Ctrl-] 1 back
0: /> ps
0 R -
1 R 0 *
B W 1
C D -
...
```

A semaphore made in the shell can be used by the tasks it starts, and by a pipeline's stages. Here the shell waits for the pipeline's first stage, which runs alongside it:

```
0: /> 0 sem .
0001
0: /> 200 sleep 1 release | 1 acquire 7 .
0007          \ a second later: when the first stage released it
```

A failed call gives `!IO ERR!`, with the reason in `ioerr` (`60` not a semaphore, `61` all 16 in use, `63` a mutex this task doesn't hold). When a task ends, the semaphores it made are freed and the mutexes it holds released.

Sound:

Word	Stack	Does
<code>sndinit</code>		Clear the YM2151
<code>sndtest</code>		Play the test song in the background: a minute of music that uses the whole YM2151 (<code>/rom/songs/test.zsm</code> : <code>play</code> plays it too; <code>sndstop</code> ends it)
<code>sndstop</code>		Stop it
<code>ywrite</code>	(<code>xxaa -- f</code>)	Write value <code>aa</code> to YM2151 register <code>xx</code> ; true if it went
<code>patch</code>	(<code>p ch --</code>)	Load patch <code>p</code> into channel <code>ch</code> (0-7): 0-127 are General MIDI's instruments (0 piano, 24 guitar, 40 violin, 56 trumpet, 73 flute ...), 128-162 drum sounds
<code>note</code>	(<code>n ch --</code>)	Play MIDI note <code>n</code> on channel <code>ch</code> (60: middle C; 69: A, 440 Hz)
<code>noteoff</code>	(<code>ch --</code>)	Key channel <code>ch</code> off
<code>play</code>	<code>play song</code> [<code>n</code>] [<code>&</code>]	Play a song (a ZSM file), in a task of its own, and wait for it (<code>&</code> : don't). Its loop is played <code>n</code> more times (none: the song once, to its end; 0: forever, until Ctrl-C or <code>kill</code>)

```
0: /> 0 0 patch 60 0 note 100 sleep 64 0 note 100 sleep 0 noteoff
```

Songs are ZSM files, the Commander X16's format: the YM2151's register writes and their timing, which the Furnace tracker exports, and X16 music comes in. `play` plays one in the ROM's player, and so does a song's name, as a program's does (`theme` for `theme.zsm`), or `run theme.zsm`. The player claims the channels the song uses, so another program can't play over them; Ctrl-C (or `kill`, for one in the background) stops it, and its channels go quiet. The exit status is 0 when it plays to its end, 130 on Ctrl-C.

```
0: /> play theme.zsm 0 &          \ the music, forever, in the background
[B]
0: /> $B kill                    \ and stopped
```

The library's other commands (a channel's volume, speakers, bend, a drum) are register numbers the chip doesn't have, sent with `ywrite` after the channel (`$02`): `$0203 ywrite drop $0640 ywrite drop` sets channel 3's volume to 64 (`$06`); `$07` is the speakers (1 left, 2 right, 3 both), `$09` the bend (64ths of a semitone), `$0A` a General MIDI drum (`$0A24`: a kick). See [/dev/snd](#).

Background tasks and exit statuses

`&` at the end of a line runs its program (or script, or pipeline) without waiting for it, as in Plan 9's `rc` or a Unix shell: the shell prints the task's number, puts it in `/env/apid`, and gives the prompt back. The program runs alongside the shell; it doesn't have the console, so it waits if it reads or writes it, until it's brought to the front (`fg`, Ctrl-] and its number) or waited for.

Exit statuses, as Plan 9's: a program ends with a code (0-255, 0 for success) and a message (up to 30 characters, or none). A program that returns has 0; Ctrl-C gives 130, `interrupt`, and a kill 137, `killed`; a C program has `main`'s value or `exit`'s, or `hy_exits`'s message. A script ends with its last program's status, or its error's (`!UNK WORD!` is 5, the message `UNK WORD`), or what `exits` gives it. The shell keeps the status of each program it waits for, and of each error at its prompt.

Word	Stack	Does
<code>status</code>	<code>(-- n)</code>	The last exit status's code (the base)
<code>exits</code>	<code>(n --)</code>	End this script, pipeline stage or command shell with exit status n (the base). At the boot shell, it only sets the status
<code>wait</code>	<code>(n --)</code>	Wait for task n (one started with <code>&</code>), giving it the console meanwhile; its exit status is the status then (<code>tasks</code>)

`/env/status` has the status as text (Plan 9's `$status`): the message, or the code if there's no message, or empty for success.

```
0: /> upper &
[B]                \ task B: the number is in /env/apid too
0: /> ps
0 R -
1 R 0 *
B W 1              \ waiting for the console
...
0: /> $B wait      \ (task numbers are hex: $B)
hiHI               \ it has the console: type, then Ctrl-D ends it
0: /> status .
0000
0: /> nosuch

!UNK WORD!
0: /> cat /env/status
UNK WORD
```

A **command shell** (`SHELL_CMD`, as Plan 9's `rc -c`) is HyForth running the lines on its stdin, with no banner or prompt, ending at its input's end with the last status: C's `system` runs one ([programs.md](#)).

Errors and keys

Message	Meaning
<code>!UNK WORD!</code>	No such word, and no program by that name (or a number it can't read)
<code>!DS PTR ERROR!</code> , <code>!RT PTR ERROR!</code>	Data / return stack under- or overflow
<code>!DIV ZERO!</code>	Division by zero
<code>!LOW MEM!</code>	Out of memory (dictionary or records)
<code>!SECURITY!</code>	A write to a protected area
<code>!IO ERR!</code>	An IO word failed, and why (<code>!IO ERR! not found</code>); <code>ioerr</code> gives the code
<code>!BREAK!</code>	Ctrl-C

Key	Does
Ctrl-C	Break: back to the prompt, keeping the dictionary. While a program runs (<code>run</code>), it stops the program

Key	Does
Ctrl-\	Kill: a fresh HyForth (the dictionary is lost)
Ctrl-D / Ctrl-Z	End of input (<code>cat</code> , <code>wc</code> , <code>key</code>)
Ctrl-] then 0-F	Switch to that task; 1 lists them
Backspace, Delete (Ctrl-D)	Erase the character before the cursor / at it
Left, Right (Ctrl-B, Ctrl-F)	Move along the line
Home, End (Ctrl-A, Ctrl-E)	To the line's start / end
Up, Down (Ctrl-P, Ctrl-N)	The lines typed before (about the last 255 characters' worth)
Ctrl-U	Erase the line

The line editing is HyForth's, at the prompt when its input is the console: it turns the console raw while a line is typed (`/dev/console`), and echoes the line itself. Programs reading the console (`cat` , `wc` , C's `fgets`) get it as before: echoed, with Backspace.

How HyForth uses memory

HyForth runs in task 1. Its code and the built-in words run from the BIOS ROM (page 1, with some words' code on page A), so only its variables are in RAM: they're copied from the paged ROM to `$0800` at startup (about 1.3K: the shell's settings, the line editor's history, the sample scripts). The build's link map (`os_rom/obj/os_rom_C02.map` , segment `FORTH_DATA`) has the size.

What	Where
Input buffer	<code>\$0200</code>
Data and return stacks	<code>\$0300-\$03FF</code>
Record stack	<code>\$0400-\$05FF</code>
Variables (the prompt's format, the shell's settings)	<code>\$0800</code>
The shell's buffers (names, a block of a file being shown, include's saved fds)	Just after the variables: not in the paged ROM
Dictionary (your words)	Grows up from the page after them (<code>\$0B00</code>), here @
Small records (strings, <code>malloc</code>)	A 2K arena from the MMU
Records of 256 bytes or more	Their own MMU blocks

HyForth keeps the MMU's page floor 2 pages above `here` , so the dictionary and the MMU's allocations never meet: `!LOW MEM!` comes first. A copy of the shell (a pipeline stage, or a script that `run` started: `TASK_CLONE`) gets all of this copied.

WOZMON, the disassembler, POST and the self tests

The machine-level tools: WOZMON (a monitor, after Steve Wozniak's Apple 1 monitor) with a built-in disassembler, the power-on self test, the ROM self tests, and the hardware test. Sources: `os_rom/monitor/`, `os_rom/tests/`, `os_rom/hwtest/`.

Getting to WOZMON

Type `bye` at HyForth's prompt. WOZMON runs in the shell task (task 1), so it has the console's fds. To go back to HyForth, press `Ctrl-\\`: the shell starts again from scratch.

The prompt shows the task, the RAM bank at `$8000` (with the shared macro-page `U` in brackets for a shared bank) and the paged ROM bank:

```
T1 00:00>          task 1, RAM bank $00, ROM bank $00
T1 F0(0):00>       shared bank $F0, U = 0
```

Commands

A line holds one or more items. Addresses and data are hex; leading zeros can be left out.

Type	Does
<code>E000</code>	Show the byte at <code>E000</code>
<code>E000.E00F</code>	Show <code>E000</code> to <code>E00F</code> (8 bytes a line)
<code>.E0FF</code>	Show from the last address shown to <code>E0FF</code>
<code>1000: 41</code> <code>42</code>	Store <code>41</code> , <code>42</code> at <code>1000</code> , <code>1001</code> (it shows the old byte at <code>1000</code>)
<code>: 43</code>	Store at the next address
<code>1000R</code>	Run the code at <code>1000</code> (a <code>jsr: rts</code> comes back to WOZMON)
<code>1000S</code>	Start the code at <code>1000</code> in a new task and wait for it to end (<code>TASK_START</code>)
<code>L</code>	List mode: show addresses as disassembled instructions from now on
<code>K</code>	Back to byte mode
<code>T</code> , <code>U</code> , <code>V</code> , <code>W</code>	Stand for the pseudo-register addresses <code>FFF0-FFF3</code> : <code>T</code> shows <code>T</code> , <code>W</code> : <code>1</code> stores <code>1</code> in <code>W</code> (careful: that switches the ROM under WOZMON's feet)
Backspace, Esc	Erase a character; cancel the line

List mode disassembles as it shows:

```
T1 00:00>L E000.E008
E000: A9 00    LDA  #$00
E002: 8D F3 FF STA  $FFF3
E005: D8      CLD
```

What WOZMON shows is the task's own view: `0000-7FFF` is task 1's RAM, `8000-9FFF` the bank in the prompt, `A000-DFFF` the paged ROM bank, and `E000-FFFF` BIOS ROM page 0 (the kernel's: WOZMON itself runs on page 4, and reads ROM as the disassembler does, from page 0 unless that was set to another). To look at another bank, store it in `0000` (RAM bank) or `0001` (ROM bank).

The disassembler is also a call: `DISASM_AY` (`$F81B`) disassembles at `.A.Y` (`C = 0`: one instruction; `C = 1`: `.X` of them). HyForth has it as `disasm ( addr n -- )`.

Self tests

The ROM has three self tests, runnable from WOZMON (or HyForth's `syscall`). Each prints its name, then `ok`, or `FAIL` with a step letter and a value to look up in its source.

Run	Test	Checks
F833R	MMU (<code>tests/mmu_test.s</code> ; HyForth: <code>mmtest</code>)	Small, chunk, page and bank allocations; reads, writes, locks; far pointers and references; the maps come back clean
F869R	Scheduler (<code>tests/sched_test.s</code>)	Three tasks interleaving; a <code>NO_PREEMPT</code> section staying together; a wait and wake
F88AR	IO (<code>tests/io_test.s</code>)	Opening, reading and writing devices; pipes between tasks; <code>IO_DUP2</code> ; namespaces; names in ROM

The scheduler test prints the tasks' letters as they run, e.g.:

```
Sched test:
mbbaambambambbaambambmbbaambamba
m[ccccccccc]mmmmmmmmmm
ww
done
```

POST: the power-on self test

POST runs first at every reset, before anything else: in task 0, with interrupts off and polled serial output. So it works even when little else does. A good board prints:

```
POST ZP:T ST:T LO:T 7D:T SH:S P1:4C
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000
```

The first line: the memory mapping the task system depends on.

Field	Good	Checks
ZP:	T	<code>\$0080</code> (zero page) is per-Task (T), not Common to all tasks (C)
ST:	T	<code>\$0180</code> (stack page) is per task
LO:	T	<code>\$0280</code> (task RAM) is per task
7D:	T	<code>\$7D80</code> (the task system page) is per task
SH:	S	Shared bank <code>\$F0</code> (U = 0) is Shared between tasks (S), or not (X)
P1:	4C	The byte at <code>forth_main</code> on BIOS page 1 (a <code>jmp</code>): page 1 is there and current

The second line: the paged RAM's lines. Each value is a hex mask of **bad** lines (bit n set = line n bad), so all zeros is good.

- `U:x`: the U register's lines U0-U3, tested on shared bank `$F0`.
- `bb:x/dd/aaaa`: bank `bb`, tested at `$8000-$9FFF`:
 - `x`: bank register lines 0-3 (writes to bank `bb` XOR 1, 2, 4 and 8 mustn't land in `bb`)
 - `dd`: data lines D0-D7 (a walking one)
 - `aaaa`: address lines A0-A12 (`$8000 + 2^n` for each line n; also catches a write landing on `$8000`)

The banks tested are the first bank of each shared RAM chip (F0, F4, F8, FC, with U = 0), then the first bank of each installed task RAM module (00, 10, 20, ...). A missing chip shows as bad lines. The tests are destructive, which is fine at reset.

A chip that fails is left unused:

- A bad shared RAM chip has every bank ID on it reserved, in all 16 macro-pages.
- A bad task RAM module is treated as not installed.
- The system's shared banks (IDs \$00 and \$09-\$0C) can't move elsewhere, so a fault on the F0 or F8 chip still needs fixing.

To find the chip and pin behind a report, see the [Hardware Reference](#): the shared bank IDs per chip (with V1's crossed bits 2/3: F4 is U28, F8 is U27), and the HM628512 pinout. For example:

- F0:0/00/0001 is A0 (pin 12) on U25;
- F8:2/00/0000 is bank line 1 (A18, pin 1) on U27.

The emulator can inject a stuck address line to see the report (`--ram-fault`, [emulator](#)). For a closer look at a fault, run the [hardware test](#).

The hardware test

A test of as much of the board as software can reach, for bringing up a board or chasing a fault. It's a program of its own in paged ROM bank 1 (`os_rom/hwtest/`). It takes the machine over: interrupts off, its own polled serial I/O at 9600 8N1, and no calls into the OS. Its memory tests overwrite everything, so it ends with a reset.

Starting it:

- From HyForth: `hwtest` .
- From POST: type `T` just after a reset, before POST's second line appears. This works when the OS can't start, for example with a bad shared RAM chip or an IRQ line held active.

The menu. Type a test's key to run it, or:

Key	Does
A	All the tests, quick: every bank's marks, and every byte of a sample of the memory (about 15 seconds)
F	All the tests, full: every byte of every bank, in every task (about 3 minutes at 3.58 MHz with three RAM modules)
L	All the tests, quick, again and again until a key: a count of the passes and failures after each (for an intermittent fault)
R	Reset: the machine starts again, through POST

Each test prints its name, what it found, then `ok` or `FAIL` and the fault. A run ends with `hwtest: all passed` or `hwtest: failed: N`:

```
CPU ..... ok
RAM modules ..... 0 1 2 ok
shared RAM ..... FAIL bank 04 8000 bits 08
CPU clock ..... 3.58 MHz ok
SPI devices ..... SD cards 0 ok
hwtest: failed: 1
```

The tests, in the order A runs them. The memory tests write a pattern that depends on the address, so a stuck or crossed address line shows up as well as a bad data line. Bits are hex masks of the bad bits.

Key	Test	Checks	A fault shows
1	CPU	A sample of the W65C02S's instructions and flags: binary and decimal arithmetic, and the 65C02's own (STZ , TSB / TRB , BRA , PHX , RMB / SMB , BBR / BBS , (zp))	check N
2	T U V W registers	Each stores and reads back all 8 bits	the register and its bad bits
3	shared RAM	Every one of the 256 banks (U 0-F, IDs \$F0-\$FF) has its own mark; every byte of the first bank of each chip (full: of every bank)	Address faults: each chip and the lines it doesn't see, with pins, e.g. U29: A18 (pin 1) (from the marks of all 256 banks: the bits a bank's ID and the mark it holds differ by). The same A13-A16 line on all four chips is the U register's line itself (test 2). no mark : a data fault or no chip. Otherwise a byte fault: bank UB (U, then the bank), where, the bad bits
4	RAM bank registers	Each task's \$00 (74LS219s IC1 and IC2): task 0 marks the 16 shared banks; then each task set to each bank and read at once; then all 16 tasks set to their own banks first and read afterwards; then lines 4-7	at once: task F: D>F (set to \$FD, it read bank \$FF's mark: a cell that doesn't hold its value); all set, then read: task F bank FF read A8 (bank FD's mark, bits 02) (right at once but wrong later: writing one register disturbs another). skipping banks FE FF (...) (information, not a failure): task 0 alone reads those banks wrong, so the shared RAM is at fault (test 3, e.g. a bad solder joint on U29, which holds banks \$FC-\$FF); the registers are tested with the other banks. (task 0 too) after a failure: task 0 read that bank wrong as well, so it's the RAM again (an intermittent fault)
5	task RAM	First, each task's RAM is its own: a mark in each (a bad task line to U7, A15-A18, makes two tasks share their RAM, which the patterns can't show). Then every task's zero page (from \$02), stack page and \$0200-\$7FFF (task F's to \$7FFF: a DS1747's clock registers may be there)	the task, where, the bad bits; or task F has task B's mark: U7 A17 (pin 30)
6	RAM modules	Which modules there are (shown); each bank of each has its mark in each task; every byte of each module's first bank (full: of every bank, in every task)	the bank, the task, where, the bad bits
7	BIOS ROM	A CRC of each 8K page against the checksums the build stores	page N, and (W line n) if it reads as another page
8	paged ROM	A CRC of each 16K bank, and the bank lines	bank NN, or bank line n
I	interrupts	No IRQ line active with the devices quiet; the vector RAM (all 16 entries, two patterns); BRK through entry V; the VIA's, the ACIA's and the YM2151's interrupts on their lines, one at a time, then all at once in priority order	an IRQ line is held active; vector entry N wrote .. read ..; the device, and no IRQ or the line it came on; unasked: IRQ on line N (one came before a device was asked). With line 4, the YM2151's status too (bit 0: timer A's flag, 1: timer B's)
V	VIA	Timer 1's latches, the shift register and IER read back; timer 1 counting, its flag (one-shot and free-running); timer 2's; the shift register's	the part
Y	sound chip (YM2151)	Its busy flag (set by a write, then clear), and its timers' flags	always busy (no chip?), never busy, timer A, ...

Key	Test	Checks	A fault shows
K	CPU clock	The CPU's clock against the YM2151's (SND_CLK , 3.58 MHz): shows 0.89, 1.79, 3.58 or 7.16 MHz (jumpers J6-J7), which must be what the ROM is built for (CPU_CLOCK_MULT). It shows its counts: first , from starting YM2151 timer A to its first overflow (judged: 16,384 at 3.58 MHz), and next , to the one after with the timer's flag reset by writing \$14 again (shown only: larger by the write's time if that restarts the timer)	the clock and what the ROM expects, or not a clock the board has and the counts
S	serial port (ACIA)	DCD and DSR active; the control and command registers; a programmed reset; a character's time at 9600 baud (Rockwell only)	the register, or the character's time in cycles
P	SPI devices	Information: each of devices 0-7 is sent an SD card's reset (CMD0); the SD cards that answer are shown	(another answer is shown as (device: answer))
C	I2C bus	SCL and SDA high when released and each low alone when pulled low (a line shorted, held low or stuck high); then information: the devices that answer, at \$08-\$77	the lines' levels
X	slot cards	Information: the slot ports (1A = slot 1, select A) that don't read as empty	
H	hold a task (probe)	Not a test, and not in A, F or L : every task's RAM bank register set to \$F0 + the task, then the task typed (0-F) held in T , its bank at \$8000 read over and over, until a key. Meanwhile, measure T0-T3 (IC1-IC4 pins 1, 15, 14, 13) and the bank register's outputs, RAMB0-7 (IC1 and IC2 pins 5, 7, 9, 11)	not a task

The ROM tests' checksums are made by the build (os_rom/tools/romsum.js , run by build.js) and kept at the end of paged ROM bank 1, so burn both images from the same build. The CPU clock and serial tests time things with VIA timer 1, so a bad VIA shows up there too; run the tests in order when chasing a fault. The emulator can inject faults to see the reports: --ram-fault, --stuck-irq, --acia-line, --clock (emulator).

Hydra-16 Programmer's Guide

For programmers writing code for the Hydra-16: ROM code (the OS, drivers, servers), programs in RAM, or HyForth words that call the OS. Using the system from its shells is in [HyForth](#) and [WOZMON](#); the machine itself is in the [Hardware Reference](#).

Chapter	Covers
ROM layout and the API index	The two ROM images and the BIOS ROM pages, calling across pages (gates), calling conventions, zero page, error codes, every public entry point
Tasks and the scheduler	The 16 tasks, scheduling, starting and ending tasks, waiting and sleeping, <code>TASK_CALL</code> , break and kill, the console's foreground, driver tasks, boot
Interrupts	The IRQ dispatcher, writing and registering handlers, software interrupts
Memory	The MMU (handles, allocation), shared memory, far pointers and references
Input and output	fds and the IO calls, stdio, the devices, the console, pipes, namespaces
Drivers and file servers	Writing a device: registration, the serve routine, the request block, waiting, the storage layer
Programs	Hydra executables (<code>.hyx</code>): the header, what a program gets, building one (in assembly, or in C with cc65), putting it on a card, how <code>run</code> loads it
The C Programmer's Guide	Programs in C (cc65): setting up, building, running, arguments and exit statuses, files, the console (stdio and conio), running other programs, time, semaphores, errors, memory, assembly, performance, debugging, limits

The system in one page

Hardware. A W65C02S where the task register `T` swaps the whole lower 32K of RAM, zero page and stack included, together with each task's RAM bank and ROM bank selections. A task switch is one register write plus the CPU registers. The 8K window at `$8000` shows a task's own RAM banks or the shared RAM. The 8K BIOS ROM at `$E000` is paged by `W`. 16 prioritised IRQ lines each have their own vector.

Kernel (BIOS ROM page 0):

- a **preemptive scheduler**, round robin at 200 Hz, with task 0 as the idle task;
- an **IRQ dispatcher** that runs each handler in the task that registered it;
- a **memory manager** per task, plus **shared memory** between tasks.

Drivers run in **resident tasks** of their own (serial `$F`, sound `$E`, pipes `$D`, storage `$C`). Their state lives in their task, and they run only from their IRQs and from calls into them.

IO, Plan 9 style: everything is a file:

- Devices are **file servers**; a task opens `/dev/cons`, `/dev/sd/0/data`, `/sd/0/games/star.frt`, ... and reads and writes fds.
- The IO layer (page 2) passes each request to the server's task with `TASK_CALL`, and the data through the client's transfer area in shared RAM.
- Reads that have to wait put the task to sleep.
- Each task has a **namespace** (mount, bind), inherited by the tasks it starts, like its open fds.

The shell is HyForth (task 1) with `|` pipelines, a current directory on the SD cards, file commands (`cd`, `ls`, `cp` ...), scripts (`.hys`) and programs (`.hyx`) run by name; it can start more shells in other tasks, and falls back to WOZMON.

```

task 1: HyForth shell      task N: programs, pipeline stages, more shells
  | IO_READ/IO_WRITE/...  (fds, namespace)
  v
IO layer (page 2) ---TASK_CALL---> server in its driver task: $F serial $E sound $D pipes $C storage
  |                                     ^
  v                                     | IRQ handlers run in the driver's task
kernel (page 0): scheduler, TASK_CALL, IRQ dispatch, MMU, shared memory

```

Where things are in the source

Folder	What
os_rom/all.s	Includes everything, page by page (<code>.scope PAGEn</code>)
os_rom/ os_rom_C02.cfg	The linker config: memory areas per ROM page, segments
os_rom/include/	Constants and macros: <code>hw.inc</code> (hardware), <code>kernel.inc</code> (tasks, errors, far pointers), <code>io.inc</code> (IO), <code>zero.s</code> (the OS zero page), <code>macros.inc</code> , <code>ascii.inc</code>
os_rom/kernel/	Reset and boot (<code>os_main.s</code>), tasks and scheduler (<code>tasks.s</code>), IRQs (<code>irq.s</code>), MMU (<code>mmu.s</code>), shared memory (<code>shared.s</code>), far pointers (<code>fp.s</code>), semaphores (<code>sem.s</code>), exit statuses (<code>exits.s</code>), the COMMON block (<code>common.s</code>), gates, thunks, printing
os_rom/io/	The IO layer (<code>io.s</code> , <code>io_p0.s</code>), namespaces (<code>ns.s</code>), the pipe server (<code>pipe_srv.s</code>)
os_rom/servers/	The console and serial port (<code>ser_srv.s</code> , <code>serctl.s</code> , <code>serfast.s</code>), <code>/dev/sd</code> (<code>sd_srv.s</code>), the RAM disks (<code>ramdisk.s</code>), <code>/proc</code> (<code>proc_srv.s</code>), <code>/env</code> (<code>env_srv.s</code>), <code>/dev/time</code> (<code>time_srv.s</code>), <code>/dev/ram</code> (<code>ram_srv.s</code>)
os_rom/fs/	HydraFS: the server (<code>hfs_srv.s</code> , <code>hfs_write.s</code> , <code>hfs_sparse.s</code> , on ROM page 6), format and check (<code>hfs_format.s</code> , <code>hfs_check.s</code> , on page 3)
os_rom/drivers/	Serial, sound, VIA, SPI, SD cards and the disks' block layer, the clock chip (<code>rtc.s</code>), the storage task
os_rom/sound/	The YM2151's library, <code>/dev/snd</code> (<code>snd_srv.s</code>), the patches, the song player
os_rom/monitor/	WOZMON and the disassembler
os_rom/hyforth/	HyForth
os_rom/shell/	The shell's page 7 part: boot (the volumes, <code>boot.hys</code>), the prompt, the file and card commands, running programs
os_rom/tests/ os_rom/hwtest/	POST and the self tests; the hardware test
os_rom/romfs/ os_rom/romfs.txt	The ROM disk's files (<code>/rom</code>)
os_rom/tools/	The cross-page call checker the build runs (<code>check_pages.js</code>), the ROM space report, the ROMs' checksums
sim/	The emulator, the regression tests, <code>tools/hydrads.js</code> and <code>tools/mkhyx.js</code> (tools)
programs/	A sample program (<code>.hyx</code>), and the header and link config for building others; <code>programs/c/</code> : the C library for cc65, and samples (programs.md)

The **plans** in [../plans](#) record the design reasoning, and what's still to come.

ROM layout, calling conventions and the API index

How the OS ROM is organised, how code calls between ROM pages, and the fixed entry points (thunks) programs use. Part of the [Programmer's Guide](#).

The two ROM images

Image	Chip	Contents
<code>os_rom/bin/os_rom_C02.bin</code> (128K)	BIOS ROM, 16 pages of 8K at <code>\$E000-\$FFFF</code> , selected by <code>W</code>	The BIOS, kernel, drivers, IO layer, HyForth, WOZMON, self tests
<code>os_rom/bin/paged_rom_C02.bin</code> (256K)	Paged ROM banks 0-15 (as many as the ROM disk's files need: up to 111, about 1.7 MB, as the hardware test's checksum table holds), at <code>\$A000-\$DFFF</code>	The whole paged ROM is one disk, the ROM disk (<code>x</code> , mounted at <code>/rom</code> : io.md), with a partition table in its first 512 bytes (bank 0, <code>\$A000-\$A1FF</code>). Bank 0, after the table: COPYTORAM (<code>\$A200</code>) and HyForth's variables (<code>\$A300</code> , copied to <code>\$0800</code> when the shell starts), HyForth's training scripts and sample binary words. Bank 1: the hardware test (<code>hwtest/</code> , scope <code>HWTEST</code>), and the ROMs' checksums at its end (<code>\$DF00</code> , 256 bytes). Banks 0-1 are the disk's partition 1, so the volume leaves them alone. Banks 2 on: the ROM disk's HydraFS volume, <code>/rom</code> 's files (the test song among them: <code>songs/test.zsm</code> , which the build makes from <code>songs/test.mml</code> with <code>sim/tools/hysong.js</code>), which <code>sim/tools/mkromdisk.js</code> adds to the image after the link, from <code>romfs.txt</code> . A bank past the image's end reads as erased (<code>\$FF</code>): <code>romsum.js</code> checks it that way

Both come from one build (`os_rom/all.s`, linked by `os_rom/os_rom_C02.cfg`); after the link, `sim/tools/mkromdisk.js` writes the ROM disk's table and volume (and reads them back to check every file), and `tools/romsum.js` writes a CRC of each BIOS ROM page and paged ROM bank into bank 1, for the hardware test. On the V1 board a bank number's bits 2 and 3, and 6 and 7, trade places on their way to the chips ([hardware.md](#)), so the image holds the bank the CPU selects as `b` at bank `swap(b)`'s place (`mkromdisk.js` and `romsum.js` do this; banks 0-3 are where they seem). The hardware test runs on its own: BIOS ROM code starts it with `_M_HWT_ENTER` (`include/hwtest.inc`), which gives every task paged ROM bank 1 and jumps to it, and it never calls back. HyForth's code in the BIOS ROM uses its variables where the paged ROM's copy puts them, and the sample binary words call BIOS ROM addresses, so most changes affect both images: burn both.

BIOS ROM pages

Page (W)	Scope	Contents	Sources
0	(global)	Reset, POST gate, the kernel (tasks, scheduler, IRQ dispatch, MMU, shared memory), serial and sound drivers, the IO layer's page 0 part, printing, thunks	<code>kernel/</code> , <code>drivers/serial.s</code> , <code>drivers/sound.s</code> , <code>io/io_p0.s</code>
1	PAGE1	HyForth: its interpreter, and its built-in words' headers and code, all run from ROM; a copy of the thunks	<code>hyforth/</code>
2	PAGE2	The IO layer: <code>fds</code> , namespaces, <code>/dev/cons</code> and <code>/dev/ser</code> (and the fast serial and tick interrupt handlers: <code>ser-fast.s</code>)	<code>io/</code> , <code>servers/ser_srv.s</code> , <code>servers/serfast.s</code>
3	PAGE3	Storage: SPI, the disks' block layer (SD cards; the ROM disk: the paged ROM, read-only; the RAM disks, and starting and stopping them), <code>/dev/sd</code> , and HydraFS's format, label, partitions and check	<code>drivers/spi.s</code> , <code>drivers/sd.s</code> , <code>servers/sd_srv.s</code> , <code>servers/ramdisk.s</code> , <code>fs/hfs_format.s</code> , <code>fs/hfs_check.s</code>

Page (W)	Scope	Contents	Sources
4	PAGE4	POST and the self tests (MMU, scheduler, IO); WOZMON (MON_START , through gates: it reads \$E000-\$FDFF from BIOS ROM page ZP_D_PAGE , 0 unless the disassembler was set otherwise, and R runs code on page 0)	tests/, monitor/wozmon.s
5	PAGE5	Far pointers and references; semaphores; exit statuses; the owner chain at a task's end (TASK_ORPHANS : its children, its RAM disk area) and TASK_MAY	kernel/fp.s, kernel/sem.s, kernel/exits.s
6	PAGE6	The HydraFS server (/sd/N/ . . .), in the storage task, on page 3's block layer: reading, writing, sparse files	fs/page6.s, fs/hfs_srv.s, fs/hfs_write.s, fs/hfs_sparse.s
7	PAGE7	The shell: the boot shell's start (the volumes found, one selected), the prompt, the file and card commands HyForth's shell words call (SH_CMD), running programs (run , the .hyx loader, arguments), redirection	shell/page7.s, shell.s, files.s, run.s, redir.s
8	PAGE8	The text editor (edit): a ROM program, run in a task of its own	shell/page8.s, shell/edit.s
9	PAGE9	The system's servers that run in their client's task: /proc (/dev/proc ; a task's memory, mem and ram , copied with MEM_COPY , which /dev/ram shares), /env (each task's environment), /dev/time (the clock: CLOCK_GET , CLOCK_SET ; a DS1747 clock chip, RTC_BOOT) and /dev/ram ; and the pipe server, /dev/pipe (in the pipe task), and the serial port's settings (/dev/ser/ctl , /dev/cons/ctl , in the serial task: ser_srv.s on page 2 reaches them through gates)	servers/page9.s, servers/proc_srv.s, servers/env_srv.s, servers/time_srv.s, servers/ram_srv.s, io/pipe_srv.s, servers/serctl.s, drivers/rtc.s
A	PAGE1::FAR	HyForth's far words (their code: the shell's and IO words, tasks, sound, memory records, multiply and divide ...; their headers are on page 1), its error messages and MALLOC , and the disassembler	hyforth/pagea.s, hyforth/farwords.s, monitor/disasm.s
B	PAGEB	Sound: the YM2151's library (the registers' shadow, volumes, notes, patches, claims), /dev/snd , the patches (the X16's General MIDI set), the console bell (YM_BEEP)	sound/
C	PAGEC	The song player (ZSM): a ROM program the shell starts in a task of its own (play), a client of /dev/snd	sound/pagec.s, sound/player.s
D	PAGED	/dev/spi , the SPI devices as files, served in the storage task (a request's bytes through page 3's SPI_XFER_N and SPI_RECV_N); /dev/gpio , the VIA's port A, served in its client's task	servers/paged.s, servers/spi_srv.s, servers/gpio_srv.s
E-F		Empty	

Every build prints the space left on each page (tools/rom_space.js ; node tools/rom_space.js --table in os_rom shows where), and warns when page 0 or COMMON is nearly full. Page 0 (the kernel) and page 1 (HyForth) have a few hundred bytes each, so new code goes on another page behind gates, and a new HyForth word's code on page A (a far word: see below), with only its header on page 1. Each page's room above COMMON, \$FE00-\$FEFF, has a segment of its own (HIGH_Pn , declared in kernel/high.s ; page 1's is FORTH_TOP) for code that fits nowhere else. The link map (os_rom/obj/os_rom_C02.map) shows each page's segments.

Fixed addresses on every page:

Address	What
\$E000	Reset entry: sets W = 0 and continues on page 0. Every page starts with it, because W isn't reset by hardware
\$F800-\$F8F8	The thunk table (pages 0 and 1): jmp s to the public calls, below
\$FD00-\$FDFF	The COMMON block: IRQ entry stubs and exit, the fast handlers' stubs (VIA, ACIA, YM2151), NMI entry, far-call trampolines, PEEK_PAGE (a byte from another page). Identical on every page (the link checks it)
\$FE00-\$FEFF	Each page's room above COMMON (HIGH_Pn): page 0's boot welcome, page 1's HyForth words
\$FFFA-\$FFFD	NMI vector (the COMMON block's NMI_ENTRY) and RESET vector (\$E000)

Calling across ROM pages

Writing `W` switches the BIOS ROM page, and the very next instruction is fetched from the new page. So page switching happens only in the COMMON block, which is the same code at the same address on every page.

Gates. Code calls a routine on another page through a *gate*, a small local stub made by a macro:

```
FAR_GATE_INLINE IO_OPEN, PAGE2::IO_OPEN, 2      ; a label IO_OPEN on this page
...
jsr IO_OPEN                                     ; calls PAGE2::IO_OPEN with W = 2, then comes back
```

- The gate is `jsr FAR_INLINE` followed by the target's address and page (6 bytes).
- `FAR_INLINE` (COMMON) reads them, pushes the caller's `W`, switches page, calls the routine, and switches back.
- **Registers:** `.A`, `.X`, `.Y`, `C` and `V` pass through both ways, and `N/Z` on return reflect `.A`.
- The stack holds the caller's page under the return address, so a routine entered through a gate can find its caller's page (`$0103,X` after `tsx` at its entry; `IO_CALLER_PAGE` does this).
- Not for IRQ handlers.

Scopes. Each page's sources are included inside `.scope PAGEn` (see `all.s`), with the page's gate file first. A page's gate labels therefore shadow the page 0 routines of the same name for that page's code: `jsr WRITE_CHAR` on page 2 goes through page 2's gate.

- Refer to page 0's own label with `::NAME`, and to another page's with `PAGEn::NAME`.
- The gate files are `hyforth/page1.s`, `io/page2.s`, `drivers/page3.s`, `tests/page4.s`, `kernel/page5.s`, `fs/page6.s`, `shell/page7.s`, `shell/page8.s`, `servers/page9.s` and `hyforth/pagea.s`.
- Page A's code is the scope `FAR` inside `PAGE1` (included from `hyforth/hyforth.s`), so it sees HyForth's names (its zero page, variables and constants), with its own gates first. Page 1 reaches it through the global aliases after `PAGE1` in `all.s` (`FW_ENTRY_PA ...`).
- The gates from page 0 outward are in `kernel/page0_gates.s`, `io/io_p0.s` and `drivers/storage.s`.

The page checker. The build ends by running `os_rom/tools/check_pages.js` on the debug info. It lists any `jsr/jmp` from one page to a routine on another that doesn't go through a gate ("No cross-page references" when there's none). It can't see pointers (a routine's address passed in registers), so:

- **Callbacks** (IRQ handlers, serve routines, `TASK_RUN` entry points, `TASK_CALL` routines) must be page 0 addresses, or come with their page where the call takes one (`TASK_RUN`, `TASK_SET_BREAK`).
- **Data pointers** (strings, buffers) are read as the reader sees them. The IO layer reads names through far pointers (`memory.md`), so they can be on the caller's page. Other buffers must be in RAM.

Gates into a task. `TASK_GATE name, target, task` makes a gate that runs `target` in another task (with `TASK_CALL`); e.g. HyForth's sound words run the sound code in the sound task.

HyForth's far words. HyForth's inner interpreter jumps straight to a word's code, so a built-in word's code has to be on page 1. A word whose code is on page A has a header made by `def_far` instead of `def_word`: its code on page 1 is `jsr FARWORD` and the address of its code on page A. `FARWORD` calls it through the gate `FW_CALL`, and it runs as it would on page 1: page A has its own `next`, `this`, `keeps`, `errrtn` and stack routines (`spush_0`, `spull_1` ...) that end the word, back on page 1 (`next`, or `errrtn` with the error in `ERRFLAG`). They put the stack back as it was when the word started, so a subroutine can use them too.

HyForth's libraries. A built-in word is in the base language or in a library (HyForth). Each library's headers are a chain of their own: the headers between `lib_begin LIBN_x` and `lib_end` (in `hywords.s` and `primitives.s`) go on library x's chain, and a library can have several such sections. `LIB_HEADS` holds each chain's head. The loaded libraries are the bits of `LIBSET`, in HyForth's zero page: `RESFIND` and words search the words in RAM, then the libraries loaded from files, then the base, then

each loaded library's chain (`LIB_NEXT`). `col` loads `LIB_BOOT` , or nothing for a bare Forth (`forth: forth_bare_main` , `BAREFLAG`). A new library needs a `LIBN_` number and bit (`hyforth.s` ; up to 8) and its name and needs on page A (`LIBNAMES` , `LIBNEEDS` in `farwords.s`).

The interpreter's line reader calls three hooks on page A (`farwords.s`): `LINE_START` (the last line's redirection and pipeline undone; `boot.hys`), `LINE_PROMPT` and `LINE_READ` (pipelines and redirection set up). Each does the shell's part (page 7) only when `LIB_SHELL` is loaded, and `RUNNAME` runs a program for an unknown word only then. So the base itself calls no shell code.

Libraries from files (`lib name: RLIBLOAD` in `farwords.s`) have `RLIB_MAX` (4) slots in `FORTH_DATA`: a name (`RLIBNAME`), a chain head (`LIB_HEADS2`) and a bit in `LIBSET2` , a second zero-page set that `LIB_NEXT` searches before `LIBSET` . `LIBSET2` 's bit 7 (`LIB2_BASE`) is the base: its chain is `LIB_HEADS2` 's entry 7. The file is found by `SH_CMD` 's `SHC_LIBOPEN` (`name.hy1` : the current directory, then `$LIBPATH` or `/lib`), and read by the include engine (`INCOPENFD`). While it's read, `LASTHEAP` starts from 0, so its definitions make a chain of their own, and the words in RAM's chain (kept in `RLIBSAVE`) aren't searched. The user's words in RAM are a chain that ends in 0 (`warm`), not in the base. At the file's end, `INCEND` calls `RLIBEND` : the slot whose `RLIBDEPTH` is that include depth gets the chain (`RLIBHAVE` , `LIBSET2`), and `LASTHEAP` is back. `INCABORT` (an error while it's read) sets `RLIBFAIL` , so `RLIBEND` frees the slot instead. `-lib` only clears the bit, so the memory isn't freed.

Calling conventions

- **Success and failure:** the kernel, MMU, scheduler and IO calls return **C = 0** on success, and **C = 1** with an error code in `.A` on failure.
- **Exceptions** (WOZMON's convention): `READ_CHAR` returns C = 1 with a key in `.A` (C = 0: no key), and `GET_CHAR` returns C = 1 with a key (C = 0: an error in `.A` , e.g. `ERR_IO_EOF` at the end of a pipe).
- **Registers:** each routine's header comment in the source says which registers it keeps ("Preserves .X, .Y"). Most calls keep `.X` and `.Y` and the caller's I flag.
- **16-bit values** go in `.A` (low) and `.Y` (high).
- **Zero-page parameters:** some calls take them in the caller's zero page (e.g. `IO_READ` takes the buffer in `ZP_IO_BUF`). The addresses are in the [API index](#).
- **Interrupts:** calls may enable interrupts only if they were enabled on entry; they use `php / sei ... p1p` for critical sections. Don't call blocking calls (IO, `GET_CHAR` , `YIELD` , `TASK_SLEEP`) with interrupts off, or from an IRQ handler.

Zero page

Every task has its own zero page, so zero page is only divided up within a task:

- **The OS zero page** grows up from `$02` (the `ZEROPAGE` segment, `os_rom/include/zero.s`), with the same layout in every task because the same ROM code runs in every task. `$00` and `$01` are the bank registers.
- **A task's own zero page** grows down from `$FF` , reserved with macros:

```
TASK_ZP_BEGIN
TASK_ZP      MY_PTR, 2          ; MY_PTR = $FE
TASK_ZP      MY_COUNT, 1       ; MY_COUNT = $FD
TASK_ZP_END          ; link error if it runs into the OS zero page
```

Each task's code (the shell, each driver) has its own block; blocks for different tasks may overlap. For example, HyForth uses `$C8-$FF` in the shell task, and the serial driver its own bytes in the serial task.

OS zero-page variables that calls take parameters in `ZP_IO_BUF` , `ZP_IO_CNT` and `ZP_IO_OFS` are at fixed addresses (programs are built against them: `zero.s` asserts them); the others are from the current build's `os_rom/obj/os_rom_C02.1b1` , and can move between builds. A program's own zero page is `$E0-$FF` (`PROGRAM_ZP` : the OS's stays below it).

Variable	Address	Used by
RAM_BANK_REG / ROM_BANK_REG	\$00 / \$01	The bank registers
TASK_STATUS_REG	\$03	The task's status bits (tasks.md)
ZP_TEMP_VEC, ZP_TEMP_VEC2	\$0A, \$0C	MEM_COPY, TASK_START
ZP_TC_VEC, ZP_TC_TASK	\$1F, \$21	TASK_CALL, DEV_REGISTER (serve routine)
ZP_FP (4 bytes)	\$42	Far pointer calls
ZP_IO_BUF, ZP_IO_CNT, ZP_IO_OFS	\$06, \$08, \$0A (fixed)	IO_READ, IO_WRITE, IO_SEEK, IO_STAT, IO_MOUNT, IO_BIND

Error codes

(os_rom/include/kernel.inc)

Code	Name	Meaning
\$02	ERR_OUT_OF_MEMORY	No memory for the allocation
\$40	ERR_MEM_NOT_ALLOC	Not allocated
\$41	ERR_MEM_NOT_VALID	Not a valid handle (or another task's RAM)
\$42	ERR_MEM_NOT_SUPPORTED	Not possible for this kind of memory (e.g. writing ROM)
\$43	ERR_MEM_BAD_ARG	A bad size or offset
\$44	ERR_MEM_NO_HANDLES	The handle table is full
\$45	ERR_MEM_LOCKED	The allocation is locked
\$50	ERR_IRQ_CHAIN_FULL	Already 2 handlers for this IRQ (or a SWI handler)
\$51	ERR_IRQ_NOT_FOUND	No such handler registered
\$70	ERR_IO_NOT_FOUND	No such file or device
\$71	ERR_IO_BAD_FD	fd not open, or out of range
\$72	ERR_IO_MODE	Not opened for that (e.g. writing a read-only fd)
\$73	ERR_IO_WOULD_BLOCK	No data yet (a non-blocking fd)
\$74	ERR_IO_EOF	End of file (IO_GETC, GET_CHAR)
\$75	ERR_IO_NO_FDS	All 12 of the task's fds are open
\$76	ERR_IO_NO_DEVS	The device table is full
\$77	ERR_IO_NAME	A bad, too long or duplicate name
\$78	ERR_IO_BAD_REQ	The server doesn't support that request
\$79	ERR_IO_DEVICE	The device didn't respond (e.g. no SD card)
\$7A	ERR_IO_BROKEN	Write to a pipe nobody reads
\$7B	ERR_IO_NO_PIPES	All pipes are in use
\$7C	ERR_IO_NS_FULL	The task's namespace is full
\$7D	ERR_IO_NS_LOOP	Too many binds in a row
\$7E	ERR_IO_NOT_READY	The device isn't started
\$7F	ERR_IO_MEDIA	The medium refused the command or data
\$80	ERR_IO_NOT_FS	The card holds no HydraFS (no superblock, or a version this can't read)
\$81	ERR_IO_FULL	The card is full
\$82	ERR_IO_EXISTS	There's a file or directory by that name already
\$83	ERR_IO_NOT_EMPTY	The directory has files in it
\$84	ERR_IO_BUSY	The file is open

Code	Name	Meaning
\$85	ERR_IO_NOT_DIR	Not a directory (<code>IO_CHDIR</code> , <code>rmdir</code>)
\$86	ERR_IO_IS_DIR	A directory, where a file was wanted (<code>rm</code> , <code>cp</code>)
\$87	ERR_IO_NOT_EXEC	A Hydra executable whose header doesn't fit task RAM (<code>run</code>)
\$88	ERR_IO_PERM	Not allowed: another task's area on the RAM disk
\$F1	ERR_NO_TASKS_AVAILABLE	All 16 tasks are busy
\$F2	ERR_TASK_BUSY	The task (or player) is busy
\$F3	ERR_BAD_TASK	Not a task that can be used that way

API index: the thunks

The thunk table at `$F800` (on BIOS pages 0 and 1) gives every public call a fixed address, so programs in RAM (and HyForth's `syscall`) can call them whatever page they're on. Each is a `jmp`; call it with `jsr`. Details are in the guide chapter for each area, and in the routine's header comment in the source.

Address	Call	In / out (summary)	Guide
\$F800	READ_CHAR	A key from stdin if there is one: <code>C = 1</code> , <code>.A</code> = key	io
\$F803	WRITE_CHAR	Write <code>.A</code> to stdout	io
\$F806	WRITE_BYTE	Print <code>.A</code> as 2 hex digits	
\$F809	WRITE_HEX	Print <code>.A</code> (0-F) as a hex digit	
\$F80C	WRITE_HEX_MASK	Print <code>.A</code> 's low nibble as a hex digit	
\$F80F	WRITE_HSTRING	Print the HString (length byte, then text) at <code>.A.Y</code>	
\$F812	WRITE_CRLF	Print CR LF	
\$F815	CLEAR_SCR	Clear the terminal (ANSI)	
\$F818	DISASM	Disassemble at <code>ZP_D_XAM</code>	WOZMON
\$F81B	DISASM_AY	Disassemble at <code>.A.Y</code> : <code>C = 0</code> one instruction, <code>C = 1</code> <code>.X</code> instructions	
\$F81E	MEM_COPY	Copy <code>.A.Y</code> bytes from <code>ZP_TEMP_VEC</code> to <code>ZP_TEMP_VEC2</code>	
\$F821	MM_ALLOC	<code>.A.Y</code> = size, <code>.X = 0</code> or <code>AI_PAGED</code> → <code>.A</code> = handle	memory
\$F824	MM_FREE	<code>.A</code> = handle	
\$F827	MM_READ	<code>.A</code> = handle, <code>.Y</code> = offset → <code>.A</code> = byte	
\$F82A	MM_WRITE	<code>.A</code> = handle, <code>.Y</code> = offset, <code>.X</code> = byte	
\$F82D	MM_LOCK	<code>.A</code> = handle → <code>.A.Y</code> = pointer, <code>.X</code> = previous RAM bank	
\$F830	MM_UNLOCK	<code>.A</code> = handle, <code>.X</code> = RAM bank from <code>MM_LOCK</code>	
\$F833	MMU_TEST	Run the MMU self test	WOZMON
\$F836	SH_ALLOC	<code>.A.Y</code> = size → <code>.A</code> = shared handle	memory
\$F839	SH_ATTACH	<code>.A</code> = shared handle: take a reference	
\$F83C	SH_DETACH	<code>.A</code> = shared handle: drop the reference	
\$F83F	SH_READ	<code>.A</code> = shared handle, <code>.Y</code> = offset → <code>.A</code> = byte	
\$F842	SH_WRITE	<code>.A</code> = shared handle, <code>.Y</code> = offset, <code>.X</code> = byte	
\$F845	SH_LOCK	<code>.A</code> = shared handle → mapped at <code>\$8000</code> ; <code>.X</code> , <code>.Y</code> = what to restore	
\$F848	SH_UNLOCK	<code>.X</code> , <code>.Y</code> from <code>SH_LOCK</code>	
\$F84B	MM_TASK_RESET	<code>.A</code> = task: free everything it has	
\$F84E	MM_FIND	<code>.A.Y</code> = address → <code>.A</code> = handle	

Address	Call	In / out (summary)	Guide
\$F851	MM_SET_FLOOR	.A = lowest page the MMU may hand out	
\$F854	YIELD	Let the other tasks run	tasks
\$F857	NO_PREEMPT	Hold the CPU (nestable)	
\$F85A	PREEMPT	Undo NO_PREEMPT	
\$F85D	TASK_WAIT	Wait until IO_WAKE	
\$F860	IO_WAKE	.A = task: wake it	
\$F863	TASK_RUN	.A.Y = entry, .X = ROM page → .A = new task	
\$F866	TASK_STATUS	.A = task → .A = its status	
\$F869	SCHED_TEST	Run the scheduler self test	
\$F86C	IO_OPEN	.A.Y = name, .X = mode → .A = fd	io
\$F86F	IO_CLOSE	.A = fd	
\$F872	IO_READ	.A = fd, ZP_IO_BUF, ZP_IO_CNT → ZP_IO_CNT = bytes read	
\$F875	IO_WRITE	.A = fd, ZP_IO_BUF, ZP_IO_CNT → ZP_IO_CNT = bytes written	
\$F878	IO_GETC	.X = fd → .A = byte	
\$F87B	IO_PUTC	.X = fd, .A = byte	
\$F87E	IO_SEEK	.A = fd, ZP_IO_OFS = 32-bit offset	
\$F881	IO_STAT	.A = fd, ZP_IO_BUF = 48-byte buffer	io
\$F884	IO_CTL	.A = fd, .X = code, .Y = argument	
\$F887	DEV_REGISTER	.A.Y = name, .X = task, ZP_TC_VEC = serve routine	servers
\$F88A	IO_TEST	Run the IO self test	
\$F88D	GET_CHAR	Wait for a key from stdin: C = 1, .A = key	io
\$F890	IO_DUP2	.A = fd, .X = new fd: make .X a copy of .A	io
\$F893	IO_PIPE	→ .A = read fd, .X = write fd	
\$F896	IO_DUP	.A = fd → .A = a new fd for the same file	
\$F899	TASK_CLONE	.A.Y = entry, .X = page → .A = new task, a copy of this one	tasks
\$F89C	IO_MOUNT	.A.Y = path, ZP_IO_BUF = device name	io
\$F89F	IO_BIND	.A.Y = path, ZP_IO_BUF = target	
\$F8A2	IO_UNMOUNT	.A.Y = path	
\$F8A5	IO_NS_LIST	Print the namespace	
\$F8A8	TASK_SET_BREAK	.A.Y = break handler, .X = its page	tasks
\$F8AB	TASK_SIGNAL	.A = break or kill flag, .X = task	
\$F8AE	CONS_SET_FG	.A = task: bring it to the front	io
\$F8B1	FP_MAKE	.A.Y = address, .X = ROM page → ZP_FP	memory
\$F8B4	FP_READ	.Y = offset → .A = byte at ZP_FP + .Y	
\$F8B7	FP_WRITE	.Y = offset, .X = byte	
\$F8BA	FP_COPY	.A.Y = destination, .X = count; C = 1 for a string	
\$F8BD	MM_REF	ZP_FP → .A = MMU handle for it	
\$F8C0	MM_FP	.A = handle → ZP_FP	
\$F8C3	SH_REF	ZP_FP → .A = shared handle for it	
\$F8C6	SH_FP	.A = shared handle → ZP_FP	
\$F8C9	IO_CREATE	.A.Y = name, .X = mode, ZP_IO_BUF = new file's mode bits → .A = fd	io

Address	Call	In / out (summary)	Guide
\$F8CC	IO_REMOVE	.A.Y = name	io
\$F8CF	IO_WSTAT	.A = fd, ZP_IO_BUF = stat record	io
\$F8D2	IO_CHDIR	.A.Y = a directory's path: the current directory	io
\$F8D5	IO_GETCWD	ZP_IO_BUF = 64-byte buffer ← the current directory	io
\$F8D8	SEM_NEW	.A = count, .Y = 0 or SEM_MUTEX → .A = semaphore	tasks
\$F8DB	SEM_ACQUIRE	.A = semaphore: take one (waits)	tasks
\$F8DE	SEM_TRY	.A = semaphore: take one, or ERR_SEM_BUSY	tasks
\$F8E1	SEM_RELEASE	.A = semaphore: give one back	tasks
\$F8E4	SEM_FREE	.A = semaphore: free it	tasks
\$F8E7	TASK_SLEEP	.A.Y = ticks (200 a second, up to 32767)	tasks
\$F8EA	TICKS_GET	→ .A.Y = the tick count	tasks
\$F8ED	CLOCK_GET	.X = a zero page address ← 4 bytes: seconds since 2000-01-01	io
\$F8F0	TASK_EXITS	.A = exit code, ZP_IO_BUF = message (high byte 0: none): end this task	tasks
\$F8F3	TASK_JOIN	.A = task: wait for it to end → .A = its code, its message at ZP_IO_BUF (high byte 0: not wanted)	tasks
\$F8F6	SHELL_CMD	An entry point for TASK_RUN (.X = 0): a command shell running the lines on its stdin (rc - c)	tasks

Calls without a thunk (for ROM code; reached with a gate from other pages): TASK_SLEEP_UNTIL, TASK_START, TASK_CALL, IRQ_REGISTER, IRQ_UNREGISTER, SWI_REGISTER, SWI_UNREGISTER, SW_INT, DRV_START, IO_FLUSH, YM_BEEP, and the server helpers IO_SRV_MAP, IO_SRV_UNMAP, IO_SRV_COUNT.

Adding a thunk: add it at the end of the list in `include/thunks.inc`, with its address: both copies of the table are made from it, page 0's (`kernel/thunks.s`) and page 1's (`hyforth/page1.s`), and the build checks every address. Give it as `THUNK_P0`: page 1's entry then goes on to page 0's (`THUNK_TO_P0`), so code running on page 1 (HyForth's binary words) can call it too; `THUNK` is for a call page 1 has a gate of its own for, which its entry jumps to. A call on another page gets its gate in `GATES_P0` (`kernel/page0_gates.s`), or, as `GATES_P0` is full, in `BIOS_THUNKS` after the thunks' `jmp s` (`FAR_GATE_INLINE`, at the end of `kernel/thunks.s`): the thunks are at a fixed `$F800`, and room below them doesn't help what's after them. Never move existing entries: programs rely on the addresses.

Adding code

- **Where:** pick the page by role (table above). If page 0 code needs it, add a gate on page 0 (`page0_gates.s`). If code on another page calls into it, add a gate in that page's gate file.
- **Build:** see [Getting started](#). The build stops at the first error, then checks for cross-page calls.
- **Test:** run `node build.js rom test` (or `makeC02 test`: build, then the regression tests). For a new feature, add a test to `sim/regress.js` ([emulator](#)).
- **Style:** match the file's formatting: mnemonics and operands in fixed columns, `;` comments aligned, CRLF line endings, a header comment on each routine (what it does, IN, OUT, what it preserves).

Tasks and the scheduler

Tasks, how they're scheduled, how to start, stop, wait for and signal them, and how drivers run in tasks of their own. Sources: `os_rom/kernel/tasks.s`, `os_rom/drivers/via.s` (the tick). Part of the [Programmer's Guide](#).

What a task is

The Hydra has **16 tasks**, `T = $0-$F`. The hardware gives each one its own:

- **RAM** `$0000-$7FFF`: its zero page, stack page and program RAM;
- **bank registers**: which RAM bank is at `$8000-$9FFF` (`$00`) and which paged ROM bank at `$A000-$DFFF` (`$01`);
- **task RAM banks** on the memory modules.

Selecting a task is one write to `T` (`$FFF0`). Everything else is shared: the ROMs, shared RAM, I/O, and the pseudo-registers `U`, `V`, `W`. So a task switch saves and restores `W` and `U` in the task's stack frame.

The software gives each task:

- an **MMU area** for its allocations ([memory.md](#));
- a **task system page** with the IRQ tables and its 12 file descriptors;
- an **IO transfer area** in shared RAM, holding its IO requests and its namespace ([io.md](#)).

Per-task memory map (all tasks alike):

Addresses	Use
<code>\$00</code> , <code>\$01</code>	Bank registers (RAM bank, ROM bank)
<code>\$02-...</code>	OS zero page (<code>include/zero.s</code>), the same layout in every task
<code>...-\$FF</code>	The task's own zero page, reserved with <code>TASK_ZP</code> (rom-layout.md)
<code>\$0100-\$01FF</code>	The stack
<code>\$0200-\$06FF</code>	Buffers (in the shell: HyForth's input buffer and stacks, WOZMON's input buffer)
<code>\$0700-\$07FF</code>	stdio buffers: stdout (<code>\$0700</code>) and the stdin read-ahead (<code>\$0780</code>)
<code>\$0800-\$7CFF</code>	Program RAM. The MMU hands out pages from the top down; below its <i>page floor</i> a program can use RAM directly (HyForth's variables are at <code>\$0800</code> , and its dictionary grows up after them)
<code>\$7D00-\$7DFF</code>	Task system page: IRQ tables (<code>\$7D00-\$7D8F</code>), unclaimed-IRQ counters (<code>\$7D90-\$7D9F</code>), fd table (<code>\$7DA0-\$7DFF</code>)
<code>\$7E00-\$7FFF</code>	MMU area: allocation maps and the handle table

Task numbers:

Task	Use
<code>\$0</code>	System task: boot, then the idle task (runs <code>wai</code> when no other task can run)
<code>\$1</code>	The shell (HyForth, then WOZMON), the foreground task at boot
<code>\$2-\$B</code>	Free: started with <code>TASK_RUN</code> , <code>TASK_START</code> , <code>TASK_CLONE</code> , HyForth's <code>shell</code> or pipelines
<code>\$C</code>	Storage driver: <code>/dev/sd</code> , HydraFS (<code>/sd</code>) and <code>/dev/spi</code> : the SPI bus is its (Resident)
<code>\$D</code>	Pipe server (Resident)
<code>\$E</code>	Sound driver: <code>/dev/snd</code> (Resident)
<code>\$F</code>	Serial driver: <code>/dev/cons</code> , <code>/dev/ser</code> , <code>/dev/ser/ctl</code> (Resident)

Task status

`TASK_STATUS_REG` (\$03 in each task's zero page) holds the task's state. `TASK_STATUS` (\$F866, `.A = task` → `.A = status`) reads another task's.

Bit	Flag	Meaning
0	<code>TASK_BUSY_FLAG</code> (\$01)	In use (0 = free)
1	<code>TASK_PAUSED_FLAG</code> (\$02)	Not to be run: being set up, or waiting for a task it started (<code>TASK_START</code>)
2	<code>TASK_WAITING_FLAG</code> (\$04)	Waiting (IO, <code>TASK_WAIT</code> , a sleep) until woken
3	<code>TASK_RESIDENT_FLAG</code> (\$08)	A driver: only runs from IRQs and calls into it
4	<code>TASK_BREAK_FLAG</code> (\$10)	A break is due
5	<code>TASK_KILL_FLAG</code> (\$20)	A kill is due
6	<code>TASK_CALLING_FLAG</code> (\$40)	In a <code>TASK_CALL</code> : its call is running in another task
7	<code>TASK_GUEST_OUT_FLAG</code> (\$80)	Switched out in the middle of another task's call

Scheduling

The tick. VIA timer 1 interrupts 200 times a second (`SCHED_TICK_HZ`, every 5 ms: `SCHED_START`). At each tick the IRQ dispatcher switches to the next runnable task, round robin over tasks 1-15. Task 0 runs only when no other task can; it runs `wai`, so the CPU sleeps until the next interrupt.

A task to run next. An interrupt that wakes a task with a deadline can put it first: the tick does this for a sleeper whose time has come (`TASK_SLEEP`: the song player's ticks), and the sound clock for its waiting player (`SCHED_URGENT_T`, in the system task's zero page: `SCHED_PICK` starts its round there, once). So a busy task's slice doesn't make a note late.

Runnable means busy, and not paused, waiting, resident or in a call. A resident task switched out in the middle of a call is runnable too, until it finishes that call.

The task frame. A task that isn't running keeps one frame on its own stack, whatever stopped it (the tick, `YIELD`, or waiting):

```
U, ZP_TC_TASK, ZP_TC_VEC+1, ZP_TC_VEC, Y, W, X, A, P, PCL, PCH    (top of the stack first)
```

Its stack pointer is kept in its zero page. A switch saves the SP, picks the next task, writes `T`, loads that task's SP and unwinds its frame.

Holding the CPU:

Way	Effect	Use for
<code>sei ... cli</code> (or <code>php / sei ... plp</code>)	No interrupts, so no task switch	Short sections only (tens of cycles): it holds off serial input and the tick
<code>NO_PREEMPT ... PREEMPT</code> (\$F857 / \$F85A)	Interrupts run, but no task switch; a switch that came due happens at <code>PREEMPT</code>	Longer sections; nestable (a counter)
<code>YIELD</code> (\$F854)	Give up the CPU now	Polling loops

How `NO_PREEMPT` behaves:

- **Blocking:** a task that blocks (IO, `TASK_WAIT`, `YIELD`) while holding it still gives up the CPU: it can't make progress anyway.
- **Calls into other tasks:** it holds across `TASK_CALL`s, such as the IO requests the task makes; the server running its request isn't switched out either. A switch that comes due in the server is the caller's when the call returns, so the caller's `PREEMPT` makes it.
- **Not runnable just now:** a switch asked for while a task holds it is noted even if the task is marked waiting (as `IO_SERVE` marks it before its call), not lost till the next tick.

- **The kernel's own use:** the MMU calls and task reset hold `NO_PREEMPT` rather than turning interrupts off.
- **The scheduler:** it scans for the next task with interrupts on between its looks at each task.

So interrupts, above all the serial port's, are rarely held off for long ([interrupts.md](#)).

Starting tasks

Call	Does
<code>TASK_RUN</code> (<code>\$F863</code>)	<code>.A.Y</code> = entry point, <code>.X</code> = its ROM page (0 for RAM or page 0 code) → <code>.A</code> = the new task. It runs alongside the caller; it ends when its entry point returns
<code>TASK_START</code>	<code>ZP_TEMP_VEC</code> = entry point (RAM or page 0) → runs it in a new task and waits for it to finish (the caller is paused)
<code>TASK_CLONE</code> (<code>\$F899</code>)	Like <code>fork</code> : a new task with a copy of the current one (below), starting at <code>.A.Y</code> on page <code>.X</code>
<code>SHELL_CMD</code> (<code>\$F8F6</code>)	Not a call but an entry point for <code>TASK_RUN</code> (page 0): a command shell , HyForth running the command lines on its stdin with no banner or prompt, as Plan 9's <code>rc -c</code> . It ends at its input's end, with the last command's exit status. C's <code>system</code> gives it a pipe with the line in it
<code>DRV_START</code>	Start a driver in a given task (below)

Every new task gets a copy of its parent's **open fds, namespace, current directory** and **environment** (each server is told: `H9_DUP`), and records its parent (`ZP_TASK_OWNER`; when a task ends, the tasks it started get its owner instead, and its area on the RAM disk is removed: `TASK_ORPHANS`). `TASK_MAY` (page 5) answers "may task A use task N's things" from the owner chain, task 0 always yes: the RAM disk's areas use it. So `TASK_RUN` from the shell gives a task that prints on the console and reads the keyboard when it's in front. Output buffered by the parent is written out first (`IO_FLUSH`), so it comes out in order.

`TASK_CLONE` copies:

- the task's RAM, `$0200-$7CFF` and the MMU area `$7E00-$7FFF`. Not copied: the stack page, the task system page, and the free pages between the MMU's page floor and its lowest allocated page;
- its task zero page (everything above the OS zero page);
- its fds and namespace.

The copy goes a page at a time through the IO transfer area, about 1/400 s per page at 3.58 MHz, before the new task runs. HyForth uses it for pipelines (each stage but the last runs in a copy of the shell) and for `run` ning a script.

Waiting for a task started alongside: `TASK_JOIN` (`$F8F3`, below) gives the task the console if the caller has it, makes the caller the task's parent (`TASK_PARENT`, in the task's zero page) and pauses, as `TASK_START` does, so the task's end wakes it; then it takes the console back and returns the task's exit status. The shell's `run` starts a program with `TASK_RUN` (or `TASK_CLONE`) and waits for it this way (`SH_WAIT` in `os_rom/shell/run.s`; [programs.md](#)).

From **WOZMON**, `addrS` starts a task at `addr` and waits for it (`TASK_START`).

Ending tasks

A task ends when its entry point returns (`TASK_EXIT`), or when it calls `TASK_EXITS` with an exit status (below). Everything it had is freed at once:

- its fds are closed (each server gets `H9_CLUNK`);
- its MMU area is reset (all its pages, chunks and banks);
- its shared memory references are dropped;
- its IRQ and software interrupt handlers are removed.

If it was the foreground task, the task that started it gets the console back (`CONS_RELEASE`). A parent waiting in `TASK_START` or `TASK_JOIN` continues.

Exit statuses

As Plan 9's `exits` and `wait`: a task ends with a **code** (0-255, 0 for success) and a **message** (up to 30 characters, `EXIT_MSG_MAX`; none for most), and the task that started it gets them when it waits for it.

Call	Does
<code>TASK_EXITS</code> (<code>\$F8F0</code>)	End this task: <code>.A</code> = the code, <code>ZP_IO_BUF</code> = the message (zero-terminated; a high byte of 0: none). Doesn't return
<code>TASK_JOIN</code> (<code>\$F8F3</code>)	<code>.A</code> = a task this one started: wait for it to end (it has the console meanwhile, if this task has it) → <code>C</code> = 0, <code>.A</code> = its code, and its message at <code>ZP_IO_BUF</code> (a buffer of 31 bytes; a high byte of 0: not wanted). A task that's ended already isn't waited for. <code>C</code> = 1, <code>ERR_BAD_TASK</code> : not a task

How a task ends	Code	Message
Its entry point returns	0	none
<code>TASK_EXITS</code>	<code>.A</code>	<code>ZP_IO_BUF</code> 's
A break (Ctrl-C) with no break handler	130 (<code>EXIT_BREAK</code>)	interrupt
A kill (Ctrl-\, <code>kill</code> , its starter's break)	137 (<code>EXIT_KILLED</code>)	killed

The status is written as the task ends, before its parent is woken, to a record for each task in the system's shared bank (`EXIT_TABLE`, `$9D20`: 16 × 32 bytes, the code and the message; `os_rom/kernel/exits.s`, page 5). It stays there until the task's number is used again. The shell keeps the status of each program it waits for: HyForth's `status`, and `/env/status` (Plan 9's `$status`: the message, or the code if there's none, or empty for success). A script's is that of its last command, or its error's number, or what `exits` gives ([HyForth](#)).

Waiting and sleeping

Call	Does
<code>TASK_WAIT</code> (<code>\$F85D</code>)	Sleep until another task or an IRQ handler calls <code>IO_WAKE</code> for it
<code>IO_WAKE</code> (<code>\$F860</code>)	<code>.A</code> = task: make it runnable again (from any task, or an IRQ handler)
<code>TASK_SLEEP</code>	Sleep for <code>.A.Y</code> ticks (200 a second, up to 32767)
<code>TASK_SLEEP_UNTIL</code>	Sleep until the tick count reaches <code>.A.Y</code> (up to 32767 ticks ahead)
<code>TICKS_GET</code>	<code>.A.Y</code> = the tick count (200 a second; wraps after about 5.5 minutes)

A sleeping task uses no CPU; the system task's tick handler wakes it (`SLEEP_CHECK`). A break or kill ends a sleep early. For steady timing (e.g. music), use `TASK_SLEEP_UNTIL` with a running target, so delays don't add up: the sound driver's tune player does this.

IO waits happen by themselves: a read with no data makes the task wait until the server wakes it ([io.md](#)).

How the kernel waits. Every wait is the same: the task's bit goes in a 16-bit **wait mask** (bit = task), it sets `TASK_WAITING_FLAG` and `YIELDs`, and when it's woken it looks again, waiting again if it has to. Waking a mask (`TASK_WAKE_MASK`) wakes every task in it and clears it; each looks again, so a wake that turns out to be for nothing costs a look and nothing more. Everything that waits is built this way: a busy server's callers (`ZP_TC_WAITERS`, `TC_WAIT_FREE`), sleepers (`ZP_SLEEPERS`, woken by the tick when their time comes), a pipe's readers and writers, the console's, and semaphores. (The serial port's interrupt wakes its masks with a copy of `TASK_WAKE_MASK` that uses no stack.)

Semaphores

For tasks that share something, or wait for each other: a semaphore is a count and the tasks waiting for it. `SEM_ACQUIRE` takes one, or waits (using no CPU) until `SEM_RELEASE` gives one back. A **mutex** is a semaphore of 1 with a holder: only the task that took it can release it.

Call	Does
SEM_NEW (\$F8D8)	.A = its count (how many can take it before a task has to wait: 0-255), .Y = 0, or SEM_MUTEX (\$80 : a mutex, count 1) → .A = the semaphore (1-16)
SEM_ACQUIRE (\$F8DB)	.A = semaphore: take one, waiting until there is one. A break or kill ends the wait
SEM_TRY (\$F8DE)	The same, but ERR_SEM_BUSY at once instead of waiting
SEM_RELEASE (\$F8E1)	.A = semaphore: give one back, and wake the tasks waiting for it
SEM_FREE (\$F8E4)	.A = semaphore: free it (any task can); the tasks waiting for it get ERR_SEM_BAD

- **All tasks see the same ones**, by number: they're in the system's shared bank (SEM_TABLE , \$8360), 16 of them. So a task can make one and hand its number to the tasks it starts, or to a pipeline's stages.
- **Errors:** ERR_SEM_BAD (\$60 : not a semaphore, or freed while waited for), ERR_SEM_NONE (\$61 : all 16 in use), ERR_SEM_BUSY (\$62), ERR_SEM_NOT_HELD (\$63 : a mutex this task doesn't hold), ERR_SEM_FULL (\$64 : its count is 255).
- **When a task ends**, the semaphores it made are freed, and the mutexes it holds are released (SEM_RESET_TASK , from MM_TASK_RESET). A counting semaphore it took one of stays one down: nothing records who took what.
- **Inside:** kernel/sem.s (page 5). Each call works on the table with IRQs off, for a few hundred cycles at most; a wait is the kernel's usual one (above): a release wakes every waiter, the first to run takes it, and the others wait again.

What semaphores don't replace. The kernel's short updates of shared tables (the MMU's, shared memory's, the environments') hold NO_PREEMPT or keep IRQs off instead. That keeps a Ctrl-C from stopping a task halfway through an update, which a lock can't do: a break ends a task's work where it is, and a table half changed would stay that way. Semaphores are for waits that can take a while, where a break is fine: a task waiting for another, or for its turn at something they share.

Running code in another task: TASK_CALL

TASK_CALL runs a routine in another task's context: its zero page, its stack (below its saved frame), its RAM bank and its MMU area. The IO layer uses it to run servers, the IRQ dispatcher to run handlers, and DRV_START to run a driver's init.

- **In:** ZP_TC_VEC = the routine (page 0), ZP_TC_TASK = the task, and .A, .X, .Y, C as the routine's inputs.
- **Out:** the routine's .A, .X, .Y and flags.
- The routine runs with the caller's I flag. The target must not be running, unless it's the current task (then it's a plain call).
- **While the routine runs**, the caller is marked in a call (TASK_CALLING_FLAG) and isn't scheduled. The target can be preempted like any task, so a long request (an SD card block takes about 40 ms) doesn't hold up the others.
- **One call at a time:** a task serves one call at a time. A task that calls it while it's busy with another waits its turn (TC_WAIT_FREE). IRQ handlers are the exception: they run at once, as an interrupt would.
- **From another page:** TASK_GATE name, routine, task makes a gate that does all this.

Signals: break and kill

TASK_SIGNAL (\$F8AB) is like a Plan 9 note:

- **In:** .A = TASK_BREAK_FLAG or TASK_KILL_FLAG, .X = task.
- The task is flagged and stops waiting, and **the tasks it started** (and theirs, 4 levels deep) get a kill.
- Task 0 and the drivers can't be signalled.
- The signal takes effect when the task next runs: it continues at BREAK_ENTRY instead of where it was. A task in the middle of another task's call finishes that first.

Signal	Effect
Break	The task continues at its break handler (<code>TASK_SET_BREAK</code>), with the stack pointer it had when the handler was set. With no handler, the task ends
Kill	The task ends. The shell (task 1) instead starts again from scratch (a fresh HyForth)

`TASK_SET_BREAK` (`$F8A8`): `.A.Y` = handler, `.X` = its ROM page (`.A.Y` = 0: no handler). Called through a far-call gate (from another ROM page), the stack pointer it keeps is inside the gate, 3 bytes deeper than the caller's: a handler that goes on as its caller would (rather than starting afresh) sets the stack pointer it wants itself, as the editor does (`shell/edit.s`: `ED_SP`). The handler never returns: it's where the task goes on after a break. HyForth's handler goes back to its prompt with `!BREAK!`, keeping the dictionary.

The console keys send them: **Ctrl-C** a break, **Ctrl-** a kill to the foreground task (`io.md`). So do `/dev/proc/N/ctl` (`break`, `kill`) and HyForth's `kill`.

The console's foreground task

One task at a time is in the **foreground**: `/dev/cons` gives the keyboard to it, and only it and the tasks it started write to the console. The others wait until they're brought to the front, like Unix job control.

- **Changing it**: `CONS_SET_FG` (`$F8AE`, `.A` = task: 1-15, busy, not a driver) brings a task to the front. So do `Ctrl-]` then the task's number, `/dev/proc/N/ctl` (`fg`), and HyForth's `fg`.
- **When it ends**, the task that started it gets the console back (or else the shell).

Drivers: resident tasks

A driver runs in a task of its own, marked **Resident**: it's never scheduled as a main loop. Its code runs from its IRQ handlers, and from calls into it (its server's requests, `TASK_CALL`). Its state lives in its task's zero page and RAM.

```
MY_DRIVER: .word MY_INIT          ; DriverInfo::init: runs in the driver's task; C = 0 OK, or C = 1, .A =
error
          .word MY_STOP          ; DriverInfo::stop: reserved (not called yet)
          .word MY_NAME          ; DriverInfo::name: an HString, for messages
```

- `DRV_START` (`.A.Y` = the `DriverInfo`, `.X` = the task) marks the task busy and resident, and runs `init` in it. `init` typically registers the driver's IRQ handlers (`IRQ_REGISTER`, `interrupts.md`) and its devices (`DEV_REGISTER`, `servers.md`); they then run in that task.
- If `init` fails, the task is freed again, and its IRQ handlers and devices are removed.

Boot (`kernel/os_main.s`):

1. POST.
2. The IRQ tables, tasks, `cli`, the MMU, the IO layer's own devices, the VIA.
3. The serial driver (task `$F`), then the welcome message.
4. The sound (`$E`), pipe (`$D`) and storage (`$C`) drivers, with `DRV_BOOT`, which prints `NAME FAIL ee` if one fails.
5. The shell is set up in task 1, the tick starts, and task 0 yields to the shell and becomes the idle task.

To add a driver, write its `DriverInfo` and `init` (page 0, or `init` on page 0 calling into its page through a gate, like `drivers/storage.s`), pick a free task number, and start it at boot in `os_main.s` with `DRV_BOOT`.

Interrupts

How the OS dispatches interrupts, and how a driver handles one. Sources: `os_rom/kernel/irq.s`, `os_rom/kernel/common.s`. The hardware side (16 prioritised lines, the vector RAM, the $n \wedge 7$ numbering) is in the [Hardware Reference](#). Part of the [Programmer's Guide](#).

IRQ lines

IRQ	Source	Registered by
0	VIA: timer 1 (the scheduler's tick), timer 2	The system task (tick); the serial driver (timer 2: sending, in WDC ACIA builds, and at 115200)
1	ACIA (serial)	The serial driver
2, 3	Slot 0, A and B	
4	YM2151	Timer B: the sound clock, a song player's tick if one asks for it (a fast handler, <code>YM_IRQ_FAST</code> : below; the song player doesn't now). The sound driver's registered handler is a placeholder
5-9	Slots 1-5, A	
10-14	Slots 1-5, B	
15	Software interrupts	<code>SWI_REGISTER</code>

Line 0 has the highest priority. In code, name a line with `IRQ_NUMBER(n)`, or with the names in `include/hw.inc` (`IRQ_NUMBER_ONBOARD_VIA`, `IRQ_NUMBER_SLOT_1_L`, ...). `IRQ_NUMBER(n)` is $n \wedge 7$: the vector RAM entry the hardware uses for line n .

How an interrupt is handled

- The stub.** At boot, `IRQ_INIT` points the vectors at the IRQ stubs in the COMMON block (all but the VIA's and the ACIA's: [below](#)) (`$FD00`, the same on every page). Each stub loads its line's number, saves `W` and switches to ROM page 0, wherever the CPU was.
- The dispatcher** (`IRQ_DISPATCH`, page 0) looks the line up in the registration table. It runs each registered handler **in the task that registered it** (`TASK_CALL`): on that task's stack, with its zero page, RAM bank and MMU area. So a driver's handler sees the driver's own state, whichever task was interrupted.
- Claiming.** A handler returns `C = 1` if it claimed the interrupt, which stops the chain. An IRQ that none of its handlers claims is counted in the interrupted task's unclaimed-IRQ counters (`$7D90-$7D9F`, by line). It's then offered to every registered handler, so a misrouted interrupt still gets cleared.
- Returning.** The dispatcher switches back to the interrupted task, or switches task if the tick asked for one. It restores `W` and returns with `rti` from the COMMON block.

The registration tables live in the task system page (`$7D00`) and are copied into all 16 tasks. So the dispatcher reads them from whichever task was interrupted, without switching. Registering updates all 16 copies.

The fast handlers: the tick, the serial port and the sound clock

The dispatcher and its `TASK_CALL` cost about 650 cycles per interrupt, far too much for the serial port at high rates (at 115200 baud a byte arrives every 320 cycles, and the 65C51 holds only one). So the busiest interrupts bypass it (`servers/serfast.s`, `sound/ymfast.s`, BIOS page 2):

- **Their vectors:** `IRQ_INIT` points the VIA's (line 0), the ACIA's (line 1) and the YM2151's (line 4) vectors at `VIA_IRQ_STUB`, `SER_IRQ_STUB` and `YM_IRQ_STUB` in the COMMON block, which switch to page 2 (`IRQ_FAST_P2`).
- **No stack switch:** instead of running in the driver's task, a fast handler briefly switches `T` to it, a "quick look": its zero page and RAM, with no stack use until `T` is back.
- `SER_IRQ_FAST` moves the received byte into the receive ring and the next byte from the transmit ring to the ACIA, and wakes the tasks waiting to read or write.
 - **Cost:** about 60–90 cycles per byte.
 - **The rest goes through the dispatcher:** the break and kill keys, console commands and the bell are recorded in `SER_PEND` for the serial driver's own handler (`SER_DO_PENDING`).
- `VIA_IRQ_FAST` counts the tick and wakes the sleepers due, in the system task's zero page.
 - **Then** it asks the dispatcher for a task switch (`IRQ_TICK`).
 - **Timer 2** paces sending: always with a WDC ACIA (its transmitter status doesn't work), and at 115200 with the Rockwell. `SER_T2_FAST` sends the next byte from the transmit ring as `SER_IRQ_FAST` would (`SER_TX_STEP`), so either chip sends at the wire's rate.
 - **Other VIA sources** go to the registered handlers as before: CA1's edges (`/dev/gpio/ca1`) to `VIA_IRQ_HANDLER`, which counts them and wakes the tasks waiting.
- `YM_IRQ_FAST` is the sound clock: timer B, run by the sound driver for a song player at the song's rate (`SND_CTL_CLOCK`, [io.md](#)). In the sound task's zero page, it resets timer B's flag, sets the period after the next (K or K + 1 units, as a 16-bit fraction carries, so the rate is exact on average), counts the tick (`SND_CLK`), and, when the waiting player's time has come (its `ZSM_AT`, a quick look into its zero page), wakes it and asks for a task switch (`IRQ_TICK`).
 - **Cost:** about 150–300 cycles: up to two register writes to the chip, each of which may wait up to 64 cycles for it (busy after the sound driver's last write).
 - **One tick an interrupt:** it counts a tick only when timer B's flag is set, and waits (256 looks at most) for the flag's reset to be done before it returns. The chip holds its IRQ line until then, longer than its busy time on some boards; returning sooner, the interrupt came straight back and counted the same tick again, and songs played nearly twice as fast.
 - **The player first:** waking the player, it makes it the task to run next ([tasks.md](#)). (The song player doesn't use the sound clock now: on the board, timer B didn't keep its period. It sleeps by the system's tick, which makes it the task to run next the same way, and it holds the CPU (`NO_PREEMPT`) from its wake-up to its next sleep, so a busy task doesn't make a note late.)
- **The registered handlers** for lines 0 and 1 (`VIA_IRQ_HANDLER`, `SERIAL_IRQ_HANDLER`) are still there, and are what the dispatcher calls for that rare work.

Registering a handler

Call	In	Out
<code>IRQ_REGISTER</code>	<code>.X = IRQ_NUMBER(n)</code> , <code>.A.Y = handler</code> (page 0)	<code>C = 0</code> ; or <code>C = 1</code> , <code>ERR_IRQ_CHAIN_FULL</code> (2 handlers per line)
<code>IRQ_UNREGISTER</code>	<code>.X = IRQ_NUMBER(n)</code> , <code>.A.Y = handler</code>	<code>C = 0</code> ; or <code>C = 1</code> , <code>ERR_IRQ_NOT_FOUND</code>
<code>SWI_REGISTER</code>	<code>.X = software interrupt number (\$0-\$F)</code> , <code>.A.Y = handler</code>	as above (one handler per number)
<code>SWI_UNREGISTER</code>	<code>.X = number</code> , <code>.A.Y = handler</code>	

The handler runs in the **calling task**, so call these from the driver's own task: from its `init` (which `DRV_START` runs there). A task's handlers are removed when it ends.

Writing a handler

```
; IN: .A = the line (0-14), or the software interrupt number; I flag set
; OUT: C = 1 claimed, C = 0 not mine
MY_IRQ_HANDLER:
    lda    MY_DEVICE_STATUS
    bpl    @not_mine          ; (e.g. bit 7 = this device interrupted)
    ...                      ; clear the cause, move the data
    sec
    rts
@not_mine:
    clc
    rts
```

Rules:

- **Page 0.** It must be a page 0 address: the dispatcher calls it on page 0. A driver whose code is on another page puts a small page 0 handler in front.
- **Clear the cause** before returning; the IRQ input is level-triggered.
- **Don't re-enable interrupts**, and don't use far gates, `YIELD`, IO calls or anything else that can wait.
- **Keep it short:** at 19200 baud a byte arrives about every 1,860 cycles (at 115200, about 320), and the ACIA holds only one.
- **What it may touch:** `.A`, `.X`, `.Y`. It may call `IO_WAKE` to wake a task waiting on the device.
- **Bank registers:** preserve `$00` and `U` if it changes them. The dispatcher gives it its own task's `$00`, but `U` is global.
- **Asking for a task switch:** a handler returns `C = 1` with `.A = SCHED_RESCHED_A` and `.Y = SCHED_RESCHED_Y`. The tick handler does this.

Software interrupts

A software interrupt is a `BRK` with `V` pointing at IRQ 15's vector. The number (`$0-$F`) goes in `V`'s upper nibble:

```
lda    #3                      ; software interrupt 3
jsr    SW_INT                  ; saves and restores V, runs the handler, returns
```

`SW_INT` disables interrupts between setting `V` and its `brk`, so another task can't change `V` in between. The dispatcher reads the number from `V` and runs the handler registered with `SWI_REGISTER`, in its task.

NMI

The NMI vector points at the COMMON block's `NMI_ENTRY`: it switches to page 0 and calls `NMI_HANDLER`, which does nothing yet. Nothing on the board drives NMI; a slot card can.

Timing notes

- The dispatcher and the `TASK_CALL` into the handler's task cost about 650 cycles per interrupt; the fast handlers about 60–90.
- **How often a driver's interrupt can come:** through the dispatcher, up to a few hundred a second (650 cycles each: 60 a second, a VERA's frames, is about 1% of the CPU; 1,000 a second would be a fifth of it). Faster than that (a raster line, a byte at a time from a fast port) needs a fast handler: a stub in COMMON, as the VIA's, the ACIA's and the YM2151's have,

and its code on its page, with no dispatcher. COMMON has little room (the build's `rom_space.js` report shows how much), so plan for it.

- **Keep sections with interrupts off short.** Anything longer than a character's time holds off the serial port and loses input: about 320 cycles at 115200 baud, 620 at 57600, 3,700 at 9600.
 - **Instead:** use `NO_PREEMPT` when only a task switch must be prevented, as the MMU calls do.
 - **For long loops that need interrupts off:** open a moment for them between iterations (`cli`, `nop`, `sei`), as `FP_COPY`, `SH_RESET_TASK` and `SCHED_PICK` do.
 - **Checked by a test:** the `irqs-off` regression test fails if any stretch after boot exceeds 5,000 cycles, and the emulator's report lists the longest.

Memory: the MMU, shared memory, far pointers

How a task allocates memory, shares it with other tasks, and refers to memory anywhere in the machine. Sources: `os_rom/kernel/mmu.s`, `os_rom/kernel/shared.s`, `os_rom/kernel/fp.s`. Part of the [Programmer's Guide](#).

The MMU: a task's own memory

Each task has its own memory manager state, the **MMU area** at `$7E00-$7FFF` of its RAM. So a task switch swaps it in with the rest of the task's RAM. All memory is reached through **handles**: 1-byte numbers (1-101) that stay valid until freed. (The MMU area stops at `$7FFF`: the last 8 bytes of task RAM are left alone in every task, as task F's are a DS1747's clock registers when one is in U7.)

What an allocation gets, by size (`MM_ALLOC`):

Size	Gets	Where
1-3 bytes	Stored in the handle entry itself	The handle table
4-64 bytes	A 4, 8, 16, 32 or 64-byte chunk	Chunk pages (256-byte pages split into chunks of one size)
65 bytes and up	Whole 256-byte pages, contiguous	Task RAM <code>\$0800-\$7CFF</code> , from the top down
any, with <code>.X = AI_PAGED (\$01)</code>	Whole 8K RAM banks	The task's banks on the RAM modules, seen at <code>\$8000-\$9FFF</code>

Allocations don't move: a pointer from `MM_LOCK` stays valid until `MM_UNLOCK`. The calls hold `NO_PREEMPT` while they work (the task isn't switched out halfway), not interrupts off: only the task's own MMU area is involved, and IRQ handlers never allocate.

The calls (all: `C = 0` on success; `C = 1` and an error in `.A` on failure):

Call	In	Out
<code>MM_ALLOC (\$F821)</code>	<code>.A.Y</code> = size (1- <code>\$FFFF</code>), <code>.X</code> = 0 or <code>AI_PAGED</code>	<code>.A</code> = handle
<code>MM_FREE (\$F824)</code>	<code>.A</code> = handle	Fails with <code>ERR_MEM_LOCKED</code> if it's locked
<code>MM_READ (\$F827)</code>	<code>.A</code> = handle, <code>.Y</code> = offset (0-255)	<code>.A</code> = the byte
<code>MM_WRITE (\$F82A)</code>	<code>.A</code> = handle, <code>.Y</code> = offset, <code>.X</code> = the byte	
<code>MM_LOCK (\$F82D)</code>	<code>.A</code> = handle	<code>.A.Y</code> = pointer, <code>.X</code> = the previous RAM bank. For a bank allocation, its first bank is selected at <code>\$8000</code>
<code>MM_UNLOCK (\$F830)</code>	<code>.A</code> = handle, <code>.X</code> = the bank from <code>MM_LOCK</code>	Restores the RAM bank
<code>MM_FIND (\$F84E)</code>	<code>.A.Y</code> = a chunk or page allocation's address	<code>.A</code> = its handle
<code>MM_SET_FLOOR (\$F851)</code>	<code>.A</code> = page	Pages below it aren't handed out (so a program can use RAM from <code>\$0800</code> up directly)
<code>MM_TASK_RESET (\$F84B)</code>	<code>.A</code> = task	Frees everything the task has (done for you when a task ends)

`MM_READ` and `MM_WRITE` check the offset against the size of small allocations and chunks. For offsets above 255, lock the allocation and use the pointer.

Example: a 300-byte buffer.

```

lda    #<300
ldy    #>300
ldx    #0
jsr    MM_ALLOC          ; .A = handle (two pages)
bcs    @error
sta    BUF_HANDLE
jsr    MM_LOCK           ; .A.Y = its address, .X = RAM bank to restore
sta    BUF_PTR
sty    BUF_PTR + 1
stx    BUF_BANK
...
; use (BUF_PTR),Y
lda    BUF_HANDLE
ldx    BUF_BANK
jsr    MM_UNLOCK
lda    BUF_HANDLE
jsr    MM_FREE

```

The RAM disk (/ram) is in the storage task's own banks (256K of them at boot), so it takes none from any other task: each task has banks of its own.

Bank memory (AI_PAGED) is only visible while locked: MM_LOCK selects the bank at \$8000-\$9FFF. A task has 16 banks per installed RAM module (bank IDs \$00-\$EF, as far as modules are fitted). An allocation of several banks gets consecutive IDs; select the next ones by writing \$00 yourself.

The page floor. The MMU allocates task RAM pages from \$7C00 down. A program that uses RAM from \$0800 up directly sets the floor above its data (MM_SET_FLOOR), so the MMU never hands those pages out. HyForth keeps its floor 2 pages above its dictionary. TASK_CLONE skips the free pages between the floor and the lowest allocation.

Layout of the MMU area (for debugging; MmuHeader in mmu.s): a version byte, the page and bank maps with their run-end maps, the chunk list heads, the low-water mark and page floor, then the handle table (4 bytes an entry: address, bank, status).

Shared memory

Shared memory lives in the shared RAM banks, which every task can map.

- **Units:** it's allocated in whole 8K banks.
- **Handles:** a **shared handle** is 1 byte, valid in every task, so a task can send it to another (in a pipe, say).
- **References:** each task that uses the memory holds a reference. The banks are freed when the last reference is dropped, so a task that ends can't leak them.

Call	In	Out
SH_ALLOC (\$F836)	.A.Y = size	.A = shared handle; the caller holds the first reference
SH_ATTACH (\$F839)	.A = shared handle	Take a reference (e.g. to a handle received from another task)
SH_DETACH (\$F83C)	.A = shared handle	Drop this task's reference
SH_READ (\$F83F)	.A = shared handle, .Y = offset (first bank)	.A = the byte
SH_WRITE (\$F842)	.A = shared handle, .Y = offset, .X = the byte	

Call	In	Out
SH_LOCK (\$F845)	.A = shared handle	Its first bank is mapped at \$8000-\$9FFF ; .X = previous RAM bank, .Y = previous U
SH_UNLOCK (\$F848)	.X, .Y from SH_LOCK	Restores the RAM bank and U

The calling task must hold a reference for all but SH_ALLOC and SH_ATTACH.

How shared banks are numbered. The 2 MB of shared RAM is 256 banks, the **shared bank ID** = $U \ll 4 \mid (\text{bank} \ \& \ \$0F)$. Bank ID \$37, for example, is $U = 3$ with $\$00 = \$F7$. U is global (not per task), so code that sets it must put it back; SH_LOCK / SH_UNLOCK do.

Reserved shared banks:

Shared bank ID	Use
\$00	System tables: the shared bank map (\$8200), the shared handle table (\$8400), the device table (\$8800), the shared reference table (\$8900)
\$09-\$0C	The IO transfer areas: 1.5K per task, four to a bank, and a copy of the system namespace in each (servers.md)
any on a bad chip	Reserved at boot if POST found the chip bad; missing U macro-pages too
a run in \$01-\$7F	The shared RAM disk's (/sram : 512K at boot, from the lower half, so SH_ALLOC , which takes from the top down, doesn't look past them; io.md)

Far pointers

A 16-bit address alone is ambiguous on the Hydra: what's at \$8000 depends on \$00 and U, at \$A000 on \$01, at \$E000 on W, and below \$8000 on T. A **far pointer** carries that context, so it means the same thing from any task and any ROM page.

FarPtr (4 bytes, [include/kernel.inc](#)):

Byte	Field
0-1	addr : the address
2	space : the kind in bits 0-1: FP_TASK (0), FP_PROM (1), FP_SHARED (2), FP_BIOS (3). Bit 2 is FP_RO (read-only). For FP_TASK, bits 4-7 hold the task whose RAM it is
3	sel : FP_TASK : the RAM bank (for \$8000-\$9FFF); FP_PROM : the paged ROM bank; FP_SHARED : the shared bank ID; FP_BIOS : the BIOS ROM page

A task's RAM can only be read through a far pointer by that task. The others get ERR_MEM_NOT_VALID : tasks can't see each other's RAM. ROM and shared RAM work from anywhere.

The calls work on the far pointer register ZP_FP, in the calling task's zero page:

Call	In	Out
FP_MAKE (\$F8B1)	.A.Y = an address as the caller sees it, .X = the caller's ROM page	ZP_FP describes it
FP_READ (\$F8B4)	.Y = offset	.A = the byte at addr + .Y
FP_WRITE (\$F8B7)	.Y = offset, .X = the byte	Refused (ERR_MEM_NOT_SUPPORTED) for ROM or FP_RO
FP_COPY (\$F8BA)	.A.Y = destination (task RAM, or the caller's bank at \$8000), .X = count (0 = 256); C = 1 to copy a string up to its 0	.X = bytes copied

```

lda    #<MY_ROM_STRING    ; a string in this page's ROM
ldy    #>MY_ROM_STRING
ldx    W_REGISTER          ; our ROM page
jsr    FP_MAKE             ; ZP_FP = it, in a form any task can use

```

The IO layer reads names through far pointers. So `IO_OPEN` and friends take a name in RAM, in the paged ROM, or on the caller's own BIOS page.

References: far pointers as handles

A **reference** wraps a far pointer in a handle, so code that takes handles can use static data (a ROM table, a string) the same way as allocated memory:

Call	In	Out
<code>MM_REF</code> (<code>\$F8BD</code>)	<code>ZP_FP</code>	<code>.A</code> = an MMU handle for it (this task). <code>MM_READ</code> , <code>MM_WRITE</code> (not ROM), <code>MM_LOCK</code> / <code>MM_UNLOCK</code> (task RAM, and the paged ROM: its bank is selected), and <code>MM_FREE</code> (drops the handle)
<code>SH_REF</code> (<code>\$F8C3</code>)	<code>ZP_FP</code> (ROM or shared RAM)	<code>.A</code> = a shared handle, usable by any task: <code>SH_ATTACH</code> , <code>SH_READ</code> , <code>SH_WRITE</code> , <code>SH_LOCK</code> (shared RAM), <code>SH_DETACH</code>
<code>MM_FP</code> (<code>\$F8C0</code>)	<code>.A</code> = MMU handle	<code>ZP_FP</code> = where it is (allocation or reference)
<code>SH_FP</code> (<code>\$F8C6</code>)	<code>.A</code> = shared handle	<code>ZP_FP</code> = where it is

A reference to shared RAM or a BIOS page can't be locked (`ERR_MEM_NOT_SUPPORTED`): the BIOS page can't be mapped while ROM code runs. Use `MM_READ`, or `MM_FP` then `FP_COPY`.

Copying

`MEM_COPY` (`$F81E`) copies `.A.Y` bytes from `ZP_TEMP_VEC` to `ZP_TEMP_VEC2`, within what the caller can see. Between tasks, use shared memory, a pipe, or the IO layer.

Input and output

All IO on the Hydra goes through **file descriptors**, Plan 9 style:

- A task opens a name (`/dev/cons` , `/dev/sd/0/data`), gets an fd, and reads and writes it.
- Devices are **file servers**: a driver registers its names, and each request runs in the driver's task.
- A read with no data makes the task wait, using no CPU, until the driver wakes it.

This chapter is the application side. Writing a server is in [servers.md](#), and the design is in [plans/IO_PLAN.md](#). Sources: `os_rom/io/`. Part of the [Programmer's Guide](#).

Files and fds

Each task has **12 fds** (0-11), in its task system page. An fd holds the device, the server's handle for the open file (the fid), the open mode, and a 32-bit offset.

Call	In	Out
IO_OPEN (\$F86C)	.A.Y = name (zero-terminated, up to 255 characters), .X = mode	.A = fd (the lowest free one)
IO_CLOSE (\$F86F)	.A = fd	(closed even if the server reports an error)
IO_READ (\$F872)	.A = fd, ZP_IO_BUF = buffer, ZP_IO_CNT = bytes wanted	ZP_IO_CNT = bytes read (0 = end of file)
IO_WRITE (\$F875)	.A = fd, ZP_IO_BUF = buffer, ZP_IO_CNT = bytes	ZP_IO_CNT = bytes written
IO_GETC (\$F878)	.X = fd	.A = byte; C = 1, ERR_IO_EOF at the end
IO_PUTC (\$F87B)	.X = fd, .A = byte	
IO_SEEK (\$F87E)	.A = fd, ZP_IO_OFS = 32-bit offset	Sets the offset for the next read or write
IO_STAT (\$F881)	.A = fd, ZP_IO_BUF = 48-byte buffer	The server's stat record (below)
IO_CTL (\$F884)	.A = fd, .X = control code, .Y = argument	.A from the server
IO_DUP (\$F896)	.A = fd	.A = a new fd for the same file
IO_DUP2 (\$F890)	.A = fd, .X = new fd	fd .X now refers to fd .A's file (closed first if open)
IO_PIPE (\$F893)		.A = read fd, .X = write fd
IO_CREATE (\$F8C9)	.A.Y = name, .X = mode, ZP_IO_BUF (low byte) = the new file's mode bits	.A = fd: a new file or directory, opened (below)
IO_REMOVE (\$F8CC)	.A.Y = name	Removes a file, or an empty directory
IO_WSTAT (\$F8CF)	.A = fd, ZP_IO_BUF = a 48-byte stat record	Renames the file, sets its mode bits
IO_CHDIR (\$F8D2)	.A.Y = a directory's path	It's the current directory (below)
IO_GETCWD (\$F8D5)	ZP_IO_BUF = a 64-byte buffer	The current directory, a zero-terminated absolute path

Names given to `IO_OPEN`, `IO_CREATE`, `IO_REMOVE` and `IO_CHDIR` are relative to the task's current directory unless they start with `/`.

Modes (`IO_OPEN`'s `.X`): `IO_MODE_READ` (`$01`), `IO_MODE_WRITE` (`$02`), `IO_MODE_RDWR` (`$03`), plus `IO_MODE_STAT` (`$04`), `IO_MODE_TRUNC` (`$08`) and `IO_MODE_NONBLOCK` (`$80`). With `IO_MODE_NONBLOCK` , a read with no data yet returns `ERR_IO_WOULD_BLOCK` instead of waiting. `IO_MODE_STAT` asks a directory for stat records instead of text, and `IO_MODE_TRUNC` (with `IO_MODE_WRITE`) empties a file as it's opened ([below](#)).

Reads and writes:

- **Units:** they move up to 256 bytes per request; the IO layer splits bigger ones.
- **A read** returns when the count is done, at the end of the file, or when the server has less (the keyboard: the keys typed so far).
- **A write** returns when the count is done. A server that takes part of it is offered the rest again.
- **The offset** advances by what was moved.
- **Buffers:** `ZP_IO_BUF` must point into task RAM (`$0000-$7FFF`), since the data is copied through shared RAM.

Names are resolved through the task's namespace ([below](#)). A name nothing matches must be `/dev/<device>` or `/dev/<device>/<rest>`: the device's server gets `<rest>` to open. Names are read as the caller sees them (through a far pointer), so a name can be in RAM, in the paged ROM, or on the caller's own BIOS page.

Inheritance. A new task gets copies of its parent's open fds, and all of a task's fds are closed when it ends.

```

lda    #<NAME                ; NAME: .byte "/dev/zero", 0
ldy    #>NAME
ldx    #IO_MODE_READ
jsr    IO_OPEN
bcs    @error
sta    FD
lda    #<BUF                  ; 16 bytes of zeros into BUF
sta    ZP_IO_BUF
lda    #>BUF
sta    ZP_IO_BUF + 1
lda    #16
sta    ZP_IO_CNT
stz    ZP_IO_CNT + 1
lda    FD
jsr    IO_READ
lda    FD
jsr    IO_CLOSE

```

stdio

fds **0, 1 and 2** are stdin, stdout and stderr. The shell opens all three on `/dev/cons` , and the tasks it starts inherit them.

Call	Does
<code>WRITE_CHAR</code> (<code>\$F803</code>)	Write <code>.A</code> to fd 1 (or straight to the serial port, for a task without fd 1: the system task and drivers)
<code>GET_CHAR</code> (<code>\$F88D</code>)	Wait for a byte from fd 0: <code>C = 1</code> , <code>.A</code> = the byte; <code>C = 0</code> with an error (e.g. <code>ERR_IO_EOF</code>)
<code>READ_CHAR</code> (<code>\$F800</code>)	A byte from fd 0 if there is one, without waiting: <code>C = 1</code> , <code>.A</code> = the byte; <code>C = 0</code> : none
<code>IO_FLUSH</code>	Write out the stdout buffer

Output buffering. `WRITE_CHAR` collects output in a 128-byte buffer (`$0700`), for every kind of fd 1. The buffer is written out:

- when it's full;
- for the console (`/dev/cons` , `/dev/ser`), at each LF too: it's line-buffered, like a terminal in C;

- before `stdin` is read (`GET_CHAR`, `READ_CHAR`), so a prompt always shows;
- before the task sleeps (`TASK_SLEEP`) or starts a task;
- before any `IO_WRITE` by the task, so output through another `fd` comes out in order;
- when `fd 1` is closed or replaced, so when the task ends.

A program that prints a partial line and then computes for a long time without any of those can call `IO_FLUSH` itself.

The console's fast paths. From the foreground task, console output and input skip the IO request entirely:

- **Output:** a flush of `/dev/cons` copies the buffer straight into the serial driver's transmit ring (`SER_CONS_PUTS`).
- **Input:** `GET_CHAR` and `READ_CHAR` take a key straight from the receive ring and echo it (`SER_CONS_GETC`).
- **Speed:** each costs well under 100 cycles a byte, where an IO request costs about 2,000. At 115200 baud a byte takes 320 cycles.
- **Otherwise the IO path:** anything they can't do (a task that isn't in the foreground, a full ring, the end-of-input and erase keys) goes through the normal IO request. So the rules (waiting for the foreground, waiting for room) are the same as before.

`/dev/cons` `fds` are marked when they're opened (`IO_FD_FLAGS`, `IO_FDF_CONS`), and the mark travels with the `fd` when it's duplicated or inherited.

Input read-ahead. When `fd 0` isn't the console, `GET_CHAR` and `READ_CHAR` read ahead 128 bytes (`$0780`). So a pipe carries a block per request instead of a byte. The read-ahead is dropped when `fd 0` is closed or replaced. Reading `fd 0` directly after `GET_CHAR` has read ahead misses what was read ahead.

Devices

Name	Server (task)	What
<code>/dev/cons</code>	Serial driver (<code>\$F</code>)	The console (below)
<code>/dev/cons/ctl</code>	Serial driver	The console's mode, as Plan 9's <code>consctl</code> : read <code>rawon</code> or <code>rawoff</code> ; write <code>rawon</code> or <code>rawoff</code> (below)
<code>/dev/ser</code>	Serial driver	The serial port as it is, for any task: no foreground rules, no echo
<code>/dev/ser/ctl</code>	Serial driver	The port's settings: read <code>b9600 18 pn s1</code> ; write commands to change them (below)
<code>/dev/snd</code>	Sound driver (<code>\$E</code>)	The YM2151 (below)
<code>/dev/sd/N/data</code>	Storage (<code>\$C</code>)	Disk N as bytes at the <code>fd</code> 's offset (<code>IO_SEEK</code>); the first 4 GB. N is one character: for an SD card, its SPI device's number as a hex digit, <code>0-7</code> (<code>8-f</code> , slot cards' SPI devices, are planned), started at the first open (<code>ERR_IO_DEVICE</code> if there's none); other disks are letters that aren't hex digits: <code>x</code> , the ROM disk, the paged ROM (<code>read-only</code>); <code>r</code> and <code>s</code> , the RAM disks
<code>/dev/sd/N/ctl</code>	Storage	Read: the disk as a line, e.g. <code>sdhc 7580 MB 15523840 blocks</code> (or <code>sdsc</code> , <code>rom</code> , or <code>none</code>), and for a HydraFS its label, free space and last check. Write: <code>init</code> starts the card again (e.g. after changing it); <code>format</code> , <code>label</code> , <code>check</code> (below)
<code>/dev/spi/N</code> , <code>/dev/spi/N/ctl</code>	Storage (page D)	SPI device N (<code>0-f</code>) as a file: a transaction a write, the bytes it sends back read after (below)
<code>/dev/gpio/N</code> , <code>port</code> , <code>ctl</code> , <code>ca1</code>	IO layer (in the reading task: page D)	The VIA's port A on J27: its pins, CA1's edges, CA2 (below)
<code>/sd/N/...</code>	Storage	The files on disk N: the HydraFS server (the device <code>hfs</code> , mounted at <code>/sd</code> ; below)
<code>/dev/pipe</code>	Pipe server (<code>\$D</code>)	Made by <code>IO_PIPE</code> , not opened by name

Name	Server (task)	What
<code>/dev/proc</code>	IO layer (in the reading task)	The tasks (below)
<code>/dev/ram</code>	The shell registers it (in the reading task)	The RAM itself, as the CPU selects it: task 0's only (any other task's open: <code>ERR_IO_PERM</code>); read-only (below)
<code>/dev/time</code>	The shell registers it (in the reading task)	The clock: read <code>2026-09-29 18:05:00</code> ; write a date and time to set it (below)
<code>/dev/null</code>	IO layer	Reads give end of file; writes are taken and dropped
<code>/dev/zero</code>	IO layer	Reads give zeros

Blocks and caching: the SD card is read and written a 512-byte block at a time through a one-block cache, and writes go straight through to the card. `IO_CTL` code `SD_CTL_INIT` (1) on either file starts the card again.

The console

`/dev/cons` is the terminal on the serial port:

- **Reads** get the keyboard input, but only for the **foreground task**; other readers wait until they're brought to the front.
- **Echo:** it echoes what it reads, like a terminal, and turns the DEL key (what many terminals send for Backspace) into a backspace. So a program reading a pipe sees no echo.
- **Writes:** only the foreground task and the tasks it started write; the others wait.
- **Buffering:** the serial driver buffers both ways (256-byte rings) and sends from its transmit interrupt.
- **Switching the foreground:** `IO_CTL` code `SER_CTL_FOREGROUND` (1), `.Y` = task, or `CONS_SET_FG` ([tasks.md](#)).
- **Settings:** the port starts at 9600 baud, 8 data bits, no parity, 1 stop bit (below).
- **Raw mode** (`/dev/cons/ctl`, as Plan 9's `consctl`): write `rawon` to it, and the console's reads get each key as it's typed, with no echo, DEL kept as it is, and Ctrl-D and Ctrl-Z as characters, not the end of input. It stays raw while the `ctl` file is open (any fd of it, in any task: its copies count), and goes back to cooked when the last one is closed, or on `rawoff`. So a program that ends, or is killed, can't leave the console raw. Ctrl-C, Ctrl-\ and Ctrl-] still work. C's `conio` uses it ([programs.md](#)).

Console keys (acted on by the serial driver as they arrive, so they work on a task stuck in a loop):

Key	Does
Ctrl-C	Break: the foreground task goes to its break handler (HyForth: back to its prompt with <code>!BREAK!</code>); the tasks it started are killed
Ctrl-\	Kill: the foreground task and the tasks it started end; the shell starts again from scratch
Ctrl-D, Ctrl-Z	End of input: a read of <code>/dev/cons</code> returns end of file
Ctrl-] then <code>0-F</code>	Bring that task to the front (prints <code>[N]</code> ; a bell if it can't be)
Ctrl-] then <code>1</code>	List the tasks that can be brought to the front (<code>[1* B]</code> : <code>*</code> = the foreground one)
Ctrl-] Ctrl-]	Type a Ctrl-]

The bell: whenever the console sends a BEL (Ctrl-G), the YM2151 beeps too (880 Hz on channel 7, skipped while the sound driver is busy).

The serial port settings

The serial port starts at 9600 baud, 8 data bits, no parity, 1 stop bit. Its settings are a text file, `/dev/ser/ctl`:

- **Reading it** gives them, e.g. `b9600 18 pn s1`.
- **Writing it** changes them, with commands separated by spaces:

Command	Sets
bN	The baud rate: 300, 600, 1200, 1800, 2400, 3600, 4800, 7200, 9600, 19200, 115200
lN	The data bits: 5-8
pX	The parity: n none, o odd, e even, m mark, s space
sN	The stop bits: 1 or 2

`b19200, 17 pe`, and `b4800 18 pn s1` are all valid. A write with a bad command changes nothing and fails with `ERR_IO_BAD_REQ`.

For programs, `IO_CTL` on any of the serial files (`/dev/cons`, `/dev/ser`, `/dev/ser/ctl`) does the same:

Code	Name	.Y
2	<code>SER_CTL_RATE</code>	The rate: <code>SER_RATE_300</code> (0) ... <code>SER_RATE_9600</code> (8), <code>SER_RATE_19200</code> (9), <code>SER_RATE_115200</code> (10)
3	<code>SER_CTL_FORMAT</code>	The format: data bits - 5 (bits 0-1), <code>SER_PAR_*</code> << 2 (bits 2-4), <code>SER_FMT_STOP2</code> (<code>\$20</code>); <code>SER_FMT_8N1</code> = 3

How a change takes effect:

- **Output first:** the change waits until what's already queued has been sent, so a message printed before it comes out whole. It then applies to both directions. Switch the terminal to match afterwards.
- **Actual speed:** the rates are 2.9% slow on the board (the ACIA's 1.79 MHz clock), which terminals accept.
- **Not possible:**
 - 2 stop bits with 8 data bits and parity: the 65C51 sends 1.
 - With a WDC ACIA build, rates below about 1200: a character's time must fit VIA timer 2.
- At 115200 (Rockwell ACIA builds), sending is paced by VIA timer 2, with idle bits between characters (`SER_PACE_GAP` in `hw.inc`; [hardware](#)). At the other rates the ACIA's TDRE interrupt sends each byte as soon as it can.
- **After a reset**, the port is back at 9600 8N1.

In HyForth: `"b19200" stty`, and `stty?` to show the settings.

Sound: `/dev/snd`

The YM2151 (8 FM channels of 4 operators, at 3.58 MHz), through the sound library in the sound task (`os_rom/sound/`, BIOS ROM page B).

Writes are register/value byte pairs. The driver waits for the chip between writes, and keeps each register as written (the chip's can't be read back). The register numbers the chip doesn't have are the **library's commands**, each for the channel `SND_R_CH` last chose (one choice for every fd: send it in the same write as the command):

Register	Name	Value
<code>\$02</code>	<code>SND_R_CH</code>	The channel (0-7) for the commands after it
<code>\$03</code>	<code>SND_R_PATCH</code>	Load patch n into the channel: 0-127 are General MIDI's instruments, 128-162 drum and percussion sounds (the Commander X16's set). The channel keeps its speakers
<code>\$04</code>	<code>SND_R_NOTE</code>	Key on MIDI note n (60: middle C; 69: A, 440 Hz), with the channel's bend
<code>\$05</code>	<code>SND_R_OFF</code>	Key off (the note's release)
<code>\$06</code>	<code>SND_R_VOL</code>	The channel's volume, 0-127 (General MIDI's curve)
<code>\$07</code>	<code>SND_R_PAN</code>	Its speakers: 1 left, 2 right, 3 both (<code>SND_PAN_*</code>)
<code>\$09</code>	<code>SND_R_BEND</code>	Its pitch bend: signed, in 64ths of a semitone (-2 to +2 semitones); the note playing moves too
<code>\$0A</code>	<code>SND_R_DRUM</code>	A General MIDI drum (MIDI channel 10's note numbers: 36 kick, 38 snare, 42 closed hi-hat ...): its patch and pitch, keyed on

Everything else goes to the chip as written (except the timers' interrupt enables, \$14's bits 2 and 3, which stay off: a register dump from another machine may have them on, and nothing here would clear the interrupt; and timer B, \$12 and its bits of \$14, while the sound clock has it), with one exception: a **carrier's level** (TL, \$60-\$7F, for the operators the channel's algorithm sounds) gets the channel's and the master volume's attenuation added. So the volumes work on raw register writes too (a song's), and a new algorithm (\$20-\$27) writes the levels again.

Reads give the registers as written (the levels before the volumes): a 256-byte file, a byte a register.

Claims: a fd can claim channels (SND_CTL_CLAIM): the other fds' writes and commands for them are dropped, and the console's bell leaves channel 7 alone while it's claimed. The claim lasts until SND_CTL_RELEASE or the fd's last close (a task's end closes its fds), which keys the channels off. So a game can have its music on some channels and its sound effects on others, each program's safe from the rest.

IO_CTL codes:

Code	Name	Does
1	SND_CTL_INIT	Stop the tune and clear the chip and the library's settings (the claims stay)
2	SND_CTL_TEST	Play the test song (/rom/songs/test.zsm: /rom is in the system namespace, which every task sees, the sound task's too) in the background, in the song player (a task of its own, which claims the channels): the caller goes on at once. ERR_TASK_BUSY if it's playing already
3	SND_CTL_STOP	Stop the tune
4	SND_CTL_CLAIM	.Y = a mask of channels (bit n: channel n), this fd's alone; ERR_IO_BUSY if another fd has one of them (none taken)
5	SND_CTL_RELEASE	.Y = a mask of channels to give back
6	SND_CTL_VOLUME	.Y = the master volume (0-127)
7	SND_CTL_CLOCK	The sound clock, a song player's tick: the YM2151's timer B, its interrupt counting ticks (SND_CLK, from 0) and waking the waiting player at its time (sound/ymfast.s). .Y = K: a period of K units of 1,024 of the chip's clocks (286 us) or K + 1, as often as the fraction written to SND_R_CLOCK_F (\$0C, and \$0D, the high byte: 65536ths) says, so the rate is exact on average; 0 stops it. This fd's while it runs (ERR_IO_BUSY: another's); its last close stops it. Timer A stays the clients' (CSM) The song player doesn't use it (on the board, timer B didn't keep its period: hardware); a read of /dev/snd shows its numbers in the shadow's spare bytes (snd_srv.s: SND_NUMBERS)

HyForth's patch, note, noteoff and ywrite (HyForth) and C's snd.h (the C guide) use it.

SPI devices: /dev/spi

The SPI bus's 16 devices (0-7 the board's headers J18-J25, 8-f the slots' cards: hardware), each a file, as Plan 9 has them. Served in the storage task, which owns the bus, so a transfer never meets an SD card's (servers/spi_srv.s, BIOS ROM page D).

Name	Read	Write
/dev/spi/N	The bytes the device sent back during the last write; or, with none kept, as many as asked, clocked in as a transaction of their own (sending \$FF)	Its bytes, sent with the device selected: one transaction a write request (up to 256 bytes), and the bytes that come back meanwhile are kept
/dev/spi/N/ctl	mode 0 or mode 3	mode 0 (SCLK idles low) or mode 3 (it idles high); 1 and 2 are ERR_IO_BAD_REQ. Kept until changed

- **A command and its answer** are one write and one read: write the command and as many dummy bytes as the answer has, then read them all back (the first bytes are what came back while the command went out). A read with nothing kept is for a device that just talks: read 4 bytes of an ADC's sample.
- **Bits:** MSB first, sent and sampled on SCLK's rising edge (modes 0 and 3), at about 100 kHz (bit-banged on the VIA's port B: drivers/spi.s). The device is deselected after each request.

- **One open at a time:** another open of the same device is `ERR_IO_BUSY` (the fd's dups share it). So is a device with an SD card started on it (`/dev/sd/N`), and while `/dev/spi/N` is open, the SD driver doesn't start a card there (`ERR_IO_BUSY`, and its `ctl` reads `none`).
- **From HyForth:** `"/dev/spi/1" 3 open`, then `write` and `read` on the fd ([files and devices](#)); from C, `open`, `write`, `read`.

GPIO: `/dev/gpio`

The VIA's port A on header J27 ([hardware](#)): 8 pins, PA0-PA7, and the handshake lines CA1 (an input, which can interrupt) and CA2. Served in its client's task (`servers/gpio_srv.s`, BIOS ROM page D), with interrupts off around each change to the VIA's registers.

Name	Read	Write
<code>/dev/gpio/N</code>	Pin N (0-7): 0 or 1 and CR LF, from the offset (<code>cat</code> reads it once; a program polling it seeks back to 0)	0 or 1: the pin's level, and the pin an output
<code>/dev/gpio/port</code>	All 8 pins, a byte (any offset)	A byte: the outputs' levels (the inputs keep theirs)
<code>/dev/gpio/ctl</code>	A line a pin (2 out 1, 3 in 0), then <code>ca1 fall 0003</code> (CA1's active edge, and the edges counted), then <code>ca2 in</code> (or <code>ca2 0</code> , <code>ca2 1</code> : an output)	A command: <code>in N</code> , <code>out N</code> , <code>ddr HH</code> (all 8 directions, hex: 1 = out), <code>ca1 rise</code> , <code>ca1 fall</code> , <code>ca2 0</code> , <code>ca2 1</code> ; anything else is <code>ERR_IO_BAD_REQ</code>
<code>/dev/gpio/ca1</code>	Waits for CA1's next active edge (one since this task last read it, or opened it), then gives the edges counted so far: <code>0003</code> and CR LF. A non-blocking fd gets <code>ERR_IO_WOULD_BLOCK</code> instead of waiting	

- **The pins start as inputs** (the VIA's reset). PA0 and PA1 are the I2C bus's SCL and SDA too, pulled up on the board; they're plain pins until an I2C driver uses them.
- **CA1's interrupt** is on only while `/dev/gpio/ca1` is open, so a floating CA1 can't flood the system. The VIA's handler (`VIA_IRQ_HANDLER`, in task 0) counts each edge and wakes the readers waiting. A switch's bounce counts too: a button wants a capacitor, or a program that ignores edges close together.
- **Port A is read and written without the handshake** (`ORA` at register `$F`), which leaves CA1's flag alone.
- **From HyForth:** `echo 1 > /dev/gpio/2`, `cat /dev/gpio/3`, `echo out 4 > /dev/gpio/ctl`; a program opens the files and reads and writes them. The [tutorial](#) wires an LED and a button.

The tasks: `/proc`

The device `proc`, which the boot shell mounts at `/proc`, as Plan 9 has it (`/dev/proc` is the same files):

Name	Read	Write
<code>/proc</code>	A line per busy task	
<code>/proc/N</code> , <code>/proc/N/status</code>	Task N's line	
<code>/proc/N/ctl</code>		<code>kill</code> , <code>break</code> or <code>fg</code> : for N's family only (below)
<code>/proc/N/cwd</code>	Its current directory	
<code>/proc/N/env</code>	Its environment, as <code>/env</code> 's list	
<code>/proc/N/pages</code>	pages <code>PP floor FF</code> (hex): the MMU pages it has (not counting the ones every task has marked: <code>\$00-\$07</code> , <code>\$7D-\$7F</code>), and its page floor; - for a free task	
<code>/proc/N/ns</code>	Its namespace, as <code>ns</code> prints it (the lines that would make it)	
<code>/proc/N/cmd</code>		A line for task N's shell to run, as if typed at its prompt: for N's family only (below)

Name	Read	Write
/proc/N/mem	Its address space as it sees it: offset = address, 64K (below)	Its RAM, \$0000-\$9FFF : for N's family only
/proc/N/ram	Its banks on the RAM modules: offset = bank * 8K + offset in it (below)	The same: for N's family only

A line is `N S O`: the task, its state (R running or runnable, W waiting, P paused, D a driver) and the task that started it (- none), then * for the foreground task. `/proc` and `/env` are served in their client's task, from ROM page 9 (`servers/proc_srv.s`, `servers/env_srv.s`); pages is counted in task N itself (`TASK_CALL`).

Who may write `ctl` and `cmd`, and use `mem` and `ram`: task N itself, a task that started N (directly or not: a shell and what it runs), and task 0; any other task gets `ERR_IO_PERM` (`TASK_MAY`, page 5), and a free task `ERR_IO_NOT_FOUND`. So a shell can't kill, drive or read another shell's programs, and only task 0 reaches a driver's memory.

`mem` and `ram`: task N's memory as a debugger, a memory dump or a core file wants it, so `cp /proc/3/mem core3` saves a task's whole address space.

- **mem's offsets are N's addresses:** `$0000-$7FFF` its task RAM (zero page, stack, program); `$8000-$9FFF` the bank it has there (its `$00`; a shared bank with its `U`); `$A000-$DFFF` its paged ROM bank (its `$01`); `$E000-$FEFF` the BIOS ROM page it's on (its `W`); `$FFF0-$FFFF` zeros (the I/O space isn't read: a read can have effects). Past `$FFFF` is the end. It writes `$0000-$9FFF`; a write anywhere else is `ERR_IO_MODE`.
- **ram's offset** `b * $2000 + o` is byte `o` of N's bank `b` (`$00-$EF`: its banks on the RAM modules, as it numbers them). Only the banks it has are there (the MMU's bank map: `MM_ALLOC` with `AI_PAGED`); another gives `ERR_IO_NOT_FOUND`. Past bank `$EF` is the end. `IO_STAT`'s size is up to its last bank (`mem`'s: 64K).
- **A copy is a snapshot:** N may run between two reads (a debugger stops it first). Each read or write is up to 64 bytes, and stops at a 256-byte page's end; ask again for the rest, as `cp` does. It's copied with `T = N` and IRQs off (`MEM_COPY`, `servers/ram_srv.s`, which `/dev/ram` uses too), so it sees N's own zero page, stack and banks, and the zero page bytes and `$00` the copy borrows read (and are written) as N has them.
- **U and W** are in N's frame on its stack while the scheduler has it switched out. A task in a `TASK_CALL` (waiting for a server), a driver, or the asker itself has none there: its `$8000` shared bank is read with `U 0`, and its BIOS page is page 0.

`cmd`: a write is one line (up to its CR, LF or 0; 39 characters at most). It waits in task N's IO transfer area until N's shell is at its prompt; if it's there now, waiting for the console (in the background, say), a break wakes it, a quiet one with no `!BREAK!`, and it runs the line, shown after its prompt. One line waits at a time (`ERR_IO_BUSY`). HyForth's `send N line` writes it. A task that isn't a HyForth shell never takes the line.

The clock: `/dev/time`

The Hydra keeps the date and time as seconds since 2000-01-01 00:00:00, counted by the scheduler's tick. It has no clock that runs while it's off, so the time starts at 2000-01-01 00:00:00 at power-up, until it's set:

- **Reading** `/dev/time` gives the date and time and CR LF: `cat /dev/time` shows `2026-09-29 18:05:00`.
- **Writing** `YYYY-MM-DD hh:mm:ss` sets it; the seconds can be left out, or the whole time (midnight): `echo 2026-09-29 18:05 > /dev/time`. 2000-01-01 to 2135-12-31; a date that isn't one (2023-02-29) is `ERR_IO_BAD_REQ`.
- **From code:** `CLOCK_GET` and `CLOCK_SET` (page 9, `servers/time_srv.s`: `.X` = a zero page address, the 4 bytes of seconds there). HydraFS stamps files with it ([plans/HYDRAFS.md](#)).
- **The clock chip:** a DS1747 in U7 keeps the time while the Hydra's off (`drivers/rtc.s`, page 9). Its clock registers are task F's `$7FF8-$7FFF` (`RTC_REGS`, `hw.inc`), which nothing else in the ROM writes, in any task. At boot the shell looks for it (`RTC_BOOT`): its registers must hold a date and time, and its seconds must change within 1.1 s; then `ZP_CLOCK` is set from it at the start of one of its seconds, and `RTC_STATE` (in the system's shared bank) says it's there. Writing `/dev/time` sets it too (`RTC_SAVE`: the W bit), and looks for it again if it wasn't found; reading `/dev/time` takes its seconds first (`RTC_LOAD`, the R bit, then `CLOCK_ADJUST`, which keeps the tick clock's place in the second). Each access is a few bytes with IRQs off and T switched to F and back for each.

The environment: /env

Each task has an environment: variables, `NAME=value`, which the tasks it starts get a copy of (so a change in a task stays in it and the tasks it starts later). They're files, as in Plan 9, under `/env`, which every task has with no mount (the IO layer sends names under it to the `env` device, as it does `/dev`):

Name	Read	Write	Create, remove
<code>/env</code>	The variables, a line each: <code>NAME=value</code>		
<code>/env/NAME</code>	Its value (not found: <code>ERR_IO_NOT_FOUND</code>)	Its value: a write at the file's start replaces it, one after that adds to it; CRs and LFs are left out	<code>IO_CREATE</code> makes it (or empties it); <code>IO_REMOVE</code> removes it

So `echo /sd/0/bin > /env/PATH` sets one, `cat /env/PATH` shows it, `rm /env/PATH` removes it, and `ls /env` lists them. A name is 1-30 characters, not `/` or `=`; a task's variables share 256 bytes (`ERR_IO_FULL` beyond). The shell uses `PATH` (directories, `:` between them, to find programs by name) and `HOME` (`cd` alone), and sets `status` (the last program's exit status: its message, or its code, or empty for success) and `apid` (the task of the last program started with `&`), as Plan 9's `rc` does. C's `getenv` and `setenv` read and write these files; `conio` reads `COLUMNS` and `LINES`.

How: each task's environment is a 256-byte block in the system's shared bank (`ENV_BLOCKS`, `$8D00`), entries one after another; `IO_INHERIT` copies the parent's block for a new task (`ENV_COPY`). An open variable is one of 16 slots (its task and name), so 16 can be open at once.

The RAM itself: /dev/ram

Every byte of RAM, as the CPU selects it, for task 0 (the system's task) and no other: a program can't read another's memory through it (a task's own family will have `/proc/N/mem`: [plans/PROC.md](#)). Read-only; `servers/ram_srv.s`, page 9.

Offsets	What
<code>\$0000000-\$007FFFF</code>	Task RAM: task <code>t</code> 's <code>\$0000-\$7FFF</code> at <code>t * \$8000</code>
<code>\$0080000-\$027FFFF</code>	Shared RAM: shared bank ID <code>s</code> at <code>\$80000 + s * \$2000</code>
<code>\$0280000-\$207FFFF</code>	The RAM modules: module <code>m</code> 's bank <code>b</code> for task <code>t</code> (its bank ID <code>m << 4 b</code>) at <code>\$280000 + m * \$200000 + t * \$20000 + b * \$2000</code>

A read gives up to 64 bytes, and stops at a 256-byte page's end (a short read: read again); past the last module, nothing (end of file). How: it switches `T`, and the bank (and `U`) the bytes are in, with IRQs off, and copies them straight into task 0's transfer area, which also holds the copy's numbers (the request block's spare bytes: the one RAM every task sees with `$00` at the IO transfer bank). The task's `$00` and the zero page bytes the copy uses are put back, and a copy of them reads as they were. The IO self test (`WOZMON F88AR`) has task 0 read a task's RAM, a module bank and a shared bank through it (`TASK_CALL`), and checks that the task itself is refused.

The ROM's files: /rom

Files kept in the paged ROM, read-only, there with no card: programs, songs and (later) libraries. The paged ROM is a disk, the **ROM disk** (the block device `/dev/sd/x`), with a HydraFS volume on it, which the boot shell mounts at `/rom` with a spec, as Plan 9's `mount` takes one: `mount -s hfs /rom x` (the HydraFS server's disk `x`), in the system namespace, so every task sees it. It's mounted if the ROM disk has a volume (a paged ROM image without one has no table: no `/rom`, and everything else works). The disks in memory aren't under `/sd`, which has the cards alone: HydraFS serves them only through a mount with a spec (below).

Name	Read
<code>/rom</code> , <code>/rom/bin</code> , ...	A directory: a line per entry, <code>name size</code> (a directory: <code>name/</code>), or stat records (<code>IO_CTL_READ_STATS: ls -l</code> , C's <code>readdir</code>)

Name	Read
/rom/README, /rom/bin/hello.hyx, ...	The file's bytes; <code>IO_SEEK</code> and <code>IO_STAT</code> work as on a card (the stamps are 2000-01-01)

Writing, creating and removing give `ERR_IO_MODE` (the shell's `!IO_ERR!`), as do `format` and `label` on `/dev/sd/x/ctl` and writes to `/dev/sd/x/data`: they're refused before anything is changed, even in the block cache. `cd /rom/bin` works, and the shell looks in `/rom/bin` last for a program by its name (and in `/rom/lib` for a library: `HyForth`). So `hello`, `jukebox` or `scom` run on a machine with no card.

The ROM disk. Block `n` of the ROM disk is paged ROM bank `n / 32`, at `$A000 + (n % 32) * 512`, as the CPU sees it: 8192 blocks, the whole 4 MB. A block never crosses a bank, or the 8K halves the board swaps, so the storage driver (`SD_ROM_READ` in `drivers/sd.s`) selects one bank, copies 512 bytes from one place, and puts the storage task's bank back. A file whose blocks are in several banks is no different from one on a card: HydraFS asks for it a block at a time, so no pointer ever runs from one bank into the next.

Blocks	Banks	What
0	0 (<code>\$A000-\$A1FF</code>)	The partition table (an MBR, as a card's), and a line saying what the disk is
1-63	0-1	Partition 1, type <code>\$DA</code> (not a file system): the system's banks: HyForth's variables, the hardware test
64-8191	2-255	Partition 2, type <code>\$7F</code> : the HydraFS volume, label <code>ROM</code> , read-only

What's in it comes from `os_rom/romfs.txt`, a list of files (their names in `/rom`, and where the build finds them). After the link, the build's `sim/tools/mkromdisk.js` makes the volume with the HydraFS PC tool (`hydrafs.js`, stamped 2000-01-01 so each build is the same), writes the table and the volume's blocks into the paged ROM image, then reads the image back the way the CPU would (through the emulator's bank mapping) and checks every file against its source: a mismatch fails the build. `node sim/tools/mkromdisk.js os_rom/romfs.txt os_rom/bin/paged_rom_C02.bin --list` (from `os_rom`, after a build) also shows which banks each file is in. The `rom-copy` test copies every file from `/rom` to a card on the emulated machine and compares them with their sources.

The RAM disks: `/ram`

Two disks in RAM, each with a HydraFS on it: fast, there with no card, and **empty after a reset** (each is quick-formatted when it starts). The plan they come from is [plans/DISKS.md](#).

Disk	Names	Memory	At boot
<code>r</code> , the RAM disk	<code>/dev/sd/r</code> ; a shell's own area at <code>/ram</code>	The storage task's own 8K banks on the RAM modules	256K (on a machine with less: half that, and so on)
<code>s</code> , the shared RAM disk	<code>/dev/sd/s</code> ; <code>/sram</code>	Shared RAM banks, from the lower half (IDs <code>\$01-\$7F</code> : <code>SH_ALLOC</code> takes from the top)	512K (the same)

Each shell mounts **its own area** of the RAM disk at `/ram` (below), and the boot shell mounts the shared disk at `/sram`, both with a spec: `ns` shows `mount hfs /ram r/1` (the shell's own namespace) and `mount -s hfs /sram s` (the system's). So `/ram` means "my files in RAM" in every shell, as Plan 9 gives each process its own names, and the tasks a shell starts (programs, scripts, pipelines) inherit it.

Areas. The RAM disk's top directories are the tasks' areas: `N`, a hex digit (`0-9`, `a-f`), is task `N`'s. A shell mounts its own when it starts (`SH_OWN_AREA`: `mount hfs /ram r/N`, `N` its task: the boot shell's is `r/1`) and makes it, with `bin` and `lib` in it. To see them all, mount the whole disk: `mount hfs /a r`, then `ls /a`.

- **Who:** anything in area `N` (opening, creating, removing, `ls`) is for task `N`, the tasks it started (and theirs, up the owner chain: `ZP_TASK_OWNER`), and task `0`, the system's (the kernel's `TASK_MAY`). Any other task gets `ERR_IO_PERM` (`$88`, the shell's `!IO_ERR!` not allowed). So a program the shell runs, or a stage of a pipeline, uses the shell's area, and two shells' families can't see into each other's.

- **Other names** in the root (`zz`) aren't areas: refused. The root's listing is everyone's. Renaming an area, or changing its mode, is refused too (but for task 0).
- **Owners:** when a task ends, the tasks it started get its owner instead (`TASK_ORPHANS` , as Unix gives orphans to `init`), so a new task in its slot isn't taken for their parent.
- **An area goes when its task ends**, with everything in it: after the task's fds are closed, and before its parent (a shell waiting for it) goes on (`TASK_AREA_END` , then `HFS_AREA_END` in the storage task). If a task it started still has a file in it open, it stays, and goes when task N ends next. (The storage task keeps a bit for each area that may be there, `RAMD_AREAS` , so a task without one ends as fast as before.)

With no card, the boot shell's current directory is its own area, `/ram` (the prompt `/ram>`), so files can be saved there (until a reset), and it runs `/rom/boot.hys` (a line about it) in place of a card's `boot.hys` .

Before `boot.hys` , the boot shell runs `/rom/lib/namespace` , the default namespace's `/bin` and `/lib` (namespaces), then the card's `lib/namespace` , if it has one: lines of `bind` and `mount` for the card's own directories (`bind -a /sd/0/tools /bin`), and `#` comments.

Program caches. The boot shell makes `/sram/bin` and `/sram/lib` (the shared caches), and each shell's area has its own `bin` and `lib` . A program typed by its name is looked for in the current directory (Plan 9's `.` first), then in `/bin` , a union (namespaces) of:

1. the shell's cache, `/ram/bin` (its `-c` member: a copy into `/bin` goes there; a `bind` to `/ram/bin` , so in each shell it's that shell's);
2. the shared cache, `/sram/bin` ;
3. the boot card's `/bin` ;
4. `/rom/bin` ;

then in `$PATH`'s directories. Caching a program is copying it there: `cp /sd/0/bin/game.hyx /bin/game.hyx` (or to `/sram/bin` , for every shell), and then `game` loads from RAM, about three and a half times as fast as from a card (a 16K program: 1.34 million cycles, against 4.69 million: [DISKS.md](#) has where the time goes). Libraries are found the same way, in the `lib` directories (`lib name`). A copy isn't checked against the card's: after rebuilding a program, copy it again (or remove the cached one).

Starting and stopping them is a write to the ctl file (`echo stop > /dev/sd/s/ctl` , or HyForth's `ctl` , which reports errors: `q^/dev/sd/s/ctl^ q^stop^ ctl`):

Write	Does
<code>start</code> <code>SIZE</code> <code>[FROM-TO]</code>	Take SIZE (8K banks; or K or M after it: 256K , 1M), from the RAM modules FROM-TO (<code>r: start 256K 1-2</code> , the storage task's banks on modules 1 and 2) or the shared bank IDs FROM-TO (<code>s: start 1M \$20-\$9F</code>), or from anywhere; then make an empty HydraFS on it. Numbers are decimal, or hex after a <code>\$</code> . <code>ERR_IO_BUSY</code> : it's started (stop it first); <code>ERR_IO_FULL</code> : no run of free banks that long there; <code>ERR_IO_BAD_REQ</code> : not a size or a range
<code>stop</code>	Give its banks back; its files are lost. <code>ERR_IO_BUSY</code> while a file on it is open

Reading the ctl file gives `ram 256 KB 512 blocks` (`sram` for the shared one), banks `$10-$2F` (its banks, or its shared bank IDs), then its HydraFS's lines; `none` when it's stopped. `format` and `check` work as on a card, and `/dev/sd/r/data` is the disk's bytes. How: the storage driver reads or writes a block by mapping its bank at `$8000` in the storage task (the RAM disk's bank in its own `$00` ; the shared one's with U), copying 512 bytes and putting them back (`SD_RAM_READ` , `SD_RAM_WRITE` in `drivers/sd.s`); the starting and stopping are in `servers/ramdisk.s` , with the memory manager's `MM_BANK_ALLOC_IN` (task banks from a range) and `SH_BANK_ALLOC` (shared banks by number, with no handle).

Pipes

`IO_PIPE` (`$F893`) makes a pipe and returns two fds: `.A` reads it, `.X` writes it.

- **Buffer:** 255 bytes. A writer waits while it's full, and a reader while it's empty.

- **Ends:** readers get end of file once every write fd is closed. Writers get `ERR_IO_BROKEN` once every read fd is closed.
- **Limit:** there are 8 pipes (`ERR_IO_NO_PIPES` beyond that).

To connect two tasks, make the pipe, then point the child's stdin or stdout at one end with `IO_DUP2` before starting it: it inherits the fds. Then close the ends each task doesn't use. HyForth's `|` does exactly this.

Namespaces

Each task has its own namespace of up to 32 entries, which the tasks it starts inherit, on top of the **system namespace**, 32 more that every task sees. It works as Plan 9's does ([NAMESPACES.md](#)): entries with the same path are a **union**, whose members are looked in, in order, for a name.

Call	Does
<code>IO_MOUNT</code> (<code>\$F89C</code>)	<code>.A.Y</code> = path (up to 13 characters), <code>ZP_IO_BUF</code> = a device's name, <code>.X</code> = flags: names under the path go to that device's server, with the rest of the name (<code>/z/sub</code> → device <code>zero</code> , name <code>/sub</code>). With <code>NS_SPEC</code> , <code>ZP_IO_CNT</code> = a spec, as Plan 9's <code>mount</code> takes one (up to 14 characters): the server gets the spec's path, then the rest (<code>mount hfs /rom x: /rom/bin</code> → <code>/x/bin</code>)
<code>IO_BIND</code> (<code>\$F89F</code>)	<code>.A.Y</code> = path, <code>ZP_IO_BUF</code> = target (up to 15), <code>.X</code> = flags: names under the path stand for the same names under the target (<code>/tty</code> → <code>/dev/cons</code>). <code>.X</code> = <code>NS_HIDDEN</code> : a hide , no target: nothing under the path is found
<code>IO_UNMOUNT</code> (<code>\$F8A2</code>)	<code>.A.Y</code> = path, <code>.X</code> = 0 (or <code>NS_SYSTEM</code>): remove all its entries. Other bits in <code>.X</code> : just the member <code>ZP_IO_BUF</code> names (a bind's target, or a mount's device). <code>.X</code> = <code>NS_FRESH</code> (the path: <code>"/</code>): a fresh namespace, as Plan 9's <code>newns</code> : every entry of the task's own goes but its <code>/ram</code> mount, so it sees the system namespace and its area, as a new shell does
<code>IO_NS_LIST</code> (<code>\$F8A5</code>)	Print the entries as the lines that would make them, as Plan 9's <code>ns</code> does: <code>mount [-acs] device /path [spec]</code> , <code>bind [-acs] /target /path</code> , <code>hide /path</code> ; the system namespace's first (<code>-s</code>), then the task's own

The flags (`.X`, `include/io.inc`): none (0) replaces the path's entries with this one, as a plain bind or mount always did; `NS_BEFORE` (`-b`) puts it before the path's members, `NS_AFTER` (`-a`) after them; `NS_CREATE` (`-c`) makes it the member a new file in the union is made in. A union with no `-c` member makes no files (`ERR_IO_MODE`). `NS_SYSTEM` (`-s`) makes the entry the system namespace's: for task 0 and the boot shell only (`ERR_IO_PERM` for others).

The system namespace is a table of 32 entries every task sees, under its own: the boot shell writes it at boot (its mounts, `/rom/lib/namespace`'s lines), and a copy of it sits in each IO transfer bank (`NS_SYS`, `$9800`), so a name resolves without switching banks. A task's own entry for a path wins over the system's, so a plain bind or mount at a system path replaces it for that task (and the tasks it starts). Changing a system union in a task (`bind -a`, `-b`, or `unmount` of one member) first copies the union's members into the task's own table (copy on write); `unmount` of a system path leaves a hide in the task's. So every task behaves as if it had a copy of the system's entries, and only what a task changes costs it entries of its own.

How `IO_OPEN` resolves a name:

- It applies the entry with the **longest matching prefix**, matching whole path elements (`/z` matches `/z/sub`, not `/zz`), in the task's table or the system's (a tie: the task's).
- After a bind it looks again, up to 4 times (`ERR_IO_NS_LOOP` beyond).
- In a union, it opens the name in the first member; if that member hasn't got it (`ERR_IO_NOT_FOUND`), the next, and so on. A create goes to the `-c` member. A hide's path gives `ERR_IO_NOT_FOUND`.
- **A union's directory** is read member by member, as in Plan 9: when an fd reading one reaches the end of a member's listing, the IO layer opens the same name in the next member that has it, on the same fd, and the read goes on (`IO_UNION_NEXT` in `io/io.s`). A name in two members shows twice. One such fd a task at a time (the last opened; the name up to 50 characters): another fd on a union's directory reads only its first member.
- A name no entry matches must be under `/dev`: the IO layer sends `/dev/<device>/...` to the device in every task, with no mount. Everything else is an entry's.
- Through a mount with a spec, the request says so (`IO_BLK_SPEC`): HydraFS serves its disks in memory (`x`, `r`, `s`) only then (or for task 0), so `/sd` has the cards alone.

The namespace at boot. In the system namespace, the boot shell mounts `/sd` (`hfs`), `/env` (`env`), `/proc` (`proc`), and `/rom` and `/sram` (`hfs`, with the specs `x` and `s`, if they're there), and binds the boot card's `bin` and `lib` at `/bin` and `/lib`; in its own, it mounts its area at `/ram`. Then, before `boot.hys`, it runs `/rom/lib/namespace` (on the ROM disk: `os_rom/romfs/namespace`), whose `-s` lines put the program caches before the card's and the ROM's after, and then the card's own `lib/namespace` if it has one (lines of the shell's `bind` and `mount`, and `#` comments). With a card, `ns` shows:

```
mount -s hfs /sd
mount -s env /env
mount -s proc /proc
mount -s hfs /rom x
mount -s hfs /sram s
bind -cs /ram/bin /bin
bind -as /sram/bin /bin
bind -as /sd/0/bin /bin
bind -as /rom/bin /bin
bind -cs /ram/lib /lib
bind -as /sram/lib /lib
bind -as /sd/0/lib /lib
bind -as /rom/lib /lib
mount hfs /ram r/1
```

That's 13 of the system's 32 entries and 1 of the shell's own 32. `/bin`'s first member is `/ram/bin`, which each task resolves in its own namespace: each shell's cache. A card's `lib/namespace` adds with `-s` for everyone (`bind -as /sd/0/tools /bin`), or without it for the boot shell and what it starts. Every task sees the same, so a C program's `fopen ("/bin/x.hyx")` finds what the shell does.

The current directory

Each task has a current directory, which the tasks it starts inherit (a copy: changing it later doesn't change theirs). It's kept in the task's IO transfer area, beside its namespace.

- **Relative names:** before a name is looked up, the IO layer puts the current directory in front of it (unless it starts with `/`) and tidies the result: `.` elements go, `..` takes the element before it away (at `/` it stays at `/`), doubled and trailing `/`s go. So `../notes` from `/sd/0/games` is `/sd/0/notes`, and a server only ever sees absolute names. The tidy, absolute name can be up to 63 characters (`ERR_IO_NAME` beyond).
- `IO_CHDIR` makes a path the current directory: a directory on a card (its stat record says so: `ERR_IO_NOT_DIR` otherwise), or `/`. A failure leaves the current directory as it was.
- `IO_GETCWD` copies it out: `/` at the top.
- A new task (`TASK_RUN`, `TASK_CLONE`) gets its parent's. The boot shell sets its own to the first card with a HydraFS on it ([hyforth.md](#)).

The files on a card

The **HydraFS** server (the device `hfs`) serves the files on the SD cards. The shell mounts it at `/sd` at startup, and the tasks it starts inherit the mount, so `/sd/0` is card 0's root directory and `/sd/0/games/star.frt` is a file on it. The format, and the host tool that makes cards, are in [plans/HYDRAFS.md](#) and [tools/emulator.md](#).

- **Names** are case-sensitive, 1-31 characters, any byte but `/` and 0. `.` and `..` are understood while walking (a path may be up to 8 elements deep); `..` at a card's root stays there.
- **Reading a file** works as it does on `/dev/sd/N/data`, except that a read stops at the end of the file.

- **Writing a file** at the fd's offset grows it past its end, and an append-only file is always written at its end. A write that *starts* past the end (after a seek) makes the file that long first, with zeros: whole 4 KB clusters of them are a **hole**, which takes no space on the card, and reads as zeros ([sparse files](#)); a write into a hole takes a cluster for it. Writing runs at about 3.5 KB/s.
- **Reading a directory** gives a line per entry, `name size` (or `name/` for a directory), then CR LF, so `"/sd/0" 1 open 0 fdup2 cat | cat` lists it (as the shell's `ls` does). Opened with `IO_MODE_STAT` it gives stat records instead; read a multiple of 48 bytes to get whole ones. Either way the listing is made again from the card at every read, so the server keeps no state for it, and a directory that changes between two reads of one listing can give a torn one.
- **Making and removing files:**
 - `IO_CREATE` makes a file (mode bits 0; `HFS_M_APPEND $40` append-only, `HFS_M_RO $01` read-only) or a directory (`HFS_M_DIR $80`) in a directory that's there, and opens it; a directory is opened for reading whatever the mode says. A *file* that's there already is emptied and opened instead, as in Plan 9.
 - `IO_REMOVE` removes a file that isn't open, or an empty directory; it borrows a free fd for the request.
 - `IO_WSTAT` renames a file in its directory (the record's name: a name, not a path; a 0 first byte keeps it) and sets its mode bits (`HFS_M_APPEND`, `HFS_M_RO`; `$FF` keeps them). The record's other fields are left alone. HyForth's `mv` fills in the record for you.
 - The mode bits are checked when a file is opened: a read-only file can't be opened for writing, but the fd that made it can write it.
- **What reaches the card when:** the data at once; the file's size when the last fd on it is closed, and whenever it gets a new 4 KB cluster. So close a file you've written before taking the card out. A crash can leave a file shorter than was written, never a damaged card.
- **Formatting:** write `format [-f] [-p] [-s size] [LABEL]` to `/dev/sd/N/ctl` to make an empty HydraFS on the card (everything on it is lost), and `label NAME` to change the label. Plain `format` is quick (a version 2 HydraFS, whose free map is written as it's used); `-f` writes the whole map now, printing its progress on the console; `-s` limits the size (megabytes, or 4G) ([plans/HYDRAFS.md](#)).
- **Partitions:** a card with a partition table has its HydraFS in its partition of type `$7F`, and the other partitions (a FAT one for a PC, say) are left alone. `format` on such a card formats that partition; `format -p` makes one on a card without one, after its other partitions (or with a new table, from block 2048). The `ctl` file then shows `partition` at `block N` ([plans/HYDRAFS.md](#)).
- **Checking:** write `check` to `/dev/sd/N/ctl`, then read the file: it counts the clusters marked in use that nothing uses (lost), in use but marked free (unmarked), and used twice, and recounts the free space. `check fix` also repairs the free map (not a cluster used twice: that's reported for a person to sort out). In HyForth: `"/dev/sd/0/ctl" "check" ctl`, then `ls /dev/sd/0/ctl` (or `0 fsck`, which does both). It takes a pass per 256 MB of card, and with 16 passes or more prints its progress on the console as it goes ([plans/HYDRAFS.md](#)).
- **Errors:** `ERR_IO_NOT_FS` (`$80`) if the card holds no HydraFS, `ERR_IO_DEVICE` (`$79`) if there's no card, `ERR_IO_NOT_FOUND` (`$70`) for a name that isn't there (or a path through a file), `ERR_IO_NO_FDS` (`$75`) when all 8 HydraFS files are open (they're shared by every task), `ERR_IO_MODE` (`$72`) for writing a directory or a read-only file, `ERR_IO_FULL` (`$81`), `ERR_IO_EXISTS` (`$82`: creating a directory where there's a name already, a file where there's a directory, or renaming to a name that's taken), `ERR_IO_NOT_EMPTY` (`$83`) and `ERR_IO_BUSY` (`$84`: removing an open file). The shell's commands add `ERR_IO_NOT_DIR` (`$85`: `cd` or `rmdir` on a file) and `ERR_IO_IS_DIR` (`$86`: `rm` or `cp` on a directory). Any other device refuses these calls with `ERR_IO_BAD_REQ`.

Stat

`IO_STAT` fills a 48-byte record from the server. A device with nothing to say returns all zeros; HydraFS fills it in from the file's directory entry:

Offset	Size	Field
0	32	The name, zero-terminated

Offset	Size	Field
32	1	Mode: bit 7 = a directory, bit 6 = append-only, bit 0 = read-only
33	1	The disk's number (0-7: the card <code>0</code> - <code>7</code> ; 8: the ROM disk, <code>x</code> ; 9 and 10: the RAM disks, <code>r</code> and <code>s</code>)
34	2	The qid's version: up by 1 at every change
36	4	The qid's id: unique on the card, never reused, and the same across renames
40	4	The size in bytes
44	4	The modification stamp: the clock's time at the last change (seconds since 2000-01-01; /dev/time)

A directory read with `IO_MODE_STAT` returns these same records, one per entry.

Writing drivers and file servers

How to add a device: a file server that tasks open by name, and optionally a driver task with IRQ handlers behind it. Sources to read alongside:

Source	What it shows
os_rom/io/io.s	NULL_SERVE : the smallest server
os_rom/servers/proc_srv.s	Names, text files and a <code>ctl</code> file
os_rom/io/pipe_srv.s	Waiting and waking
os_rom/servers/sd_srv.s	A server on its own ROM page, with a cache
os_rom/drivers/serial.s	A driver with IRQ handlers

Part of the [Programmer's Guide](#).

The model

```

client task          IO layer (BIOS page 2)          server (its task)
IO_WRITE fd, buf, n  -----> fd -> (device, fid, offset)
                        copy <= 256 bytes of buf into
                        the client's transfer area
                        fill in the request block -----> serve routine (TASK_CALL)
                                                                reads the transfer area,
                                                                does the IO
                                                                C = 0 and a count, or an error
                        advance the offset, loop <-----
```

- **The server** is a **serve routine** that runs in a task: the driver's task, or the calling task. The IO layer calls it with `TASK_CALL`, so it runs with the server task's zero page, stack and RAM.
- **One request at a time**: a task that calls a busy server waits its turn. The server can be preempted in the middle of a request like any code.
- **Data** passes through the client's **transfer area** in shared RAM, which the server maps while it works on the request.
- **Requests** follow a small subset of Plan 9's 9P ("H9P").

Registering a device

`DEV_REGISTER ($F887)` adds a name to the device table:

In	
<code>.A.Y</code>	The name: 1-8 characters, e.g. "null" for <code>/dev/null</code>
<code>.X</code>	The task the serve routine runs in: usually the driver's own (<code>T_REGISTER</code>), or <code>IO_DEV_CALLER_TASK (\$FF)</code> to run it in whichever task makes the request (<code>/dev/proc</code> , <code>/dev/null</code>)
<code>ZP_TC_VEC</code>	The serve routine: a page 0 address. A server on another page gets a page 0 gate

It returns the device's index, or `ERR_IO_NAME` (bad or duplicate name) / `ERR_IO_NO_DEVS` (16 devices already).

A driver registers from its `init`, which `DRV_START` runs in the driver's task ([tasks.md](#)). If the `init` fails, the task's devices are removed again.

```

; drivers/storage.s: the storage task's init (page 0), for a server on page 3
FAR_GATE_INLINE      SD_SERVE,          PAGE3::SD_SERVE,          3

STORAGE_INIT:
    jsr      STORAGE_INIT3              ; (its own setup, on page 3)
    bcs      @done
    LOAD_ADDR SD_SERVE, ZP_TC_VEC        ; the page 0 gate is the serve routine
    lda      #<SD_NAME                  ; "sd"
    ldy      #>SD_NAME
    ldx      #STORAGE_TASK_NUM
    jmp      DEV_REGISTER
@done:
    rts

```

The serve routine

```

; IN: .A = request (H9_*), .X = client task, .Y = fid
; OUT: C = 0 (.A = the new fid, for H9_OPEN); or C = 1, .A = error

```

Request	Value	The server should
H9_OPEN	1	Look at the name (the rest after the device name, in the data area) and the mode; return a fid : a byte of its own choosing that identifies the open file in later requests
H9_READ	2	Put up to <i>count</i> bytes from <i>offset</i> in the data area; set the count to what it gave (0 = end of file)
H9_WRITE	3	Take up to <i>count</i> bytes from the data area at <i>offset</i> ; set the count to what it took, or leave it for all
H9_CLUNK	4	The fd is closed: release the fid
H9_STAT	5	Put a 48-byte stat record in the data area (STAT_ZERO for all zeros; the layout is in io.md)
H9_CTL	6	Device-specific control: the code and argument are in the request block
H9_DUP	7	Another fd now refers to the fid (IO_DUP2, or a new task inherited it): count it if it keeps counts
H9_CREATE	8	A filesystem's: make the file named in the data area (its mode bits in the request block's IO_BLK_PERM), and open it
H9_REMOVE	9	A filesystem's: remove the file named in the data area (no fid)
H9_WSTAT	10	A filesystem's: change the fid's file as the stat record in the data area says

Return `ERR_IO_BAD_REQ` for requests it doesn't support. **Refuse the filesystem's requests** (H9_CREATE and up) unless it's a filesystem: a server that treats an unknown request as done would make `IO_REMOVE` of its name "succeed", or `IO_CREATE` hand out an fd. The servers so far start with `cmp #H9_CREATE / bcs` to their refusal.

Fids are the server's business. Common patterns:

- one fid per kind of file (`/dev/cons 0, /dev/ser 1, /dev/ser/ctl 2`);
- a fid that encodes the file (`/dev/proc: kind | task; /dev/sd: kind | card`);
- an index into the server's own table (pipes).

A server that counts its users keeps counts per fid. It gets an `H9_DUP` for every extra fd, and an `H9_CLUNK` for every close.

The request block and the transfer area

Each task has a 1.5K **IO transfer area**, four to a shared bank: in bank ID $\$09 + \text{task} / 4$ ($\$09-\$0C$), at $\$8000 + (\text{task} \& 3) * \600 when mapped (`IO_XFER_OF`). Its request block is at the start, the data at $+ \$100$, and the task's namespace after it; each bank's $\$9800-\$9BFF$ holds a copy of the system namespace:

Offset	Field (<code>include/io.inc</code>)
0	<code>IO_BLK_TYPE</code> : the request
1	<code>IO_BLK_FID</code> : the fid
2	<code>IO_BLK_MODE</code> : the fd's mode (on every request, so a server can tell a pipe's ends apart)
3	<code>IO_BLK_CLIENT</code> : the client task
4-7	<code>IO_BLK_OFS</code> : the 32-bit offset (read, write)
8-9	<code>IO_BLK_COUNT</code> : bytes requested (in), bytes done (out); 1-256
11	<code>IO_BLK_CTL_CODE</code> (<code>H9_CTL</code>)
12	<code>IO_BLK_CTL_ARG</code> (<code>H9_CTL</code>)
$\$20-\FF	The task's namespace (not the server's business)
$\$100-\$1FF$	<code>IO_BLK_DATA</code> : the data, or the name for <code>H9_OPEN</code> (zero-terminated)

Mapping it:

- `IO_SRV_MAP` (`.X` = the client) maps the client's area into $\$8000-\$9FFF$ in the server's task, and points `ZP_IO_REQ` at its request block (the data is at $\text{ZP_IO_REQ} + \$100$).
- `IO_SRV_UNMAP` undoes it; call it before returning.
- `IO_SRV_COUNT` sets the count to `.A` and unmaps. It's on page 2 only; servers elsewhere write the count themselves.

While the area is mapped, the server's own bank at $\$8000$ isn't: copy through a buffer in task RAM if the server's data is in a bank.

The smallest server (`/dev/null`, `io/io.s`):

```

NULL_SERVE:
    cmp        #H9_READ
    beq        @read
    cmp        #H9_STAT
    beq        @stat
    cmp        #H9_CTL
    beq        @bad

@ok:                                ; H9_OPEN (fid 0), H9_WRITE (all taken), H9_CLUNK, H9_DUP
    lda        #0
    clc
    rts

@read:                                ; end of file: count = 0
    jsr        IO_SRV_MAP
    lda        #0
    ldy        #IO_BLK_COUNT
    sta        (ZP_IO_REQ),Y
    iny
    sta        (ZP_IO_REQ),Y
    jsr        IO_SRV_UNMAP
    bra        @ok

@stat:
    jsr        STAT_ZERO
    bra        @ok

@bad:
    lda        #ERR_IO_BAD_REQ
    sec
    rts

```

Waiting: when there's no data yet

A server that can't complete a request now:

1. **Remembers the client.** `.X` on entry is the client; the usual way is to set its bit in a 16-bit wait mask.
2. **Returns** `C = 1` with `.A = ERR_IO_WOULD_BLOCK`. The IO layer marks the client waiting, and it sleeps (no CPU). A non-blocking fd gets the error instead.
3. **Wakes the client** when data comes in, usually in the driver's IRQ handler: it calls `IO_WAKE` (`.A = task`) for each task in the mask, and clears the mask. The IO layer then sends the **same request again**.

Waking a task that has already been woken, or isn't waiting any more, is harmless: it just tries again. The IO layer holds off task switches between marking the client waiting and yielding, so a wake can't be lost.

Names and ctl files

Names. For `H9_OPEN`, the data area holds the rest of the name after the device. Opening `/dev/proc/3/ctl` sends `/3/ctl` to the `proc` server, and a mount sends what follows the mount point. A server walks it however it likes: `proc_srv.s` and `sd_srv.s` parse `/N/name`.

ctl files. A text control file is friendlier than `IO_CTL` codes, because anything that can write text can use it (HyForth, a script). Examples:

- `/dev/proc/N/ctl` takes `kill`, `break`, `fg`;

- `/dev/sd/N/ctl` takes `init`, and reads back the card's details;
- `/dev/ser/ctl` takes `b9600 18 pn s1`-style settings, and reads back the current ones (`servers/serctl.s`).

The parsing in `PROC_WRITE` / `SD_CTL_WRITE` takes a command word followed by the end of the write, a space, CR, LF or 0.

Text reads. A server that returns generated text (a status line) makes the text in the data area for each read, then hands over what's after the fd's offset, up to the count. `PROC_READ` and `SD_CTL_READ` do this, so `cat` gets the text once and then end of file.

Checklist for a new device

1. **Choose where it runs:**
 - a driver task, for a device with state or IRQs;
 - or the calling task (`IO_DEV_CALLER_TASK`), for a stateless one like `/dev/proc`.
2. **Write the serve routine**, on the page that suits it, with a page 0 gate if it isn't on page 0.
3. **Write the driver's** `DriverInfo` **and** `init`:
 - set up the hardware;
 - `IRQ_REGISTER` its handler ([interrupts.md](#));
 - `DEV_REGISTER` its names.
4. **Start it at boot:** `DRV_BOOT` in `kernel/os_main.s`, in a free task. Task numbers `$2-$B` are free; drivers so far use `$C-$F`, downwards.
5. **Add HyForth words** if it's useful from the shell (`hyforth/hywords.s`: `open`, `read`, `write` and `ioctl` already reach any device).
6. **Add a regression test** that opens, reads and writes it in the emulator ([emulator](#)), and model the device in `sim/hydrasim.js` if it's new hardware.

The storage layer (for the filesystem server)

The storage task (`$C`) owns the SPI bus. Its block layer (`drivers/sd.s`, page 3) works on card `SD_DEV` (0-7), in the storage task's zero page:

Routine	Does
<code>SD_INIT</code>	Start the card: SDHC/SDXC (block addresses) or SDSC (byte addresses). Reads its size from the CSD register into <code>SD_CARD_BLOCKS</code>
<code>SD_READ_BLOCK</code>	Read block <code>SD_LBA</code> (32 bits) into the 512 bytes at <code>SD_BUF</code>
<code>SD_WRITE_BLOCK</code>	Write the 512 bytes at <code>SD_BUF</code> to block <code>SD_LBA</code>

Errors: `ERR_IO_DEVICE` (no card, or no answer), `ERR_IO_NOT_READY` (not started), `ERR_IO_MEDIA` (the card refused).

Throughput is about 300 CPU cycles a byte (12 KB/s at 3.58 MHz), and roughly 60% of that is the bit-banged SPI receive loop (`SPI_RECV`, 18 cycles a bit). The VIA's shift register can't help: it uses CB1/CB2, which aren't wired to the SPI lines. The `sd-speed` regression test holds the figure to a budget.

The cache: `sd_srv.s` keeps a one-block cache (`SD_CACHE_LOAD`: block `SD_LBA` of card `SD_DEV`).

The HydraFS server

`fs/hfs_srv.s` and `fs/hfs_write.s` (ROM page 6: page 3 was full) are a second device, `hfs`, in the same storage task, on the same block layer and cache (through gates to page 3, one far call a block): it serves the **files** on the cards (`/sd/N/...`, the format in [plans/HYDRAFS.md](#); using it is in [io.md](#)). Worth knowing if you write a server of your own:

- **Two devices, one task.** A server task serves one call at a time, so `hfs_srv.s` borrows the SD server's zero page (`SD_POS`, `SD_LEFT`, `SD_N`, ...) rather than having its own, and uses `SD_LBA` as its block number (`SD_CACHE_LOAD` leaves it alone). Its own state goes in the storage task's RAM, next to the SD driver's.
- **Its own fids.** A HydraFS fid is an open-file slot (0-7), shared by every task; `H9_DUP` counts up the fids that share one and `H9_CLUNK` counts down, so an inherited or `IO_DUP2`'d fd doesn't free it early.
- **Generated text, again.** A directory listing is made from the card at every read and the bytes before the fd's offset thrown away, exactly as `SD_CTL_READ` does for its status line, so the server keeps nothing between reads.
- **Short reads.** Unlike `/dev/sd`, it can return fewer bytes than asked for (the end of the file), so it writes the count done as *what was wanted, less what's left*.
- **Every request ends the same way** (`HFS_FINISH`): what it changed in the metadata buffer, and the card's counters, go to the card before the reply, so the card is consistent between requests, and nothing is cached between them that a raw `/dev/sd` write could leave stale.
- **Several copies of one entry.** Each open file holds a copy of its directory entry; any change to one is copied to the others at once (`HFS_SYNC`), and a walk takes an open file's copy over the card's, since the file's size goes to the card only when it's closed or grows a cluster.
- **Restarting a card** (`SD_CTL_INIT` on `/dev/sd/N/ctl`) calls `HFS_FORGET`, which drops the card's superblock numbers and frees any HydraFS file open on it, since a different card may be in the socket now. Those fids give `ERR_IO_BAD_FD` from then on.

Two warnings from writing it:

- A 32-bit subtraction whose loop ends with `cpx #4` loses the carry between the `sbc s`. `HFS_TAIL` is unrolled for that reason; check any multi-byte arithmetic in a loop for the same thing.
- **Scripted input and long commands:** the emulator types a key every 20,000 cycles and the serial task buffers 255 of them, whether the shell is reading or not, so a regression test that types past a long command (a 20 KB write takes about 30 million cycles) loses the rest of its input. Put `w(n)` waits after anything slow.

Programs: Hydra executables (.hyx)

A Hydra executable is a program on a card that the shell loads and runs in a task of its own: `run hello.hyx`, or just `hello` (HyForth). It's machine code linked for a fixed address in task RAM, with a 16-byte header in front. Write one in assembly (a sample is in `programs/`), or in C (below: `programs/c/`).

The file

Offset	Size	Field
0	4	Magic: HYX1
4	2	Load address
6	2	Length of the code (the bytes after the header)
8	2	Entry point
10	2	Flags: 0
12	2	BSS: the bytes after the code that the loader clears (0: none)
14	2	Top: the end of the RAM the program uses directly (its heap and stack's too); 0: the end of its code and BSS

Then the code, which is loaded at the load address as it is: a program isn't relocatable. The code, its BSS and its top must fit in `$0800-$7BFF` (`ERR_IO_NOT_EXEC`, `$87`, otherwise). As in Plan 9's `a.out` header, the BSS isn't in the file: the loader clears it. An older header, with 4 zero bytes at offset 12, still loads. The constants are in `os_rom/include/shell.inc` (`HYX_*`).

`run` tells a program from a script by the header, not by the name: a file without `HYX1` at its start is run as a HyForth script, except a **song** (a ZSM file, which starts with `zm`), which the ROM's song player plays in a task of its own (HyForth's `play`). Name them `.hyx` (`.zsm`) anyway, so the shell finds them by name.

What a program gets

- **A task of its own**, started with copies of the shell's fds (stdin, stdout and stderr: fds 0-2), namespace and current directory. So a program can read and write files by relative names, and be a stage of a pipeline (`run hello.hyx | wc`).
- **Its RAM**: task RAM `$0800-$7BFF`. Its code is loaded at its load address, its BSS is cleared, and the MMU's page floor is set just above the higher of its BSS's end and its top, so the MMU's allocations come from the pages above them. For RAM beyond its end, a program allocates it (`MM_ALLOC`), or raises the floor (`MM_SET_FLOOR`) to use it directly ([memory.md](#)).
- **Its arguments**: at the entry point, `.A.Y` points to them: the rest of the line after its name (`prog a b` gives `a b`), zero-terminated, at `HYX_ARGS` (`$0110`, 63 characters at most; empty if there are none). They travel from the shell through a pipe on fd 10 (`SH_ARGS_FD`), which the loader reads and closes.
- **Its name**: as it was run (`hello`, `bin/hello.hyx ...`), zero-terminated, at `HYX_NAME` (`$0150`, 31 characters at most): C's `argv[0]`. It comes through the same pipe, after the arguments.
- **Its environment**: a copy of the shell's: `/env/NAME` files it can read (and change, for itself and the tasks it starts).
- **Zero page** `$E0-$FF` (`PROGRAM_ZP`: the OS's stays below it; the bytes between the OS's end, `$B7` in this build, and `$E0` may go to the OS in a later ROM). The calls' zero-page parameters (`ZP_IO_BUF ...`) are at fixed addresses ([the API index](#)). And the stack page: the loader has used a few bytes of the stack, `$0100-$010F` held the header, the arguments are at `$0110-$014F`, and the name at `$0150-$016F`.
- **The OS calls** through the `$F8xx` thunks ([the API index](#)): the entry point is called on ROM page 0, where they are.
- **The console** while it runs, if the shell had it: its keys come to the program, and **Ctrl-C ends it** (a task without a break handler ends on a break; `TASK_SET_BREAK` sets one: [tasks.md](#)). The shell's prompt comes back when it ends.

Ending: the program returns (`rts` from its entry point), or ends with an exit status (`TASK_EXITS`), or it's killed (Ctrl-C, Ctrl-\, kill). Either way its task ends: its fds are closed (stdout's buffered output goes out first), and its memory is freed.

Its exit status (Plan 9's `exits`): a code, 0-255, and a message of up to 30 characters. `TASK_EXITS` ends the task with them (`.A` = the code, `ZP_IO_BUF` = the message, or 0 for none). A program that returns ends with 0; a break with 130, `interrupt`; a kill with 137, `killed`. The shell keeps it: HyForth's `status`, and `/env/status` (the message, or the code if there's none; empty for 0: Plan 9's `$status`). A task that started another gets it from `TASK_JOIN` ([tasks.md](#)).

Building one

`programs/asm/` has a sample, `samples/hello.s`, and what it's built with (C programs are in `programs/c/`: below):

File	What
<code>hyx.inc</code>	<code>HYX_HEADER</code> entry: the header, in its own segment, with the load address and length filled in by the linker
<code>hyx.cfg</code>	The ld65 config: the header, then <code>CODE</code> , <code>RODATA</code> and <code>DATA</code> at <code>\$0800</code> . There's no BSS segment: everything is in the file (variables go in <code>DATA</code>)
<code>make.bat</code>	Runs <code>build.js asm:ca65</code> , then ld65 with <code>hyx.cfg</code> , for each <code>samples*.s</code> : <code>bin\hello.hyx</code>

```
.include "hyx.inc"
WRITE_CHAR      = $F803
                HYX_HEADER  start

.code
start:    lda      #'!'
                jmp      WRITE_CHAR          ; its rts ends the program
```

From another assembler, make a raw binary for a fixed address, then put the header on it with `sim/tools/mkhyx.js` (Node.js):

```
node sim/tools/mkhyx.js prog.bin prog.hyx $0800 [ENTRY]    the header on a binary (ENTRY: default the load
address)
node sim/tools/mkhyx.js --info prog.hyx                  show a .hyx file's header
```

C programs

`programs/c/` has a C library for [cc65](#): cc65's own C library (stdio, strings, malloc, time ...) on the Hydra's calls, and the Hydra's own (`hydra.h`). A C program is a `.hyx` like any other. [The C Programmer's Guide](#) covers writing one in depth; this section is the summary.

```
programs\c\make.bat                build the library (lib\hydra.lib) and the samples (bin\*.hyx)
programs\c\hyc.bat prog.c [more.c] [more.s]    build a program of your own: bin\prog.hyx
```

They run `build.js` (`node build.js c`, `node build.js prog prog.c ...` on any OS), which finds cc65 in `CC65_HOME`, on the `PATH`, or in `C:\source\cc65\win64_snapshot`. `hyc.bat` compiles with `cc65 -t none --cpu 65C02 -O` and links with `hydra.cfg` and `lib\hydra.lib`.

File	What
<code>hydra.cfg</code>	The ld65 config: the header, then the program at <code>\$0800</code> ; zero page <code>\$E0-\$FF</code>
<code>include/hydra.h</code>	The Hydra's own calls and constants (below)

File	What
lib/hydra.inc	The calls' addresses, zero-page parameters and constants, for the library's assembly (the <code>c-programs</code> test checks them against the ROM)
lib/crt/	<code>crt0.s</code> : the header (with its BSS and top), the start (the stack, the constructors, <code>main</code>), and the end (<code>exit</code> : the destructors, then <code>TASK_EXITS</code> with the status); <code>mainargs.s</code> : <code>argc</code> and <code>argv</code>
lib/io/	Files: <code>read</code> , <code>write</code> , <code>open</code> , <code>close</code> , <code>lseek</code> , <code>remove</code> , <code>rename</code> , <code>mkdir</code> , <code>rmdir</code> , <code>chdir</code> , <code>getcwd</code> , <code>stat</code> , <code>fstat</code> , <code>isatty</code> , <code>opendir</code> and the rest of <code>dirent.h</code> , <code>errno</code> and <code>_oserror</code>
lib/env/	The environment: <code>getenv</code> , <code>putenv</code> , <code>setenv</code> , <code>unsetenv</code>
lib/conio/	<code>conio.h</code> on the console (an ANSI terminal)
lib/snd/	<code>snd.h</code> : the YM2151, through <code>/dev/snd</code> and the ROM's sound library; songs (<code>snd_play</code>)
lib/sys/	<code>system</code> , <code>clock</code> , <code>clock_gettime</code> (so <code>time</code>), <code>sleep</code> ; <code>hydra.h</code> 's calls (tasks, exit statuses, semaphores, ticks)
samples/	<code>hello.c</code> (arguments, arithmetic, the heap, the clock), <code>upper.c</code> (a filter: stdin to stdout in capitals), <code>code.c</code> (an exit status from its argument), <code>keys.c</code> (<code>conio</code> : the screen and raw keys), <code>tones.c</code> (<code>snd.h</code> : patches, notes, volume, a bend, drums), <code>jukebox.c</code> (a song in the background, stopped), <code>ctest.c</code> (the library's test)

A module in `lib/` replaces cc65's module of the same name (`make.bat` adds them to a copy of cc65's `none.lib`), so name a new one after the cc65 module it replaces, or something cc65 doesn't have.

What a C program gets:

- **The standard library:** `printf` and the rest of `stdio` on the Hydra's files (`stdin`, `stdout`, `stderr` are fds 0-2, as the shell gives them, so `prog < in.txt | wc` works), `malloc`, `strings`, `time` and `localtime` (the Hydra's clock: to the second, as local time), `sleep`, and the file and directory calls above. Names are relative to the current directory unless they start with `/`.
- **Its RAM:** `$0800-$6FFF`: the program, its BSS, the heap, and a 2 KB C stack at the top (`__RAMTOP__` and `__STACKSIZE__` in `hydra.cfg`). The header gives the loader the BSS's size (it clears it) and `__RAMTOP__` as the top (the MMU's floor goes there), so the file holds only the code and data, and the MMU's `MM_ALLOC` pages come from above `$7000`.
- **argc and argv:** `argv[0]` is the program's name as it was run; the arguments are split at spaces, "a b" as one (15 at most).
- **The environment** (Plan 9's: each variable is a file, `/env/NAME`, in a copy of the shell's environment): `getenv` reads one (into a static buffer, up to `HY_ENV_MAX` characters), and `setenv`, `putenv` (`"NAME=value"`; `"NAME"` alone removes it) and `unsetenv` change it. The changes are the program's own, and the commands it runs get them.
- **Files and directories:** `stat` and `fstat` (`st_size`, `st_mode`: `S_ISDIR`, `S_IREAD`, `S_IWRITE`; `st_mtime` from HydraFS's stamps), `opendir/readdir/closedir` (`d_name`; `hy_dirstat` gives the entry's stat without opening it), `isatty`.
- **The console** through `conio.h` (below), or `stdio`. `stdio` on the console acts as a Unix terminal: an LF written goes out as CR LF, and Enter (CR) reads as `\n` (`read.c`, `write.c`).
- **Commands:** `system` (`"ls /bin | wc"`) runs a command line as the prompt would (in a command shell, `SHELL_CMD`: Plan 9's `rc -c`), waits, and returns its exit code; `hy_spawn` starts one without waiting, and `hy_wait` waits for it and gets its code and message.
- **Time:** `time` and `localtime` (the Hydra's clock, to the second, as local time); `clock` (ticks since the program started: `CLOCKS_PER_SEC` is 200); `sleep`.
- **Errors:** a failed call returns -1 and sets `errno`; `_oserror` has the Hydra's own error ([error codes](#)).
- **Ending:** `main` returns, or `exit(code)`: the destructors (and `atexit`'s) run, and the task ends with that exit status. `hy_exits` (`"message"`) ends it with a message too (and code 1; `NULL` or `""`: success). The shell keeps it (`status`, `$status`).
- **Not yet:** `rename` only within a directory; `FILENAME_MAX` is cc65's 17 for this target, so use `HY_PATH_MAX` (65) for paths, `HY_NAME_MAX` (32) for a name.

`hydra.h`:

Call	Does
<code>hy_sem_new (count), hy_mutex_new ()</code>	A semaphore (1-16), or a mutex (semaphores)
<code>hy_sem_acquire (s)</code>	Take one, waiting until there's one
<code>hy_sem_try (s)</code>	Take one if there is one: 1; else 0, at once
<code>hy_sem_release (s), hy_sem_free (s)</code>	Give one back; free it
<code>hy_ticks (), hy_sleep_ticks (n)</code>	The scheduler's tick count (<code>HY_TICKS_PER_SEC</code> , 200 a second); sleep n ticks
<code>hy_yield ()</code>	Let the other tasks run
<code>hy_clock ()</code>	The clock: seconds since 2000-01-01
<code>hy_task ()</code>	This task's number (1-15, as <code>ps</code> shows it)
<code>hy_spawn (cmd)</code>	A command line in a task of its own, not waited for: its task, or -1
<code>hy_wait (task, msg)</code>	Wait for a task this one started to end: its exit code (0-255), and its message into <code>msg</code> (<code>HY_STATUS_MAX</code> bytes; <code>NULL</code> : not wanted)
<code>hy_exits (msg)</code>	End the program with a message (Plan 9's <code>exits</code>)
<code>hy_kill (task)</code>	End a task and the tasks it started (its status: 137, killed)
<code>fstat (fd, st), hy_dirstat (dir, st), isatty (fd)</code>	What POSIX has and cc65's headers don't for this target
<code>setenv, unsetenv</code>	(Also POSIX's)
<code>COLOR_*, CH_*</code>	conio's colours (ANSI's 16) and keys

conio: the console is an ANSI terminal on the serial port, so `conio.h` works through its sequences: `clrscr`, `gotoxy` and the rest, `wherex/wherey` (of what conio writes), `textcolor`, `bgcolor` (`COLOR_*`), `revers`, `cursor`, `chline/cvline` (- and |), `cclear`, `cputs`, `cprintf`, `cscanf`. `screenize` is `$COLUMNS x $LINES`, or 80 x 24. `cgetc` and `kbhit` read the keys raw: conio opens `/dev/cons/ctl` and writes `rawon` ([Plan 9's consctl](#)): no echo, each key as it's typed, no Ctrl-D. The terminal's cursor, editing and function keys come as one code each (`CH_CURS_UP` ... `CH_F4`). Raw ends when the program does. conio's output and stdout's keep their order (both go out through `WRITE_CHAR`).

```
#include <stdio.h>

int main (int argc, char* argv[])
{
    printf ("Hello from C: %d arguments\n", argc - 1);
    return 0;
}
```

Putting it on a card

- **In the emulator, or on a real card's image:** `node sim/tools/hydrafs.js mkdir card.img bin`, then `node sim/tools/hydrafs.js put card.img programs/asm/bin/hello.hyx bin` ([the card tool](#)).
- **On the Hydra:** copy it from one card to another with `cp`.

A program in the current directory, or in `/bin` on the current directory's card, runs by its name: `hello`.

How `run` does it (`os_rom/shell/run.s`, BIOS ROM page 7)

1. The shell opens the file, and reads its start: an `HYX1` header is checked (it has to fit in task RAM); anything else is a script.
2. The file goes on fd 11 (`SH_RUN_FD`), back at its start, and the arguments into a pipe on fd 10 (`SH_ARGS_FD`); the shell starts a task (`TASK_RUN`) at the loader, `SH_LOAD`, which inherits both.

3. The shell waits (`TASK_JOIN`): it hands the console to the new task if it has it (`CONS_SET_FG`), makes itself the task's parent, and pauses, as `TASK_START` does. The task's end wakes it, and `CONS_RELEASE` gives the console back (and the shell takes it back itself too, for a task that ended before it had it). It keeps the task's exit status (`status`, `/env/status`). A line ending in `&` isn't waited for: the shell prints the task's number (`[B]`), puts it in `/env/apid`, and goes on; HyForth's `wait` waits for it later (HyForth).
4. The loader, in the new task, reads the header and the code into place, clears the BSS, sets the MMU's page floor above it (or the header's top), closes fd 11, reads the arguments and the name from fd 10 into `HYX_ARGS` and `HYX_NAME` and closes it, and calls the entry point on ROM page 0 with `.A.Y` pointing to the arguments. When that returns, the loader returns, and the task ends (exit status 0).

A script (`.hys`) is run by HyForth itself: a copy of the shell's task (`TASK_CLONE`) reads it from fd 11 as `include` would, and ends at its end; the shell waits for it the same way.

The C Programmer's Guide

How to write programs for the Hydra-16 in C: setting up cc65, building and running a program, what a C program gets from the system, the library (the standard one and the Hydra's own), the console, files, processes and time, mixing in assembly, performance, debugging, and the limits to keep in mind. It's for programmers who know C but not the Hydra or cc65. Part of the [Programmer's Guide](#); [programs.md](#) has the executable format underneath.

Contents

1. Quick start
2. The machine, as C sees it
3. Building
4. A program's life: arguments, environment, exit status
5. Files and devices
6. The console: stdio and conio
7. Running other programs
8. Time
9. Tasks working together: semaphores
10. Errors
11. Memory
12. Assembly in a C program
13. Performance
14. Debugging and testing
15. Limits and gotchas
16. Reference: [hydra.h](#)
17. Working on the library

1. Quick start

What you need:

Tool	For	Where
cc65 (V2.19 or later)	The C compiler, assembler and linker	cc65.github.io : the Windows snapshot zip, or build it from source. The build looks in <code>CC65_HOME</code> , then on the <code>PATH</code> , then in <code>C:\source\cc65\win64_snapshot</code>
Node.js	The emulator and the card image tool	nodejs.org (no packages needed)
The ROM images	The emulator runs them	Built already in <code>os_rom/bin</code> , or build them (getting started)

Build the library (once, and after pulling changes to `programs/c/lib`). It builds `lib\hydra.lib` and the samples in `programs/c/samples`:

```
set CC65_HOME=C:\path\to\cc65      (if it isn't in the default place)
programs\c\make.bat
```

Write a program, `hello.c`:

```
#include <stdio.h>

int main (int argc, char* argv[])
{
    printf ("Hello from %s, with %d arguments\n", argv[0], argc - 1);
    return 0;
}
```

Build it into `programs\c\bin\hello.hyx`:

```
programs\c\hyc.bat hello.c
```

Run it in the emulator. Make a card image, put the program in its `bin` directory, and boot with the card (from `sim/`; in Git Bash, leave out a card path's first `/`):

```
node tools/hydrafs.js mkfs work.img 16 WORK
node tools/hydrafs.js mkdir work.img bin
node tools/hydrafs.js put work.img ../programs/c/bin/hello.hyx bin
node hydrasim.js -i --sd work.img
```

At HyForth's prompt, a program runs by its name:

```
0: /> hello a "b c"
Hello from hello, with 2 arguments
0: /> status .
0000
```

Ctrl-A x quits the emulator ([its keys](#)); `--speed 0` runs it as fast as the PC can.

On the real Hydra, the card is the same format: read the card into an image with a disk imager (Win32 Disk Imager, `dd`), put the program into the image, and write it back. Or put it on one card and `cp` it to another on the Hydra. The emulator is the quickest loop, and what the Hydra does with a program there it does on the board.

Not on Windows? The two `.bat` files run `build.js` (at the top of the repository), which works on any OS: `node build.js c` builds the library and the samples, and `node build.js prog hello.c` a program of your own (`programs/c/bin/hello.hyx`). By hand, the library is each file in `lib/crt`, `lib/io`, `lib/env`, `lib/conio`, `lib/snd` and `lib/sys`, assembled or compiled, and added (ar65 a) to a copy of cc65's `none.lib` as `lib/hydra.lib`; a program is then:

```
cc65 -t none --cpu 65C02 -O -I programs/c/include -o obj/hello.s hello.c
ca65 --cpu 65C02 -o obj/hello.o obj/hello.s
ld65 -C programs/c/hydra.cfg -o hello.hyx obj/hello.o programs/c/lib/hydra.lib
```

2. The machine, as C sees it

- **The CPU** is a W65C02S at 3.58 MHz. cc65's code runs roughly 10-20 times slower than careful assembly, so a C program on the Hydra is about as fast as compiled BASIC on a home computer of the 1980s: fine for tools, games and logic, slow for heavy number crunching ([performance](#)).
- **Types:** `int` and pointers are 16 bits, `long` is 32. `char` is **unsigned** (cc65's default; `-j` makes it signed). **No float or double** (cc65 says "Floating point type is currently unsupported"), and **no** `long long`: use integers, `long` or fixed point.

- **A task of its own.** A program runs in one of the 16 tasks, preemptively scheduled at 200 Hz alongside the shell, drivers and other programs. A read that has to wait (the keyboard, a pipe) puts the task to sleep; so does `sleep`. Nothing a program does can stop the others, short of turning interrupts off.
- **Its own 32K.** Each task has its own RAM from `$0000` to `$7FFF`, zero page and stack included: the hardware swaps it on a task switch. So a program's memory is its own, and a crash can't corrupt another task's.
- **No screen.** The console is a terminal on the serial port (ANSI, 9600 baud to start), so text I/O is the interface; `conio` drives it with ANSI sequences ([below](#)).
- **The OS,** Plan 9 style: everything is a file. The cards (`/sd/0/...`), devices (`/dev/cons`, `/dev/time`), the environment (`/env/NAME`) and the tasks (`/dev/proc`) are all read and written through file descriptors. The C library puts the standard calls on top of that.

A program's memory (the defaults in `programs/c/hydra.cfg`):

Addresses	What
<code>\$0000-\$00DF</code>	The OS's zero page: don't touch
<code>\$00E0-\$00F9</code>	cc65's runtime zero page (<code>sp</code> , <code>ptr1-ptr4</code> , <code>tmp1-tmp4</code> , <code>regbank</code> , ...)
<code>\$00FA-\$00FF</code>	Free: 6 bytes of zero page for your assembly
<code>\$0100-\$01FF</code>	The 6502 stack (return addresses; the C stack is separate). At the start, <code>\$0110</code> has the arguments and <code>\$0150</code> the name (copied by the start-up code)
<code>\$0200-\$07FF</code>	The OS's per-task buffers (<code>stdout</code> 's at <code>\$0700</code> , <code>stdin</code> 's read-ahead at <code>\$0780</code>): leave them alone
<code>\$0800-...</code>	The program: code, read-only data, data, then the BSS (cleared at the start)
<code>...-\$67FF</code>	The heap (<code>malloc</code>): from the BSS's end up to the stack
<code>\$6800-\$6FFF</code>	The C stack (2K, <code>__STACKSIZE__</code>), growing down from <code>\$7000</code> (<code>__RAMTOP__</code>)
<code>\$7000-\$7CFF</code>	The MMU's pages (<code>MM_ALLOC</code>)
<code>\$7D00-\$7FFF</code>	The task system page (the fd table, IRQ tables) and the MMU's state
<code>\$8000-\$9FFF</code>	An 8K window: the task's own RAM banks, when it maps one (memory)
<code>\$A000-\$FFFF</code>	ROM and I/O

So code, data, heap and stack share 26K. A small program (`hello.c`: `printf`, `strings`, `time`) is about 8K, which leaves some 16K of heap; a program with only `puts` has over 20K.

3. Building

Script	Does
<code>programs\c\make.bat</code>	Builds <code>lib\hydra.lib</code> (cc65's <code>none.lib</code> with the Hydra's modules added or in place of cc65's) and every <code>samples*.c</code> into <code>bin\</code>
<code>programs\c\hyc.bat</code> <code>FILE.c</code> [<code>MORE.c ...</code>] [<code>MORE.s ...</code>]	Builds one program, <code>bin\FILE.hyx</code> (named after the first file), and its map, <code>bin\FILE.map</code>

`hyc.bat` compiles each `.c` with `cc65 -t none --cpu 65C02 -O` and the include path `programs\c\include` (for `hydra.h`), assembles each `.s` with `ca65` (with `programs\c\lib` on its include path, for `hydra.inc`), and links with `hydra.cfg` and `lib\hydra.lib`. Give each file its own name: `prog.c` and `prog.s` would both make `obj\prog.o`.

Options go in two environment variables:

Variable	Goes to	E.g.
<code>HYC_CFLAGS</code>	cc65	<code>--check-stack</code> (stop on a stack overflow: below); <code>-Oi</code> (inline more), <code>-Or</code> (register variables), <code>-Os</code> (inline some library functions); <code>-DDEBUG=1</code>
<code>HYC_LDFLAGS</code>	ld65	<code>-D __STACKSIZE__=\$0400</code> (a 1K C stack); <code>-D __RAMTOP__=\$7C00</code> (all of task RAM: see memory)

```
set HYC_CFLAGS=--check-stack -Or
programs\c\hyc.bat game.c map.c sound.s
```

The **map file** (`bin\NAME.map`) lists each segment's size and where every function and variable went: the place to look when a program gets too big, and to find an address for the debugger.

The **executable** (`.hyx`): a 16-byte header, then the code and data, loaded at `$0800`. The header also tells the loader how big the BSS is (it clears it, so it isn't in the file) and where the program's RAM ends (the MMU's pages start above it). The start-up code (`lib/crt/crt0.s`) fills all of that in; there's nothing to set by hand. The format is in [programs.md](#).

4. A program's life: arguments, environment, exit status

Running it. HyForth runs a program by name: a word it doesn't know is looked for as `name.hyx` (then `name.hys`, a script, and `name.zsm`, a song) in the current directory, then in `/bin` (a union in the namespace of the program caches, the boot card's `/bin` and the ROM's: [io.md](#)), then in the directories of `$PATH`. `run file.hyx args` runs one by its file name. **HyForth's own words come first:** a program called `fg`, `ls` or `wait` never runs by name (use `run`, or another name; `words` lists them).

What it starts with:

- **argc and argv.** `argv[0]` is the name as it was typed (`hello`, `bin/hello.hyx`). The rest of the line is split at spaces; `"two words"` is one argument (without the quotes). At most 15 arguments, and 63 characters of line.
- **stdin, stdout and stderr** (fds 0-2): the shell's, usually the console. `prog < in.txt`, `prog > out.txt`, `prog >> log.txt` and pipelines (`a | b`) change them before the program starts.
- **The current directory, namespace and environment:** copies of the shell's. What the program changes (`chdir`, `setenv`) is its own, and passes to the programs it starts.
- **The console**, if the shell had it: the program's reads get the keys, and Ctrl-C stops it.

Ending. `main` returns, or the program calls `exit` or `hy_exits`. The functions registered with `atexit` run, the files are closed (buffered output goes out), and the memory is freed. The **exit status**, as Plan 9 has it, is a code (0-255) and, optionally, a message:

How it ends	Code	Message
<code>return n; from main, exit (n)</code>	<code>n</code> (its low byte)	none
<code>hy_exits ("bad input")</code>	1	bad input (30 characters at most)
<code>hy_exits (NULL), hy_exits ("")</code>	0	none
Ctrl-C (a break)	130	interrupt
Killed (Ctrl-\\, <code>kill</code> , its shell's Ctrl-C)	137	killed
A stack overflow, built with <code>--check-stack</code>	4	none

atexit functions don't run on Ctrl-C or a kill: the task just ends (its files are still closed, and output written with `write` or `printf` is never lost). Don't rely on an `atexit` function to save state.

The shell keeps the status of each program it waits for. HyForth's `status` gives the code, and `/env/status` has it as text: the message, the code if there's no message, or nothing for 0 (Plan 9's `$status`). So scripts can check a step:

```
0:./> mytool in.txt
0:./> status .
0001
0:./> cat /env/status
bad input
```

Use 0 for success, 1 with a message for a failure, and 2 for a usage error, as Unix tools do.

The background: `prog &` starts it without waiting: the shell prints its task (`[B]`), puts it in `/env/apid`, and gives the prompt back. A background program waits if it reads or writes the console, until it's brought to the front (`fg`, `Ctrl-]` and its task) or waited for (`$B wait`: task numbers are hex). `Ctrl-C` at the prompt kills the programs the shell started, the background ones too (the console keys).

5. Files and devices

Names. A name is relative to the current directory unless it starts with `/`. The namespace:

Path	What
<code>/sd/0/...</code> , <code>/sd/1/...</code>	The files on SD card 0, 1, ... (HydraFS). The shell's <code>0:/</code> prompt is <code>/sd/0</code>
<code>/dev/cons</code>	The console
<code>/dev/null</code> , <code>/dev/zero</code>	As on Unix
<code>/dev/time</code>	The clock, as text: <code>2026-09-30 14:05:00</code> ; write one to set it
<code>/dev/spi/N</code>	SPI device N (<code>0-f</code>): write a transaction, read what came back (io.md)
<code>/dev/gpio/N</code> , <code>/dev/gpio/port</code>	Port A's pins: <code>"0"</code> / <code>"1"</code> each, or the 8 as a byte; <code>ctl</code> for directions, <code>ca</code> to wait for an edge (io.md)
<code>/rom/...</code> , <code>/ram/...</code> , <code>/sram/...</code>	The ROM's files; the shell's own area of the RAM disk; the shared RAM disk
<code>/bin</code> , <code>/lib</code>	The programs and libraries: unions of <code>/ram</code> 's, <code>/sram</code> 's, the card's and <code>/rom</code> 's
<code>/proc/N/...</code>	Task N: its <code>status</code> , <code>cwd</code> , <code>env</code> , <code>pages</code> , <code>ns</code> ; and for a task this program started (or itself), its memory: <code>mem</code> (its addresses) and <code>ram</code> (its banks)
<code>/dev/snd</code>	The YM2151 sound chip (below)
<code>/env/NAME</code>	An environment variable (below)

Size buffers for names with `HY_NAME_MAX` **and** `HY_PATH_MAX`, **not** `FILENAME_MAX`. A file's name is up to 31 characters (`HY_NAME_MAX`, 32 with its 0), and a path up to 64 (`HY_PATH_MAX`, 65); cc65's `FILENAME_MAX` is 17 for this target, so a buffer sized with it cuts names short. `/` and `/dev` aren't directories you can list; a card's directories are. See io.md for the devices in full.

What works:

Header	Calls
<code>stdio.h</code>	<code>fopen</code> (<code>r</code> , <code>w</code> , <code>a</code> , <code>r+</code> , <code>w+</code> , <code>a+</code> ; there's no text mode: bytes are bytes), <code>fread</code> , <code>fwrite</code> , <code>fgets</code> , <code>fputs</code> , <code>fprintf</code> , <code>fscanf</code> , <code>fseek</code> , <code>ftell</code> , <code>rewind</code> , <code>fflush</code> , <code>feof</code> , <code>ferror</code> , <code>fclose</code> , <code>remove</code> , <code>rename</code> , <code>perror</code>
<code>fcntl.h</code> , <code>unistd.h</code>	<code>open</code> (<code>O_RDONLY</code> , <code>O_WRONLY</code> , <code>O_RDWR</code> , <code>O_CREAT</code> , <code>O_TRUNC</code> , <code>O_APPEND</code> , <code>O_EXCL</code>), <code>read</code> , <code>write</code> , <code>lseek</code> , <code>close</code> , <code>chdir</code> , <code>getcwd</code> , <code>rmdir</code> , <code>isatty</code>
<code>sys/stat.h</code>	<code>stat</code> , <code>fstat</code> (<code>st_size</code> , <code>st_mode</code> , <code>st_mtime</code>), <code>mkdir</code>
<code>dirent.h</code>	<code>opendir</code> , <code>readdir</code> , <code>closedir</code> , <code>rewinddir</code> , <code>telldir</code> , <code>seekdir</code>

Listing a directory, with each entry's size and kind (`hy_dirstat` gives the entry's `stat` without opening it):

```

#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <hydra.h>

void ls (const char* path)
{
    DIR* d = opendir (path);
    struct dirent* e;
    struct stat st;

    if (d == 0) {
        perror (path);
        return;
    }
    while ((e = readdir (d)) != 0) {
        hy_dirstat (d, &st);
        printf ("%s %8ld%s\n", e->d_name, st.st_size, S_ISDIR (st.st_mode) ? " dir" : "");
    }
    closedir (d);
}

```

Details:

- `st_mode` has `S_IREAD`, `S_IWRITE` (the file isn't read-only) and `S_IFDIR` (`S_ISDIR`, in `hydra.h`); `st_mtime` is the time of the last change, from HydraFS's stamps. Devices give zeros.
- `rename` renames a file within its directory; it can't move one to another directory yet.
- `mkdir`'s mode is ignored.
- A task has 12 fds; `stdin`, `stdout` and `stderr` take 3. An open directory takes one.
- Writes to a card go straight through to it (there's a one-block cache for reading), so a `fclose` is all it takes to be safe.

The environment is Plan 9's: each variable is a file, `/env/NAME`. `getenv` reads one, `setenv`, `putenv` and `unsetenv` change it. A name is 1-30 characters (no `/` or `=`); a value up to 255 (`HY_ENV_MAX`); a task's variables share 256 bytes. `getenv` returns a static buffer, overwritten by the next `getenv`: copy a value to keep it.

```

char* home = getenv ("HOME");          /* NULL if it isn't set */
setenv ("MODE", "fast", 1);           /* This program's, and the programs it starts */

```

The shell uses `PATH`, `LIBPATH` and `HOME`, and sets `status` and `apid`. `conio` uses `COLUMNS` and `LINES`. Reading `/env/NAME` with `fopen` works as well.

Sound: `snd.h`

The YM2151 has 8 channels (0-7), each a voice of 4 FM operators. `snd.h` plays them through `/dev/snd` and the ROM's sound library (`io.md`): a channel plays a **patch** (0-127 are General MIDI's instruments, 128-162 drum and percussion sounds) at a **MIDI note** (60 is middle C), with its own volume, speakers and bend.

Call	Does
<code>snd_claim (mask),</code> <code>snd_release (mask)</code>	The channels (bit n: channel n) this program's alone: other programs' writes to them are dropped. <code>EBUSY</code> if another has one. Given back when the program ends

Call	Does
<code>snd_patch (ch, p)</code>	Load patch p
<code>snd_note (ch, n),</code> <code>snd_off (ch)</code>	Key a MIDI note on; key off (the note's release)
<code>snd_vol (ch, v),</code> <code>snd_volume (v)</code>	A channel's volume, the master volume (0-127)
<code>snd_pan (ch,</code> <code>SND_PAN_LEFT _RIGHT </code> <code>_BOTH)</code>	Its speakers
<code>snd_bend (ch, b)</code>	Its bend, in 64ths of a semitone (-128 to 127)
<code>snd_drum (ch, n)</code>	A General MIDI drum (35-36 kick, 38 snare, 42 closed hi-hat, 46 open hi-hat, 49 crash ...)
<code>snd_write (reg, val),</code> <code>snd_writes (pairs, n)</code>	The chip's own registers (the YM2151's datasheet), one or n at a time
<code>snd_regs (buf)</code>	All 256 registers, as written
<code>snd_reset ()</code>	Clear the chip and the settings
<code>snd_play (song, loops)</code>	Play a song (a ZSM file: the Commander X16's format, which the Furnace tracker exports) in the ROM's player, a task of its own: its task, at once. <code>loops</code> : its loop that many more times (0: the song once; <code>SND_FOREVER</code>). <code>hy_wait</code> waits for it, <code>hy_kill</code> stops it

```

#include <hydra.h>
#include <snd.h>

int main (void)
{
    static const unsigned char tune[] = { 60, 64, 67, 72 };
    unsigned char i;

    if (snd_claim (1) < 0) {                /* Channel 0 */
        hy_exits ("sound busy");
    }
    snd_patch (0, 73);                      /* A flute */
    for (i = 0; i < sizeof tune; ++i) {
        snd_note (0, tune[i]);
        hy_sleep_ticks (40);                /* 0.2 s */
        snd_off (0);
    }
    return 0;
}

```

Each call is one request to the sound driver (a couple of thousand cycles), fine for music at human speeds. For music, a song is easier: `snd_play ("background.zsm", SND_FOREVER)` plays it while the program goes on (a game's sound effects can use the channels the song doesn't), and `hy_kill` on its task stops it; `samples/jukebox.c` shows how. Songs come from the Furnace tracker (it exports ZSM) or from a score compiled on the PC (`sim/tools/hysong.js`: [the emulator's tools](#)). For timing, sleep between events (`hy_sleep_ticks`: 5 ms steps), and measure with `hy_ticks` so delays don't add up. `samples/tones.c` shows the rest.

6. The console: stdio and conio

stdio. `printf` and `puts` go to `stdout`, and `fgets`, `scanf` and `getchar` read `stdin`, as on Unix. When `stdin` or `stdout` is the console, the library acts as a Unix terminal driver does:

- **Output:** each LF (`\n`) goes out as CR LF, so lines start at the left on a real terminal. (Files and pipes get the bytes as they are.)
- **Input:** the terminal's Enter key (CR) comes to the program as `\n`, so `fgets` and `scanf` see ends of line; the console echoes each key as it's typed.
- **Backspace:** the console erases the character on the screen, but the program still gets the backspace (`\b`, 8): `fgets` keeps it in the line. For typed input that can be corrected, read a line yourself:

```
/* A line from stdin, Backspace handled: its length, or -1 at the end of input */
int readline (char* buf, int size)
{
    int n = 0;
    char c;

    for (;;) {
        if (read (0, &c, 1) != 1) {
            return n ? n : -1;          /* Ctrl-D, or the end of a file or pipe */
        }
        if (c == '\n') {
            break;
        }
        if (c == '\b' || c == 127) {     /* (Erased on the screen already) */
            if (n) {
                --n;
            }
        } else if (n < size - 1) {
            buf[n++] = c;
        }
    }
    buf[n] = 0;
    return n;
}
```

- **The end of input** is Ctrl-D (or Ctrl-Z) at the keyboard: `fgets` returns `NULL`, `read` returns 0.
- A program in the background waits for the console, and a program started from a script with its input redirected reads the file. `isatty (0)` says whether `stdin` is the console.

Filters work as on Unix: read `stdin` to its end, write `stdout`. `samples/upper.c` is one: `hello | upper`, `upper < notes.txt > NOTES.TXT`.

conio (`#include <conio.h>`) is cc65's console library, here on an ANSI terminal:

Call	Does
<code>clrscr</code> , <code>gotoxy</code> , <code>gotox</code> , <code>gotoy</code> , <code>wherex</code> , <code>wherey</code>	Clear the screen, move the cursor, where it is (as far as conio's own output goes: <code>printf</code> doesn't move it)
<code>cputc</code> , <code>cputs</code> , <code>cprintf</code> , <code>cputcxy</code> , <code>cputsxy</code>	Output (<code>\n</code> is a new line)

Call	Does
<code>textcolor</code> , <code>bgcolor</code>	Colours: <code>COLOR_BLACK</code> ... <code>COLOR_WHITE</code> (0-7), and the bright ones, <code>COLOR_GRAY</code> ... <code>COLOR_BRIGHTWHITE</code> (8-15) in <code>hydra.h</code>
<code>revers</code> , <code>cursor</code>	Reverse video; show or hide the cursor
<code>chline</code> , <code>cvline</code> , <code>cclear</code> (and their <code>xy</code> forms)	Lines (- and), blanks
<code>screensize</code>	<code>\$COLUMNS</code> x <code>\$LINES</code> , or 80 x 24
<code>cgetc</code> , <code>kbhit</code>	A key, raw: as it's typed, with no echo; whether one's waiting

Keys: conio switches the console to raw (writing `rawon` to `/dev/cons/ctl`) the first time it reads one, so keys come as they're typed, unechoed, and Ctrl-D is a key like any other. The terminal's cursor, editing and function keys come as single codes: `CH_CURS_UP`, `CH_CURS_DOWN`, `CH_CURS_LEFT`, `CH_CURS_RIGHT`, `CH_HOME`, `CH_END`, `CH_PAGE_UP`, `CH_PAGE_DOWN`, `CH_INS`, `CH_DELETE`, `CH_F1-CH_F4` (in `hydra.h`); Enter is `CH_ENTER` (13), Esc `CH_ESC`, Backspace `CH_DEL` (127, on most terminals) or 8. The console goes back to normal when the program ends, however it ends. Ctrl-C still stops the program.

```
#include <conio.h>
#include <hydra.h>

int main (void)
{
    unsigned char w, h, x = 40, y = 12;
    char c;

    screensize (&w, &h);
    clrscr ();
    textcolor (COLOR_YELLOW);
    cputsxy (0, 0, "Arrow keys move the *, q quits");
    textcolor (COLOR_WHITE);
    for (;;) {
        cputcxy (x, y, '*');
        c = cgetc ();
        cputcxy (x, y, ' ');
        if (c == 'q') {
            break;
        }
        switch (c) {
            case CH_CURS_UP:    if (y > 1) --y;    break;
            case CH_CURS_DOWN:  if (y < h - 1) ++y; break;
            case CH_CURS_LEFT:  if (x > 0) --x;    break;
            case CH_CURS_RIGHT: if (x < w - 1) ++x; break;
        }
    }
    clrscr ();
    return 0;
}
```

`conio` and `printf` can be mixed: their output stays in order. `samples/keys.c` shows the keys' codes.

7. Running other programs

A command line runs as it would at the prompt, in a **command shell** (HyForth reading the line from a pipe, with no banner or prompt: Plan 9's `rc -c`), in a task of its own, with this program's fds, current directory and environment. So it can be a program, a script, a pipeline or a HyForth word.

Call	Does
<code>system (cmd)</code>	Runs it and waits: its exit code (0-255), or -1. <code>system (NULL)</code> is 1: there's a shell
<code>hy_spawn (cmd)</code>	Starts it and returns at once: its task (1-15), or -1
<code>hy_wait (task, msg)</code>	Waits for a task this program started: its exit code, and its message into <code>msg</code> (HY_STATUS_MAX bytes; NULL if not wanted). While it waits, the task has the console, if this program has it

```
#include <stdio.h>
#include <stdlib.h>
#include <hydra.h>

int main (void)
{
    char msg[HY_STATUS_MAX];
    int task, code;

    if (system ("ls bin") != 0) {
        return 1;
    }
    task = hy_spawn ("mytool in.txt > out.txt");
    /* ... other work while it runs ... */
    code = hy_wait (task, msg);
    if (code != 0) {
        printf ("mytool failed: %d %s\n", code, msg);
    }
    return code;
}
```

A command line is at most 250 characters. Its output comes out wherever this program's goes (redirect it in the line: `"prog > file"`). A spawned task that's never waited for runs to its end on its own.

8. Time

Call	Gives
<code>time</code> , <code>localtime</code> , <code>strftime</code> , <code>mktime</code> , <code>difftime</code>	The Hydra's clock, to the second, as local time (the Hydra keeps local time; cc65's time zone is UTC, so <code>localtime</code> shows it as it is). Set it by writing <code>/dev/time</code> (<code>echo 2026-09-30 14:05 > /dev/time</code>)
<code>clock</code>	Ticks since the program started: <code>CLOCKS_PER_SEC</code> is 200. It keeps counting past the tick counter's wrap (5.5 minutes) as long as it's called at least that often
<code>sleep (s)</code>	Sleeps <code>s</code> seconds: other tasks run meanwhile
<code>hy_ticks ()</code> , <code>hy_sleep_ticks (n)</code>	The scheduler's 16-bit tick count (200 a second; it wraps), and a sleep of <code>n</code> ticks (up to 32767)
<code>hy_clock ()</code>	The clock as seconds since 2000-01-01 00:00:00

Call	Gives
hy_yield ()	Gives up the rest of this tick to the other tasks

Don't busy-wait. A loop that polls something (a key, the time) should `hy_sleep_ticks (1)` or `hy_yield ()` each time round: otherwise it takes the CPU from every other task, the shell included.

```
#include <stdio.h>
#include <time.h>

int main (void)
{
    time_t now = time (0);
    char when[32];

    strftime (when, sizeof when, "%Y-%m-%d %H:%M:%S", localtime (&now));
    printf ("%s\n", when);
    return 0;
}
```

9. Tasks working together: semaphores

The Hydra has 16 semaphores for the whole system, known by number (1-16), so one program can make a semaphore and hand its number to the programs it starts (as an argument or in the environment). A **mutex** is a semaphore of 1 that only the task holding it can release.

Call	Does
hy_sem_new (n)	A semaphore of n (0-255): n acquires before one has to wait. Its number, or -1 (EAGAIN: all 16 in use)
hy_mutex_new ()	A mutex
hy_sem_acquire (s)	Take one, waiting (using no CPU) until there is one. Ctrl-C ends the wait
hy_sem_try (s)	Take one if there is one (1), or 0 at once
hy_sem_release (s)	Give one back (a mutex: only its holder can)
hy_sem_free (s)	Free it: tasks waiting for it get -1

When a program ends, however it ends, the semaphores it made are freed and the mutexes it holds are released, so a crash can't leave one held. Semaphores are for coordinating tasks (one waits for another's step, or they take turns at a device or a file); a program's tasks don't share memory, since each has its own RAM.

```
int s = hy_sem_new (0);                /* Nothing to take yet */
char cmd[40];

sprintf (cmd, "worker %d", s);         /* The worker releases s when it's ready */
hy_spawn (cmd);
hy_sem_acquire (s);                    /* Wait for it */
```

10. Errors

A failed library call returns -1 (or `NULL`) and sets `errno`, as usual; `perror` and `strerror` describe it. `_oserror` (in `errno.h`) keeps the Hydra's own error code ([the list](#)), which says more, and `_stroserror` (`_oserror`) (`string.h`) gives it as text, as

HyForth's !IO ERR! does ("not found", "disk full" ...; _poserror ("prog") prints it after prog:):

The Hydra's error	errno
ERR_IO_NOT_FOUND (\$70)	ENOENT
ERR_IO_BAD_FD (\$71)	EBADF
ERR_IO_MODE (\$72), ERR_IO_NOT_EMPTY (\$83), ERR_IO_IS_DIR (\$86), ERR_SEM_NOT_HELD (\$63)	EACCES
ERR_IO_WOULD_BLOCK (\$73), ERR_SEM_NONE (\$61), ERR_SEM_BUSY (\$62)	EAGAIN
ERR_IO_NO_FDS (\$75)	EMFILE
ERR_IO_NAME (\$77), ERR_IO_NOT_DIR (\$85), ERR_SEM_BAD (\$60), ERR_MEM_*	EINVAL
ERR_IO_BAD_REQ (\$78)	ENOSYS
ERR_IO_DEVICE (\$79), ERR_IO_NOT_READY (\$7E), ERR_IO_NOT_FS (\$80)	ENODEV
ERR_IO_BROKEN (\$7A), ERR_IO_MEDIA (\$7F)	EIO
ERR_IO_FULL (\$81)	ENOSPC
ERR_IO_EXISTS (\$82)	EEXIST
ERR_IO_BUSY (\$84)	EBUSY
ERR_IO_NOT_EXEC (\$87)	ENOEXEC
ERR_OUT_OF_MEMORY (\$02)	ENOMEM
ERR_SEM_FULL (\$64)	ERANGE
Anything else	EUNKNOWN

```
FILE* f = fopen ("data/scores", "r");

if (f == 0) {
    perror ("data/scores");           /* data/scores: No such file or directory */
    hy_exits ("no scores");
}
```

11. Memory

- **The heap** is malloc, calloc, realloc and free, between the program's BSS and its stack. _heapmemavail () gives the free bytes, _heapmaxavail () the biggest block. malloc returns NULL when it's out: check it.
- **The C stack** (locals, arguments) is 2K by default. cc65 doesn't check it unless asked: a stack that overflows runs into the heap and corrupts it, with confusing results. Build with HYC_CFLAGS=--check-stack while developing, and an overflow ends the program with exit status 4. Large locals (char buf[512]) and deep recursion are what fill it: make big buffers static, or malloc them. A different size: HYC_LDFLAGS=-D __STACKSIZE__=\$0C00.
- **More room:** -D __RAMTOP__=\$7C00 gives the program all of its task RAM (29K), but then MM_ALLOC has no task RAM pages left to hand out (bank memory, below, still works).
- **Bank memory:** a task can have up to 16 banks of 8K per installed RAM module (typically well over 100K), seen one at a time at \$8000-\$9FFF. The MMU hands them out (MM_ALLOC with AI_PAGED, then MM_LOCK to map one: [memory.md](#)). The library has no C calls for them yet; reach them with a little assembly ([below](#)). Two rules: the window only shows the bank while it's mapped, and **never hand the IO calls a buffer in the window** (the IO layer maps its transfer area there): copy through a buffer below \$7000.

12. Assembly in a C program

Put the assembly in a `.s` file and name it on `hyc.bat`'s line after the C files. `lib/hydra.inc` has the OS's calls (the `$F8xx` thunks), their zero-page parameters and constants.

cc65's calling convention (`__fastcall__`, the default):

- The last argument is in `.A` (low) and `.X` (high); the others are on the C stack, which `popa` (a byte) and `popax` (a word) take off, last first. A `long` argument or result has its high word in `sreg`.
- The result is in `.A` and `.X` (an `int`: set `.X`, even for a `char`).
- A function may change `.A`, `.X`, `.Y`, `ptr1`-`ptr4`, `tmp1`-`tmp4`, `sreg` and `regsave`; it must leave `sp` (the C stack pointer) and `regbank` as they were.
- Its C name gets an underscore: `getkey` is `_getkey`.

The OS's calls keep their own conventions ([the API index](#)): registers in, `C = 1` for an error with the code in `.A`. They're callable from any task.

A non-blocking key from stdin (`READ_CHAR`: `C = 1` with a key in `.A`):

```
; getkey.s - int getkey (void): a key from stdin if one has been typed, or -1, without waiting
```

```
.export      _getkey
#include     "hydra.inc"

.code

_getkey:
    jsr      READ_CHAR          ; C = 1: .A = the key
    bcc      @none
    ldx      #0                 ; (An int: .X is the high byte)
    rts
@none:
    lda      #$FF               ; -1
    tax
    rts
```

```
int getkey (void);                /* getkey.s */

while ((k = getkey ()) < 0) {
    hy_sleep_ticks (2);           /* (Don't spin: let the others run) */
}
```

```
programs\c\hyc.bat game.c getkey.s
```

`lib/sys/hydra.s` and `lib/io/fileio.s` have more examples: calls with arguments on the C stack, returning a `long`, setting `errno` on an error (`jmp __mappederrno` with the OS's error in `.A`).

13. Performance

The CPU does about 3.6 million simple operations a second, and cc65's code isn't dense or fast, so it pays to know what costs:

- **Printing** is limited by the serial line first: at 9600 baud a character takes 1 ms, so 100 lines of 20 characters take 2 s whatever the program does. After that, formatting: a `printf` of a short line costs about 4 ms of CPU, `sprintf` and one `write` about 2.5 ms (each `write` is an IO request, about 2,000 cycles). Faster lines: `stty` a higher rate on the Hydra and the terminal (HyForth's "b19200" `stty`).
- **Types:** `unsigned char` is the fastest type, then `int`; `long` arithmetic is several times slower, and multiply and divide are subroutines. Count loops with `unsigned char` when they're under 256.
- **Locals and arguments** live on the C stack and are reached through a pointer: statics are faster (`static` locals, or `-C1` to make every local static, if nothing is recursive). `-Or` puts `register` variables in the zero page.
- **Arrays of structs:** indexing multiplies; walking a pointer is cheaper.
- **IO:** each `read` or `write` is a request to a server task (about 2,000 cycles plus the data); read and write in blocks, not a byte at a time. `fgetc` and `fputc` are one request per byte.
- **Inner loops** that matter go into assembly ([above](#)).

14. Debugging and testing

- `printf` to `stderr` (`fprintf (stderr, ...)`): it goes to the console even when `stdout` is redirected.
- **The exit status** says how a program ended: 130 (Ctrl-C), 137 (killed), 4 (a stack overflow, with `--check-stack`).
- **The emulator** runs the same ROM and program, with tools the board hasn't got ([emulator.md](#)):
 - scripted runs: `--input` types a line (`'\w\w\wmyprog a b\r'`: `\w` waits for the boot) and `--cycles` sets how long, so a run can be repeated exactly;
 - `--watch ADDR@TASK` reports every write to an address in a task's RAM (find a variable's address in `bin\NAME.map`), `--dump ADDR:LEN@TASK` shows memory after the run, `--pc ADDR` the registers each time the PC gets there, `--trace N` the last instructions;
 - a program runs in the highest free task (usually B): `ps` shows it, and `hy_task ()` tells the program.
- **Tests:** `sim/regress.js` boots the emulator with a card image and checks the output. Its `c-programs` test runs the samples (`C_SAMPLES`); a test for a program of your own follows the same pattern ([emulator.md](#)).

15. Limits and gotchas

No float, double or long long	Integers, long, fixed point
<code>char</code> is unsigned	Compare with (<code>signed char</code>) casts, or build with <code>-j</code>
26K for code, data, heap and stack	Watch the map file; <code>malloc</code> returns <code>NULL</code> when it's out
A 2K C stack, unchecked	<code>--check-stack</code> while developing; big buffers <code>static</code>
15 arguments, 63 characters of line	Longer input: a file, or <code>stdin</code>
Names 31 characters, paths 64	<code>HY_NAME_MAX</code> , <code>HY_PATH_MAX</code> (not <code>FILENAME_MAX</code> , 17)
12 fds a task, 3 used	Close what you open
<code>rename</code> within a directory	Copy and remove to move a file
<code>getenv</code> 's buffer is shared	Copy the value before the next <code>getenv</code>
<code>atexit</code> functions don't run on Ctrl-C or a kill	Don't save state in them
HyForth's words shadow program names	<code>run prog.hyx</code> , or another name
A function's locals: 256 bytes at most ("Too many local variables")	Make big arrays <code>static</code> , or <code>malloc</code> them
Backspace reaches <code>fgets</code>	A <code>readline</code> of your own (above)
<code>/</code> and <code>/dev</code> can't be listed	List a card: <code>/sd/0</code> , <code>.</code>
No signals, no <code>fork</code>	<code>hy_spawn</code> , <code>system</code> ; Ctrl-C ends the program
No threads	Tasks (<code>hy_spawn</code>) and semaphores

A buffer in \$8000-\$9FFF can't go to IO

Copy it below \$7000 first

16. Reference: hydra.h

Call or constant	Does
HY_PATH_MAX (65), HY_NAME_MAX (32)	A path's and a name's buffer, with the 0
HY_ENV_MAX (255), HY_STATUS_MAX (31), HY_STAT_SIZE (48)	A variable's value; an exit message's buffer; the OS's stat record
HY_TICKS_PER_SEC (200), CLOCKS_PER_SEC	The scheduler's tick
unsigned hy_ticks (void)	The tick count (it wraps)
void hy_sleep_ticks (unsigned ticks)	Sleep, up to 32767 ticks
void hy_yield (void)	Let the other tasks run
unsigned long hy_clock (void)	The clock: seconds since 2000-01-01
unsigned char hy_task (void)	This program's task (1-15)
int hy_spawn (const char* cmd)	Start a command line: its task, or -1
int hy_wait (int task, char* msg)	Wait for a task: its code, and its message
int hy_kill (int task)	End a task and the tasks it started (its status: 137)
int hy_bind (new, old, flags), int hy_mount (dev, old, flags, spec)	The namespace, as Plan 9's bind and mount: names under old stand for names under new (or are device dev's, such as "zero"; spec: NULL, or what the server serves there, as Plan 9's, hy_mount ("hfs", "/a", HY_MREPL, "r")). flags: HY_MREPL (replace old's entries), HY_MBEFORE, HY_MAFTER (a union's member, before or after the others), HY_MCREATE (files made in the union go to it). 0, or -1
int hy_unmount (new, old), int hy_hide (path)	Take old's member new out (NULL: all of old's entries); make nothing under path found. 0, or -1
int hy_newns (void)	A fresh namespace, as Plan 9's newns: the program's own entries go, but its /ram, so it sees the system namespace. 0, or -1
void hy_exits (const char* msg)	End with a message (code 1), or success (NULL, "")
int hy_sem_new (unsigned char count), int hy_mutex_new (void)	A semaphore (1-16), or -1
int hy_sem_acquire (s), hy_sem_try (s), hy_sem_release (s), hy_sem_free (s)	Semaphores
int fstat (int fd, struct stat* st)	stat for an open file
int hy_dirstat (DIR* dir, struct stat* st)	The last readdir entry, as stat
int isatty (int fd)	Is it the console?

Call or constant	Does
<code>int setenv (name, value, overwrite), int unsetenv (name)</code>	The environment
<code>S_IFDIR, S_ISDIR (m)</code>	A directory, in <code>st_mode</code>
<code>COLOR_BLACK ... COLOR_BRIGHTWHITE</code>	conio's 16 colours
<code>CH_ENTER, CH_ESC, CH_DEL, CH_CURS_UP ... CH_F4</code>	<code>cgetc</code> 's keys
<code>CH_HLINE, CH_VLINE</code>	<code>chline</code> 's and <code>cvline</code> 's characters

17. Working on the library

programs/c/:

Path	What
<code>hydra.cfg</code>	The linker config: the header, the memory, <code>__STACKSIZE__</code> , <code>__RAMTOP__</code> , the zero page
<code>include/hydra.h</code> , <code>include/snd.h</code>	The Hydra's own calls and constants; the sound chip's
<code>lib/hydra.inc</code>	The OS's calls, zero page and constants for the library's assembly. The <code>c-programs</code> test checks each name against the ROM's build, so keep it in step with <code>os_rom/include</code>
<code>lib/crt/</code>	<code>crt0.s</code> (the header, start-up and <code>exit</code>), <code>mainargs.s</code> (<code>argc</code> , <code>argv</code>)
<code>lib/io/</code>	Files: <code>fileio.s</code> (the raw IO calls), <code>read.c</code> and <code>write.c</code> (the console's line ends), <code>open.c</code> , <code>lseek.c</code> , <code>stat.c</code> , <code>dirent.c</code> , <code>isatty.c</code> , <code>sysfile.s</code> (remove, rename, mkdir, rmdir), <code>_cwd.s</code> , <code>oserror.s</code> (errors to <code>errno</code>), <code>oserrlist.s</code> (the errors as text: <code>_stroserror</code>)
<code>lib/env/</code>	<code>getenv.c</code> , <code>putenv.c</code>
<code>lib/conio/</code>	<code>conio.c</code> (ANSI output, keys), <code>conglue.s</code> (the entry points cc65's own conio code calls), <code>cursor.c</code>
<code>lib/snd/</code>	<code>snd.c</code> : <code>snd.h</code> over <code>/dev/snd</code> ; <code>sndplay.c</code> : <code>snd_play</code>
<code>lib/sys/</code>	<code>hydra.s</code> (tasks, semaphores, ticks, sleep), <code>system.c</code> , <code>gettime.c</code> , <code>clock.s</code>
<code>samples/</code>	<code>hello.c</code> , <code>upper.c</code> , <code>code.c</code> , <code>keys.c</code> , <code>tones.c</code> , <code>jukebox.c</code> , <code>ctest.c</code> (the library's own test)

- **A module replaces cc65's module of the same file name** (`make.bat` adds the objects to a copy of `none.lib`): `getenv.c` makes `getenv.o`, which takes the place of cc65's. Name a new module after the cc65 module it replaces, or something cc65 doesn't have (`ar65 t lib\hydra.lib` lists them). Two files with the same base name (`conio.c` and `conio.s`) would make the same object.
- **cc65's common code calls some routines expecting `ptr1-ptr3` and `tmp1` to survive** (conio's `cputs` and `cprintf` call `cputc` and `gotoxy`): a C version of one of those needs an assembly wrapper that keeps them, as `conglue.s` does.
- **A new call in the ROM** gets a line in `lib/hydra.inc` (the test checks it) and, for C, a function in `lib/sys/hydra.s` and a prototype in `hydra.h`.
- **Test it:** `ctest.c` checks the library from the inside (its last line is `N failed`), and `sim/regress.js` `c-programs` runs it in the emulator with the other samples. Run the whole suite (`node sim/regress.js`) before committing.

The emulator and tools

A minimal Hydra-16 emulator for debugging the OS ROM without the hardware. It boots the real ROM images built by `build.js` (`os_rom/bin/os_rom_C02.bin` and `os_rom/bin/paged_rom_C02.bin`).

Requires [Node.js](#). No other dependencies. The emulator, the regression tests and the card tool are in `sim/`; the commands below are run from there.

Using the Hydra from your terminal

```
node hydrasim.js -i
node hydrasim.js -i --sd card.img          with an SD card (a file: see tools/hydrafs.js to make one)
```

`-i` (`--interactive`) makes the terminal the Hydra's serial terminal.

- **What happens:** it boots (POST, then the HyForth prompt) and runs in real time. What you type goes to the serial port and the output comes straight back, so it's like sitting at the board.
- **Keys:** the console keys work as on the board: Ctrl-C breaks, Ctrl-\ restarts the shell, Ctrl-] then a task number switches tasks, Ctrl-D ends input. `bye` leaves HyForth for WOZMON.
- **Emulator commands** (Ctrl-A is the emulator's own prefix key, as in QEMU or `screen`):

Keys	Does
Ctrl-A x	Quit
Ctrl-A r	Press the reset button (RAM and the SD cards keep their contents, as on the board)
Ctrl-A s	Show the state: time, task, ROM page, PC
Ctrl-A h	List these
Ctrl-A Ctrl-A	Type a Ctrl-A

- **Serial speed:** output arrives at the Hydra's serial rate, as on the board: 9600 baud at boot, about 930 characters a second. `q^b115200^ stty` speeds it up; the emulated terminal follows any rate and format, so nothing needs switching.
- **Speed:** `--speed N` runs N times real time (`--speed 0`: as fast as the PC can go, about 20 times). Timings the Hydra shows (sleep, a song's tempo) keep the Hydra's time either way.
- **Other options:** most options below work too, e.g. `--modules`, `--acia wdc`, `--seed`. `--cycles` stops it after that many cycles.
- **Piped input:** input can be piped in, e.g. `printf '1 2 + .\n' | node hydrasim.js -i`. Line ends become Enter, and it stops 3 seconds after the input runs out.
- **Not modelled:** there's no sound; the YM2151 is timed but silent, so the bell only reaches you through the terminal's own BEL.

Usage

```
node hydrasim.js [options]
```

Option	Description
<code>-i</code> , <code>--interactive</code>	Use the Hydra from the terminal, in real time (above)
<code>--speed N</code>	Interactive: N times real time (default 1; 0 = as fast as it goes)

Option	Description
<code>--paste</code>	Type the input at the ACIA's full line rate, back to back like a paste, whether the ROM keeps up or not: bytes that arrive while the last one is still unread are lost, as on the chip, and counted in the report (default: each key waits until the ROM has read the last)
<code>--rom DIR</code>	ROM images directory (default: <code>../os_rom/bin</code>)
<code>--cycles N</code>	CPU cycles to run (default 20,000,000; about 5.6 seconds at 3.58 MHz)
<code>--input TEXT</code>	Serial input to type, from cycle 200,000 on, a key every 20,000 cycles; <code>\r</code> = CR, <code>\xNN</code> = the byte NN (e.g. <code>\x03</code> = Ctrl-C), <code>\w</code> = wait 2M cycles before the next key (booting to the HyForth prompt takes about 0.9M cycles, so start with one), <code>\p</code> = wait for a prompt: a new one (<code>></code> , or WOZMON's <code>></code>), with the output quiet for 300,000 cycles after it
<code>--stop-after-input N</code>	Stop <code>N</code> cycles after the last key of <code>--input</code> is typed (after its waits), or at <code>--cycles</code> : with a final <code>\p</code> , a run ends soon after its last command is done
<code>--clock 3.58 7.16</code>	The CPU clock in MHz, as the ROM was built for (<code>CPU_CLOCK_MULT</code> in <code>os_rom/include/hw.inc</code> ; default 3.58). It sets the ACIA's and the YM2151's timing in CPU cycles, and the seconds in the report
<code>--modules N</code>	RAM modules installed: banks <code>\$00</code> to <code>N*16-1</code> (default 3)
<code>--shared-u N</code>	Shared RAM installed for <code>U</code> macro-pages 0 to <code>N-1</code> (default 16; each 512K chip is 4 macro-pages)
<code>--acia-line N</code>	IRQ line the ACIA interrupts on (default 1)
<code>--acia rockwell wdc</code>	The ACIA chip: the Rockwell R65C51 (default), or the WDC W65C51N with its transmitter bug (TDRE always reads 1, no TX interrupt; for a ROM built with <code>SER_ACIA = SER_ACIA_WDC</code> , which paces sending with VIA timer 2). In WDC mode the emulator counts bytes written while one is still being sent (they'd be garbled on the chip) and reports them at the end. The report also gives the shortest idle time on the line between characters sent, in bits, since the rate was last set (0: back to back) (VIA timer 2 is modelled too: one-shot)
<code>--stuck-irq N</code>	Hold IRQ line <code>N</code> active the whole time
<code>--sd [N:]FILE[@B]</code>	An SD card (SDHC) on SPI device <code>N</code> (0-7, the board's SPI headers J18-J25; default 0), backed by the image <code>FILE</code> (512-byte blocks; writes go to the file). Up to 8 cards, one per device, e.g. <code>--sd card0.img --sd 3:C:/images/card3.img</code> . <code>@B</code> : the card says it has <code>B</code> blocks, more than the file (a big card from a small file: blocks past the file's end read as zeros, and writing one makes the file longer), e.g. <code>--sd card.img@500170752</code> for a 244 GB card. Models the VIA's port B SPI bit by bit (device select as the board's 74HC138 does it), and the SD commands the ROM uses (CMD0, 8, 9, 16, 17, 24, 55, 58, ACMD41; CMD9's CSD gives the image's size)
<code>--sdsc N</code>	Make the card on device <code>N</code> a standard capacity one (SDSC): byte addresses, and a v1 CSD register
<code>--spi-echo N</code>	A test device on SPI device <code>N</code> (0-f, hex; 8-f are the slots'), for <code>/dev/spi</code> : it answers each byte with the one it got before; its first after a select is <code>\$A0</code> , or <code>\$A3</code> if SCLK was high then (mode 3)
<code>--ym-reset-delay N</code>	The YM2151 takes <code>N</code> cycles more to reset a timer flag (and let its IRQ line go) after the write's busy time: a slower chip, as some boards' seem to be (the <code>songs</code> test runs with 150)
<code>--gpio-in HH</code>	The VIA's port A inputs (J27: <code>/dev/gpio</code>), as levels, hex (default <code>FF</code> : the pull-ups); the report gives its directions, outputs and PCR at the end (<code>--- VIA port A</code>)
<code>--ca1 C[,C...]</code>	Pulse CA1 (J27 pin 11) low at each cycle <code>C</code> , high again 500 cycles on (<code>/dev/gpio/ca1</code>)
<code>--ram-fault BANK:An:high low</code>	Address line <code>An</code> (0-12) stuck high or low on the RAM chip holding <code>BANK</code> (a shared chip holds 4 bank IDs, e.g. <code>F0-F3</code> ; a task RAM module 16 banks), e.g. <code>F0:A0:high</code> . The POST RAM line should report it
<code>--model M</code>	Hardware what-ifs: <code>sharedlow</code> , <code>nostack</code> , <code>zponly</code> , <code>noshared</code>
<code>--raw</code>	Print serial output as-is (by default ESC shows as <code><ESC></code>)
<code>--trace N</code>	Show the last <code>N</code> instructions (default 25)
<code>--dump ADDR[:LEN] [@TASK]</code>	Hex dump task RAM after the run, e.g. <code>--dump 7D90:16@1</code>
<code>--watch ADDR[@TASK]</code>	Report every write to a task RAM address: the old and new value, and the PC that wrote it

Option	Description
<code>--mark TEXT</code>	Report the cycle each time the serial output ends with <code>TEXT</code> (<code>\r</code> = CR), e.g. <code>--mark "/> "</code> to time a command from prompt to prompt
<code>--profile-top N</code>	How many routines the profile lists (default 30)
<code>--profile N[-M]</code>	From cycle <code>N</code> on (to <code>M</code>), count the cycles and instructions each task runs in each routine (named from the build's debug info, <code>os_rom/obj/os_rom_C02.dbg</code>), and report the busiest by cycles (with <code>--mark</code> , the window can be one command: its line's mark to the next prompt's), e.g. <code>--profile 2800000 --input '\wwords wc . . .\r'</code>
<code>--ym-log</code>	List every YM2151 key-on (channel and cycle) in the report, not just the first 8. The report also gives the longest gap between key-ons and the time from the first to the last (a late note shows as a long gap)
<code>--u7-fault An:high low</code>	Task RAM line <code>An</code> (A15-A18: T0-T3) stuck high or low at U7, e.g. <code>A17:low</code> : tasks that differ in that bit share their RAM (in the emulator, their bank registers too, which the board keeps apart)
<code>--ym-dump</code>	Show the YM2151's registers at the end, 16 a line (as the chip has them: the carriers' levels with the volumes)
<code>--ym-vgm FILE</code>	Write what the ROM wrote to the YM2151 as a VGM file (with the time between writes), to hear it in any VGM player (VGMPay, foobar2000 with its VGM plugin, ...)
<code>--rtc TIME</code>	A DS1747 in U7 (a task RAM with a clock): its clock registers are task F's <code>\$7FF8-\$7FFF</code> . <code>TIME: YYYY-MM-DDThh:mm:ss</code> or <code>now</code> (the PC's time), the time it has at power-up, running; <code>stopped</code> : its oscillator off (at 2000-01-01); <code>unset</code> : junk in its registers, as a part never set may have. Modelled from the datasheet: R and W halt its updates, clearing W sets it from its registers (and the century), BF can't be written. The report ends with its time. Without it, U7 is a plain HM628512
<code>--rtc-battery-low</code>	The DS1747's battery flag reads 0 (a flat battery)
<code>--seed N</code>	Power up RAM and the pseudo-registers from random number seed <code>N</code> , so a run repeats exactly (by default each run powers up differently)
<code>--pc [PAGE:]ADDR</code>	Report the registers each time the PC reaches <code>ADDR</code> (on BIOS ROM page <code>PAGE</code> , if given); addresses are in <code>os_rom/obj/os_rom_C02.lbl</code>

Example: boot to Forth and run a command (Forth starts after `COPYTORAM`, so allow plenty of cycles):

```
node hydrasim.js --cycles 60000000 --input "1 2 + .\r"
```

The report shows the serial output, the last instructions executed (`W T PC A X Y S P`), the hottest PCs (a stuck loop shows up at the top), the longest stretches with IRQs off from the first key typed (where they start and end: what holds off the serial port), each task's lowest stack pointer (its free stack bytes, and the `W:PC` that got it there), and the final pseudo-register and vector RAM state.

Regression tests

`regress.js` boots the ROM in the emulator once per test, types each test's input, and checks the serial output for what it expects: POST, the self tests (MMU, scheduler, IO; also with 1 RAM module and 1 shared macro-page), POST with hardware faults, the hardware test (all of it; and from POST, with faults injected), HyForth and its libraries, pipelines, files and namespaces, tasks and console switching, Ctrl-C, background sound and the bell, `sleep`, the serial settings, `/dev/sd` (on a blank card image; also two shells reading it at once), HydraFS reading, writing, checking and quick formatting (on fixture card images kept in `sim/cards`, and on cards made by `tools/hydrasfs.js`, and checked with it afterwards), partitions, the clock (`/dev/time`) and files' stamps, sparse files, and the shell: the volume chosen at boot, `boot.hys`, `cd`, the prompt, the file commands, `include`, running programs (`.hys` executables and `.hys` scripts, by name and from `/bin`, their arguments, Ctrl-C), redirection, `echo`, the editor, and each task's environment (`/env`, `PATH`, `HOME`, `/dev/proc`), and `/rom`. One test runs a small program of its own instead of the ROM, and checks the CPU's cycle counts against WDC's table. Four watch timing: a 1000-character paste at 57600 with nothing lost, console output at 115200 inside a cycle budget, SD read throughput inside a

cycles-a-byte budget, and a limit on how long the ROM ever holds interrupts off. The emulators run in parallel; the whole set takes under a minute.

```
node regress.js          all the tests (exit code 1 if any fails)
node regress.js pipes io  only the tests whose names contain "pipes" or "io"
node regress.js --list    what each test checks
node regress.js --random  a new random power-up each run (default: --seed 1, so runs repeat exactly)
node regress.js --verbose show every test's serial output, not just the failures'
```

Or `node build.js rom test` builds the ROM and then runs them. A failure shows what was missing (or found when it shouldn't be), the `hydrasim.js` command that reproduces it, and the serial output. Every test also fails if a task's stack got within 32 bytes of its bottom, and the summary shows the deepest stack of the run (about 70 of the 256 bytes so far, with IRQ frames on top of far calls). To add a test, add an entry to one of the files in `sim/tests/` (by area: `system`, `forth`, `devices`, `storage`, `shell`; their helpers are in `tests/common.js`, and the fields are described at the top of `regress.js`). In a test's input, `P` waits for the prompt (the last command done) and `W(n)` for a fixed time: use `P` between commands, and `W` only where time has to pass (a song playing, a background task working, a key typed into a program). A test's run ends 3M cycles after the prompt that follows its last key (`--stop-after-input`), or at its `--cycles` if that never comes; `fullRun: true` makes it run to `--cycles` (for a check of what happens later). A test that wants SD cards lists them under `sd`; a card with `hfs` gets a HydraFS made on it (`quick`: as the Hydra's quick format makes one), and the function is handed the volume (the `Volume` class below) to put files in; `claim` makes a card say it's bigger than its image (`--sd FILE@B`); `image` starts a card from one of the fixture images in `sim/cards` (a copy), cards made before that the ROM must go on reading ([their README](#)).

HydraFS card images

`tools/hydrafs.js` makes and reads HydraFS images (the Hydra's SD card filesystem, [plans/HYDRAFS.md](#)) on the PC, for the emulator's `--sd` or for writing to a real card with a disk imager:

```
node tools/hydrafs.js mkfs card.img 64 GAMES      a new, empty 64 MB image (-q: as the Hydra's quick format)
node tools/hydrafs.js mkfs card.img 64 GAMES -p 32 ... in a partition, after a 32 MB FAT one (unformatted)
node tools/hydrafs.js import card.img myfiles      copy a folder tree in
node tools/hydrafs.js put card.img star.frt games  copy a file into /games
node tools/hydrafs.js ls card.img games           list a directory (ls card.img -l games: with dates)
node tools/hydrafs.js get card.img games/star.frt star.frt
node tools/hydrafs.js check card.img              check the free map against the files
```

The Hydra reads the same images through its `hfs` server ([io.md](#)): `node hydrasim.js -i --sd card.img` boots with the card's root as the current directory (`0:/>`), and `ls` lists it.

`node tools/hydrafs.js` alone lists every command. Card paths start at the card's root; in Git Bash, leave out their first `/` (Git Bash turns `/games` into a Windows path). From Node, `require('./tools/hydrafs.js')` gives `mkfs` and `Volume`.

Hydra executables

`tools/mkhyx.js` puts the 16-byte `.hyx` header on a raw binary linked for a fixed address, so the shell can run it ([programs.md](#)). (Programs built with `ca65` and `programs/asm/hyx.cfg` have the header already.)

```
node tools/mkhyx.js prog.bin prog.hyx $0800      the header on a binary (entry point: the load address)
node tools/mkhyx.js prog.bin prog.hyx $0800 $0810 ... with another entry point
node tools/mkhyx.js --info prog.hyx              show a .hyx file's header
```

From Node, `require('./tools/mkhyx.js')` gives `hyx(load, code, entry)`, which `regress.js` uses for its test programs.

The ROM disk

`tools/mkromdisk.js` makes the ROM disk, the paged ROM as one disk with `/rom`'s files on it (`io.md`). The build runs it after the link; run from `os_rom`, it takes the list of files and the paged ROM image:

```
node ../sim/tools/mkromdisk.js romfs.txt bin/paged_rom_C02.bin --list ... and show each file's size and banks
```

It makes a HydraFS volume of the files with `hydrafs.js` (stamped 2000-01-01, so each build is the same), puts a partition table in bank 0's first block and the volume from bank 2, in the chips' order (the halves and the V1 board's bank bits swapped), then reads the image back as the CPU would, through the emulator's own bank mapping (`lib/machine.js`), and fails if any file isn't its source byte for byte. Running it again on an image it has already written is harmless: the blocks are written over.

Songs: the score compiler

`tools/hysong.js` compiles a score (a text file: instruments, and a line of MML for each of the YM2151's eight channels) into a ZSM song, which the Hydra's player plays (`play`, `HyForth`). The patches and volumes are worked out on the PC, so the song is plain register writes: it plays on a Commander X16 or in any ZSM player too. The score language is described at the top of `hysong.js`; `os_rom/songs/test.mml` (the ROM's test song) and `programs/songs/scom.mml` are examples.

```
node tools/hysong.js song.mml song.zsm           compile; report its size, length and notes by channel
node tools/hysong.js song.mml song.zsm --vgm song.vgm ... and a VGM, to hear it on the PC (VGMPPlay,
foobar2000)
node tools/hysong.js song.mml song.zsm --rom song.s ... and a ca65 source of its bytes (for a ROM of your
own)
```

The build compiles `os_rom/songs/test.mml` this way before it assembles the ROM (into `os_rom/songs/test.zsm`, a file on the ROM disk). To hear what the Hydra itself plays, run the emulator with `--ym-vgm`.

What it models

- W65C02S CPU, including the WDC additions (`STZ`, `BRA`, `PHX/PLY`, `TSB/TRB`, `BBR/BBS`, `RMB/SMB`, `WAI`, `STP`, (`zp`) addressing, the 1-byte NOPs). Cycles are counted from WDC's table, with its extras: +1 for an indexed read that crosses a page, +1 for a branch taken and +1 more if it lands on another page, +1 for `ADC/SBC` in decimal mode, and 7 for an interrupt. The board has no wait states (`RDY` only has a pull-up), so every access is at full speed. A read or write of an I/O device happens at the instruction's last cycle (the devices are brought up to that cycle first), and `WAI` sleeps until the next device event (which also makes idle time fast to simulate).
- `T`: each task has its own `$0000-$7FFF` (zero page, stack, task RAM) and `$00/$01` bank registers.
- RAM bank window `$8000-$9FFF`: banks `$00-$EF` per task (only the installed modules; others read back floating-bus values), banks `$F0-$FF` shared, 16 macro-pages selected by `U`.
- Paged ROM `$A000-$DFFF` (16K banks selected by `$01`), including the board's A13 half-swap (`$A000` reads ROM offset `$2000`, `$C000` reads ROM offset `$0000`) and the V1 board's swap of the bank number's bits 2 and 3, and 6 and 7 (the image holds bank `b` at bank `swap(b)`'s place).
- BIOS ROM `$E000-$FFFF` in 8K pages selected by `W`; I/O at `$FF00-$FFEF`; `T/U/V/W` at `$FFF0-$FFF3`.
- IRQ vector RAM at `$FFFE/$FFFF`: written at index `V[0..3]`, read at index `IRQ_NUMBER(n)` ($n \wedge 7$) of the lowest active IRQ line, or `V[0..3]` when no line is active (and for `BRK`).

- Rockwell 65C51 ACIA at \$FF10 on IRQ line 1: transmit and receive with interrupts, output captured, input from `--input`. A character takes the time set by the registers the ROM programs: the baud rate (from its clock, the board's `SER_CLK`, 1.790 MHz, so every rate is 2.9% slow as on the board; or that clock / 16 for 115200), the word length, parity and stop bits. A programmed reset (a status write) clears the command register's bits 0-4. The transmit interrupt comes as TDRE goes on (when a byte has gone), as on the board: turning it on while TDRE is already on doesn't interrupt.
- VIA timer 1 (one-shot and free-running, latches, interrupt flag and enable registers) on IRQ line 0: the scheduler's tick; timer 2 (one-shot); the shift register's timing and flag (its CB1/CB2 lines aren't brought out: shifting in reads 1s); port B as the SPI bus, its 16 devices (see `--sd`, `--spi-echo`); port A's inputs read as `--gpio-in` gives them (high: the I2C bus's pull-ups; no I2C devices), and CA1 as `--ca1` pulses it (its edge sets its flag; `ORA` with the handshake clears it). CA2 and the CB lines aren't modelled.
- YM2151: busy (status bit 7) for 64 of its clocks (3.58 MHz) after each data write. A write while it's busy would be lost on the chip: the report counts them. Key-ons are reported (`--ym-log`). Its timers A and B (registers \$10-\$14): an enabled timer's overflow sets its status flag (bits 0, 1), which holds IRQ line 4 until it's reset.
- RAM and the pseudo-registers power up with random values, like the hardware.

It is a model, not the hardware: anything it doesn't simulate (the YM2151's sound, the SD card's own delays, card slots, the timing of each bus cycle inside an instruction) can still behave differently on the board.

Inside the emulator

`hydrasim.js` is the command line: its options, the ROM and card image files, the report, the interactive terminal. The machine itself is in `sim/lib/`, with no Node.js in it (no files, no console), so it can run in a web page too:

File	What
<code>machine.js</code>	<code>createMachine(opt)</code> : memory (tasks, banks, the paged ROM and its bit swaps), <code>T/U/V/W</code> , the vector RAM, I/O, IRQ lines, the run loop (<code>run(cycle)</code>), the reset button, and what's watched as it runs (the trace, hot PCs, IRQs-off stretches, stack depth, the profile)
<code>cpu65c02.js</code>	The W65C02S: <code>step()</code> , <code>interrupt()</code> , <code>reset()</code> , the cycle count
<code>acia.js</code> , <code>via.js</code> , <code>ym2151.js</code> , <code>ds1747.js</code>	The devices: each has <code>read/write</code> , <code>tick</code> (cycles gone), <code>irqActive</code> , <code>nextEvent</code> (for <code>WAI</code>) and <code>reset</code>
<code>sd.js</code>	The SPI bus on the VIA's port B, and SD cards and echo devices (<code>--spi-echo</code>) on it: each card's blocks come from a block device ({ <code>blocks</code> , <code>read(n)</code> , <code>write(n, data)</code> }): a file in <code>hydrasim.js</code>

A new device (a card in a slot: the VERA, [plans/VIDEO.md](#)) is a module like these, which `machine.js` connects to its I/O ports and IRQ line.

Next steps: making the Hydra fun and useful

What the Hydra-16 has, what's missing for hobbyists and programmers, and an order to do it in. It builds on [IDEAS.md](#) (the running list), [VIDEO.md](#) (the Vera X card) and the [code review](#).

Contents

1. [Where it stands](#)
2. [What a newcomer runs into](#)
3. [The gaps, by theme](#)
4. [Milestones](#)

Where it stands

The Hydra already does more than most homebrew 65C02 machines:

- **The system:**
 - 16 hardware tasks, preemptive scheduling, per-task memory and shared memory;
 - Plan 9-style IO: files, pipes, namespaces, `/env`, `/proc`, exit statuses, background jobs, semaphores.
- **Files:** SD cards with a real filesystem (HydraFS: directories, sparse files, a checker, partitions), a clock chip, and files built into the ROM (`/rom`).
- **The shell and languages:**
 - a shell that's also a language (HyForth: pipelines, redirection, scripts, libraries from files);
 - a C toolchain (cc65, with a library for the Hydra's calls and a guide);
 - a line editor;
 - WOZMON with a disassembler.
- **Sound:** the YM2151, with a library, `/dev/snd`, a ZSM song player and a score language.
- **Tools:** an emulator that runs the real ROM, 74 regression tests, a hardware test, and documentation for every part.

What's missing is mostly **on-ramps** (ways to start without the author's setup) and **things to make** (a screen, input, games, hardware projects, languages beginners know).

What a newcomer runs into

A newcomer wants to ...	Today	What would help
Try it without building a board	Install Node and cc65, clone, build, run the emulator in a terminal	A web emulator: a page that boots the ROM, with a sample card
Build it on Linux or macOS	Windows <code>.bat</code> files only	<code>build.js</code> for any OS; CI; release images
Plug in a monitor and keyboard	A serial terminal is the only console	Vera X video and a keyboard (VIDEO.md)
Get programs onto it	Take the SD card out and copy with <code>hydrafs.js</code>	A PC folder as a filesystem over serial (<code>/pc</code>), then XMODEM
Type a first Forth program	<code>: x 65 . ;</code> silently breaks, and <code>.</code> prints hex	HyForth fixes: literals compiled, decimal output, history (review)
Write BASIC	No BASIC	hy-basic , adapted to run as a program

A newcomer wants to ...	Today	What would help
Blink an LED, read a sensor	The VIA's port A and I2C are on a header, but there's no driver	<code>/dev/i2c</code> , <code>/dev/spi</code> , <code>/dev/gpio</code>
Make a game	No graphics, no joystick	Vera X sprites and tiles, pads, a small game library
Learn how it works	Complete references, which assume a lot	Tutorials first, then the references

The gaps, by theme

Sizes are rough: **S** a few days, **M** a few weeks, **L** longer.

1. Easy to try and to build

- **A web emulator (M).** `hydrasim.js` is JavaScript already. Separate its core from Node ([review](#)), and serve:
 - the ROM images and a sample card image in a static page, with `xterm.js` as the terminal;
 - later, the VERA's canvas.

Host it with the project's GitHub Pages. This is the single biggest way to reach people.

- **build.js and CI (S-M).** One build script for any OS, finding cc65 by `CC65_HOME` or the `PATH`. A GitHub Actions job builds and runs the tests on every push. Tagged releases carry the ROM images, a card image, and the C library.
- **A starter card image (S).** A HydraFS image with:
 - `boot.hys` and `PATH/HOME` set up;
 - the samples in `/bin`, libraries in `/lib` and songs in `/songs`;
 - the tutorials' files.

Make it from a folder in the repository with `hydrafs.js import`, at every build.

- **Tutorials (M, ongoing).** Before the references:
 - the first hour with HyForth;
 - a first C program, from the PC to the card to running it;
 - a first assembly program;
 - writing a small device server;
 - once Vera X exists, the first sprite.
- **A contributor's guide (S).** `CONTRIBUTING.md`:
 - the code style: the column layout, `IN: /OUT:` headers, CRLF;
 - the recipes: how to add a thunk, a gate, a server, a HyForth word or a test;
 - the rules: `board/` is read-only, run the tests before a push.

Much of it is in the Programmer's Guide already; this gathers the rules in one place.

2. A standalone computer

- **Video: the Vera X card (L).** 640x480 VGA, text and tiles, 128 sprites, the screen as the console ([VIDEO.md](#)).
- **Keyboard, mouse and game pads (M).** Vera X's input controller on IRQ line 3, over I2C ([VIDEO.md](#)).
- **Files from the PC without the card (M).** Two ways, both worth having:
 - **XMODEM** ([IDEAS.md](#) item 6): send and receive single files with any terminal program.
 - `/pc`: a small Node program on the PC serves a folder over the serial port, and the Hydra mounts it as a file server (9P-style requests, which is what the IO layer speaks inside). Then `cc65` output is runnable at once (`/pc/bin/game`), with no copying. The emulator can serve a folder the same way. For a programmer this is the biggest workflow win.

- **A full-screen editor (M).** `edit` is a line editor. A small screen editor (nano-like: arrows, insert, search, save) using `conio` works on a terminal today and on the Vera X screen later.

3. Languages and tools on the machine

- **HyForth fixes (S-M):** (Done: [CODE_REVIEW.md](#).)
 - numbers compiled inside definitions;
 - decimal output (`decimal`, `hex`, `u.`);
 - line history and editing;
 - readable error messages;
 - `syscall` fixed ([review](#)).
- **BASIC: EhyBASIC (M).** What most hobbyists expect, and there's one already: EhyBASIC ([burntcouch/ehybasic](#)), Microsoft BASIC from the mist64/msbasic and Ben Eater line with Applesoft touches, built today as a paged ROM image started from WOZMON (`A000R`) against the 1.8Ce ROM. To be adapted to the system as it is now: its zero page into what a program has (`$E0-$FF`: Microsoft BASIC uses well over 100 bytes, `CHRGET` among them), its calls through the current thunks, and `EXIT` back to the shell. As a `.hyx`:
 - `INPUT/PRINT` through `stdio` (so it works in pipelines and from scripts);
 - `LOAD/SAVE` on HydraFS files (and `/pc`'s, once it's there);
 - a `/rom/bin/basic`, so it's there with no card.

Later, `SOUND`, `SPRITE` and `PLOT` statements that use `/dev/snd` and Vera X.

- **An assembler on the Hydra (M).** A 65C02 assembler as a program (source file in, `.hyx` out), or as HyForth words (`code ... end-code`). Then programs can be written without a PC at all. WOZMON could get a one-line mini-assembler like the Apple II monitor's.
- **A debugger through /proc (M).** Plan 9's way ([PROC.md](#)): `/proc/N/mem` (read and write a task's RAM), `/proc/N/regs` (its saved frame), `/proc/N/ctl` (`stop`, `start`, `step`, `break ADDR`). A debugger is then just a program, in C or HyForth, and it debugs other tasks while the system runs.
- **Tools as programs (S each).** [IDEAS.md](#) item 10, and more:
 - reading files: `more`, `head`, `tail`, `grep`, `xd` (a hex dump);
 - managing them: `cp -r`, `find`, `sort`, `diff`, `df`, `du`, `touch`;
 - the system: `date`, `which`, `top` (from `/dev/proc`).

Most are small C programs. They make good examples for newcomers and good `/rom/bin` residents.

- **More C (S-M):** `MM_ALLOC` from C (paged memory for bigger programs). (`_stroserror` gives the OS's messages now.)

4. Making things: hardware projects

The board already has the connections: the VIA's port A on J27 (6 free pins and CA1/CA2), I2C on J27 and every slot, 8 SPI device headers, and 6 slots. It needs drivers and examples, in this order:

- **/dev/i2c (M):** the bit-banged bus as a file per address (`/dev/i2c/50` an EEPROM, `/dev/i2c/68` a sensor), with reads and writes as transfers. Vera X's input controller needs it anyway.
- **/dev/spi (S-M):** raw access to SPI devices 0-f (8-f are decoded by cards), for displays, ADCs and radio modules. (Built: `/dev/spi/N`, a transaction a write and the bytes that came back read after, modes 0 and 3: [io.md](#).)
- **/dev/gpio (S):** (Built: [io.md](#), and the [tutorial](#)'s LED and button.)
 - `echo 1 > /dev/gpio/2` lights an LED;
 - `cat /dev/gpio/3` reads a button;
 - `/dev/gpio/ctl` sets directions;
 - CA1 can be an interrupt (`/dev/gpio/ca1`).

- **Networking (L).** A WIZnet W5500 module (Ethernet with its own TCP/IP, on SPI) on one of the SPI headers, served as Plan 9's `/net` (`/net/tcp/clone, ctl, data`). Then:
 - telnet into the Hydra (a shell per connection: the Hydra has 16 tasks);
 - fetch files over HTTP;
 - an IRC client.

An ESP32 modem card (a serial port in a slot with AT commands) is a simpler alternative.

- **A prototyping guide (S):** how to build a slot card, using the breakout card, what the bus timing is, and the open-collector rule for IRQs. The [hardware reference](#) has the facts; a guide turns them into a first project.

5. Sound and music

- **Importing music** from other machines (VGM, MIDI files, trackers) into ZSM: planned in [SOUND.md](#).
- **The VERA's PSG and PCM (VIDEO.md):** 16 more voices, sampled sound, and X16 songs that play in full.
- **A tracker (M-L)** once there's a screen: write songs on the Hydra itself, saving ZSM.
- **MIDI (M):** a slot card with a UART at 31,250 baud, as `/dev/midi`; play the YM2151 from a keyboard, or record into the score language.

6. Games and demos

- **A game library (M)** for C and HyForth:
 - sprites, tiles, input and sound under one small API, in the manner of a fantasy console;
 - on Vera X, with `/dev/snd`;
 - a frame loop paced by the VERA's frame interrupt.
- **Samples to learn from and play:** Snake, Breakout, Tetris, a scrolling shooter, a Mandelbrot, a paint program, the music visualiser. Each is a few hundred lines, and good in `/rom/bin`.
- **Multitasking is the Hydra's party trick:** a game runs while the music plays in another task and a shell stays at Ctrl-]'s reach. Show it off in the demos.

7. The OS

From [IDEAS.md](#), still open:

- **Notes with handlers** (Plan 9's `notify`): a program catches Ctrl-C to clean up;
- **notices** when background tasks end, and `wait` for all of them;
- `rename` across directories.

New:

- `/dev/sysname` **and a version** (the build's stamp: [review](#));
- `/dev/proc/N/ctl` (`kill`, `stop`, `start`), on the way to the debugger above;
- **memory statistics** for `top`.

8. Board V2

[IDEAS.md](#) has the RDY wait states, and the V1 errata are listed in the [hardware reference](#). For a V2 wish list:

- **wait states**, so the CPU can run at 7-14 MHz with the YM2151;
- **a 1.8432 MHz clock for the ACIA**, so the standard baud rates are exact;
- **the bank register bits** in order;
- **the audio jack's channels** the usual way round;
- **a PS/2 or USB keyboard controller on the board** (or keep it on Vera X);
- **a footprint or header for the VERA module** on slot 0's position, if the card proves itself.

Milestones

Each milestone leaves the project in a state worth showing.

1. Easy to start (mostly software, no new hardware):

1. (Done with the code review's fixes: [CODE_REVIEW.md](#).) HyForth's fixes and `syscall`; `build.js` and CI; the ROM budget report and room on every page; the emulator in modules, with no Node.js in its core; the first tutorial ([tutorial.md](#)).
2. Releases with ROM and card images (a starter card), once the CI runs on GitHub.
3. The web emulator with the starter card.
4. [CONTRIBUTING.md](#).

2. Storage and names, then programming and making things (the order set for it):

1. RAM and ROM disks ([DISKS.md](#)). *Done*: the paged ROM is the disk `x`, a HydraFS volume mounted at `/rom`; the RAM disks `r` and `s` (each shell's own area at `/ram`, and `/sram`), the tasks' areas, the program caches; the test song a file; with no card, `/ram` and `/rom/boot.hys`; `/dev/ram` for task 0.
2. Namespaces the Plan 9 way ([NAMESPACES.md](#)). *Done*: 32 entries a task and 32 in the system namespace every task sees (`-s`), mounts with a spec, unions (`bind -a`, `-b`, `-c`) and their listings, `hide`, `unmount new old`, `ns` as `bind` lines; the default namespace (`/rom/lib/namespace`, a card's `lib/namespace`), `.` then `/bin` in place of search paths; C's `hy_bind` and the rest; `newns [file]`, a fresh namespace (C's `hy_newns`).
3. `/proc` ([PROC.md](#)). *Done*: `/proc` mounted, `ns`, `pages`, `ctl` for the family only, and `/proc/N/cmd` (`send N line: shell N` runs it as if typed); `/proc/N/mem` and `/proc/N/ram`, a task's memory as files, for its family and task 0. *To do*: `regs` and `fd`, then the debugger's `ctl` commands with the debugger.
4. `/dev/spi`. *Done*: `/dev/spi/N` and its `ctl` (modes 0 and 3), shared with the SD cards (a device with a card started is busy, and a card isn't started on an open one).
5. `/dev/gpio`, with a hardware project tutorial. *Done*: the pins (`/dev/gpio/N`, `port`), `ctl` (directions, CA1's edge, CA2), `/dev/gpio/ca1` (a read waits for CA1's edge: its interrupt), and the tutorial's LED and button.
6. **Next**: `/pc`: a PC folder over the serial port, then XMODEM.
7. Not yet placed in the order: EhyBASIC, adapted to the system (a `.hyx`, and `/rom/bin/basic`), and `/dev/i2c`.

3. A computer on its own:

1. The emulator's VERA, then the Vera X carrier card.
2. The screen console; `/dev/vid`; HyForth's `video` words with turtle graphics.
3. The input controller (on `/dev/i2c`), keyboard input.
4. A full-screen editor.
5. The game library and the first games; the VERA's PSG and PCM.
6. Tools as programs; an assembler.

4. Connected:

1. `/net` on a W5500; telnet and a file fetcher.
2. The debugger through `/proc`.
3. Music import; a tracker.

Hydra 16: Ideas and future work

Ideas worth coming back to, with the reasoning so far. Plans that are being built live in `MMU_PLAN.md` and `IO_PLAN.md`.

Next features, in order

1. **A battery-backed clock: a DS1747 in U7.** (Done: the reserved bytes, the boot's probe and report, `/dev/time` with it, the emulator's `--rtc`, and tests. Still to do: a hardware test of the chip, and a regular reload while the system runs, not only on a `/dev/time` read.) The DS1747 (the 5 V part; the DS1747W is 3.3 V) is a 512K RAM with a clock in its top 8 bytes, pin compatible with the HM628512 task RAM. In U7 its registers (`$7FFF8-$7FFFF`) are **task F's** `$7FFF8-$7FFF`: `T0-T3` come from U48 (74F573) through U21 (74F541, `T_M0-T_M3`) to U7's A15-A18 in order, with nothing inverted or swapped. What it needs:
 - Keep the ROM off those 8 bytes, in every task: the MMU area's clear (`MM_TASK_INIT`) and its handle table (103 handles become 101), and the hardware test's task RAM test. Any stray write there could stop the oscillator or change the time.
 - At boot: if its seconds count along with the tick (a plain HM628512 there doesn't), load `ZP_CLOCK` from it. Setting the time (`/dev/time`) writes it too (the century, the day of the week, and OSC cleared: parts often come with the oscillator stopped). Load it again now and then, so the tick clock doesn't drift. Warn at boot when its battery flag says the battery is low.
 - Each access with interrupts off, like `CLOCK_GET` (`T = F`, the R or W bit, the registers, `T` back), so it needs no lock.
 - The emulator: a DS1747 in task F's RAM (`--rtc`), and regression tests. The hardware test: is it counting, and is its battery good.
2. **Semaphores.** (Done: `kernel/sem.s`, thunks `$F8D8-$F8E4`, HyForth's `sem`, `mutex`, `acquire`, `acquire?`, `release`, `-sem`; [tasks.md](#). The kernel's waits already share one mechanism, a wait mask; semaphores use it too.) Nothing lets programs share memory safely today (`NO_PREEMPT` stops every task). Kernel calls on a small table (16): `SEM_NEW` (a count; 1 = a mutex), `SEM_ACQUIRE` (waits, using no CPU), `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`. Each has a count, a mask of the tasks waiting (woken with `TASK_WAKE_MASK`) and its holder. The check and the wait happen with interrupts off, so no wakeup is lost. A break or kill ends a wait with an error, and a task's end releases what it holds and frees what it made. HyForth words for them. Later, maybe named ones as files (`/dev/sem/NAME`).
3. **A sound library for the YM2151**, and a test song that uses all of the chip: its 8 voices and 4 operators each, the algorithms and feedback, LFO (vibrato and tremolo), noise, stereo, and the timers. A semaphore (or one per voice) for sharing the chip between tasks: the bell, `/dev/snd` and a tune can't cut into each other's register writes. (The library is done: `os_rom/sound/` on page B, `/dev/snd`'s commands, volumes, the X16's General MIDI patches, and channels claimed by an fd instead of semaphores; HyForth's `patch`, `note`, `noteoff`; C's `snd.h`. The song player too: ZSM files, `play`, a song by its name, C's `snd_play`; and the test song in the ROM (`sndtest`), written in a score language (`sim/tools/hysong.js`). The chip's timer B times the player (the sound clock). Still to do: importing music from other machines: [SOUND.md](#).)
4. **C programs:** a cc65 target (start-up code, and a library over the `$F8xx` calls and `MM_ALLOC`). (Done: `programs/c/`, [programs.md](#): `stdio` and the file calls, `stat/fstat`, `dirent.h`, the environment (`getenv`, `setenv`, `putenv`, `unsetenv` on `/env`), `conio` (ANSI, raw keys through `/dev/cons/ctl`), `system/hy_spawn/hy_wait` (a command shell, `SHELL_CMD`), `exit` statuses, `argv[0]` (the loader's `HYX_NAME`), the header's BSS and top, `clock`, `isatty`, `semaphores`. Still to do: `rename` across directories; `MM_ALLOC` from C.)
5. **/rom: programs and libraries in the paged ROM.** A read-only file server over the paged ROM banks (4 MB, mostly free), made from a directory by a PC tool; `PATH` and `LIBPATH` fall back to `/rom/bin` and `/rom/lib`. So programs, scripts and libraries run without a card. (Done, and since redone as the ROM disk: the paged ROM is the disk x, a read-only HydraFS volume mounted at `/rom` ([DISKS.md](#)), made by `sim/tools/mkromdisk.js` from `os_rom/romfs.txt`, the C samples and a song in it; [io.md](#). Still to do: libraries for `/rom/lib`.)

6. **XMODEM** send and receive, to move files over the serial port without taking the card out. Its checksums and retries also get past the bad characters at 115200.
7. **Exit status** for programs and scripts (through `TASK_EXIT`), and a word to read it, so a script can check a step. (*Done, as Plan 9's exits: a code and a message, `TASK_EXITS` and `TASK_JOIN` (kernel/exits.s), HyForth's status and exits, /env/status; [tasks.md](#).*)
8. **Running a program in the background** (`&`). (*Done: `[B]`, /env/apid, HyForth's wait; [hyforth.md](#). Still to do: a note when a background task ends, and wait with no number for all of them.*)
9. **Notes with handlers** (Plan 9's `notify`), as `IO_PLAN.md` step 10 has it.
10. **Tools as programs**: paging output, `head`, `grep`, a hex dump of a file, copying a directory.

Slow devices on a faster CPU clock (board V2)

The CPU runs at 3.58 MHz; the board can also run it at 7.16 MHz, and the W65C02S goes to 14 MHz. Some devices can't keep up with a faster bus: the YM2151 runs on its own 3.58 MHz clock, and slower 65C51/65C22 grades and ROMs have similar limits. Until there's hardware for this, the CPU clock is a build-time setting (`CPU_CLOCK_MULT` in `os_rom/include/hw.inc`), and above 3.58 MHz the sound chip mustn't be used.

Planned: RDY wait states

The more versatile option, and the plan for board V2 (possibly earlier on an expansion card).

- A **wait table** with an entry per I/O port or memory area. On an access, the entry for the port/area is copied into a **RDY hold counter**; RDY is held low while it counts down, and released at zero.
- A new pseudo-register, `$FFF9`, is the interaction point: the value the RDY hold counter loads from. It's **initialised to 15 at boot** (the maximum wait), so everything is slow and safe until the ROM sets the table up.
- A clock jumper register to read the CPU clock from, replacing the build-time setting.
- RDY has to be open-drain with a pull-up: the W65C02S drives it low itself during `WAI` (the ROM uses `WAI`).
- Transparent to software: every access to a slow port or memory area gets its waits, including instruction fetches from a slow ROM, and existing code needs no changes.
- OS support: set the table first thing at reset (before the self test touches the ACIA); per-device defaults; drivers declare their port's wait states (a `DriverInfo` field that `DRV_START` programs) and an `IO_WAIT_SET` call; the self test reports the clock and the table.

Alternative: switch the clock divider

Set the CPU clock divider through `$FFF9`: a driver writes a slower divider before touching its device, and clears it (back to the system default) when done.

- Reuses the board's existing divider chain (74F191s): no wait table or RDY counter.
- Needs **glitch-free switching** (only on an edge where the fast and slow clocks line up), so the change takes effect a cycle or two after the write: a driver needs a `nop` or a read-back before touching the device. `$FFF9` should be readable (to save/restore it, and to report the jumper setting), with "the jumper speed" as the reset default.
- Slows **everything** while set: other tasks, IRQ handlers, and the VIA timer (which counts `PHI2`), so the scheduler tick stretches. A slow section can't be preempted (the next task would inherit the slow clock): run it with IRQs off, and/or save the divider in each task's frame (like `w` and `u`).
- Only protects code that switches explicitly: the ROM (and anything executed) must be fast enough for the full clock.

Serial output errors at 115200 (open)

Long output at 115200 (a WOZMON dump of the whole BIOS ROM, about 32,000 characters) gets a few bad characters on the board: about 20 in 160,000. Nothing is lost or reordered, and in each bad character bit 7, the last data bit, arrives as 1: the

receiver reads the start of the stop bit, as if the character's end came early. 19200 is clean.

What's been ruled out:

- **The ROM's data:** the emulator's output at 115200 matches the ROM exactly, and so do all the other characters on the board.
- **The rate:** "115200" is really 111,861 baud (the ACIA's 1.79 MHz clock / 16), but a terminal set to exactly 111861 (an FTDI cable, which can do it) gets the same errors.
- **The ACIA:** a logic analyzer on its TxD (U3 pin 10) shows every bit 8.94-8.96 us, including the last data bit before the stop bit.
- **The MAX232's supplies and wiring:** V+ about +9 V, V- about -8.4 V; U5 pin 14 to DE-9 pin 2 and DE-9 pin 5 to ground about 1 ohm.

What helps: sent back to back (as the ROM first did), one bad bit also threw the terminal off the character boundaries, and whole lines were garbled. So at 115200 the ROM paces sending with VIA timer 2, with idle bits after each character (`SER_PACE_GAP` in `os_rom/include/hw.inc`, 2: `SER_PACED`, `servers/serfast.s`): the terminal finds the next start bit, and an error stays one character. It costs about a third of the rate (about 7,000 characters a second).

What's left: the MAX232's output stage (U5), or the cable's RS-232 receiver. Next tests:

- The cable alone: join pins 2-3 (and 7-8) at its DE-9, send a big file from the terminal at 115200, and compare what comes back.
- The Hydra alone: a loopback plug on its DE-9 (2-3, 7-8) and a hardware test that sends a few thousand bytes at 115200 and reads them back through its own receiver (not written yet).
- A scope on DE-9 pin 2 (the edges' times and levels), or a MAX232A / MAX3232 in U5. Once it's fixed, try `SER_PACE_GAP` at 0 (the full 11,000 characters a second).

Namespaces, the Plan 9 way: one tree put together from many

A plan to make the namespace the way names are found, as Plan 9 does, in place of search paths. Directories are mapped onto other paths (bound, mounted) and stacked into **unions**, so `/bin` is one directory that holds the program caches, the cards' programs and the ROM's, and the shell just looks in `/bin`. It goes with the [RAM and ROM disks](#), which give it most of what it puts together. Built: steps 1-6 below, `news` included.

Contents

1. Where it stands
 2. What Plan 9 does
 3. Unions
 4. The namespace a task starts with
 5. Finding programs and libraries
 6. Room: a bigger namespace
 7. Hiding
 8. Calls and words
 9. Costs
 10. Tests
 11. Steps
-

Where it stands

Each task has a namespace of up to 32 entries in the IO transfer areas' banks, copied to the tasks it starts, on top of the **system namespace**, 32 entries every task sees ([io.md](#)): a **mount** sends names under a path to a device's server, and a **bind** makes a path stand for another; the entries with the longest matching prefix apply, and several with the same path are a **union**, looked in in order (`bind -a`, `bind -b`, `-c` for creates; `umount new old` takes one member out; `hide`); reading a union's directory reads each member's. Names nothing matches must be under `/dev`. In the system namespace, the boot shell mounts `/sd`, `/env`, `/proc`, and `/rom` and `/sram` (the HydraFS server with a spec, as Plan 9's `mount` takes one: `mount -s hfs /rom x`), and binds the boot card's `bin` and `lib`; in its own, it mounts its area at `/ram` (each shell mounts its own: `mount hfs /ram r/N`). Then it runs `/rom/lib/namespace` (the caches before the card's, the ROM's after, with `-s`) and the card's `lib/namespace` ([io.md](#)). The shell looks for a program in `.` then `/bin`, then `$PATH`; libraries in `/lib`, then `$LIBPATH`. C has `hy_bind`, `hy_mount`, `hy_umount` and `hy_hide`.

Before this, programs and libraries were found by **search paths** in the shell's code (the current directory, the caches up the owner chain, `$PATH` or the card's `/bin`, then `/rom/bin`): a C program that opened `/bin/x` saw none of it, each new place was another case in the shell, and `ls /bin` showed one of the places.

Room. Each task's IO transfer area is 1.5K, four to a shared bank (banks `$09-$0C`), with 32 entries of its own; each bank's top also holds a copy of the system namespace's 32 (`NS_SYS`, `$9800`), so a name resolves with no bank switch. The default namespace is 13 system entries with a card (11 without) and 1 of the shell's own (`/ram`); a task's own entry for a path wins over the system's, and changing a system union in a task copies it to the task's table first, so each task behaves as if it had a copy, and only its changes cost it entries.

What Plan 9 does

There are no search paths. Each process has a namespace, built from **binds** (`bind new old`: the directory `new` is also seen at `old`) and **mounts** (a file server's tree at `old`), and a bind or mount can **add to** what's at `old` instead of replacing it: `bind -a /usr/me/bin /bin` puts my programs after the system's in `/bin`, `bind -b` before. So `/bin` is a **union directory**: a lookup tries each member in order and takes the first that has the name; a listing shows them all. The shell has `path=(. /bin)` and nothing else: every program, from anywhere, is in `/bin`. A process starts with its parent's namespace (a copy, or shared), and a user's namespace is built at login from a file of `bind` and `mount` lines (`/lib/namespace`, then the user's own). Permissions stay with the file servers: the namespace only says what a process can *name*.

Unions

An entry gets **flags**, as Plan 9's:

Flag	bind / mount	Meaning
(none)	<code>bind new old</code>	<code>new</code> replaces whatever was at <code>old</code> (as now)
<code>-b</code>	<code>bind -b new old</code>	<code>new</code> goes before what's at <code>old</code> : looked in first
<code>-a</code>	<code>bind -a new old</code>	<code>new</code> goes after what's at <code>old</code> : looked in last
<code>-c</code>	with <code>-b</code> or <code>-a</code>	a file created in the union goes into this member (the first such)

- **Looking up** `old/name`: each member in order (a bind's target, or a mount's server), the first that has `name` wins. So `/bin/game` is the first `game` in the stack.
- **Listing** `old`: each member's entries, in order (an fd on a union directory reads one member, then the next). A name in two members shows twice, as in Plan 9; `ls` can leave out the later ones.
- **Creating** `old/name`: in the first member bound with `-c` (none: `ERR_IO_MODE`).
- **Removing, renaming**: in the member that has the name.
- `unmount new old` takes one member out; `unmount old` takes them all.

The namespace a task starts with

The boot shell builds its namespace, every task's at first, from `/rom/lib/namespace`, a file of binds and mounts on the ROM disk (Plan 9's `/lib/namespace`), then runs the card's `/lib/namespace` if there is one, then `boot.hys`. Only the lines that reach that file are built in (`/sd` and `/rom`, as the boot shell makes them now). The built-in names become plain entries (no more special cases in the IO layer):

```
mount /dev    ...           the devices (as now)
mount /env    env
mount /proc   proc          the tasks (PROC.md)
mount /sd     hfs            the cards' volumes: /sd/0-f
mount /rom    hfs x          the ROM disk (a spec: the boot shell's, before the file is read)
mount /sram   hfs s          the shared RAM disk
mount /ram     hfs r/N       this shell's own area on the RAM disk (each shell mounts its own)
bind -bc /ram/<the shell>/bin /bin  /bin: this shell's cache first (created in: cp) ...
bind -a /sram/bin /bin            ... the shared cache ...
bind -a /sd/<the boot volume>/bin /bin  ... the card's programs ...
bind -a /rom/bin /bin              ... the ROM's
(and /lib the same, with lib)
```

- **A new task gets a copy** of its parent's, as now. So a program started from the shell has the shell's `/bin`, its cache included, and the tasks it starts the same.
- **Two shells** start with the same list, each with its own area's cache first: `/bin`'s first member is `/ram/bin`, and each shell mounts its own area at `/ram` (`r/1` for the boot shell, `r/3` for a shell in task 3).
- **A task's area** is named by number only in a spec (`r/N`); in a shell's namespace it's always `/ram`.
- **Adding a tool directory** is a line in `boot.hys` or the card's `/lib/namespace: bind -a /sd/0/tools /bin`.

Finding programs and libraries

- **The shell looks in two places:** the current directory, then `/bin` (the union: every cache, card and ROM in their order), as Plan 9's `path=(. /bin)`. So a program in the folder you're in runs before a cached copy: after a rebuild, the new one runs from its folder without removing the cached copy.
- **Libraries:** `lib name` reads `/lib/name.hyl`, the union `/lib`.
- `$PATH` and `$LIBPATH` are still read, after `/bin` and `/lib`, while scripts move to binds; then they go.
- **C programs** see the same `/bin` (`system`, `hy_spawn` and `fopen("/bin/x")` all through the namespace).
- **Caching** a program is a copy into the first member with `-c`, the task's own cache (`cp game.hyx /bin/game.hyx`).

Room: a bigger namespace

(Done, as below; now 32 entries a task in a 1.5K transfer area, and the system namespace's 32: see *Where it stands*.) The default namespace above is about 12 entries, and a union member is an entry, so 5 wasn't enough. Each task's namespace was 5 entries of 32 bytes in its IO transfer area (`$20-$BF`, with the current directory after it). It moved to a namespace bank of its own:

- **A shared bank**, 512 bytes a task (16 tasks: one 8K bank): **16 entries of 32 bytes** each; or 8 entries of 64 bytes for longer names (a prefix and a target of 31 characters each, against 13 and 15 now). Which is better depends on the default list's names: `/sram/bin` fits in 15, `/sd/0/tools/bin` doesn't.
- **An entry:** the type (mount, bind, hide), the flags (`-b`, `-a`, `-c`), the device or the target, the prefix. The union's members are entries with the same prefix, in order.
- **Copying** a task's namespace for a new task is a 512-byte copy in the bank (`IO_INHERIT`).
- The transfer area's freed bytes go to a longer current directory (63 characters now).

Hiding

A **hide** entry makes a path not exist for the task and the tasks it starts: `hide /sd`, then `/sd/0/...` is `ERR_IO_NOT_FOUND`, however it's named. It's the namespace's part in fencing a program in: start an untrusted program with `/sd` hidden, and only its own area (`/ram`) and `/rom` to work in. (Plan 9 does it with the namespace a process is given, and `RFNOMNT` stops it mounting more.) Permissions themselves stay in the servers: a hide can't grant anything, only take a name away.

Calls and words

- `IO_BIND`, `IO_MOUNT` take the flags in `.X` (0: replace, as now, so programs keep working): `NS_BEFORE`, `NS_AFTER`, `NS_CREATE`.
- `IO_UNMOUNT` takes the member too (`ZP_IO_BUF`: the target or device; 0: all of old).
- **Hide:** `IO_BIND` with `.X = NS_HIDDEN` (`.A.Y` = the path, no target).

- `IO_NS_LIST` prints the entries as the lines that would make them (`bind -a /sram/bin /bin`), as Plan 9's `ns` does, so a namespace can be saved and read back.
- **HyForth:** `bind [-abc] new old`, `mount [-abc] device old`, `unmount [new] old`, `hide path` take their arguments from the line, as the shell's commands do (stack forms `(bind)`, `(mount)`, `(unmount)`, with no flags); `ns`; `#` for comments; `newns [file]` (built) starts a fresh namespace: the task's own entries go but its `/ram`, so it sees the system namespace, then the file's lines run (`IO_UNMOUNT` with `NS_FRESH`; C's `hy_newns`). The namespace files are scripts of those lines, run by the boot shell.
- **C:** `hy_bind(new, old, flags)`, `hy_mount`, `hy_unmount`, `hy_hide`.

Costs

- **Lookups:** a name in a union is tried member by member: `/bin/game` with 4 members is up to 4 opens, and an unknown word (HyForth tries it as a program) tries `.hyx`, `.hys` and `.zsm` in each. The caches come first and are fast; a miss on a card costs a directory search there. If unknown words get slow, HyForth can stop trying the card for a name that isn't one (a word it couldn't parse), or the IO layer can remember misses.
- **ROM:** the union reads and the flags are in the IO layer (page 2: about 970 bytes free); the namespace bank needs `NS_RESOLVE` and `IO_INHERIT` changed, not much else.
- **Programs** that `bind` or `mount` keep working: no flags is the old behaviour.

Tests

In the emulator (`sim/tests/devices.js`): unions (`-b`, `-a`, the order a name is found in, a listing of all the members, a create into the `-c` member, unmounting one member); the default namespace (`ns`), a new task's copy; programs found in `/bin` from a cache, a card and the ROM, in that order; `$PATH` still read; `hide`; 16 entries (and the 17th refused); the existing namespace tests unchanged.

Steps

1. **(Done)** The namespace bank: 16 entries a task, `NS_RESOLVE` and `IO_INHERIT` on it, longer names; every test passing.
2. **(Done)** Unions: the flags, lookups through the members, creates, `unmount` of one member, union directory reads (one fd a task); `hide`, `ns` printing `bind` lines, the words (`ns-unions` in `sim/tests/devices.js`).
3. **(Done)** The default namespace: `/env` and `/proc` are mounts (the IO layer's special prefix is gone), made by the boot shell with `/sd`, `/rom`, `/ram` and the boot card's `bin` and `lib` (they're needed to read any file, and with no ROM disk there's no file); `/rom/lib/namespace` adds the caches and the ROM's to `/bin` and `/lib`, then the card's `lib/namespace` runs (`ns-default`). `newns [file]` gives a task a fresh one: the system namespace and its `/ram`, then the file's lines (`ns-newns`).
4. **(Done)** The shell: programs in the current directory then `/bin`, libraries in `/lib`, `$PATH` and `$LIBPATH` after them; a copy into `/bin` goes to the `-c` member.
5. **(Done)** C's calls: `hy_bind`, `hy_mount`, `hy_unmount`, `hy_hide` (`ctest`).
6. **(Done)** Docs: `io.md` (namespaces), `hyforth.md` (the words, finding programs), `c.md`, the tutorial.

/proc: tasks as files, their memory included

A plan to move `/dev/proc` to `/proc`, as Plan 9 has it, and to add a task's **memory as files**: `/proc/N/mem` (its address space as it sees it) and `/proc/N/ram` (its banks on the RAM modules). Then a debugger, a memory dump or a core file is just a program reading files. Who may read and write them is the same rule as the RAM disks' areas ([DISKS.md](#)), and **task 0, the system task, may use everything**. Built so far: `/proc` mounted, `pages`, `ns`, `cmd` and `ctl` checked by `TASK_MAY` (step 2 below, and `ns` from step 4), and the memory files, `mem` and `ram` (step 3); `regs` and `fd` are next.

Contents

1. [Where it stands](#)
2. [What Plan 9 does](#)
3. [The files](#)
4. [/proc/N/mem](#)
5. [/proc/N/ram](#)
6. [Who can use it](#)
7. [How it works](#)
8. [What it gives](#)
9. [Tests](#)
10. [Steps](#)

Where it stands

`/proc` ([io.md](#)), the device `proc` mounted by the boot shell (`/dev/proc` is the same files), lists the busy tasks, and for each task `N` has `status`, `ctl` (`kill`, `break`, `fg`), `cwd`, `env`, `pages` (a **summary**: `pages PP floor FF`; it was `mem`), `ns` (its namespace, as `ns` prints it), `cmd` (a line for its shell to run: HyForth's `send`), and its memory, `mem` and `ram`. It's served in its client's task, from BIOS ROM page 9 (`servers/proc_srv.s`). Any task can read the status files; `ctl`, `cmd`, `mem` and `ram` are for the task's family and task 0. Far pointers still refuse another task's RAM (`FP_READ`: the MMU test's step 9): the files are the one way to it, and they check who asks.

What Plan 9 does

`/proc/N/` has `mem` (the process's memory, at the offsets of its addresses), `regs` (its registers), `ctl` (`stop`, `start`, `kill`, ...), `status`, `ns` (its namespace, as `bind` and `mount` lines), `fd` (its open files), `note` (signals). The debugger (`acid`) and `ps` are ordinary programs reading those files, and a process's owner (or the host owner, Plan 9's system user) may read and write its `mem`.

The files

Name	Read	Write	Who
<code>/proc</code>	A line per busy task (as now)		Every task
<code>/proc/N</code> , <code>/proc/N/status</code>	Task N's line (as now)		Every task

Name	Read	Write	Who
/proc/N/cwd, /proc/N/env	As now		Every task
/proc/N/ns	Its namespace, as the lines that would make it (ns 's)		Every task
/proc/N/fd	Its open fds: a line each, the fd, its mode and its name		Every task
/proc/N/pages	pages PP floor FF: today's mem summary, renamed		Every task
/proc/N/ctl		kill, break, fg; later stop, start, step, break ADDR (the debugger)	Its family, task 0
/proc/N/regs	Its saved registers: A X Y S P PC and W, its RAM and ROM banks	Later: the debugger sets them	Its family, task 0
/proc/N/mem	Its address space, as it sees it: offset = address (below)	Its RAM	Its family, task 0
/proc/N/ram	Its banks on the RAM modules: offset = bank * 8K + offset in the bank (below)	The same	Its family, task 0

The name: /proc is a mount of the proc device in the default namespace (as /env will be: NAMESPACES.md); /dev/proc stays as a bind to it until the programs and docs have moved.

/proc/N/mem

The 64K that task N's code sees, with its registers as they are while it's switched out:

Offsets	What	Write
\$0000-\$7FFF	Its task RAM: zero page, stack, its program and data	Yes
\$8000-\$9FFF	The RAM bank it has selected (its \$00), or the shared bank it has mapped	Yes
\$A000-\$DFFF	The paged ROM bank it has selected (its \$01)	ERR_IO_MODE
\$E000-\$FFFF	The BIOS ROM page it's on (W)	ERR_IO_MODE
\$FF00-\$FFFF	The I/O space: reads give zeros (a read can have effects: the ACIA's data, say)	ERR_IO_MODE

- **A read or write is a snapshot.** The task may run between two reads; a debugger stops it first (stop in ctl), or reads a running one knowing that.
- **Offsets past \$FFFF** are the end of the file. IO_STAT gives a size of 64K.
- cp /proc/3/mem core3 saves a task's whole address space; cp of a 32K slice back is how a debugger loads a patch. HyForth's dump and WOZMON's R keep working on the reading task's own memory; this is for another task's.

/proc/N/ram

Task N's banks on the RAM modules, by the bank numbers it uses (each task numbers its own banks, \$00 up, and sees them at \$8000 through its \$00): offset $b * \$2000 + o$ is byte o of its bank b .

- **Only the banks it has:** the memory manager's bank map says which (MM_ALLOC with AI_PAGED). A read of a bank it hasn't got gives ERR_IO_NOT_FOUND; a listing (IO_STAT) gives the size up to its last bank.
- So a task's whole state is /proc/N/mem and /proc/N/ram; shared RAM is reached by its handles, as now.

Raw RAM: /dev/ram (built). Neither file above reads memory as the hardware has it; /dev/ram does: every task's 32K, the shared RAM by bank ID, every RAM module's banks for every task, read-only, and **for task 0 only**, the system's task (any other

task's open is `ERR_IO_PERM`). Its offsets and how it reads are in [io.md](#).

Who can use it

The status files (`status`, `cwd`, `env`, `ns`, `fd`, `pages`) are for every task, as now: `ps` and `top` read them. The rest (`ctl`, `regs`, `mem`, `ram`) need the client (task A, from the request's `IO_BLK_CLIENT`) to be:

- **task 0**, the system task, which passes every check, here and on the RAM disks ([DISKS.md](#));
- **task N itself**;
- **a task that started N**, directly or not: a shell and the programs it runs, a debugger and the program it started.

So two shells can't read or kill each other's programs, and a program can't change its shell. This also closes a gap: today any task can `kill` any other.

The check is the kernel's `TASK_MAY` (built: page 5), shared with the HydraFS server: "may A use N's things", walking N's owner chain (`NP_TASK_OWNER`) up for `/proc` (is A an ancestor of N?) and A's for the RAM disks (is N an ancestor of A?), task 0 passing both. Two things it needs first:

- **Owners kept right when a task ends** (done: `TASK_ORPHANS`): its children's owner becomes its own owner, as Unix gives orphans to `init`, so a new task in its slot isn't taken for their parent.
 - **The whole chain**: `TASK_SIGNAL` looks 4 owners up; `TASK_MAY` follows the chain to its end (at most 16 steps). For `/proc` it's called with `.Y = 1`: the asker started the task (or is it).
-

How it works

(Built, as below. The plan was a copy routine run in task N through `TASK_CALL`, with IRQs on; but that call writes task N's zero page and stack while it runs, so the copy wouldn't read them as N has them.)

The proc server runs in its client's task (page 9), as before. Reading task N's memory has to happen with `T = N`, since every task's `$0000-$7FFF` and its banks are only seen while `T` is that task:

- `MEM_COPY` (`servers/ram_srv.s`, shared with `/dev/ram`) sets `T` to N with IRQs off and no stack use, and N's `$00` to the client's IO transfer bank, so the client's data area and the copy's numbers (`MC_AREA`, after the system namespace in each transfer bank) are seen. It copies up to 64 bytes (about 2,000 cycles: the `irqs-off` test's limit is 5,000), then puts N's `$00` and the 8 zero page bytes it borrowed back; a read of them gives N's values, and a write to them lands in them.
 - **N's bank window** (`$8000-$9FFF`): the copy switches N's `$00` to its bank and back for each byte (and `U` for a shared one), since the transfer bank uses the window too.
 - **ROM areas**: `$A000-$DFFF` is read with `T = N`, so it's N's paged ROM bank; the BIOS page (`$E000-$FEFF`) is N's `w`, read with IRQs on through `PEEK_PAGE`. `U` and `w` come from N's frame on its stack, where the scheduler left them; a task in a `TASK_CALL`, a driver or the asker itself has none there (`U 0`, page 0).
 - `/proc/N/ram` is the same copy with N's `$00` set to bank `b`, if N's MMU bank map has it.
 - **A free task** gives `ERR_IO_NOT_FOUND`; a driver task's memory, only task 0 (`TASK_MAY`).
-

What it gives

- **The debugger** ([NEXT_STEPS.md](#)) as a program: `regs`, `mem` and `ctl`'s `stop`, `step` and `break ADDR`, in C or HyForth.
- **Core dumps**: the shell can save `/proc/N/mem` when a program dies (`ERR_` exit statuses), for later.
- `ps`, `top`, which read the status files (as now), and `fd` and `ns` show what a task has open and sees.
- **Tools on files**: `cmp`, `od`, a hex editor work on another task's memory, as on a card's file.
- **Fewer special calls**: reading another task's RAM needs no new kernel call for programs, only the files.

Tests

In the emulator (`sim/tests/devices.js`): `/proc` as `/dev/proc` was (the existing tests, renamed); a shell reading a background program's `mem` (a known value at a known address) and writing it (the program prints the change); `$8000` through the program's bank; a ROM area read and a write refused; `ram` for a program with two banks; another family's task refused (`ERR_IO_PERM`), `kill` included; task 0's request allowed; the owner chain after a task ends and its slot is reused; the `irqs-off` limit while copying.

Steps

1. *Done*: the owner chain kept right at task end; `TASK_MAY` (task 0 passes).
2. **(Done)** `/proc` as a mount (`/dev/proc` stays: the same device); `pages` for the old `mem`; `ctl` checked by `TASK_MAY`; and `cmd` (`send`), a line for another running shell (the `send` and `proc` tests).
3. **(Done)** `/proc/N/mem` and `/proc/N/ram`, reads and writes: `MEM_COPY` with `T = N` and IRQs off (the `proc-mem` test: another shell's RAM and bank read and written, the ROM and I/O areas, refusals, a 64K copy).
4. `regs`, `fd` (`ns`: done in step 2).
5. The debugger's `ctl` commands (`stop`, `start`, `step`, `break ADDR`) with the debugger itself.
6. Docs: `io.md` (`/proc`), `tasks.md` (owners, task 0), `hyforth.md` (`ps`).

RAM and ROM disks: /ram and /rom as HydraFS volumes

Disks in memory, and what they let the rest of the system drop:

- **The ROM disk:** the whole paged ROM (4 MB) as one read-only disk, so everything in ROM that isn't code is a file. **Built** (the disk `x`, mounted at `/rom`).
- **RAM disks:** a program cache and scratch files, fast, with no card: an area for each task, and a shared one. **Built** (the disks `r` and `s`: each shell's own area mounted at `/ram`, and `/sram`).

All of them are block devices under the HydraFS server, as the cards are, and [io.md](#) describes them as they are. The names put together from them (`/bin` over the caches, the cards and the ROM) are the [namespace plan](#)'s, and a task's memory as files is the [/proc plan](#)'s.

Contents

1. What it looks like
2. Disks in memory
3. The ROM disk (built)
4. Pointers and banks: what can't go wrong
5. RAM disks
6. Who can use which area
7. How the RAM disks work
8. Booting: finding the disks
9. What the disks simplify
10. The program caches
11. Tests
12. Steps

What it looks like

Name	What	On	Lasts
<code>/rom</code>	The ROM's files: programs, songs, libraries, help (<code>mount hfs /rom x</code>)	The ROM disk, <code>x</code>	Read-only
<code>/ram</code>	A shell's own area on the RAM disk (<code>mount hfs /ram r/N</code> , <code>N</code> its task): a directory for it and the tasks it starts	The RAM disk, <code>r</code>	Until task <code>N</code> ends (removed then), or reset (or <code>stop</code>)
<code>/ram/bin</code> , <code>/ram/lib</code>	The shell's own program and library cache	The RAM disk	
<code>/sram</code>	The shared RAM disk, every task's (<code>mount hfs /sram s</code>)	The shared RAM disk, <code>s</code>	Until reset (or <code>stop</code>)
<code>/sram/bin</code> , <code>/sram/lib</code>	The shared program and library cache	The shared RAM disk	

```

/> ls /rom/bin           the programs in ROM
/> cp game.hyx /ram/bin/game.hyx  into this shell's cache (/ram/bin) ...
/> cp game.hyx /sram/bin/game.hyx ... or the shared one, for every task
/> game                  found in a cache: loaded from RAM, not the card

```

RAM disks lose their contents on reset (and at power-off): they're formatted each time they're started.

Disks in memory

A memory disk is a block device, like a card. Its memory is a run of banks, 8K RAM banks or 16K paged ROM banks, shown as one run of 512-byte blocks, 0 to N-1: block n is at bank $n / 16$ (RAM) or $n / 32$ (ROM) of the run, at that offset in the bank. The banks don't show above that: the file system sees **one large partition**, so 1.5 MB of shared RAM is one 3,072-block disk, not 192 pieces of 8K.

HydraFS goes on it, as on a card: the same server, so a memory disk has everything a card has (directories, `ls -l`, stamps, `fsck`; sparse files on RAM disks), and there's no second file system. The storage driver's disks are named the same way in `/dev/sd` and `/sd`: an SPI device by its number as one hex digit, any other disk by a letter that isn't one:

Name	What	Memory	Also seen as	Disk number
0-7	SD cards on the board's SPI devices 0-7		<code>/sd/0 ... /sd/7</code>	0-7
8-f	SD cards on slot cards' SPI devices \$8-\$F (planned)		<code>/sd/8 ... /sd/f</code>	11-15, handed out
x	The ROM disk (built)	The paged ROM, all 256 banks	<code>/rom</code>	8
r	The RAM disk (built)	The storage task's banks on the RAM modules	<code>/ram</code> (a shell's own area, <code>r/N</code>)	9
s	The shared RAM disk (built)	Shared RAM banks (the board's 2 MB)	<code>/sram</code>	10

The disks in memory are never under `/sd`, which has the cards alone: they're mounts of the HydraFS server with a spec, as Plan 9's `mount` takes one (`mount hfs /rom x`), and the server serves them only through such a mount (the request says so: `IO_BLK_SPEC`), or for task 0. Each is also a block device, `/dev/sd/x`, `/dev/sd/r`, `/dev/sd/s`, as a card is.

- **A hex digit is the SPI device's, always:** the card on SPI device \$A is `/sd/a`. A slot card can decode SPI devices \$8-\$F (SPI_A3: [hardware.md](#)), so `/sd/8 ... /sd/f` are theirs, and nothing else may take those names. The other disks are letters that aren't hex digits (x, r, s), so the two can never meet, and every name is one character.
- **A name isn't a number.** The tables are indexed by a disk number, 0-15 (`DISK_MAX`: a fid's low 4 bits), and `DISK_FROM_NAME` (`include/io.inc`) turns a name into it, in `/dev/sd`'s server and in HydraFS's. Cards 0-7 are disks 0-7, the ROM disk is disk 8.
- **Cards on slot cards** (planned) need a slot map: disks 11-15 handed out to SPI devices \$8-\$F at their first open (five at once), and `SPI_SELECT` given the device from the map, not the disk number. The map is the driver's own: names, listings (`voIs`, the prompt's volume) and errors always show the SPI device's hex digit, and `DISK_FROM_NAME` reads 8-f (either case) too.

So the card tools work on them: `voIs` lists them, check on `/dev/sd/s/ct1` checks the shared cache, `/dev/sd/s/data` is the raw disk.

The ROM disk

Built. The paged ROM is the disk x (`/dev/sd/x`; disk 8 in the tables): block n is bank $n / 32$, at $\$A000 + (n \% 32) * 512$, as the CPU sees it (8,192 blocks). It starts with a partition table, as a card can:

Blocks	Banks	What
0	0, \$A000-\$A1FF	The partition table (an MBR), and a line saying what the disk is

Blocks	Banks	What
1-63	0-1	Partition 1, type <code>\$DA</code> (not a file system): the system's banks: HyForth's variables and <code>COPYTORAM</code> (bank 0, from <code>\$A200</code>), the hardware test (bank 1)
64-8191	2-255	Partition 2, type <code>\$7F</code> : the HydraFS volume, label <code>ROM</code> , read-only

- **The storage driver** (`drivers/sd.s`, page 3) reads a block by selecting its bank in the storage task's own ROM bank register, copying 512 bytes into the block cache, and putting the register back (`SD_ROM_READ`). Writes are refused (`ERR_IO_MODE`) in the driver, in `/dev/sd`'s data and ctl files (`format`, `label`), and in the HydraFS server (`create`, `write`, `remove`, `wstat`), each before anything changes, the block cache included.
- **The HydraFS server** finds the `$7F` partition as on a card, so nothing in it is special to the ROM but `HFS_RO_DISK`.
- `/rom` is a mount of the HydraFS server with the spec `x` (`mount -s hfs /rom x`), in the system namespace, which the boot shell makes; every task sees it. The IO layer's built-in `/rom` prefix, the `/rom` server (`servers/rom_srv.s`, BIOS ROM page D), its gate and its image format (`HYROM1`, `mkromfs.js`) are gone.
- **The build** (`sim/tools/mkromdisk.js`, after the link): the volume made with the HydraFS PC tool from the manifest (`os_rom/romfs.txt`), stamped 2000-01-01 so each build is the same; the table and the volume's blocks written into the image in the chips' order (the A13 half-swap, the V1 board's bank-bit swaps); then the image read back as the CPU sees it and every file checked (below).
- **One disk, not a disk per chip.** The table is in bank 0 for all 4 MB. A chip can't be burned on its own with a volume of its own (a cartridge), but the server needs one partition per disk, there's no union to build, and a file can be any size. A slot card with a ROM of its own could be another disk, with a letter of its own.

Pointers and banks: what can't go wrong

A file on the ROM disk can lie across banks (`jukebox.hyx` is in banks 4 and 5 now). None of these can break it:

1. **A block never crosses a bank, or the board's 8K halves.** 512 divides `$2000`, and a block's bank and offset come from its number alone (asserted in `sd.s`). So a read selects one bank and copies from one place.
2. **No CPU pointer walks through a file.** Every read goes through HydraFS, a block at a time into the block cache, so a file's blocks can be in any banks, in any order, and its extents decide where. Nothing assumes bank `b + 1` follows bank `b`.
3. **The CPU always sees logical banks.** The V1 board's swaps (bank bits 2 and 3, 6 and 7, and the A13 halves) are in the hardware; only the image writer (`mkromdisk.js`, `romsum.js`) and the emulator (`romBank` in `sim/lib/machine.js`) know about them. So code that selects bank `n` gets the build's bank `n`.
4. **The build checks it.** `mkromdisk.js` rebuilds the disk from the finished image through the emulator's own bank mapping, opens it as a HydraFS volume, and compares every file with its source byte for byte: a mismatch fails the build. It prints how many files lie across banks (`--list`: which).
5. **The machine checks it.** The `rom-copy` test copies every file in `/rom` to a card on the emulated machine (through `SD_ROM_READ`) and compares the card's copies with the sources.
6. **Code and data used by address stay in the system banks**, linked by `ld65` into a memory area per bank half, so a segment can't spill into the next bank (the link fails). The one place that walked a pointer from bank to bank was the song player reading the ROM's test song (`ZSM_BYTE` incremented the ROM bank past `$DFFF`); it's gone: `sndtest` plays `/rom/songs/test.zsm` as a file (below).
7. **A task's banks are its own.** The bank registers are in each task's zero page; the storage task saves and restores its own around a read. A new task now starts with banks 0 (`RESERVE_TASK`): before, it kept its slot's last banks, and a shell started where the song player had been ran bank 2's song data as HyForth's `COPYTORAM` (found by the `irqs-off` test when bank 0's layout moved).
8. **The hardware test's bank lines** compare bank 0's first page with bank `2^n`'s: the table's block 0 starts with a line of text, so bank 0's first page can't be all zeros like an empty block elsewhere.

RAM disks

The ROM starts both at boot (`RAMD_BOOT`, `servers/ramdisk.s`), with the sizes in `os_rom/include/io.inc`, and a write to a `ctl` file stops one and starts it again with another size, from other memory (`boot.hys` can do it):

```
echo start 256K 1-2 > /dev/sd/r/ctl    the RAM disk: 256K, from RAM modules 1 and 2 (the areas: /ram)
echo start 1M $20-$9F > /dev/sd/s/ctl  the shared RAM disk: 1 MB, from shared bank IDs $20-$9F (/sram)
echo stop > /dev/sd/s/ctl              stop one (its files go; not while one is open)
cat /dev/sd/r/ctl                      its size, its banks, its HydraFS: "ram 256 KB 512 blocks", "banks
$10-$2F"
```

- **The defaults:** the RAM disk 256K (`RAMD_BANKS`) of the storage task's banks, from any module; the shared one 512K (`SRAMD_BANKS`) from the lower half of the shared banks (`$01-$7F`). On a machine with less memory, half, and half again, rather than none.
- `start SIZE FROM-TO on r`: the modules the RAM disk's banks come from. A task's banks on the memory cards can only be seen while `T` is that task (each task has its own bank numbers), and the HydraFS server runs in the storage task, so this disk is in the storage task's own banks: its slice of each module, 128K of a 2 MB module, taken with `MM_BANK_ALLOC_IN` (a run of banks from a range, with no handle). The areas share it: there's no limit for one area but the disk's size.
- `start SIZE FROM-TO on s`: the shared bank IDs it comes from (`U << 4` | bank, as `memory.md` numbers them: `$20-$9F` is macro-pages 2-9, 1 MB), taken with `SH_BANK_ALLOC` (a run, with no handle: nothing else hands them out, and `stop` gives them back). The system's own banks (`$00`, the IO transfer banks `$09-$0C`) and banks in use are never taken. Why the lower half by default: `SH_ALLOC` takes from the top down with IRQs off, looking past every bank in use above a free one, and 64 of them there made a shared allocation hold IRQs off too long (the `irqs-off` test).
- **Sizes** take `K` and `M`; without one, they're 8K banks (1-255). Numbers are decimal, or hex after a `$`.

Who can use which area

The Hydra has no users, so the identity a server can check is the task asking: every request carries it (`IO_BLK_CLIENT`), and the kernel knows which task started which (the owner chain, `ZP_TASK_OWNER`). The HydraFS server checks it on the RAM disk (`HFS_AREA_CHECK`), before a walk and before a create cuts its name:

Area	Who	What
Area <code>r/N</code>	Task <code>N</code> , and the tasks it started (and theirs: its family)	Read and write
Area <code>r/N</code>	Any other task	Nothing: <code>ERR_IO_PERM</code> (<code>\$88</code> , "not allowed")
The RAM disk's other names (<code>r/zz</code>)	Every task but 0	Nothing: not an area
The RAM disk's root (its listing), <code>/sram</code> and all in it	Every task	Read and write
<code>/rom/...</code>	Every task	Read only (<code>ERR_IO_MODE</code> for the rest)
Everything	Task 0, the system task	Read and write (the ROM disk still read-only)

- **A child has its parent's area:** a program started from the shell reads and writes the shell's area (and its cache), and so do the tasks it starts. Two shells' families can't see into each other's areas.
- **Task 0 passes every check.** It's the system's own task (the boot, the kernel's work for other tasks), so a request from it is never refused for permission, here or (planned) in `/proc` (`PROC.md`). Only code the system runs in task 0 has that: a

program never runs there.

- **An area itself** can't be renamed, or its mode changed, but by task 0 (`HFS_AREA_WSTAT`): renaming `r/1` to `r/2` would take task 2's.
- **Owners kept right:** when a task ends, the tasks it started get its owner instead (`TASK_ORPHANS`, page 5, from `TASK_EXIT_NOTED`), as Unix gives orphans to `init`, so a new task in its slot isn't taken for their parent.
- **One check for both:** "may task A use task N's things?" is the kernel's `TASK_MAY` (page 5: `.A = A`, `.X = N`), which the HydraFS server calls through a gate. `.Y = 0` is the areas' way (A is N, or a task N started); `.Y = 1` the other way, for the `/proc` server (A started N: a debugger and its program, [PROC.md](#)). Task 0 is always yes.
- **An open file stays usable** by whoever has the fd, as in Unix and Plan 9.
- **Any mount of the disk** (`mount hfs /a r`, then `/a/2`) is checked the same: the server checks the area.
- **Permissions are the server's**, not the namespace's (Plan 9's way): a bind or a mount can rename an area, but the server still checks who's asking. The namespace can add hiding, for any device ([NAMESPACES.md](#)).

How the RAM disks work

In the storage driver (BIOS ROM page 3, with `/dev/sd` and the block cache), as the ROM disk is:

- `SD_READ_BLOCK` and `SD_WRITE_BLOCK` for `r` and `s` (`SD_STATE_RAM`, `SD_STATE_SRAM`) map the block's bank at `$8000` (`SD_RAM_MAP`: the storage task's own `$00` for `r`; `U` and `$00` for a shared bank ID), copy 512 bytes between it and the block cache (in task RAM), and put the bank and `U` back. Block `n` is bank `n / 16` of the disk's run, from its first (`RAMD_FIRST`), at `$8000 + (n % 16) * 512`.
- **Starting one** (`RAMD_START`) takes the banks, writes zeros to block 0 (so RAM from before, or from power-up, can't look like a partition table), then makes a HydraFS on it as `format` does (a quick one, labelled `RAM` or `SRAM`). **Stopping one** (`RAMD_STOP`) gives the banks back, and is refused while a file on it is open.
- `DISK_FROM_NAME` (`include/io.inc`) knows `r` and `s` (disks 9 and 10).

In the HydraFS server (page 6):

- (`/r/s` was the shared disk too, so that one bind gave both; with each shell's own area at `/ram`, the shared disk has a bind of its own, `/sram`, and that's gone.)
- the areas' check (`HFS_AREA_CHECK`, `HFS_AREA_WSTAT: ERR_IO_PERM`);
- **an area removed as its task ends:** the task-end path (`TASK_EXIT_NOTED`, after `MM_TASK_RESET` has closed the task's fds) calls `TASK_ORPHANS` (page 5), which gives the task's children its owner, then looks at the storage task's `RAMD_AREAS` (a bit for each area that may be there, set by `HFS_AREA_CHECK` when one is used) and, if the task's bit is set, runs `HFS_AREA_END` in the storage task with `TASK_CALL` (no fd, no request: the storage task is between requests then). That walks a path of its own (`RAMD_AREA_PATH`, 256 bytes) from the area down to the first file or empty directory, removes it (`HFS_REMOVE_AT`, the remove request's own), and starts again, until the area itself is gone and its bit cleared. A file still open (a task it started may have one) stops it, and the bit stays for the next time; a parent waiting for the task (`TASK_JOIN`) goes on only after it.

Names: the boot shell mounts `/sram` (`mount -s hfs /sram s: SH_MOUNTS`) and makes the shared caches' directories, `/sram/bin` and `/sram/lib` (`SH_RAM_DIRS`). Each shell mounts its own area at `/ram` (`mount hfs /ram r/N`, in its own namespace) and makes it, with `bin` and `lib` (`SH_OWN_AREA`: the boot shell's from `SH_BOOT`, one started with `shell` from `SHELL_MAIN`); the programs, scripts and pipelines it runs inherit that.

Speed. A block from RAM or ROM is a 512-byte copy, about 7,000 cycles; from a card it's the bit-banged SPI, about 150,000 (the `sd-speed` test's budget). The rest of a load is the same either way, and it's most of the RAM disk's time. Profiled (`sim/hydrasim.js --profile N-M`), a 16K program from the shared RAM disk takes 1.34 million cycles, against 4.69 million from a card (3.5 times as fast: `ram-speed`):

- **Three copies of every byte, 55%:** from the disk's bank to the block cache (`SD_RAM_READ`), to the client's transfer area (`HFS_FILE_READ`), and to the program (`IO_COPY_OUT`), each a page four bytes a turn of the loop (`_M_COPY_PAGE`: 13.75

cycles a byte). The RAM disk's bank and the transfer area are both seen at `$8000`, so the cache's copy can't be skipped without switching banks for every byte, which costs about as much.

- **Each request, 31%:** the file is read 256 bytes at a time (`IO_UNIT`), 64 requests, each with its task calls, gates and scheduling, HydraFS's place in the file (`HFS_EXT_TRY`, `HFS_FILE_BLOCK`) and the IO layer's bookkeeping: about 6,000 cycles a request. Bigger requests would halve it.
- **Starting the program, 14%:** the shell's word lookup and name search, the new task (its namespace: only the entries in use are copied), the MMU.

The ROM disk's blocks are read where they are: the paged ROM is at `$A000`, apart from the transfer area, so HydraFS copies a file's bytes from it straight into the transfer area (`HFS_BLOCK_AT`), with no copy into the cache (a ROM block never changes), about 16 cycles a byte less. A program found by name in a cache pays for the places looked in first (the current directory, on the card).

Booting: finding the disks

1. **The storage driver starts the disks it can:** the ROM disk needs no start (`SD_ROM_INIT`: its state and size); the RAM disks are started with the defaults and formatted (done); the cards are started at their first open, as now.
2. **The boot shell finds the volumes:** `/sd` mounted, then each disk's partition table read (HydraFS finds its partition, as on a card). `/rom` is bound if the ROM disk has a volume (an image without one, or a blank paged ROM, has no table: no `/rom`, and everything else works). *Done:* the boot shell binds a name only if what it stands for opens (`SH_BINDS`; the `rom-none` test).
3. **(Done) The namespace from a file:** the boot shell reads `/rom/lib/namespace`, the default list of binds and mounts (`/ram`, `/bin` and `/lib` unions), then a card's `/lib/namespace` ([NAMESPACES.md](#)). The list is a file in ROM, not code: changing it is a rebuild of the ROM disk, not of a BIOS page.
4. **The current directory:** the first card with a HydraFS, as now; with no card, the shell's own area, `/ram`, so files can be saved (until reset) on a machine with no card at all. *Done* (`SH_VOLUMES`).
5. **The boot script:** a card's `boot.hys`; with no card, `/rom/boot.hys` (now a line saying so; later it can copy the ROM's programs into `/sram/bin`, set `PATH`, print help). *Done:* `SH_VOLUMES` says which (`BOOTFLAG` 1 or 2).

What the disks simplify

Every piece of the ROM that isn't code can be a file, and every place that searches can be a bind:

Now (or before)	With the disks	Status
<code>/rom</code> : its own server (page D), image format and PC tool, an IO-layer prefix, a gate	A HydraFS volume read by the storage driver; a bind	Done. Page D is free (7.5K)
The test song in bank 2, and the player's ROM mode (<code>ZSM_PLAY_ROM</code> , <code>SND_SONG_BANK</code> , the bank walk in <code>ZSM_BYTE</code> , <code>songs/test_rom.s</code>)	<code>sndtest</code> plays <code>/rom/songs/test.zsm</code> (already on the disk) as <code>play</code> does any file	Done: bank 2 is the volume's, and the only cross-bank pointer is gone
HyForth's training scripts and sample binary words in bank 0 (<code>ftrain</code> , words compiled at <code>COPYSTART</code> offsets)	Files in <code>/rom/forth</code> , read with <code>include</code>	Planned: bank 0 keeps only <code>COPYTORAM</code> and the variables
New HyForth words: assembly on page 1 (196 bytes free) or page A (530)	Forth source libraries in <code>/rom/lib</code> (<code>lib name</code> , which reads files already)	From now on, where speed allows
The shell's search: the current directory, <code>\$PATH</code> , the card's <code>/bin</code> , then <code>/rom/bin</code> (and <code>\$LIBPATH</code> , <code>/lib</code> , <code>/rom/lib</code>)	<code>.</code> then <code>/bin</code> , a union of the caches, the card and the ROM (NAMESPACES.md)	Done
A built-in default namespace (code in the boot shell)	<code>/rom/lib/namespace</code> , a file	Done (the mounts and the card's binds stay the boot shell's)

Now (or before)	With the disks	Status
No card: nowhere to save	/ram (the shell's area) as the current directory	Done
Slow program loads from a card, and no resident programs	The caches: a program copied to RAM once	Done
A task's memory: summaries in /proc/N/pages; far pointers refuse other tasks' RAM	/proc/N/mem, the bytes, for the family and task 0 (PROC.md)	Planned
ROM programs on BIOS pages (the editor, page 8; the song player, page C)	Could be .hyx files in /rom/bin, loaded into task RAM	An option: BIOS pages aren't short, but files can change without a BIOS rebuild
Two PC tools for images (hydrafs.js, mkromfs.js)	One, hydrafs.js, for cards and the ROM	Done

What stays in system banks: what has to run without the OS (the hardware test, bank 1), and what HyForth copies by address before any IO is up (COPYTORAM and its variables, bank 0).

The program caches

Two caches: the boot shell's own (/ram/bin, its area's, and its children's, since they share its namespace) and the shared one (/sram/bin). For a program typed with no /, the shell looks in the current directory (Plan 9's . first: path=(. /bin)), then /bin (SH_FIND, shell/run.s), a union in the namespace (NAMESPACES.md) of:

1. the shell's cache, /ram/bin (-c: a copy into /bin goes there);
2. the shared cache, /sram/bin;
3. the boot card's /bin;
4. /rom/bin;

then \$PATH's directories. Libraries the same, in /lib, then \$LIBPATH. A name with a / is used as given, so /sd/0/bin/game always loads the card's copy.

- **Caching is copying:** cp game.hyx /sram/bin/game.hyx, and rm takes it out. (The cache and uncache commands this plan had are left out: a copy says the same, and HyForth's page 1 has under 200 bytes.)
- **Not checked against the card:** after rebuilding a program, copy it again (or remove it), or the old copy keeps running. (A program in the current directory runs before a cached copy, so the new one runs from the build's folder.)
- boot.hys can copy the programs used most into /sram/bin, for every task.

Tests

In the emulator:

- **The ROM disk (done):** /rom listed, read, a program run by name from /rom/bin, read-only everywhere (rom); every file copied to a card and compared with its source (rom-copy); the build's read-back check; /dev/sd/x/ctl (rom 4 MB 8192 blocks), the ROM disk not under /sd, ns showing the mount, /sd/8 no disk (rom).
- **The RAM disks (done):** the defaults at boot, their ctl files, /ram bound, the caches' directories, a file bigger than a bank through each and back to a card, compared (ram-disks); stop (busy with a file open) and start with sizes and ranges, busy, no room, bad text, the ROM disk refused (ram-ctl); less on a small machine (ram-small).
- **The areas and caches (done):** the shell's area, a pipeline's stage in it, another task's area and a name that isn't one refused, programs by name from both caches and from a stage, the current directory first (ram-areas).
- **An area removed as its task ends (done):** a script, a task of its own, makes an area with directories in directories and files and leaves one open; when it ends, the area is gone and the space back (ram-area-end).

- **Task 0's requests (done):** an area's removal runs as task 0 (`SD_CLIENT = 0`) and walks into another task's area, so `ram-area-end` passes only if task 0 passes the check.
- **An area kept while a task its task started has a file in it open (done):** a script starts `upper` in the background, writing into the script's area; the area stays when the script ends, and goes at the next end of a task in that slot (`ram-area-busy`).
- **The speed (done):** a 16K program by its full path from the card, then from `/sram/bin`: at least 2.5 times as fast (`ram-speed`).
- **The boot (done):** no `/rom` bind, and no `/rom/boot.hys`, with a paged ROM image that has no volume (`rom-none`); `/rom/boot.hys` run with no card (`rom`), not with one (`ram-disks`); `/ram` as the current directory with no card (every test without a card: the prompt `/ram>`).
- **The test song as a file (done):** `sndtest` plays it from the ROM disk (`sound`).

Steps

1. **The ROM disk.** *Done:* the disk `x` (disk 8) in the storage driver, read-only in the driver, `/dev/sd` and HydraFS; `/rom` as a bind; `mkromdisk.js` with its read-back check; the `rom-copy` test; the old server and format gone; new tasks starting on bank 0.
2. **The RAM disks.** *Done:* the memory manager's `MM_BANK_ALLOC_IN` and `SH_BANK_ALLOC/SH_BANK_FREE`; `r` and `s` (disks 9 and 10) in the storage driver, `start` and `stop`, the defaults at boot; `/ram` (and `/sram` through HydraFS); the areas' check and `ERR_IO_PERM`; `TASK_ORPHANS`; the caches in the shell's search and their directories; the tests; the docs.
3. **An area removed when its task ends.** *Done:* `TASK_AREA_END`, `HFS_AREA_END`, `RAMD_AREAS`.
4. **The test song as a file.** *Done:* `sndtest` plays `/rom/songs/test.zsm` (`ZSM_PLAY_TEST`); the player's ROM mode and bank 2 are gone, and the volume starts at bank 2.
5. **The boot.** *Done:* `/rom` (and `/ram`) bound only when there; `/rom/boot.hys`; with no card, `/ram` as the current directory.
6. **TASK_MAY as a kernel routine.** *Done* (page 5), with the way `/proc` will need.
7. Cards on SPI devices `$8-$F`, with the slot map, whenever a slot card has one.

The [namespace plan](#) then turns the search and the default names into binds, and the [/proc plan](#) uses `TASK_MAY` for task memory.

Video: the Vera X card in slot 0

A plan for the Hydra-16's supported video card: a card in **slot 0** carrying the **VERA** (the Versatile Embedded Retro Adapter, the Commander X16's video chip: an iCE40UP5K FPGA with 128K of video RAM, VGA out, a 16-voice PSG and PCM audio). The card is called **Vera X** here. Its 32 registers fill slot 0's **I/O ports 2 and 3** (\$FF20-\$FF3F). Its interrupt is slot 0's **IRQ A, line 2**, and **IRQ B, line 3**, is for its keyboard and mouse controller. The VERA's source (the module's PCB, gateway v0.9 and its programmer's reference) is in `c:\source\vera-module`. Nothing here is built yet.

Contents

1. [Why the VERA](#)
2. [The card](#)
3. [The registers on the Hydra](#)
4. [Sharing one chip between 16 tasks](#)
5. [The software, in steps](#)
6. [The emulator](#)
7. [Risks and open questions](#)
8. [Order of work](#)

Why the VERA

- **It fits slot 0 exactly.** The VERA has 32 registers, `A0-A4`. Slot 0's two port selects are ports 2 and 3, `$FF20-$FF2F` and `$FF30-$FF3F`: together 32 bytes in a row, with `A4` picking the port. So the card's chip select is the two selects ANDed, and the VERA's `A0-A4` come straight from the bus. The hardware reference already marks ports 2 and 3 "intended for video".
- **It's made for an 8-bit bus** (the X16's 65C02 at 8 MHz), with 5 V level shifting on the module and its own 25 MHz clock: the Hydra's 3.58 MHz bus is slow for it.
- **It's a lot of fun for a 65C02:** 640x480 VGA, two layers (text, tiles or bitmaps, 1-8 bits per pixel), hardware scrolling, 128 sprites, a 256-colour palette from 4096, line and frame interrupts, and sprite collisions. Also 16 PSG voices and a 4K PCM FIFO.
- **It brings software with it.** The Hydra's song format, ZSM, is the X16's. An X16 ZSM's PSG writes are skipped today (`sound/player.s`); with the VERA they play, so X16 music plays in full. cc65 has VERA support for the X16: headers, `vpeek/vpoke`, and a TGI graphics driver (`cx16-320-8`). These are mostly a change of base address, from `$9F20` to `$FF20`. The X16's documentation, demos and tools teach the chip.
- **It's open:** the gateway, the module's PCB and the reference are all in the repository.

The card

The module's connector

The VERA module (rev 4, `pcb/rev4`) plugs in through a 2x12 header, J1:

J1 pin	Signal	J1 pin	Signal
1	+5 V	2	GND
3-10	D7-D0	11	CS#
12	RES# (to the FPGA's <code>CRESET_B</code> : it reloads its gateway)	13	WR#

J1 pin	Signal	J1 pin	Signal
14	IRQ#	15	A4
16	RD#	17, 18	A2, A3
19, 20	A0, A1	21, 22	GND
23	Audio left (line level)	24	Audio right

On the module, the data bus goes through a 74LVC4245A (enabled by CS#, direction from RD#). The control lines go through a 74CBTD3861 bus switch. The gateway's bus interface (fpga/source/top.v) works like this:

- A read is CS# and RD# low with WR# high. The data comes from a register mux, and DATA0/DATA1 give the byte fetched ahead of time.
- A write is CS# and WR# low. **The data is latched as the write strobe ends** (negedge bus_write).
- IRQ# is driven both ways ((ISR & IEN) == 0), not open-collector.

Option A: a carrier card for the stock module (first)

A small slot card with a 2x12 socket for an unmodified VERA module, which can be bought as an X16 part or built from pcb/rev4, plus a little glue logic:

VERA	From the slot	How
CS#	nIOA_S (pin 51), nIOB_S (pin 49)	AND (74ACT08): low for either port. Both are already qualified by PHI2 on the board
RD#	PHI2 (pin 39), R/W (pin 21)	NAND(PHI2, R/W) (74ACT00)
WR#	PHI2, R/W	NAND(PHI2, NOT R/W)
A0-A4	A0-A4 (pins 62, 60, 58, 56, 54)	Straight through
D0-D7	D0-D7 (pins 18-4)	Straight through (the module buffers them)
IRQ#	IRQA (pin 55): line 2	Through an open-collector buffer (74LS07 or 74HCT07) or a Schottky diode: the board pulls the line up, and slot IRQ outputs must be open-collector
RES#	RESB (pin 3)	Straight through. A reset reloads the FPGA from its flash, which takes a while: the ROM waits for it (below)
Audio L/R	SND_CL0, SND_CR0 (pins 26, 24)	Through coupling capacitors and a divider: the WM8524 DAC gives up to 2 V rms, so match it to the mixer (IC9)
+5 V, GND	Pins 5, 57 / 1, 19, 61	The module draws about 200 mA

Why RD# and WR# come from PHI2, not just R/W: the VERA latches write data when the strobe ends. If WR# were only R/W, the write would end when CS# goes high. CS# comes through the board's decoding (U14, then the 74LS154 U19), so it rises some tens of nanoseconds after PHI2 falls, by which time the CPU may have stopped driving the data (the W65C02S holds it about 10 ns). A fast NAND on PHI2 ends the strobe within a few nanoseconds of PHI2 falling, while the data is still there. This is how the X16 does it. Check it on the bus breakout card with a logic analyzer before trusting it; the fallback is a 74ACT574 latching the data on PHI2's falling edge.

IRQ B (line 3): the input controller. A Hydra with a screen wants a keyboard, and the board has none (its console is the serial port). The card adds a small microcontroller (an RP2040, or an AVR like the X16's SMC) for a PS/2 keyboard and mouse, and optionally two SNES pads:

- It talks over I2C, which every slot already has (pins 28 and 30, bit-banged by the VIA, PA0/PA1). It needs no I/O port, and ports 2 and 3 are the VERA's.

- It pulls `IRQB` (**line 3**) while it has keys, mouse moves or pad changes waiting, so nothing polls. Line 3 is just below the VERA in priority, and above the YM2151 (4).
- A keystroke is a few bytes at I2C speed, which is plenty fast for typing. Mouse packets come at most 100 times a second.
- PS/2 can't be read directly from the VIA instead: its bits come every 60-100 μ s, and the ROM sometimes keeps interrupts off for longer than that (the emulator reports runs of over 1,000 cycles, about 300 μ s).

The VERA's SD card slot is on its own SPI controller, at 12.5 MHz with auto-transfer. That's much faster than the Hydra's VIA-driven SPI. It could later carry a second HydraFS card (an SPI back end for `drivers/sd.s`), which makes it a fast card for programs and assets.

Option B: Vera X on one board (later)

The same circuit laid out on a full Hydra slot card with the glue logic and the input controller: one board, no module. That circuit is the iCE40UP5K (with its 128K of video RAM inside), its flash, the audio DAC, the VGA resistor DACs and the level shifters. `pcb/rev4` is the reference. Do this only after option A has proven the timing.

The gateway

Start with the module's gateway as it is (v0.9 in `c:\source\vera-module`; the Commander X16 community has later releases). Running it unchanged is itself a feature, because it keeps X16 code and documentation valid. Possible changes later, since the source is here:

- An ID/version register to detect the card by. For v0.9, the ROM detects the card by writing and reading back `ADDR0_L` and `DC_BORDER` instead.
- An optional second interrupt pin on option B, for raster (line) interrupts on their own.

The gateway already fills most of the UP5K, so check the utilisation report before adding anything bigger, such as a PS/2 controller.

The registers on the Hydra

The VERA's register `n` is at `$FF20 + n`:

Hydra	VERA	Port	Hydra	VERA	Port
<code>\$FF20</code>	<code>ADDRx_L</code>	2	<code>\$FF30</code>	<code>L0_HSCROLL_L</code>	3
<code>\$FF21</code>	<code>ADDRx_M</code>	2	<code>\$FF31</code>	<code>L0_HSCROLL_H</code>	3
<code>\$FF22</code>	<code>ADDRx_H</code> (increment, DECR, bit 16)	2	<code>\$FF32</code>	<code>L0_VSCROLL_L</code>	3
<code>\$FF23</code>	<code>DATA0</code>	2	<code>\$FF33</code>	<code>L0_VSCROLL_H</code>	3
<code>\$FF24</code>	<code>DATA1</code>	2	<code>\$FF34</code>	<code>L1_CONFIG</code>	3
<code>\$FF25</code>	<code>CTRL</code> (reset, DCSEL, ADDRSEL)	2	<code>\$FF35</code>	<code>L1_MAPBASE</code>	3
<code>\$FF26</code>	<code>IEN</code>	2	<code>\$FF36</code>	<code>L1_TILEBASE</code>	3
<code>\$FF27</code>	<code>ISR</code>	2	<code>\$FF37 - \$FF3A</code>	<code>L1_HSCROLL_L ...</code> <code>L1_VSCROLL_H</code>	3
<code>\$FF28</code>	<code>IRQLINE_L / SCANLINE_L</code>	2	<code>\$FF3B</code>	<code>AUDIO_CTRL</code>	3
<code>\$FF29 - \$FF2C</code>	<code>DC_VIDEO</code> , <code>DC_HSCALE</code> , <code>DC_VSCALE</code> , <code>DC_BORDER</code> (DCSEL 0); <code>DC_HSTART ... DC_VSTOP</code> (DCSEL 1)	2	<code>\$FF3C</code>	<code>AUDIO_RATE</code>	3
<code>\$FF2D</code>	<code>L0_CONFIG</code>	2	<code>\$FF3D</code>	<code>AUDIO_DATA</code>	3
<code>\$FF2E</code>	<code>L0_MAPBASE</code>	2	<code>\$FF3E</code>	<code>SPI_DATA</code>	3
<code>\$FF2F</code>	<code>L0_TILEBASE</code>	2	<code>\$FF3F</code>	<code>SPI_CTRL</code>	3

In `include/hw.inc`: `VERA_BASE = $FF20` and the register names (`VERA_ADDR_L ...`), as `VIA_*` and `ACIA_*` are now. `IRQ_NUMBER(2) (5)` is its vector index, and `IRQ_NUMBER(3) (4)` the input controller's.

Speed at 3.58 MHz: an unrolled copy loop (`lda (zp),y / sta VERA_DATA0 / iny`) moves about 300K a second into VRAM. A full 80x60 text screen (9,600 bytes) takes about 30 ms, two frames. A 320x240 bitmap at 8 bits per pixel (76,800 bytes) takes about a quarter of a second. Scrolling is free: a write to `VSCROLL`.

Sharing one chip between 16 tasks

The VERA has one set of address pointers (`ADDR0`, `ADDR1`, chosen by `CTRL`'s `ADDRSEL`) and one `DCSEL`. A task that's switched out between setting an address and writing `DATA0` loses it to whichever task touches the chip next. So:

- **A driver owns it.** The `vid` driver (a resident driver task, like the sound task) sets the chip up, runs the screen console, and serves `/dev/vid`.
- **Programs claim it for direct access.** A program that wants speed opens `/dev/vid` and claims the chip (an `IO_CTL`, as `/dev/snd` claims channels). Until it closes it, or ends (a task's end releases its claims, as with semaphores and sound channels), it may write the registers directly; the driver leaves them alone. Console output meanwhile still goes to the serial port; the screen console picks up again on release. Only one claim at a time; others get `ERR_IO_BUSY`, or wait.
- **The interrupt handler keeps the program's state.** The line 2 handler always reads and clears `ISR`. Anything that also touches VRAM (moving sprites at the frame's start, say) first saves `CTRL` and `ADDR1` and puts them back afterwards. The registers read back, so that's about 40 cycles. The convention is that interrupt code uses data port 1 and programs use port 0, as on the X16.
- **VRAM is shared by agreement.** The console keeps its map and font at the top of VRAM, and programs get the rest:

VRAM	What
<code>\$00000-\$1AFFF</code> (108K)	The program's: bitmaps (320x240x8 bits is 75K), tiles, sprite images
<code>\$1B000-\$1EFFF</code> (16K)	The console's text map: 128x64 entries of 2 bytes (80x60 shown; the extra rows make scrolling a <code>VSCROLL</code> write)
<code>\$1F000-\$1F7FF</code> (2K)	The console's font: 256 characters, 8x8, 1 bit per pixel
<code>\$1F800-\$1F9BF</code>	Free
<code>\$1F9C0-\$1FFFF</code>	The PSG, palette and sprite attributes (the chip's)

A program that wants all 128K says so when it claims (`VID_CLAIM_ALL`). The driver then reloads the font and clears the map on release.

The software, in steps

Each step is usable on its own and has regression tests in the emulator.

1. The emulator's VERA (first: everything else is developed against it). See [below](#).

2. Detection and the driver (BIOS ROM **page E**, which is empty).

- **POST** looks for the card. After a reset the FPGA reloads its gateware for a while: wait up to about half a second for `ADDR0_L` to read back what was written. Then the boot report says `VERA` (or nothing, with no card).
- **vid, a resident driver task**, registers a handler on IRQ line 2 and sets up the chip:
 - it clears the register area `$1F9C0-$1FFFF`, as the reference advises, since VRAM powers up random;
 - it loads the font (a file: `/rom/lib/font8x8`, so a card's `/lib/font8x8` can replace it);
 - it sets layer 1 to 80x60 text, 16 colours;
 - it shows a boot logo (also a `/rom` file).

- **The frame interrupt** counts frames and wakes the tasks waiting for one. A 60 Hz handler through the normal dispatcher (`TASK_CALL_IRQ` into the driver's task) is cheap enough.

3. `/dev/vid`, Plan 9 style. Its files:

File	Read	Write
<code>/dev/vid/ctl</code>	The mode, as text (<code>80x60 text</code> , <code>320x240x8 bitmap</code> ...)	Commands: <code>mode text</code> , <code>mode bitmap 320 8</code> , <code>layer 0 on</code> , <code>border 6</code> , <code>claim</code> , <code>release</code> ...
<code>/dev/vid/vram</code>	VRAM: the offset is the address (<code>IO_SEEK</code>)	VRAM, through <code>DATA0</code> with auto-increment, 256 bytes a request
<code>/dev/vid/pal</code>	The palette: 2 bytes an entry	The palette
<code>/dev/vid/sprites</code>	The sprite attributes: 8 bytes a sprite	The sprite attributes
<code>/dev/vid/frame</code>	Waits for the next frame, then gives its number (4 bytes)	

So `cp /sd/0/pics/logo.pal /dev/vid/pal`, or a C program's `fopen("/dev/vid/vram")`, `fseek` and `fwrite`, load images with no special code. For speed, programs claim the chip and write it directly (step 5).

4. **The screen console.** A terminal emulator on the text layer, understanding the ANSI sequences the console already uses: `conio's (lib/conio)`, the editor's, HyForth's `Ac1s/Ascr/Aco1`. So every program that works on a serial terminal works on the screen, colours included. It has:

- a cursor (blinked by the frame interrupt);
- scrolling by `VSCROLL`;
- `/dev/cons` output mirrored to the screen and the serial port, or sent to one of them (`/dev/cons/ctl: screen, serial, both`);
- keys from the input controller (step 6) and the serial port both reaching `/dev/cons`.

With a keyboard, the Hydra is a standalone computer: switch on, get a prompt on the monitor.

5. Programming it.

- **HyForth, a video library:**
 - the chip: `vpoke` and `vpeek` (a 17-bit address as an address and a bank bit), `pal`, `vmode`, `cls`;
 - drawing: `plot`, `line`, `box`, `circle`;
 - sprites and tiles: `sprite`, `spimg`, `tile`;
 - timing: `vsync` (waits for a frame);
 - **turtle graphics**: `fd`, `bk`, `rt`, `lt`, `pu`, `pd`, `home`, the classic way into graphics for a new programmer.
- **C, vera.h:**
 - the chip: the register block as a struct at `$FF20` (cc65's `cx16.h` VERA layout), `vpoke/vpeek` (from cc65's `cx16` library);
 - sharing it: `vera_claim()/vera_release()`, `vera_wait_frame()`;
 - data: `vera_load(path, addr)` to load a file into VRAM;
 - sprite helpers;
 - a **TGI driver** made from cc65's `cx16-320-8`, so cc65's portable graphics programs run.
- **Assembly:** `hw.inc`'s names, and a page in the Programmer's Guide (`docs/programming/video.md`): the registers, the sharing rules, the VRAM map, and the interrupt conventions.

6. Keyboard, mouse and pads (the input controller on IRQ line 3):

- **The firmware**, for an RP2040 (or an AVR, after the X16's SMC firmware), turns PS/2 scan codes into the console's key codes, including the cursor and function keys `conio` decodes (`CH_CURS_UP` ...).
- **An input driver** on line 3 reads the controller over I2C (the first I2C driver: `/dev/i2c` is useful on its own). It feeds keys to `/dev/cons`, and offers `/dev/mouse` (Plan 9's format: `m x y buttons`) and `/dev/pads`.

- **The mouse** can drive a sprite as its pointer.

7. Sound.

- **The PSG:** 16 more voices for `/dev/snd` (channels 8-23: frequency, waveform, pulse width, volume, pan). The ZSM player plays a song's PSG writes instead of skipping them (`sound/player.s`). HyForth's `note` and C's `snd.h` reach them as they reach the FM channels.
- **PCM:** the `AFLOW` interrupt (the FIFO below a quarter full) refills the 4K FIFO from a buffer, so `play` takes WAV files (8 or 16 bits, mono or stereo, up to 48 kHz) and ZSM's PCM extension.
- **Both chips at once:** the YM2151 on the board and the VERA's PSG and PCM, mixed on the board through slot 0's audio pair.

8. Fun things to make with it (samples in `programs/`, and in `/rom/bin` once they're small and good):

- a sprite demo (bouncing balls, using collision interrupts);
- a smooth tile scroller (a map bigger than the screen);
- Snake, Breakout and Tetris in C, with the PSG for effects;
- a paint program with the mouse;
- a Mandelbrot in 256 colours;
- a ZSM jukebox with the song's channels drawn as bars, from the frame interrupt;
- and ports of X16 programs, which mostly need the new base address and the Hydra's file calls.

The emulator

`hydrasim.js` gets a VERA on slot 0 (`--card 0:vera`), modelled from the reference. The X16's own emulator (`x16-emulator, video.c`) is a good guide to the details. It covers:

- the registers;
- the 128K of VRAM and the address increments;
- the palette;
- the layers: 1 bpp text in both colour modes, 2/4/8 bpp tiles, and bitmaps;
- the sprites;
- the frame (60 Hz: about 59,700 cycles at 3.58 MHz) and line interrupts, `ISR` clearing, and sprite collisions;
- the PCM FIFO's level and `AFLOW`.

Seeing it:

- **In tests:** `--screen` prints the text layer as text in the report, so regression tests check the screen just as they check serial output, with no image comparison. `--frame-png FILE` saves the last frame as a PNG (Node's `zlib` does the compression) for tests that need pixels and for documentation.
- **Live:** the emulator's core gets separated from Node's terminal and file system (see the [code review](#)), so the same core runs in a web page: a canvas for the VERA, a terminal (`xterm.js`) for the serial port, and keys for the input controller. That page is also how most people will first try the Hydra.

Tests: detection (with and without the card); the console's text, colours and scrolling (`--screen`); `/dev/vid`'s files; a claim and its release (the console coming back); the frame interrupt waking `/dev/vid/frame`; the PSG voices (writes counted, as the YM2151's are); a ZSM's PSG part played; the emulator's own interrupt timing.

Risks and open questions

- **Write timing** (above): confirm `WR#` from `PHI2` with a logic analyzer on the breakout card before building more than one.
- **Reset time:** the FPGA reloading after `RESB` must be waited for (POST), and a VERA `CTRL` reset (bit 7) does the same.
- **The CPU at 7.16 MHz:** the VERA is fine (the X16 runs at 8 MHz); the YM2151 still isn't (no wait states on `V1`).

- **Audio level** into the board's mixer: measure, and size the divider.
 - **ROM space:** the driver and terminal fit BIOS page E (8K). Fonts, the logo and help text go in `/rom`. Page 0 and COMMON are full, so the driver uses gates. A fast interrupt path (for raster effects at line rate) would need a stub in COMMON (1 byte free). Merging `PEEK_D_XAM` and `FP_PEEK_PAGE` into one routine would make room ([code review](#)).
 - **The namespace:** `/dev/vid` lives under `/dev` (no mount), so it costs no namespace entries.
 - **Which gateware:** v0.9 from this repository first. Later X16 releases add features but must still fit the module's flash and the Hydra's expectations.
-

Order of work

1. The emulator's VERA, `--screen`, and its tests.
2. The carrier card (option A) with the glue logic: check the timing on the breakout card.
3. Detection, the `vid` driver, the font and the screen console (output only). Now the Hydra shows its prompt on a monitor.
4. `/dev/vid`, claims, HyForth's `video` library, C's `vera.h`.
5. The input controller and its firmware, `/dev/i2c`, keyboard input: a standalone Hydra.
6. PSG and PCM in `/dev/snd` and the player.
7. Demos and games; the TGI driver; the web emulator.
8. Option B: the one-board Vera X.

Hydra 16: Sound (the YM2151)

The plan for the Hydra's sound: a library for the YM2151, a song player, a test song that uses everything the chip can do, and importing music made for other machines. Phases 1-4 are done; importing is planned. How to use what exists is in [io.md](#), [HyForth](#) and [the C guide](#).

The chip on the Hydra

- A **YM2151** (OPM: 8 channels, 4 operators each, 8 algorithms, feedback, an LFO with 4 waveforms, noise on channel 7, 2 timers) and a **YM3012** stereo DAC, then op-amps to `AUDIO_L` / `AUDIO_R` and the mixer ([hardware](#)). CT1/CT2 (two output bits, \$1B) are on a header.
- **Clock:** `SND_CLK`, 3.579545 MHz, the same as the Commander X16's and many arcade boards' (Capcom CPS1, Sega System 16, Konami, Namco), so their key codes give the same pitches: MIDI note 69 (A, 440 Hz) is key code \$4A. The X68000 ran its YM2151 at 4 MHz: its music needs retuning (below).
- **I/O:** port 4, \$FF40 (register) and \$FF41 (data, and the status: busy, the timers' flags). After a data write the chip is busy for 64 of its clocks (about 64 CPU cycles at 3.58 MHz); `YM_WRITE` waits for it. No wait states on board V1, so the chip can't be used with the CPU at 7.16 MHz.
- **IRQ line 4:** the timers' interrupt: timer B is the sound clock (phase 4).

Phase 1: the library (*done*)

Where: BIOS ROM page B (`os_rom/sound/`), in the sound task; page 2 gave up the sound code it had (about 800 bytes back). Page B has about 750 bytes left.

The interface is `/dev/snd`, as Plan 9 would have it: register/value pairs written to a file.

- **The shadow.** The chip's registers can't be read, so the library keeps each as written (`SND_SHADOW`, 256 bytes of the sound task's RAM), and reads of `/dev/snd` give it back.
- **Commands at the registers the chip doesn't have** (`$02-$07`, `$09`, `$0A`): a channel, a patch, a note, key off, volume, speakers, bend, a drum. One protocol carries both raw register writes and the library's commands, so anything that can write pairs (HyForth's `ywrite`, a C program, a song's register stream) can use either, in one request.
- **Volume without a special path.** A carrier's level (TL) gets the channel's and the master volume's attenuation as it's written; the shadow keeps the level as written. So the volumes work on raw streams too (a song played from someone else's register dump), and changing a volume writes the channel's four levels again. Which operators are carriers depends on the algorithm, so a new algorithm writes them again too. The curve is General MIDI's: $40 \log_{10} (v / 127)$ dB, in the chip's 0.75 dB steps.
- **Pitch:** MIDI notes, and a bend in 64ths of a semitone, which is the chip's own key fraction.
- **Patches:** the Commander X16's General MIDI set (128 instruments and 35 drum and percussion sounds, 26 bytes each, and its General MIDI drum map), 2-clause BSD, with its notice in `sound/patches.s`.
- **Claims, tied to an open file:** a fd claims channels (`SND_CTL_CLAIM`); other fds' writes to them are dropped; the last close gives them back. A task's end closes its fds, so a program that dies can't hold a channel. The console's bell (channel 7) stays off a claimed channel. This is how music and sound effects share the chip, and how two programs can't cut into each other's notes.
- **Not in interrupts:** register writes run in the server, with interrupts on. Each takes about 64 cycles of the chip's busy time, and a song's tick can have dozens; with interrupts off that would lose serial bytes at 115,200 baud (a byte every 320 cycles, and the 65C51 holds one).
- **Also:** HyForth's `patch`, `note` and `noteoff`; C's `snd.h`; the emulator's `--ym-dump` and `--ym-vgm` (listen to what the ROM played); the `sound-lib` test.

Phase 2: the song player (*done*)

Built: `os_rom/sound/player.s` on BIOS ROM page C (627 bytes; it uses only gates, so it has a page of its own and page B keeps its room). `play song [n] [&]`, a song by its name (theme finds `theme.zsm`, after `.hyx` and `.hys`), run `song.zsm` (the shell knows a song by its `zm`, as it knows an executable by `HYX1`), and C's `snd_play` (with `hy_kill` to stop one). `n` plays the song's loop `n` more times; none: the song once, to its end; 0: forever. Tests: `songs` (a song made in the test: its timing, loops, a claim, Ctrl-C, `jukebox`).

Measured in the emulator with songs from the X16's tools (zsound's `BGM.ZSM`, 564 notes at 60 Hz; zsmkit's `SONG1.ZSM`, 935 notes at 62 Hz, with PSG parts skipped): every note played, on average 1.5 ms from the song's time, with no drift over a minute (under 2 ms at the end). A note is late when its tick carries a lot of register writes before its key-on (the song's first tick, setting up every channel, 40-50 ms; a big patch change, 15 ms): the chip takes the writes one at a time. What made it that close:

- the system tick was 0.35% fast (the timer's latch had 64 cycles taken off for "the handler's time", but a free-running VIA timer counts latch + 2 cycles whatever the handler does), which also made the clock gain 5 minutes a day; fixed (`TIMER_TASK_INT`, `kernel.inc`), and the `sleep` test's window narrowed so it can't come back;
- the song's time in 8.16 fixed point ($200 \times 65536 / \text{the rate}$), from a half, so each tick rounds to the nearest system tick;
- the song's start on a tick boundary, two ticks after the player starts (the shell that started it is waiting by then);
- the file read ahead while the player waits (a card's block can take 10 ms), not when a tick's writes run out.

The design, as planned:

The format: ZSM, the Commander X16's "Zsound" music format ([spec](#)), as it is:

- It's a stream of YM2151 register writes and delays, made for this chip at this clock: a 16-byte header (`zm`, a version, a loop point, the channels it uses, a tick rate in Hz, 60 by default), then commands: `$41-$7F` `n` register/value pairs, `$81-$FF` a delay of `n` ticks, `$80` the end, `$00-$3F` a VERA PSG write, `$40` an extension (PCM, sync events).
- **Furnace exports it**, and so do the X16's tools, so a composer can write for the Hydra today in [Furnace](#) (a free multi-system tracker with a YM2151 system), and X16 music plays at the right pitch.
- The Hydra plays the FM part. PSG writes and PCM are skipped (the Hydra has neither); sync events could become notes to the player's client later.
- A register stream needs no instrument logic in the player, so the player is small, and anything a composer can make the chip do plays back exactly.

The player runs in a task of its own, as a **client of** `/dev/snd` (not in the driver): `play song.zsm`, `play song.zsm &`, or C's `snd_play`. Like `run`: the file on an fd, the player (ROM, page B) in a new task, the shell waiting unless `&`. So the usual machinery applies: `wait`, `kill`, Ctrl-C (status 130), the exit status, a background song while the shell works.

- It claims the channels the header lists (`SND_CTL_CLAIM`): a second song, or a program's sound effects on other channels, can't collide with it. Its end (or a kill) closes `/dev/snd`, which keys its channels off.
- It reads the file in 256-byte blocks, collects each tick's register pairs, and sends them in one write (one request a tick), then sleeps to the next tick.
- **Timing:** the system tick is 200 Hz (5 ms), so a 60 Hz song's ticks land 15 or 20 ms apart; with a running target (`TASK_SLEEP_UNTIL` and a fraction), the tempo is exact on average. Phase 4 made each tick exact.
- **Cost:** a request is about 2,000 cycles, and the server about 150 a pair: a 60 Hz song with 10 writes a tick is about 6% of the CPU. Songs are streams: 0.5-2 KB a second, so a 3-minute song is 100-300 KB, read from the card as it plays (well within the SD card's rate).
- **Loops:** the header's loop point, a number of times (`play song.zsm 3`), or forever until stopped.
- A text `/dev/snd/ctl1 (play, stop, volume 90`, a status line) may follow, as Plan 9's audio devices have one.
- The driver keeps the timers' interrupt enables (`$14`) off whatever a song writes: a register dump from an arcade game has its driver's timers in it, and an interrupt nothing clears would stop the machine.

Phase 3: a test song that uses the whole chip *(done)*

Built:

- **The score compiler**, `sim/tools/hysong.js` ([emulator.md](#)): a score is instruments (the ROM's patches by number, or a voice's registers by name: `alg`, `fb`, and each operator's `mul`, `dt1`, `t1`, `ar` ...) and a line of MML for each channel (notes, rests, lengths, octaves, legato, slides, repeats, volume, speakers, detune, the LFO, noise, General MIDI drums, and raw register writes). It works out the patches and volumes itself, so its ZSM is plain register writes; it can write a VGM for listening, and the ROM's source.
- **The test song**, `os_rom/songs/test.mml`: 31 bars (66 s) in A minor, 852 notes on all 8 channels, 14 KB. The build compiles it to `songs/test.zsm`, a file on the ROM disk (`/rom/songs/test.zsm`), and `sndtest` (`SND_CTL_TEST`) plays it: the song player (page C) opens it as any song file, in its own task (`ZSM_PLAY_TEST`). (It was in paged ROM bank 2, read by a mode of the player's own, until the ROM disk.) In the emulator all 852 notes play, 1.3 ms from the song's time on average.
- **The old riff** (the first `sndtest`, `snd_test.s`) is `programs/songs/scom.mml` (`scom.zsm`: on a card, it runs as the command `scom`), a demo of the score language; its ROM code is gone (page B has about 1,200 bytes free).

What the song uses (each checked in its register stream when it was written: all 8 algorithms, feedback 0-7, the four LFO waves, left, right and both, noise, CSM; no key-on on a silent channel):

- **all 8 channels** at once, with **all 8 algorithms**, from 1 carrier (a 4-operator stack) to 4 (organ-like);
- **feedback** (M1's self-modulation, 0-7), **detune** (DT1, and DT2's inharmonic ratios for bells), **multipliers**, **key scaling** (brighter, faster notes higher up);
- **envelopes**: fast and slow attacks, the two decay rates and sustain level, releases;
- **the LFO**: vibrato (PMD, PMS) and tremolo (AMD, AMS, and each operator's AMS-EN), its four waveforms (saw, square, triangle, noise) and its rate;
- **noise** on channel 7 (its frequency: hi-hats, snares, wind);
- **stereo**: channels panned left, right and both, and moving;
- **the timers**: CSM (timer A keying all operators, the chip's speech trick) in the coda: an A major chord pulsed at 110 Hz, then 82 (the emulator doesn't model CSM's sound, so this is for the board, or a VGM player: `--ym-vgm`); the player's tick is timer B (phase 4), so timer A is free for this;
- pitch bends and slides (key fractions), and a drum part on the General MIDI drum patches.

The instruments were written for it (`@lead` algorithm 4, `@bell` 5, `@pad` 6, `@organ` 7, `@bass` 0, `@synbass` 1, `@brass` 2, `@pluck` 3, and noise ones for channel 7), and the drums are the ROM's General MIDI kit. It hasn't been heard on the board yet: its voices are first drafts, to adjust by ear.

Phase 4: the chip's own clock *(done, then set aside)*

On the board, the YM2151's timer B didn't keep its period while the driver re-armed it each tick (its flag reset through `$14`, which there seems to restart it): songs ran up to twice as fast, by a different amount each run. So the player times songs by the system's tick again, as before this phase, with the scheduler's fixes (the woken player runs next, and holds the CPU through its tick's work); the sound clock stays in the driver (`SND_CTL_CLOCK`), unused.

Built: the **sound clock**: the YM2151's timer B, at the song's rate, drives the player (`SND_CTL_CLOCK`; `sound/ymfast.s`, the player's `ZSM_CLOCK_START` and `ZSM_CLOCK_WAIT`).

- **Timer B, not A**: timer A's steps are finer (18 us), but songs use it for CSM (the test song's coda does), and a song from a VGM may program it; timer B's are 286 us, and the rate comes out exact anyway: each period is K or $K + 1$ units, as a 16-bit fraction carries (Bresenham's way), so the ticks are never more than one unit (0.3 ms) from where they belong and the average is exact. Range: 14-3,495 Hz.
- **A fast handler**, as the VIA's and the ACIA's (the dispatcher's 650 cycles would be too long beside serial input at 115,200): a stub in the COMMON block (`YM_IRQ_STUB`: 6 bytes found by letting the last IRQ stub and `FAR_INLINE` run on instead of

jumping), then `YM_IRQ_FAST` on page 2: timer B's flag reset, the next period, the tick counted, and the player woken with a task switch when its time comes. About 150-300 cycles, as long as the chip keeps it waiting to be written.

- **The player** asks for the clock after it claims its channels, and sleeps until the clock reaches its time (its `ZSM_AT`, which the interrupt looks at). If another program has the clock, or the rate is out of its range, it times the song by the system tick as before. The clock is the fd's that started it, stops at its last close, and survives `SND_CTL_INIT`. A song's own writes to timer B are dropped while it runs.
- **Measured** (emulator): zsound's BGM.ZSM (60 Hz), 564 notes, 0.63 ms from the song's time on average (1.42 by the system tick), 0.2 ms at the end; the test song (200 Hz, the system tick's rate) as before, 1.4 ms. The worst notes are still the ones behind a tick of many register writes. The `irqs-off` test passes (the longest stretch is still the task-end cleanup's).
- Page 0 moved the bell's gate after the thunks (for `IRQ_INIT`'s 9 bytes); COMMON has 1 byte left.

Importing music from other machines (*investigation: nothing built yet*)

The approach: convert on the PC, to ZSM, with a Node tool (`sim/tools/hymusic.js`, no packages: Node's own `zlib` reads `.vgz`). The Hydra only ever plays ZSM. Converting on the PC keeps the ROM small, and the hard parts (retuning, translating other chips, choosing voices) need memory and arithmetic the Hydra hasn't got to spare. Every route below ends in the same file, testable in the emulator (`--ym-vgm` renders it) and on the board.

Source	Where it comes from	Route	How faithful
ZSM (<code>.zsm</code>)	Commander X16 music; Furnace's export	Plays as it is	Exact for the FM part; its PSG and PCM parts are lost, or (an option) the PSG voices moved onto spare FM channels as simple square-ish patches
Furnace (<code>.fur</code>), DefleMask (<code>.dmf</code>)	Furnace, with a YM2151 system (or the X16's)	Export ZSM from Furnace	Exact. The best route for new music. Furnace also opens many other trackers' files, so it's a hub for music in other formats
VGM (<code>.vgm</code> , <code>.vgz</code>) with YM2151 writes	vgmrips.net : thousands of arcade (Capcom CPS1, Sega System 16/18/X, Konami, Namco System 1/2, Atari) and X68000 soundtracks	<code>hymusic</code> <code>vgm2zsm</code>	The FM exactly. Their samples (drums on the OKI MSM6295/6258, Sega PCM) are lost: the Hydra has no PCM. Other chips' writes are skipped
MDX (<code>.mdx</code> , <code>.pdx</code>)	Sharp X68000 music (MXDRV), a very large library	<code>mdxtools'</code> <code>mdx2vgm</code> , then <code>vgm2zsm</code>	As VGM from an X68000: retuned from 4 MHz; the PDX drums (ADPCM) lost
VGM for other FM chips (YM2612/OPN2: Mega Drive; YM2203, YM2608: PC-88/98; YM2413)	vgmrips.net	<code>vgm2zsm</code> with a translation	Close for the FM: OPN2 and OPM are both 4-operator chips with the same algorithms, but their registers, detune and frequency encodings differ, and OPN2's SSG-EG and DAC channel have no OPM equivalent. A second stage of the tool
MIDI (<code>.mid</code>)	Everywhere	<code>hymusic</code> <code>mid2zsm</code>	General MIDI: the X16 patch set (as in the ROM), 8 voices allocated from the song's notes (oldest note stolen first; drums on their own channel or two), velocity and channel volume as levels, pan as left/right/both, pitch bend as key fractions, modulation as the LFO's vibrato. As good as the patches and the 8-voice limit allow: a MIDI file with 20 simultaneous notes loses some
Patches (<code>.opm</code> VOPM, <code>.dmp</code> DefleMask, <code>.fui</code> Furnace, <code>.tffi</code> , <code>.y12</code>)	Instrument libraries	<code>hymusic</code> <code>patch</code>	Exact for OPM patches; OPN patches translated. To YMP (26 bytes): into a song, as register writes, or (later) into user patch slots that <code>SND_R_PATCH</code> can load
Sample-based (MOD, XM, S3M, IT, WAV), SID, NES, SNES, AY		Not a direct route	The Hydra can't play samples, and other synthesis doesn't map onto FM. A person can remake one in Furnace

The converter's parts:

- **VGM reading:** the header (`Vgm`, version, the YM2151 clock at `0x30`, the data at `0x34` + its value, or `0x40` before version 1.50, the loop offset at `0x1C`, the GD3 tag's title and author); commands `0x54 aa dd` (YM2151), waits `0x61 nn nn`, `0x62` (1/60 s), `0x63` (1/50 s), `0x7n` (n + 1 samples), `0x66` (the end), data blocks (`0x67`, skipped), and every other chip's command skipped by its length. Two YM2151s (bit 30 of the clock) can't be played on one: the second's writes are dropped, or its channels merged onto unused ones where there are some.
- **Timing:** VGM counts 44,100ths of a second. The tool finds the song's own rate (the waits' common step: 60 Hz, 50 Hz or a tempo's) and quantizes to it, choosing the ZSM tick rate so nothing moves by more than a millisecond or two. Writes in one tick go out together.
- **Retuning** for a clock that isn't 3.579545 MHz (the X68000's 4 MHz is 1.92 semitones sharp at the same key code): each channel's key code and fraction are tracked and rewritten, shifted by the ratio in 64ths of a semitone (123 for 4 MHz). The LFO rate, the noise frequency and the timers scale too. Notes past the top of the range fold down an octave.
- **Size:** repeated register values are dropped (the tool keeps its own shadow), and loops found in the VGM become ZSM's loop point.
- **Checking:** the emulator plays the ZSM and writes `--ym-vgm`; the tool compares that stream with the source's (after retiming), and reports the differences.

Fidelity limits to expect: no samples (a lot of arcade drums, X68000 percussion, Mega Drive voice clips); 8 channels.

Copyright: music ripped from games and computers belongs to its composers and publishers. The tools convert files the user has; the repository and the ROM carry only music written for the Hydra (the test song) or under a license that allows it.

Order of work

1. ~~Phase 2, the player~~ (done).
2. ~~Phase 3: the score compiler and the test song~~ (done).
3. The importers, in the order they pay off: `vgm2zsm` for YM2151 VGMs (arcade and, with retuning, X68000), then `mid2zsm`, then patch conversion, then the OPN family.
4. ~~Phase 4, the chip's timer as the player's clock~~ (done).

HydraFS: the Hydra's SD card filesystem

A small filesystem for SD cards, designed for the 65C02 and the Hydra's Plan 9-style IO layer (see `IO_PLAN.md`): directories, files with a Plan 9-style qid, text and binary directory listings. It replaces FAT32 on the Hydra's cards. **All five build steps are done:** the SD layer's cards and sizes, the PC tool, and the server reading, writing and checking, on ROM page 6 (`os_rom/fs/hfs_srv.s`, `hfs_write.s`, `hfs_sparse.s`; the format, label and check are on page 3: `hfs_format.s`, `hfs_check.s`), with HyForth words. Since then: [partitions](#), [time stamps from a clock](#), and [sparse files](#).

Goals and trade-offs

- Easy for a 65C02: fixed-size directory entries, contiguous runs of clusters (extents) instead of a FAT chain, 32-bit arithmetic at most.
- Plan 9-like: every file and directory is reached by walking a path; stat records; a qid (unique id + version) that survives renames; create, remove, and rename by wstat; no hard links.
- Simple and robust enough: writes go in a safe order (data, then the free map, then the directory entry), but there's no journal.
- **Not readable on a PC.** Cards are made and read with a host tool (`sim/tools/hydrafs.js`), as disk images; a disk imager writes an image to a real card. The same images work in the emulator (`--sd`).

Names

Name	What it is
<code>/dev/sd/0/data</code> ... <code>/dev/sd/7/data</code>	Card 0-7 (SPI device 0-7) as raw bytes, at the fd's offset (the first 4 GB)
<code>/dev/sd/0/ctl</code> ... <code>/dev/sd/7/ctl</code>	Read: the card's details, as text (e.g. <code>sdhc 7580 MB hydrafs label=GAMES free=6100 MB</code>). Write: text commands: <code>init</code> (start the card again, e.g. after changing it), <code>format [-f] [-p] [-s size] [label], label <text>, check [fix]</code>
<code>/sd/0/...</code> ... <code>/sd/7/...</code>	The files on card 0-7: the HydraFS server (device <code>hfs</code>), mounted at <code>/sd</code> in each task's namespace (the shell mounts it at startup; the tasks it starts inherit it). <code>/sd/0</code> is card 0's root directory

Both servers run in the storage task (`$C`), which owns the SPI bus and the block cache; the filesystem uses the block layer directly, not `/dev/sd/N/data`. The filesystem is the whole card, or its HydraFS partition ([partitions](#)).

- Names: 1-31 characters, **case-sensitive**, any byte except `/` and `0`. `.` and `..` are understood when walking a path (the server keeps each open file's path), but aren't stored.

On the card

Blocks are the card's 512-byte sectors, counted from the HydraFS's first (block 0 of the card, or of its partition). Space is allocated in **clusters of 8 blocks (4 KB)**; cluster 0 is the first block of the data area. Numbers are little-endian.

Blocks	Contents
0	The superblock
1 ...	The free map: 1 bit per cluster (1 = in use), 4096 clusters per block (16 MB of card per block; a 32 GB card has 2048 map blocks)
after the map	The data area: clusters of file and directory data, and extent blocks

The superblock (block 0)

Offset	Size	Field
0	8	Magic: HYDRAFS1
8	1	Version: 1 (the whole free map written), or 2 (written as it's used: offset 44)
9	1	Cluster size, as a shift: 3 (8 blocks)
10	2	Reserved (0)
12	4	Clusters in the data area
16	4	The free map's first block (1)
20	4	Its size in blocks
24	4	The data area's first block
28	4	Free clusters (a hint: <code>check</code> recounts it)
32	4	Where to look for free clusters next (a hint)
36	4	The next qid id to give out
40	4	The latest modification stamp given (the clock's time; before the clock, this was a counter's next)
44	4	Version 2: the free map's blocks written so far, from its first; the rest read as all free. (Version 1: all of them are written; the Hydra keeps the map's size here, and it's ignored)
48	16	Reserved (0)
64	64	The root directory's entry (below)
128	32	The volume label, zero-terminated (31 characters at most)
160	352	Reserved (0)

Directory entries (64 bytes)

A directory is a file whose data is an array of 64-byte entries (8 per block). The entry *is* the file's metadata: there are no separate inodes. A free entry has a 0 first name byte; new entries fill free ones first, and a directory only grows when it's full.

Offset	Size	Field
0	32	Name, zero-terminated (0 first byte: a free entry)
32	1	Mode: bit 7 = directory, bit 6 = append-only, bit 0 = read-only
33	1	Reserved (0)
34	2	Qid version: goes up by 1 at every change (write, truncate, rename)
36	4	Qid id: unique on the card, never reused, the same across renames
40	4	Size in bytes (up to 4 GB)
44	4	Modification stamp: the clock's time at the last change (time stamps)
48	6	Extent 1: first cluster (4), clusters (2)
54	6	Extent 2: first cluster (4), clusters (2)
60	4	The first extent block, for more extents (a block number; 0 = none)

- An **extent** is a run of contiguous clusters: up to 65535 clusters (256 MB). A file's data is its extents in order. Most files need one or two; a fragmented file's further extents go in a chain of extent blocks.
- A **hole** is an extent whose first cluster is `$FFFFFFFF`: that many clusters of zeros, with nothing on the card ([sparse files](#)).
- The allocator tries to grow a file's last extent (the next cluster free?) before starting a new one, so files stay contiguous.

Extent blocks (512 bytes)

Offset	Size	Field
0	4	The next extent block (0 = the last)
4	2	Extents used in this block (0-84)
6	2	Reserved (0)
8	504	84 extents of 6 bytes (first cluster, clusters)

Stat records (48 bytes)

What `IO_STAT` returns, and what a binary directory read returns per entry: the directory entry without its extents.

Offset	Size	Field
0	32	Name, zero-terminated
32	1	Mode (as the entry)
33	1	Card (0-7)
34	2	Qid version
36	4	Qid id
40	4	Size
44	4	Modification stamp

(`IO_STAT`'s block grows from 16 to 48 bytes; `/dev/null` and friends return all zeros, as now.)

Reading a directory

Open it for reading, then read:

- **Text** (the default): a line per entry, `name size` (a `/` after a directory's name, and no size), then CR LF. E.g. `games/` and `star.fr 1234`. Good from HyForth: `q^/sd/0^ 1 open then cat`.
- **Binary**: open with `IO_MODE_STAT` (new mode bit): each read returns whole 48-byte stat records.

Calls and requests

Existing calls work as they do on devices: `IO_OPEN` walks the path (`IO_MODE_TRUNC`, new, empties a file as it's opened), `IO_READ`, `IO_WRITE` (writing past the end grows the file), `IO_SEEK`, `IO_CLOSE`, `IO_STAT`. New:

Call	H9P request	Does
<code>IO_CREATE</code> (.A.Y = name, .X = open mode, <code>ZP_IO_BUF</code> = the new file's mode: directory, append-only, read-only)	<code>H9_CREATE</code>	Creates a file or directory in an existing directory, and opens it. A file that's there already is emptied, as in Plan 9
<code>IO_REMOVE</code> (.A.Y = name)	<code>H9_REMOVE</code>	Removes a file, or an empty directory
<code>IO_WSTAT</code> (.A = fd, <code>ZP_IO_BUF</code> = a stat record)	<code>H9_WSTAT</code>	Renames (within its directory) and sets the mode bits; other fields are left alone

HyForth words: `create ( sz mode -- fd )`, `remove ( sz -- )`, `rename ( sz-old sz-new -- )`, `ls ( sz -- )` (a directory's text listing), `mkdir ( sz -- )`.

The filesystem's requests are numbered from `H9_CREATE` (8) on, and every other server refuses them (`ERR_IO_BAD_REQ`), so `IO_REMOVE` of `/dev/null`, say, is an error, not a no-op. `IO_REMOVE` borrows a free fd for the request and gives it back.

How writing behaves (as built):

- **Mode bits are checked when a file is opened**, as in Plan 9: opening a read-only file for writing (or with `IO_MODE_TRUNC`) is `ERR_IO_MODE`, and so is opening a directory for writing. The fd that creates a file can write it even if its mode is read-only, and a file made read-only while it's open for writing can still be written through that fd.
- **An append-only file** is written at its end, wherever the fd's offset is.
- **A write past the end of a file** makes it that long first, with zeros: whole clusters of them are a hole, taking no space ([sparse files](#)).
- **Creating a name that's there**: a file, made again as a file, is emptied and opened; a directory, or a file where a directory was asked for, is `ERR_IO_EXISTS`. A new directory is opened for reading, whatever the open mode says.
- **Removing** needs the file not to be open (`ERR_IO_BUSY`) and a directory to be empty (`ERR_IO_NOT_EMPTY`); a card's root can't be removed.
- **Renaming** is within the file's directory (the name is a name, not a path); a name that's taken is `ERR_IO_EXISTS`. A 0 first name byte keeps the name, and a mode of `$FF` keeps the mode bits.
- **A full card** is `ERR_IO_FULL` (`$81`); what was written up to it stays.
- **Qid versions and stamps** change once per open for writing (at the first write), and at a truncate, create, remove (the directory's) or wstat.

What reaches the card when, in an order that's safe without a journal:

- A file's data goes to the card at once, a block at a time (a 256-byte write is a whole block write: about 1,000 CPU cycles a byte, 3.5 KB/s). A block that's entirely past the end of the file isn't read first.
- When a file gets a cluster: the free map (the cluster marked in use), then the extent block if the new extent goes in one, then the entry (with the file's size so far).
- Its size otherwise changes in RAM, in every open fd's copy of the entry, and goes to the card when the last fd on it is closed. So after a crash, a file being written can be shorter than what was written, but its clusters are never lost or in use twice.
- Emptying or removing a file writes its entry first (no extents), then frees its clusters: a crash between leaves lost clusters (`check` will find them), never a cluster in use twice.
- A new entry is written before its directory's entry (the directory's size, when the entry is at its end), so a crash never leaves a half-made entry in a directory.
- The superblock's counters (free clusters, the hint, the next qid and stamp) are kept in RAM and written at the end of each request that changes them; the next qid before the entry that uses it.

The storage task's state

- The one-block cache (512 bytes, shared with `/dev/sd`), for files' data and for reading directories. A second, the **metadata buffer** (512 bytes), for the free map, extent blocks, entries being written and the superblock, so a file's data and its allocation don't evict each other. A block is changed there and written back when another block is wanted, and at the end of every request, which also forgets it: nothing is kept in it between requests, so a raw `/dev/sd` write or a changed card can't leave it stale.
- Per card (8): its state, and the superblock's numbers and counters (`HFS_V_*`, 37 bytes each, as parallel arrays). The server's scratch variables are in a page (`$0800`) the storage task keeps out of its MMU's reach by raising its page floor. `init` on a card's ctl file forgets them, and lets go of any HydraFS file open on it.
- Open files: 8, shared by every task. Each keeps a 16-byte header (the card, the open mode, how many fds share it, where its directory entry is, and where its directory's entry is, for a rename) and a **copy of the entry itself** (64 bytes), so a read finds the file's extents without an extra block read. Every copy of one entry is kept the same, and a walk takes an open file's copy over the card's (its size may not be on the card yet). Reads walk the extent list from the start each time: that's arithmetic alone for the two extents in the entry, and costs a block read per chunk only for a file fragmented into three or more.
- The allocator takes the cluster after a file's last if it's free (so files stay in one piece), else scans the free map from the hint, a byte (8 clusters) at a time past full ones.

- A walk keeps the entry it's on (`HFS_ENT`, 64 bytes) and a stack of the directories above it (`HFS_STK`, 8 levels of 5 bytes), so `..` needs no stored path. A path deeper than that gives `ERR_IO_NAME`.

Formatting and tools

`format [-f] [-p] [-s size] [label] on /dev/sd/N/ctl` (the shell's `mkfs`, `mkfs-full`, `mkfs-size` and `mkfs-part`), in `os_rom/fs/hfs_format.s` on ROM page 3:

- **Where:** on a card with a HydraFS partition, in the partition (the partition table and the other partitions are left alone). `-p` makes one on a card without one: after its other partitions, to the card's end (a card with no partition table gets one, with the partition from block 2048). Otherwise the HydraFS is the whole card.
- **A quick format** (the default) clears block 0, then writes the superblock: a **version 2** volume, with none of its free map written (offset 44 = 0). It takes a moment whatever the card's size (a 244 GB card: about a quarter of a second).
- `-f`, **a full format**, writes the whole free map (all free) between the two: a **version 1** volume, as the first ROMs wrote and read. A 32 GB card's map is 2048 blocks, about a minute and a half at 3.58 MHz; 244 GB, 15,264 blocks and about 13 minutes. It shows its progress on the console (`10% 20% ... 100%`).
- `-s size` makes the volume that size, in megabytes (`-s 4096`), or gigabytes with a G (`-s 4G`), if the card is bigger: the rest of the card isn't used.
- The label is the rest of the line (31 characters at most), so it can't start with `-`.

A version 2 free map is written as it's used. The superblock counts the map's blocks written so far (from the first); a block past the count isn't read, and is all free, however the card's old contents read. When a cluster in such a block is first marked in use (or freed), the server first writes zeros to it and to any unwritten blocks before it, then the superblock with the new count, and only then the block with its change (`HFS_MAP_WRITTEN`). So the card never has a map block with bits in it past the count, and a crash can at most leave a counted block of zeros. The allocator works through the card from its hint, so the map is normally written in order, a block for each 16 MB used. The PC tool reads and writes both versions the same way (`hydrafs.js mkfs ... -q` makes a version 2 image).

- `format on /dev/sd/N/ctl` makes an empty HydraFS (above): from the card's size (its CSD register: the SD layer gets `CMD9`, and the emulator's card too) it clears block 0, writes the free map (a full format only), then the superblock (with an empty root directory: size 0, no clusters yet), so a format that's cut short leaves no half-made HydraFS. The layout is the one `hydrafs.js mkfs` makes. `label <text>` sets the label (31 characters at most).
- `sim/tools/hydrafs.js` (Node, on the PC): `mkfs <image> <MB> [label] [-q] [-p FATMB] (-p: in a partition, after a FAT one of FATMB megabytes, left for the PC to format; 0: none), ls <image> [-l] [path] (-l: with dates), put <image> <file> <path>, get <image> <path> <file>, mkdir, rm, and import <image> <folder> (a whole folder tree), for making test cards and moving files to and from the Hydra. It finds a card's HydraFS partition as the Hydra does, stamps what it writes with the PC's time, and reads holes as zeros.`

Build order

1. **(Done)** The SD layer: `CMD9` (the card's size); per-card state for devices 0-7 (`/dev/sd/N/data`, `/dev/sd/N/ctl`). The emulator: `CMD9` (and `--sdsc N`).
2. **(Done)** `sim/tools/hydrafs.js` (`mkfs`, `info`, `ls`, `put`, `get`, `mkdir`, `rm`, `import`, `check`): test images first, from the PC side. (A new directory entry goes in the first free one, else at the end; a file grows its last extent when the next cluster is free; extent blocks take a cluster each, their first block.)
3. **(Done)** Read-only HydraFS (`os_rom/fs/hfs_srv.s`, ROM page 3, the `hfs` device, mounted at `/sd` by the shell): walk (with `.` and `..`), open, read, both directory formats, `stat`. `IO_STAT`'s record grew from 16 to 48 bytes and `IO_MODE_STAT` was added; the `hydrafs` regression test covers it. A card that holds no HydraFS gives the new `ERR_IO_NOT_FS` (\$80).
4. **(Done)** Writing (`os_rom/fs/hfs_write.s`): write and grow, `IO_CREATE`, `IO_REMOVE`, `IO_WSTAT`, `IO_MODE_TRUNC`; format and `label`. The server moved to its own ROM page (6): page 3 was full. New errors `ERR_IO_FULL` (\$81), `ERR_IO_EXISTS`

(`$82`), `ERR_IO_NOT_EMPTY` (`$83`), `ERR_IO_BUSY` (`$84`); new thunks `$F8C9-$F8CF`. The `hydrafs-write` regression test covers it, and checks the cards afterwards with the PC tool.

5. **(Done)** HyForth words (`ls`, `create`, `mkdir`, `remove`, `rename`, written with step 4 to test it, and `ctl`: a command to a `ctl` file; `remove` and `rename` became the shell's `rm`, `rmdir` and `mv`: [SHELL.md](#)), and words for the cards themselves: `vol`s, `mkfs`, `relabel`, `fsck`, `fsfix`; check on the `ctl` file (`os_rom/fs/hfs_check.s`), and the card's HydraFS details in its `ctl` file's text. The `hydrafs-check` regression test covers it.

The check

`check` on `/dev/sd/N/ctl` walks every directory from the root, marks in a bitmap every cluster that a file, a directory or an extent block uses, and compares that with the free map:

- **lost**: marked in use, but nothing uses them (harmless, but the space is wasted; a crash while a file was being emptied or removed leaves these);
- **unmarked**: in use, but marked free (dangerous: a new file could be given them);
- **twice**: used by two files, or twice by one (one of them is damaged).

It recounts the free clusters into the superblock. `check fix` also makes the free map say what the files use: lost clusters are freed and unmarked ones marked. A cluster used twice needs a person to decide which file keeps it, so it's only reported (the PC tool's `check` names the files).

Reading the `ctl` file then shows the results, after the card's label and free space:

```
sdhc 1 MB 2048 blocks
hydrafs label=GAMES
free 1012 KB of 1020 KB
check: lost 0, unmarked 0, twice 0
```

(, fixed after `check fix`; the check line after a check of that card, until another card is checked.)

How: the bitmap is 8 KB, for 65,536 clusters (256 MB of card) at a time, so a bigger card takes a pass for each 256 MB, each walking the directories again: a 32 GB card takes 128, and 244 GB 954. With 16 passes or more it shows its progress on the console (`10% 20% ... 100%`). Empty space is quick: 8 clusters neither used nor marked are counted free at once, a pass whose walk marked nothing isn't cleared again, and a map block that's all free (one a quick format hasn't written yet, or one that's all zeros) with none of its clusters used is 4096 free clusters at once. So an empty quick-formatted 4 GB volume checks in under a second, and a 244 GB one in about a minute and a half; a full-formatted one has its whole map to read, about 7 minutes for 244 GB. The walk keeps a copy of each directory's entry it's in, and where it is in it, after the bitmap, and follows directories 24 deep (deeper: `ERR_IO_NAME`, and no results). The buffer (about 9.7 KB of the storage task's RAM) is taken from the MMU at the first check, and kept. Clusters a file claims past the card's end aren't counted (the PC tool's `check` reports them).

Partitions

A card can have a partition table (an MBR, in block 0: four entries, and `$55 $AA` at its end). Its HydraFS is then in its partition of type `$7F` (the type set aside for an OS in development), and every block number in the HydraFS counts from the partition's first block; the other partitions are left alone. So a card can hold a HydraFS next to a FAT partition a PC reads.

- **Finding it**: when a card is first used, the server reads block 0: a superblock (`HYDRAFS1`) means the whole card is HydraFS; a partition table with a `$7F` entry means the HydraFS is there (and its block 0 must be a superblock). Each card's partition start is kept with its other numbers (`HFS_V_BASE`), and page 6's block calls add it (`SD_CACHE_LOAD`, `SD_READ_BLOCK`, `SD_WRITE_BLOCK` in `hfs_srv.s`), so the rest of the server never sees it. (A table is taken as one only if each entry's status byte is `$00` or `$80`: a FAT volume that isn't partitioned has boot code there.)

- **Making one:** `format -p` (the shell's `mkfs-part`). On a card a PC partitioned (a small FAT partition, say), the HydraFS partition goes after the last partition, on a 1 MB boundary, to the card's end, in the table's first free entry (none free, or no room: `ERR_IO_FULL`). A card with no table gets one, with just the HydraFS partition, from block 2048. Formatting again (`mkfs`) keeps the partition.
- **The `ctl` file** shows `partition at block N` after the label line. The PC tool reads and makes partitioned images (`mkfs ... -p FATMB`).

Time stamps

A file's modification stamp is the Hydra's clock at the change: seconds since 2000-01-01 00:00:00, 32 bits (to 2136). The clock is counted by the scheduler's tick (`ZP_CLOCK`, in the system task: `VIA_IRQ_FAST` on page 2, a second every 200 ticks), read and set with `CLOCK_GET` and `CLOCK_SET` (page 9, `os_rom/servers/time_srv.s`), and as text through `/dev/time`: reading gives `2026-09-29 18:05:00`, and writing `YYYY-MM-DD hh:mm[:ss]` (or just the date) sets it: `echo 2026-09-29 18:05 > /dev/time`. The Hydra has no clock that keeps the time while it's off, so the clock starts at 0 at power-up: until it's set, stamps are early on 2000-01-01. The superblock keeps the latest stamp given (offset 40). The shell's `ls -l` shows each entry's date and time, as does the PC tool's; the PC tool stamps with the PC's local time.

Sparse files

A file can have **holes**: runs of clusters that read as zeros and have nothing on the card. A hole is an extent whose first cluster is `$FFFFFFFF`, with its clusters, in the extent list with the others (`os_rom/fs/hfs_sparse.s`).

- **A write past the end** of a file (after a seek) makes the file that long first, with zeros (`HFS_EXTEND`): the rest of its last cluster is written with zeros, the whole clusters after it become a hole (65535 clusters an extent), and zeros are written from the start of the write's cluster to where it starts. So every byte before a file's end reads as written, or as zero, and a 64 MB file with 4 bytes at its end takes one cluster.
- **A write into a hole** (`HFS_FILL`) gives its cluster a cluster of the card, written with zeros first, and splits the hole's extent round it: `{hole, k}`, `{the cluster, 1}`, `{hole, the rest}` (the empty ones left out). The extents after it move along the list a place for each new one, through the extent blocks, the last into a new place at the end (`HFS_EXT_INSERT`).
- **Reading** a hole gives zeros. Freeing a file, and the check, pass holes by: they have no clusters. The size, the stat record and `ls` count a file's holes as bytes.
- The PC tool reads holes as zeros, and checks them; it doesn't make them (it writes zeros instead), nor write into them.

Later

fsck-style repair of damaged directories, a GPT partition table (SDXC cards over 2 TB), a battery-backed clock.

Hydra IO Plan

A Plan 9-style IO subsystem: **all IO goes through file handles**. Devices, drivers and (later) filesystems are *file servers*; a task opens a name, gets a file descriptor, and reads and writes it. The console, the serial port and the sound chip are files; so, later, are an SD card filesystem, pipes between tasks, and per-task information.

This plan builds on the MMU work (`MMU_PLAN.md`): tasks, driver tasks, `TASK_CALL`, the IRQ dispatcher and shared memory.

Goals

- One interface for all IO: `open`, `read`, `write`, `close` (plus `stat` and `seek`), the same for every device and file.
- Drivers are file servers running in their own (resident) tasks, so a driver's state and IRQ handlers stay in its task.
- A task blocks (waits) on IO instead of busy-waiting, once there is a scheduler.
- Per-task names (Plan 9 namespaces) in a later phase; a global `/dev` to start with.
- Small enough for a 65C02: fixed-size tables, 1-byte handles, no dynamic allocation in the IO path.

Plan 9 ideas, and what they become on the Hydra

Plan 9	Hydra
Everything is a file; devices are file servers	Drivers register as file servers (<code>DEV_REGISTER</code>); <code>/dev/cons</code> , <code>/dev/ser</code> , <code>/dev/snd</code> , ...
Per-process file descriptors (0 = stdin, 1 = stdout, 2 = stderr)	Per-task fd table in the task system page; fds 0-2 inherited by spawned tasks
9P protocol between the kernel and file servers	"H9P": a small request block (<code>open</code> , <code>read</code> , <code>write</code> , <code>clunk</code> , <code>stat</code> , ...) passed to the server's <code>serve</code> routine
The kernel moves data between the client and the server	Each task has a 256-byte IO transfer area in shared RAM; the server reads/writes it
<code>iounit</code> : the largest read/write per message	256 bytes; <code>IO_READ</code> / <code>IO_WRITE</code> split bigger transfers
Per-process namespace, <code>bind</code> and <code>mount</code>	Phase 3: a per-task mount table (inherited on spawn)
A read with no data blocks the process	The task sets "Awaiting I/O" and yields; the server wakes it (needs the scheduler)
Notes (signals)	Later: <code>interrupt</code> / <code>kill</code> notes (the console keys' <code>break</code> and <code>kill</code> are a first step: <code>TASK_SET_BREAK</code>)

Architecture

```

client task          IO layer (BIOS page 2)          server (driver task)
-----
IO_WRITE fd, buf, n  ----->  fd -> (server, fid, offset)
                                copy <= 256 bytes of buf into
                                the task's IO transfer area
                                fill in the H9P request      ----->  TASK_CALL serve (in its task)
                                                                reads the transfer area,
                                                                does the IO
                                advance offset, loop          <-----  C = 0, count  (or error / "would block")

```

- **Transport:** requests go to the server with `TASK_CALL` (a procedure call into the server's task, like the IRQ dispatcher uses), not as messages through the rings. It's synchronous, and the server still runs with its own ZP, stack and RAM. The server can be preempted in the middle of a request like any task (so a long one, e.g. an SD card block, doesn't hold up the others), and serves one call at a time: another task that calls it meanwhile waits (see step 2b). The request is a real H9P message in memory, so a later message-based transport (or a remote one) can reuse the servers unchanged.
- **Data:** tasks have separate RAM, so data goes through the client's IO transfer area in shared RAM, which both tasks can map. The IO layer copies between the caller's buffer and the transfer area (`MM_*` / `SH_*` style bank switching, IRQ-safe).
- **Where the code lives:** the IO layer and the device table code on BIOS ROM **page 2** (`W = 2`), reached through gates and thunks; page 0 is nearly full (about 340 bytes free), so new code goes on other pages behind gates (page 1 about 2.3K free, page 2 about 2.2K, page 3 about 6.4K, page 4 about 3K, page 5 about 6.7K, pages 6-F empty). Server `serve` routines live with their drivers.

File descriptors

The fd table lives in the task system page, after the IRQ tables: `$7DA0-$7DFF`, **12 fds x 8 bytes**.

Offset	Size	Field
0	1	Server (device table index; <code>\$FF</code> = fd not open)
1	1	Fid: the server's own handle for the open file
2	1	Mode: read / write / read-write, flags (e.g. append)
3	1	Flags: <code>IO_FDF_CONS</code> (the fd is <code>/dev/cons</code> : <code>WRITE_CHAR</code> from the foreground task puts bytes straight into the serial TX ring, <code>SER_CONS_TRY</code>)
4	4	Offset (32-bit, for filesystems; devices ignore it)

- **fd 0, 1, 2** are stdin, stdout and stderr. The shell starts with all three on `/dev/cons`. `SPAWN_TASK/TASK_PREPARE` copy the parent's open fds (and tell each server with a `dup`-style H9P request so its reference counts stay right).
- `MM_TASK_RESET` closes every fd of the task (a clunk to each server), so a finished task can't leak open files.

Names

The layout below is Plan 9-style for now. It may change later: once the base IO system works, the filesystem layout will be revisited (a separate design document will describe the ideas).

- **Phase 1: a global `/dev`.** `DEV_REGISTER` (called from a driver's `init`, in its task) adds `name -> (task, serve routine)` to the device table in shared bank ID `$00` (`$8800-$88FF`, 16 devices). `IO_OPEN "/dev/cons"` finds `cons` in the table. Names are short (up to 8 characters) and case-sensitive.
- **Phase 3: per-task namespaces (done).** A mount table per task, inherited on spawn: 7 entries of 32 bytes in the task's IO transfer area (`IO_BLK_NS`, next to the data area names are resolved in; layout in `include/io.inc`). `IO_MOUNT` attaches a device's server at a path (e.g. the SD card's HydraFS server at `/sd`); `IO_BIND` makes one name refer to another. `IO_OPEN` applies the longest matching prefix (whole path elements), rewriting binds (up to 4, `ERR_IO_NS_LOOP` beyond), and falls back to the global `/dev`. The server that owns a mount gets the rest of the name (`/games/star.frt`) to walk. (A mount names a device, not a fid within one: mounting a server's subdirectory can come with the filesystem servers.)

H9P: the request block

The first 16 bytes of the task's IO transfer area; the data (up to 256 bytes) follows.

Byte	Field
0	Type (below)
1	Fid

Byte	Field
2	Mode (open)
3	Client task
4-7	Offset (read / write)
8-9	Count: bytes requested (in) / done (out)
10	Error (out)
11-15	Reserved

Type	Request	Plan 9
H9_OPEN	name (in the data area), mode -> fid	walk + open
H9_CREATE	name, mode -> fid (filesystems)	create
H9_READ	fid, offset, count -> data, count	read
H9_WRITE	fid, offset, count, data -> count	write
H9_CLUNK	fid: close	clunk
H9_STAT	fid -> size, type, flags (in the data area)	stat
H9_DUP	fid -> another reference (fd inherited by a child)	(kernel)
H9_FLUSH	cancel a blocked request	flush
H9_CTL	device-specific control (baud rate, capture, ...)	writes to a <code>ctl</code> file

Servers return `C = 0` with the count (or fid), or `C = 1` with an error: `ERR_IO_NOT_FOUND`, `ERR_IO_BAD_FD`, `ERR_IO_MODE`, `ERR_IO_WOULD_BLOCK`, `ERR_IO_EOF`, ... Plan 9 uses `ctl` files for control; a Hydra device can offer both `H9_CTL` and a `/dev/<name>ctl` file.

IO transfer areas

Shared bank ID `$09` (reserved at boot): **512 bytes per task** at `$8000 + task * $200` (16 tasks = 8K). `$00-$0F` is the request block, `$100-$1FF` the data (256 bytes = the iounit). Only the IO layer and the server handling the task's request touch a task's area, one request at a time per task.

Blocking and the scheduler

Reads often have to wait (no key pressed yet). So this plan starts with the scheduler:

- **Preemptive scheduler:** the VIA T1 timer IRQ (the system task's `VIA_IRQ_HANDLER`) sets a reschedule flag; at the end of `IRQ_DISPATCH`, if it's set, the dispatcher switches tasks (`SCHED_SWITCH`, which picks the next runnable one: `SCHED_PICK`) instead of returning to the interrupted task. Task switches save and restore `w` per task (a task can be interrupted while on ROM page 1 or 2). `YIELD` switches voluntarily.
- **Critical paths (holding the CPU):** two ways, depending on how long.
 - `sei ... cli` (or `php / sei ... plp`): no IRQs at all, so no task switch. Only for short sections (tens of cycles): it also holds off serial RX (a byte arrives every ~0.5 ms at 19200 baud, and the ACIA holds only one), the timer tick and sound. This is what the MMU, shared memory and message code already do.
 - `NO_PREEMPT ... PREEMPT`: a nestable per-task counter. While it's non-zero, IRQs and their handlers run normally (drivers keep up), but the dispatcher doesn't switch tasks; it only notes that a switch is due. `PREEMPT` brings the count back down, and switches right away if one was due. For longer sections, like timing-sensitive IO or updating several related structures. A task that blocks on IO (or calls `YIELD`) while holding it still gives up the CPU, since it can't make progress anyway.

- **Blocking IO:** a server that can't finish a request returns `ERR_IO_WOULD_BLOCK` and remembers the waiting task. The IO layer marks the task "Awaiting I/O" (`TASK_STATUS_REG` bit 2) and yields; the scheduler skips waiting tasks. When data arrives (usually in the server's IRQ handler), the server calls `IO_WAKE` task, and the request is retried.
- **Non-blocking mode** (`O_NONBLOCK` in the fd mode) returns `ERR_IO_WOULD_BLOCK` to the caller instead, which is how `READ_CHAR`'s "is there a key?" test keeps working.
- Until the scheduler exists, a blocked request just waits (`wai`) and retries.

Clock speed and wait states

The CPU runs at 3.58 MHz; the board can also run it at 7.16 MHz (the W65C02S goes to 14 MHz). Some devices can't keep up with a faster bus: the YM2151 runs on its own 3.58 MHz clock, and slow 65C51/65C22 grades and ROMs have similar limits.

- **Now (board V1): a build-time clock setting.** `CPU_CLOCK_MULT` in `include/hw.inc` (1 = 3.58 MHz, 2 = 7.16 MHz); every timing constant is derived from it: the scheduler tick (`TIMER_TASK_INT` = 5 ms), `YM_TIMEOUT` and `YM_DELAY_64` (note lengths), and the self test delays. There is no wait-state hardware, so above 3.58 MHz the sound chip mustn't be used (software can space out accesses, but can't stretch a bus cycle).
 - SPI is bit-banged, so its clock speeds up with the CPU. (**Done**) `SPI_RECV` is unrolled to 18 cycles a bit (199 kHz at 3.58 MHz), and `_M_SPI_PAD` pads it at 7.16 MHz to keep `SCLK` at about 275 kHz, under the 400 kHz an SD card allows while it starts up.
- **Board V2: RDY wait states** (the hardware design, to be finalised with V2; possibly available earlier on an expansion card; see also `IDEAS.md`, which also records the alternative of switching the clock divider):
 - A **wait table per I/O port or memory area**, whose entry for the accessed port/area is copied into a **RDY hold counter**; RDY is held low while the counter counts down, and released at zero.
 - A new pseudo-register, `$FFF9`, is the interaction point: the value the RDY hold counter loads from. It is **initialised to 15 at boot** (the maximum wait), so everything is slow and safe until the ROM sets the table up.
 - A clock jumper register to read the CPU clock from, replacing the build-time setting.
 - RDY has to be open-drain with a pull-up: the W65C02S drives it low itself during `WAI` (the ROM uses `WAI`).
- **OS support (with V2):** set the wait table first thing at reset, before the self test touches the ACIA; per-device defaults for the on-board devices; drivers declare their port's wait states (a `DriverInfo` field that `DRV_START` programs) and an `IO_WAIT_SET` call; the self test reports the clock and the table; timing constants read the clock register instead of `CPU_CLOCK_MULT`.

Servers

Name	Server	Notes
<code>/dev/cons</code>	Serial driver (task <code>\$F</code>)	The console: read = keyboard, write = screen. Replaces the "serial-capture task": whoever reads <code>cons</code> gets the input
<code>/dev/ser</code> , <code>/dev/ser/ctl</code>	Serial driver (fast paths: <code>serfast.s</code>)	Raw serial port; its settings (done : <code>serctl.s</code> : <code>b9600 18 pn s1</code> read and written as text, <code>IO_CTL SER_CTL_RATE / SER_CTL_FORMAT</code> , HyForth <code>stty / stty?</code> ; 9600 8N1 at boot; a change waits for the queued output to go)
<code>/dev/snd</code>	Sound driver (task <code>\$E</code>)	Write YM2151 register/value pairs; <code>IO_CTL SND_CTL_INIT / SND_CTL_TEST</code> (the test tune, played by a background player task) / <code>SND_CTL_STOP</code> (instead of a <code>/dev/sndctl</code> file); HyForth's sound words use it
<code>/dev/null</code> , <code>/dev/zero</code>	IO layer	The usual
<code>/dev/sd/N/data</code> , <code>/dev/sd/N/ctl</code>	Storage task (<code>\$C</code>)	SD card N (SPI device 0-7) as bytes (block cache, writes straight through); its type and size, and <code>init</code>

Name	Server	Notes
Later: /sd/...	SD card filesystem server (HydraFS)	Uses SPI in the BIOS; first real use of offsets, H9_CREATE and directories
/dev/pipe	Pipe server (task \$D)	IO_PIPE: two fds on a 255-byte ring in the pipe task's RAM (8 pipes); end of file when the writers are gone. HyForth's runs a pipeline through them
Later: /proc/ <task>/..., / env/...	System servers	Task status, memory use, notes; per-task environment variables

Serial driver changes: it becomes a file server. RX bytes go into a buffer in its own task RAM (instead of the capture task's message ring); H9_READ on cons takes from it, blocking when it's empty. TX goes through a TX buffer drained by the ACIA's transmit IRQ, so WRITE_CHAR no longer busy-waits. READ_CHAR / WRITE_CHAR (and the \$F800 thunks) become one-byte reads and writes on fd 0 / fd 1, so WOZMON and HyForth work unchanged, and follow redirection.

API

All calls: C = 0 on success, C = 1 with the error in .A. Thunks after \$F853.

Call	In	Out
IO_OPEN	.A.Y = zero-terminated name, .X = mode	.A = fd
IO_CLOSE	.A = fd	
IO_READ	.A = fd, ZP_IO_BUF = buffer, ZP_IO_CNT = count	ZP_IO_CNT = bytes read (0 = end of file)
IO_WRITE	.A = fd, ZP_IO_BUF = buffer, ZP_IO_CNT = count	ZP_IO_CNT = bytes written
IO_GETC / IO_PUTC	.X = fd (.A = byte for put)	.A = byte (get)
IO_SEEK	.A = fd, ZP_IO_OFS = offset	
IO_STAT	.A = fd, ZP_IO_BUF = 16-byte buffer	stat block
IO_DUP2	.A = fd, .X = new fd	redirection (e.g. stdout to a file)
IO_DUP	.A = fd	.A = a new fd (the lowest free) for the same file
IO_PIPE		.A = read fd, .X = write fd
IO_MOUNT	.A.Y = path, ZP_IO_BUF = device name	a mount in the task's namespace
IO_BIND	.A.Y = path, ZP_IO_BUF = target path	a bind in the task's namespace
IO_UNMOUNT	.A.Y = path	removes the path's entry
IO_NS_LIST		prints the task's namespace
TASK_CLONE	.A.Y = entry, .X = its ROM page	.A = a new task: a copy of this one (fork)
GET_CHAR		.A = a key from fd 0 (waits)
IO_CTL	.A = fd, .X = control code, .Y = argument	device-specific
DEV_REGISTER	.A.Y = name, ZP_TC_VEC = serve routine	registers the calling (driver) task
IO_WAKE	.A = task	server: a blocked request can be retried
YIELD		give up the CPU (scheduler)
TASK_SLEEP	.A.Y = ticks (200 a second, up to 32767)	sleeps that long (the other tasks run, or the system idles)
TASK_SLEEP_UNTIL	.A.Y = a tick count (TICKS_GET)	sleeps until then
NO_PREEMPT / PREEMPT		hold the CPU (nestable) while IRQs keep running; see Critical paths

HyForth words: `open ( sz mode -- fd )` (the name is a HyForth `q^...^` string, e.g. `q^/dev/cons^ 3 open`), `close ( fd -- )`, `read ( fd addr n -- n' )`, `write ( fd addr n -- n' )`, `ioctl ( fd code arg -- )`, `ioerr ( -- n )` (a failed call gives `!IO ERR!`; `ioerr` is the IO layer's error code); `emit /key` on fds 1 and 0. Later: `type`, and a `redirect ( fd -- )` for output to a file.

Build order

Phase 1 - Scheduler (done)

1. **(Done)** One frame format for every switched-out task (the IRQ dispatcher's frame plus `U`, so `W` and `U` follow each task); `SCHED_SWITCH`/`SCHED_PICK` (round-robin over tasks 1-15, task 0 idle); `YIELD`; `NO_PREEMPT`/`PREEMPT`; the VIA T1 tick (the VIA handler returns a reschedule marker) and preemption at the end of `IRQ_DISPATCH`, never while a task runs a `TASK_CALL` routine for another task (`ZP_TC_GUEST`). `TASK_START` (spawn and wait) and the new `TASK_RUN` (background) start tasks through `TASK_TRAMPOLINE`/`TASK_EXIT`. `SWITCH_TO`/`NEXT_TASK` are gone.
2. **(Done)** `TASK_WAIT`/`IO_WAKE` (the "Awaiting I/O" bit), `TASK_STATUS`; `SCHED_TEST` (`$F869`): interleaved tasks, a `NO_PREEMPT` block, wait/wake. 2b. **(Done)** Preemptible servers and sleeping. A `TASK_CALL` routine (a server's serve routine, a new task copying itself in for `TASK_CLONE`) can be switched out like any code: the caller is marked in a call (`TASK_STATUS_REG` bit 6, `TASK_CALLING_FLAG`) and isn't run until the call returns; the server, switched out mid-call, gets bit 7 (`TASK_GUEST_OUT_FLAG`), which makes it runnable (even resident or paused) until it's resumed, and a break or kill waits until it's back in its own code. A server serves one call at a time: a task that calls it while it's busy waits in `TC_WAIT_FREE` (the server's `ZP_TC_WAITERS` mask, woken by `TASK_WAKE_MASK` when the call returns); the IRQ dispatcher's calls (`TASK_CALL_IRQ`) run at once, on the stack below the switched-out call. A server can now also wait itself (`TASK_WAIT`, or an IO request of its own to another server) in the middle of a request. `TASK_SLEEP` (ticks) / `TASK_SLEEP_UNTIL` (a tick count), HyForth `sleep`: the task waits, and the system task's tick handler wakes it (`SLEEP_CHECK`, the `ZP_SLEEPERS` mask); the test tune sleeps between notes instead of spinning on `YIELD`, so the system idles (`wai`) while it plays.

Phase 2 - IO core and the console

3. **(Done)** BIOS ROM page 2 (`page2.s`, `io.s`) with its gates and thunks (`$F86C`-`$F88A`); compact 6-byte gates (`FAR_GATE_INLINE`, via `FAR_INLINE` in the `COMMON` block); the IO transfer bank (`$09`); the fd tables (`$7DA0`, set up by `TASKS_INIT`); the device table (`$8800`) and `DEV_REGISTER`, `IO_SRV_MAP`/`IO_SRV_UNMAP`, `IO_INIT` (`io_p0.s`, page 0). Page 0 is nearly full (about 55 bytes left): converting its older 15-byte gates to compact ones would free a few hundred bytes.
4. **(Done)** `IO_OPEN`/`CLOSE`/`READ`/`WRITE`/`GETC`/`PUTC`/`SEEK`/`STAT`/`CTL`, H9P dispatch with race-free blocking (the task marks itself waiting before calling the server), `/dev/null` and `/dev/zero`, and `IO_TEST` (`$F88A`).
5. **(Done)** The serial driver as the `cons` / `ser` server (`serial.s`: init, IRQ handler, `SER_TX_TRY`; `ser_srv.s` on page 2: the requests), with 256-byte RX and TX rings in its task and IRQ-driven TX (Rockwell TDRE interrupt); `cons` reads only for the foreground task (`SER_CTL_FOREGROUND`, `SER_CALL_SET_CAPTURE`), others wait; readers and writers wait (not spin) on an empty RX / full TX ring. `READ_CHAR` (non-blocking, as before) / `WRITE_CHAR` on fds 0 and 1, or the rings directly for tasks without them; `IO_STD_OPEN` gives the shell fds 0-2; `TASK_BUILD_FRAME` copies the parent's open fds (`IO_INHERIT`); `MM_TASK_RESET` closes fds (`IO_CLOSE_ALL`). `IO_WRITE` now offers a server the rest after a short write. The serial-capture message ring is gone. To make room on page 0, SPI moved to page 2. Not done yet: `/dev/serctl`, `H9_DUP` (no server counts references yet), and making blocking reads the default for `key` (step 6).
6. **(Done)** HyForth words `open`, `close`, `read`, `write`, `ioctl`, `ioerr` (error `!IO ERR!`); `emit /key` through fds 1 and 0. `GET_CHAR` (`$F88D`): wait for a key on fd 0 (echoed), sleeping instead of polling; HyForth's line input and `key`, and `WOZMON`, use it, so an idle shell doesn't use the CPU.

Phase 3 - More servers and names

7. **(Done)** `/dev/snd` (`snd_srv.s`: register/value pairs, `IO_CTL` init and test). `H9_DUP`, sent by `IO_DUP2` and `IO_INHERIT`; every request now carries the fd's mode (`IO_BLK_MODE`). `IO_DUP2` (`$F890`, HyForth `fdup2`) for redirection. Pipes (`pipe_srv.s`, a Resident task `$D`): `IO_PIPE` (`$F893`, HyForth `pipe`) opens the read end and adds a write end (`IO_FD_COPY` with the write mode); the server counts each end's fds, readers and writers wait for each other, a reader gets end of file

when no writers are left, a writer `ERR_IO_BROKEN` when no readers are. `IO_TEST` checks a pipe between two tasks (a child writes 600 bytes). Fixed: `IO_SERVE` holds `NO_PREEMPT` from marking the task waiting until it yields or is done; a task switch in between left it waiting with nobody to wake it. (| in the shell: step 7b.) 7b. **(Done)** | in the shell: HyForth splits a line at each | (outside `q^...^` strings); each stage but the last runs in a copy of the shell's task (`TASK_CLONE`, like fork: task RAM, task ZP, namespace and fds; HyForth's `child_start`) with stdout into a pipe, and ends at the end of its part of the line; the shell runs the last stage with stdin from the pipe, and puts stdin back at the end of the line. HyForth `cat` and `wc` (-- lines words chars); key gives -1 at end of file. `/dev/cons` does the echo now (not `READ_CHAR/GET_CHAR`), so piped input isn't echoed; `WRITE_CHAR` drops output when stdout fails (a pipe nobody reads). `IO_DUP`. 7c. **(Done)** Console control keys. Ctrl-D / Ctrl-Z: end of input (`/dev/cons` returns end of file; `SER_KEY_EOF`, `SER_KEY_EOF2`). Ctrl-C / Ctrl-\: break / kill, acted on by the serial IRQ handler as the key arrives (`SER_BREAK`): the foreground task gets `TASK_BREAK_FLAG` or `TASK_KILL_FLAG` and the tasks it started (`ZP_TASK_OWNER`, set by `TASK_BUILD_FRAME`) the kill flag, and they stop waiting; when the scheduler next resumes a flagged task (`SCHED_RESUME`), it rewrites its frame to continue at `BREAK_ENTRY`, which cleans up (waiting, `NO_PREEMPT`, the RAM bank) and goes to the task's break handler (`TASK_SET_BREAK`, `$F8A8`: HyForth's goes back to the prompt) or ends the task (the shell starts again). A precursor to Plan 9 notes. (Page 0 room: the sound test tune moved to page 2, `snd_test.s`.)

8. **(Done)** Per-task namespaces (`ns.s`): `IO_MOUNT`, `IO_BIND`, `IO_UNMOUNT`, `IO_NS_LIST` (HyForth `mount`, `bind`, `unmount`, `ns`), inherited by new tasks and cleared when a task ends; `IO_OPEN` resolves names through them. Servers that walk longer paths come with the filesystem (step 9).

9. The SD card filesystem server (`/sd`): HydraFS (`HYDRAFS.md`).

- **(Done)** The storage layer, on BIOS ROM page 3 (`page3.s`, `spi.s`, `sd.s`, `sd_srv.s`; `storage.s` on page 0), in a Resident storage task (`$C`) that owns the SPI bus. SPI: bit-banged on the VIA's port B, mode 0, device select through the board's 74HC138 (device 0 = header J18; `SD_SPI_DEVICE`). SD card: `SD_INIT` (`CMD0`, `CMD8`, `ACMD41`, `CMD58`: SDHC/SDXC block addresses or SDSC byte addresses), `SD_READ_BLOCK` / `SD_WRITE_BLOCK` (512-byte blocks, with timeouts). `/dev/sd`: the card as bytes at the fd's offset (`IO_SEEK`; the first 4 GB), through a one-block cache, writes straight through; the card starts at the first open (`ERR_IO_DEVICE` without one), `SD_CTL_INIT` starts it again. The emulator has an SD card on device 0 (--sd image). HyForth `seek`.
- **(Done)** HydraFS build steps 1-2 (`HYDRAFS.md`). Up to 8 cards, one per SPI device: `/dev/sd/N/data` (the card as bytes, as `/dev/sd` was) and `/dev/sd/N/ctl` (read: `sdhc` 7580 MB 15523840 blocks, `sdsc` ... or none; write: `init`). `SD_INIT` reads the card's size from its CSD register (`CMD9`; v1 and v2), into per-card state in the storage task's RAM (`SD_CARD_STATE`, `SD_CARD_BLOCKS`); the block layer works on card `SD_DEV`, and the cache remembers its card. The emulator answers `CMD9` and can make a card standard capacity (--sdsc N: byte addresses, CSD v1). `sim/tools/hydrafs.js` makes and reads HydraFS images on the PC. Fixed: `IO_SERVE` returned C = 0 for a server's errors below `ERR_IO_WOULD_BLOCK` (`$70`-`$72`: not found, bad fd, wrong mode), so e.g. opening `/dev/proc/z` "worked".
- **Next:** the HydraFS server (`HYDRAFS.md`), in the storage task on the block layer (so it reaches the whole card): directories, reading and writing files; mounted at `/sd`. (HydraFS was chosen over FAT32: see `HYDRAFS.md`.)

10. `/proc` and `/env`; notes.

- **(Done)** `/dev/proc` (`proc_srv.s`, page 2, served in the reading task): the task list, `/dev/proc/N/status`, and `/dev/proc/N/ctl` (`kill`, `break`, `fg`). Notes, so far: `TASK_SIGNAL` (a break or a kill for a task, and a kill for the tasks it started), used by the console keys, the ctl files and HyForth's `kill`. Task switching: `CONS_SET_FG` brings a task to the front (HyForth `fg`, the ctl files, Ctrl-] then the task); only the foreground task and the tasks it started write to `/dev/cons` (the others wait, like Unix job control); a foreground task that ends hands the console to the task that started it (`CONS_RELEASE`). HyForth `shell` starts another shell, `ps` lists the tasks.
- **(Done)** stdio buffering: `WRITE_CHAR` buffers stdout and `GET_CHAR` / `READ_CHAR` read stdin ahead (128 bytes each, page `$07` of each task) when the fd isn't the console (`STDOUT_PUT`, `STDIN_GET`, `IO_FLUSH` in `io.s`), with fast paths on pages 0 and 1 (`_M_STDOUT_FAST`, `_M_STDIN_FAST`) so a buffered byte takes no far call. `words` | `wc` went from about 18M to 2M cycles. (Found with the emulator's --profile.)
- **Next:** `/env`; more notes (a note text, a handler per note, like Plan 9's `notify`).

Decisions

- **Transport:** direct calls (`TASK_CALL` into the server's task), preemptible, one call at a time per server. The H9P block stays a real message in memory, so a message-based transport (requests through the rings, servers as scheduled tasks) can be added later without changing the servers.
- **Scheduler:** preemptive, round-robin on the VIA T1 tick (`TIMER_TASK_INT_H/L` in `include/kernel.inc` : about 5 ms). A task can hold the CPU with `sei/cli` for short sections, or `NO_PREEMPT/PREEMPT` for longer ones (IRQs keep running).
- **Sizes:** 12 fds per task, 16 devices, a 256-byte iounit, 8-character device names, 32-bit offsets.
- **Console ownership:** keyboard input from `/dev/cons` goes to the **foreground task** only, set by the shell (`IO_CTL_SER_CTL_FOREGROUND` on a `cons` fd, or `SER_CALL_SET_CAPTURE`). Other tasks reading `cons` block until they're brought to the foreground; any task may write to it.

Hydra MMU Plan

Goals

- Per-task heap allocations, using tables stored in that task's own RAM. A task switch (`T`) swaps them in automatically.
- Shared allocations for cross-task messaging and driver transfer buffers, using tables stored in shared RAM.
- HyForth (the future shell) gets its large memory blocks from the MMU.
- All access goes through **handles** via the MMU API, plus a lock call that returns a raw pointer for speed-critical code.

Tiers of allocation (per task)

Tier	Size	Source	Tracking
Small	1-3 bytes	Stored inside the handle entry itself (<code>AI_SMALL</code>)	Handle table
Chunk	4-64 bytes	Fixed-size chunk allocators (4, 8, 16, 32, 64) carved from 256-byte pages (no 128: with the header in the first chunk, a 128-byte chunk page would hold only one chunk)	Chunk page header + free list
Page	129 bytes - ~16K	Contiguous 256-byte pages in <code>\$0800-\$7CFF</code> , allocated top-down	Page bitmap (16 bytes)
Bank	8K units	Task banks <code>\$00-\$EF</code> , seen through <code>\$8000-\$9FFF</code>	Bank bitmap (30 bytes)

Requests are rounded up to the next chunk size. Callers can force a tier with flags (e.g. `AI_PAGED` for bank memory).

Where the MMU area lives: top of Task RAM (`$7E00-$7FFF`)

The low end of Task RAM is already full: `BUFFERS` packs `$0200-$07FF` (serial input, the Forth TIB and stacks, `MEMSTK`, the `WOZMON` `IN` buffer), and the Forth RAM dictionary starts at `$0800`. The top of Task RAM, directly under the paged window, is better for the MMU area:

- ROM code in every task can use the same fixed address, whatever the task is running and however the Forth/shell layout changes later.
- Page `$7D` is the **task system page** (IRQ/SWI registration tables, spurious counters, driver state; see the IO subsystem section). MMU pages are allocated **top-down** from `$7C`, and the Forth dictionary grows **up** from `$0800`. The MMU header keeps a **low-water mark** (the lowest allocated page), and the dictionary may grow up to it.
- Pages `$00-$07` and `$7D-$7F` are pre-marked in use in the page bitmap.

Per-task layout (Task RAM, starting at `$7E00`)

Offset	Size	Description
<code>\$00</code>	1	MMU status / version (non-zero = initialized)
<code>\$01</code>	16	Page bitmap for pages <code>\$00-\$7F</code> (pages <code>\$00-\$07</code> and <code>\$7D-\$7F</code> pre-marked in use)
<code>\$11</code>	30	Bank bitmap for banks <code>\$00-\$EF</code>
<code>\$2F</code>	6	Chunk allocator heads, one page number per size class (0 = none)
<code>\$35</code>	16	Page run-end bitmap (marks the last page of each page allocation)
<code>\$45</code>	30	Bank run-end bitmap (marks the last bank of each bank allocation)
<code>\$63</code>	1	Low-water mark: lowest allocated page (<code>\$7D</code> when empty)
<code>\$64</code>	..	Handle table (fills the rest of the MMU area)

MMU area size = `MMU_TASK_PAGES` (build-time constant). **Suggested default: 2 pages (\$7E00-\$7FFF), about 100 handles.** 1 page gives about 36 handles, which is enough for small tools. 3 pages give about 160, for Forth or programs that allocate a lot. With the chunk allocators, most tasks should need far fewer than 100 live allocations.

Chunk page header (first bytes of each chunk page): `size_class`, `free_count`, `free_head` (byte offset into the page, 0 = full), `next_page` (next page of the same class). Free chunks are linked through their first byte. That's one byte per link, because every chunk sits inside a single page.

Handle = 1-byte index into the handle table (0 = invalid). Each entry is 4 bytes, the same as the current `SmallAllocInfo`: `PageAddress` (`addr.w + bank`), then `status`. Sizes aren't stored in the entry. The allocator gets them from the chunk page header (chunks) or the run-end bitmaps (pages and banks). For small allocations, the data bytes sit where the `PageAddress` would be, and the length (1-3) goes in status bits 0-1, which are unused when `AI_SMALL` is set. A 1-byte handle fits in a register, and because it's an index, the MMU can move blocks later without breaking handles.

Shared memory (256 banks: \$F0-\$FF × U 0-F)

- **Shared bank ID** = `U << 4 | (bank & $0F)`. The existing `PageAddress::page` encoding already works this way.
- **Bank ID \$00 holds the system data** (tables and message rings), and no one allocates from it:
 - Shared bank bitmap (32 bytes) and an owner/refcount table (256 bytes: owner task in the high nibble, reference count in the low nibble).
 - Shared handle table (4-byte entries: `status`, `PageAddress`). Shared handles are 1 byte too, so they can be sent in messages.
 - Head/tail pointers for the 256 message rings (512 bytes).
- **Message rings live in shared bank IDs \$01-\$08**. There's one 256-byte ring for each receiver/sender pair ($16 \times 16 = 64K$):
 - Bank ID = `1 + (receiver >> 1)`, address = `$8000 + (receiver & 1) * $1000 + sender * $100`.
 - Each ring has a single writer and a single reader, so senders never contend. 8-bit pointers wrap for free.
- Any task can call the shared API. A `php/sei...plp` critical section protects table updates, so the "system task only" limit can go away (it currently blocks drivers).
- Shared allocations are in banks (8K) or 256-byte pages inside a bank. Chunk allocators aren't needed here.

Messaging and buffered transfers

(The message rings were built (`msg.s`), then removed in the code cleanup: `pipes` and `/dev/cons` replaced them (see `IO_PLAN.md`), and their shared bank IDs \$01-\$08 are free for `SH_ALLOC` now. The notes below are the original design.)

- **Message** = `type`, `len`, then up to a few payload bytes, written into the receiver's ring for that sender. The sender is implied by which ring it's in. `MSG_RECV` scans the 16 rings round-robin, or reads one sender's ring directly. Larger data goes in a shared buffer, and the message carries only its **shared handle**.
- When a task waits on an empty inbox, `TASK_STATUS_REG` bit 2 (Awaiting I/O) is set, and `MSG_SEND` clears it.

HyForth integration

HyForth keeps its own allocator for small records and gets everything large from the MMU.

- **Small-record arena:** at `cold`, Forth gets its arena from the MMU (`FORTH_ARENA_PAGES` pages, e.g. 8 = 2K) and points `MEMTOP`/`MEMLAST` at the top of it, replacing the fixed `MEMTOP = $8000`. `MALLOC`, `MEMSTK`, `m1en`, `mktemp` and `purge0` keep working as they are inside the arena.
- **Large blocks:** `malloc` requests at or above `FORTH_LARGE_MIN` (e.g. 256 bytes) call `MM_ALLOC` (page tier) instead:
 - The block starts with the same 3-byte `type/len` header, so `m1en` and existing words that use records don't change.
 - Type bit \$40 marks the record as MMU-backed. Bit \$80 (temp) keeps its current meaning.

- The `MEMSTK` slot holds the block's address, the same as for arena records. Page-tier blocks don't move, so the address stays valid.
- When a record is freed (`purge0` or a new `mfree`), MMU-backed records call `MM_FIND` (address → handle) and then `MM_FREE`.
- **Bank memory (more than ~16K, or explicit):** new words `halloc` (`bytes flags -- h`), `hfree` (`h --`), `hlock` (`h -- addr`) and `hunlock` (`h --`). Bank memory is only visible in the `$8000-$9FFF` window while it's locked.
- `ALTBUF`: allocated from the MMU (16 pages) at `cold`, or when first used, instead of the fixed `$6000-$6FFF`.
- **Dictionary limit:** here growth and the `free` word check against the MMU low-water mark instead of `MEMLAST`.
- **Task exit:** `MM_TASK_RESET` frees the arena, large blocks and `ALTBUF` with no Forth cleanup needed.
- **Later:** `MEMSTK` could become MMU handles too, which frees `$0500-$06FF`. That isn't needed for this phase.

Task completion (`MM_TASK_RESET`)

Called from `TASK_START`'s `@task_complete`. It frees all of the task's memory without walking its handles:

1. Re-initialize the task's MMU area: clear the bitmaps, chunk heads and handle table. All task pages and banks become free at once.
2. Shared memory: scan the owner/refcount table. The task's references are dropped, and any bank whose count reaches 0 is freed. Banks still in use by other tasks survive until they're released.
3. Reset the head and tail of the 32 message rings the task is part of: its 16 inboxes, and its ring in each other task's inbox.

API (all: **C = 0** success, **C = 1** with error in **.A**)

Call	In	Out
<code>MM_ALLOC</code> (\$F821)	<code>.A.Y</code> = size (<code>.A</code> = low), <code>.X</code> = 0 or <code>AI_PAGED</code>	<code>.A</code> = handle
<code>MM_FREE</code> (\$F824)	<code>.A</code> = handle	fails if locked
<code>MM_FIND</code>	<code>.A.Y</code> = block address	<code>.A</code> = handle
<code>MM_LOWWATER</code>		<code>.A</code> = lowest allocated page
<code>MM_READ</code> / <code>MM_WRITE</code> (\$F827 / \$F82A)	<code>.A</code> = handle, <code>.Y</code> = offset 0-255 (<code>.X</code> = byte for write)	<code>.A</code> = byte
<code>MM_LOCK</code> (\$F82D)	<code>.A</code> = handle	<code>.A.Y</code> = pointer, <code>.X</code> = previous RAM bank; selects the bank (<code>AI_PAGED</code>); pins the block
<code>MM_UNLOCK</code> (\$F830)	<code>.A</code> = handle, <code>.X</code> = bank from <code>MM_LOCK</code>	restores the bank
<code>MMU_TEST</code> (\$F833)		runs the handle calls in the current task and prints <code>MMU test: ok</code> or <code>FAIL x ee</code> (WOZMON: F833R)
<code>SH_ALLOC</code> / <code>SH_FREE</code> / <code>SH_READ</code> / <code>SH_WRITE</code> / <code>SH_LOCK</code>	same shapes, shared handles	
<code>MSG_SEND_BYTE</code>	<code>.A</code> = byte, <code>.X</code> = receiving task (sender = current task)	<code>C</code> = 1 if the ring is full
<code>MSG_RECV_BYTE</code>	<code>.X</code> = sending task, or <code>\$FF</code> for any	<code>.A</code> = byte, <code>.X</code> = sender; <code>C</code> = 1 if empty
<code>MSG_PEEK</code>	<code>.X</code> = sending task	<code>.A</code> = bytes waiting

All calls go through `BIOS_THUNKS`, after `MEM_COPY`.

Zero-page convention

Every task has its own zero page, so ZP only has to be divided up **within one task**, not across the system.

- **OS/BIOS ZP grows up from** `$02` (the `ZEROPAGE` segment, `zero.s`). It has the same layout in every task, because the same ROM code runs in every task. It currently ends at `$7A`.
- **Task ZP grows down from** `$FF`. Each task's code (shell, sound driver, serial driver, ...) reserves its ZP with a macro, instead of hard-coded addresses (`sound.s $A0/$B0`) or `.org` (HyForth `$C8`):
 - `TASK_ZP_BEGIN` resets a per-task counter to `$100`.
 - `TASK_ZP name, size` lowers the counter by `size` and defines `name` at the new value.
 - `TASK_ZP_END` adds a link-time `.assert` that the counter is still above the end of the `ZEROPAGE` segment. The linker checks this for every task image.
- Drivers and the shell run in different tasks, so their task ZP can overlap freely. For example, sound and Forth can both use `$C8-$FF`.
- A driver that shares a task with other code (not planned) would have to allocate its ZP after that code's, from the same counter.

Build order

Phase 1 - MMU basics (done, stage 1)

1. `MMU_INIT` / `MM_TASK_INIT`: clear each task's MMU area and bitmaps (shared tables still a stub). `jsr MMU_INIT` is on at reset.
2. Page and bank bitmap allocators (`MM_PAGE_ALLOC / FREE`, `MM_BANK_ALLOC / FREE`, `BM_*`).

Phase 2 - Drivers and shell in their own tasks (done; pulls work forward from the IO subsystem)

3. **(Done) ZP convention**: add the `TASK_ZP` macros, then convert `sound.s` and HyForth's ZP to them.
4. **(Done) ROM layout**: the MMU and OS stay on page 0. HyForth and the disassembler moved to page 1 (`w = 1`), with far-call gates in both directions (`page0_gates.s`, `page1.s`) and a `COMMON` block at `$FD00` on every page (IRQ/NMI entry, far-call trampolines). Each ROM page is its own 8K linker memory area.
5. **(Done) IRQ dispatcher**: per-IRQ stubs on every BIOS page, registration tables in the task system page (`$7D00`), and `IRQ_REGISTER / SWI_REGISTER`. See the IO subsystem section.
6. **(Done for sound; serial in step 8) Resident driver tasks**: `TASK_STATUS_REG` Resident bit, and `DRV_START` (run a driver's `init` in its task). Boot (task 0) starts:
 - the **sound** task: `SOUND_INIT` and the YM IRQ handler.
 - the **serial** task: `SERIAL_INIT` and `SERIAL_IRQ_HANDLER`.
7. **(Done, msg.s; removed later: replaced by pipes) Message rings (byte streams)**: fixed shared banks `$01-$08` with ring pointers in shared bank `$00`, and `MSG_SEND_BYTE / MSG_RECV_BYTE / MSG_PEEK`, plus `MSG_RESET_TASK` for task reset. Framed messages (type, length) can be layered on top later. This doesn't need the shared allocator.
8. **(Done) Serial through messages**: the serial driver runs in task `$F`; the serial task's RX handler writes to the capture task's ring, and `READ_CHAR` reads from the ring. TX stays polled/direct at first. Retire `INPUT_BUFFER`.
9. **(Done) Shell in its own task (task 1, via `TASK_PREPARE + SWITCH_TO`)**: task 0 finishes boot, starts the shell task (Forth/WOZMON) as the serial-capture task, and hands over to it with `SWITCH_TO`. This needs no preemptive scheduler: the only running task is the shell, and the drivers run from IRQs.

Phase 3 - Rest of the MMU

10. **(Done, stage 2)** Handle table plus `MM_ALLOC / MM_FREE` for the small, page and bank tiers, then `MM_READ / WRITE / LOCK / UNLOCK`, the `$F821-$F835` thunks and `MMU_TEST`. Until chunks exist (step 11), 4+ byte allocations take whole pages.

11. **(Done, stage 3)** Chunk allocators: 4, 8, 16, 32 and 64-byte chunks; 65+ bytes take whole pages. A chunk page that becomes empty goes back to the page allocator.
12. **(Done, stage 4)** Shared bank allocator and shared handles (`shared.s`): whole 8K banks, tables in shared bank ID `$00` (`$8200` bitmaps, `$8400` handle table of 255 entries). Each entry keeps a mask of the tasks holding a reference instead of a count, so a task reset drops exactly that task's references. `SH_ALLOC`, `SH_ATTACH`, `SH_DETACH`, `SH_READ`, `SH_WRITE`, `SH_LOCK`, `SH_UNLOCK` (`$F836-$F848`). Bank IDs `$00-$08` and missing `U` macro-pages are reserved at boot. 256-byte pages inside a shared bank are left for the IO subsystem.
13. **(Done, stage 4)** `MM_TASK_RESET` (`$F84B`): resets the task's MMU area, drops its shared references, empties its message rings, and unregisters its IRQ/SWI handlers. Hooked into task completion; `TASK_START` now saves the parent's context so the parent resumes correctly. Driver `stop` is still to come (no driver task completes yet).
14. **(Done, stage 5)** `HyForth`: `cold` resets the task's MMU area and gets a 2K arena for small `malloc` records (`FORTH_ARENA_SIZE`); `MALLOC` now fails cleanly (out of memory) instead of overrunning. Records of 256+ bytes get their own MMU block (type bit `MEM_MMU` = `$40`), freed by `purge0` via `MM_FIND` + `MM_FREE` . The dictionary is protected by the MMU page floor (`MM_SET_FLOOR`): `DICTCHK` keeps it `FORTH_DICT_MARGIN` pages above here, checked before each compiled word and in `:`, `var`, `cons` and `bload` ("out of memory" instead of corruption). New words `halloc`, `hfree`, `hlock`, `hunlock`, `mmtest`; `free` reports the room between here and the lowest MMU page. `ALTBUF` was unused, so it's left as is.
15. Tests in the style of `MEM_TEST` that you can run from `WOZMON` or `Forth`.
16. **(Done)** Far pointers and references (`fp.s`, BIOS ROM page 5). A far pointer (`FarPtr`: address, kind, selector) says what's mapped at its address (a task's RAM and RAM bank, a shared bank, a paged ROM bank, a BIOS ROM page), so it reads the same from any page and any task (except a task's own RAM, from another task): `FP_MAKE`, `FP_READ`, `FP_WRITE`, `FP_COPY` (a BIOS page is read through `FP_PEEK_PAGE` in the `COMMON` block). References are handles for far pointers: `MM_REF` (an MMU handle entry with `AI_SMALL` | `AI_BLOCK` set, the kind in bits 0-1; `MM_READ`, `MM_WRITE`, `MM_LOCK` / `MM_UNLOCK` for task RAM and the paged ROM, `MM_FREE`) and `SH_REF` (a shared handle whose count is `SH_REF_MARK`, its far pointer in `SH_REF_TBL` at `$8900` of shared bank ID `$00`; `SH_READ`, `SH_WRITE`, `SH_LOCK` for shared RAM, `SH_ATTACH` / `SH_DETACH`); `MM_FP` / `SH_FP` give any handle's far pointer. The IO layer reads callers' names through far pointers (`IO_OPEN`, `IO_MOUNT`, `IO_BIND`, `IO_UNMOUNT`, `DEV_REGISTER`), taking the caller's ROM page from the far call's stack frame. Tested by the MMU test (steps 5-9) and the IO test (step k). (To make room on page 0, `POST` moved to page 4: `tests/post.s`.)

Later: the preemptive scheduler (VIA T1 tick plus a reschedule flag), serial TX through messages, driver loading from the Forth shell (`drv-load`, `capture`, ...), and shared ring buffers for bulk transfers.

IO subsystem (IRQ dispatch and drivers: Phase 2; the rest later)

The IRQ dispatcher, resident driver tasks and serial-through-messages are built in Phase 2 (see Build order). The rest comes after the MMU work.

Current IRQ handling (what gets replaced)

- The 16-entry vector RAM (`$FFFE`, indexed by `v[0..3]` on write) points **directly at handlers**, set with `IRQ_SET_VECTOR`. Handlers run in **whatever task is current**, using that task's ZP and stack. For example, `SERIAL_IRQ_HANDLER` writes into the interrupted task's `INPUT_BUFFER` / `ZP_WRITE_PTR`.
- The vectors are copied onto every BIOS page, but the handlers only exist on page 0. An IRQ while `w <> 0` jumps into whatever code is on that page.
- Known bugs:
 - `IRQ_VECTOR_INIT` defaults every unused IRQ to `SERIAL_IRQ_HANDLER`.
 - `IRQ_SET_VECTOR` and `SERIAL_INIT` always turn interrupts back on.
 - `SW_IRQ_HANDLER` does nothing.
 - `SW_INT` puts `rts` directly after `brk`, and `brk` returns past the following signature byte, so that `rts` is skipped.
 - `NMI_HANDLER` returns immediately, and the scheduler code after its `rti` never runs.

New design: every IRQ runs a registered handler in a registered task

1. **Per-IRQ entry stubs, on every BIOS page.** Each of the 16 vectors points to a stub in a new `IRQ_TRAMP` segment that sits at the **same offset on every BIOS page** (like `RESETVEC_Pn`). A stub is `pha / phx / ldx #irq / jmp IRQ_ENTRY`. `IRQ_ENTRY` is also on every page: it saves `W`, sets `W = 0` and continues into the dispatcher on page 0. The exit path mirrors this (restore `W`, then `rti` from the same offset on every page).
2. **Registration table, copied into every task.** The table goes in a new per-task **system page at \$7D00**:
 - The layout is `IRQ# × IRQ_MAX_CHAIN (2)` entries of `{task, handler.w}` = 96 bytes. It's indexed by the logical IRQ number (the `IRQ_NUMBER()` errata mapping only applies when writing `V`).
 - `IRQ_REGISTER` writes the entry into all 16 tasks' copies (switching `T` in a loop, the way `TASKS_INIT` does). So the dispatcher can read it from whichever task was interrupted, with no bank switching. Registration is rare, and dispatching happens constantly.
 - Software interrupts get their own 16-entry `{task, handler.w}` table (48 bytes), indexed by `V[4..7]`.
3. **Dispatcher (`IRQ_DISPATCH`), with `I` set throughout:**
 1. `phy`, then save the current `T`. `tsx / stx STACK_SAVE_REG` stores the interrupted task's SP in its own ZP, which is the right value for the scheduler too.
 2. For each chain entry of this IRQ: if its task differs from the current task, set `T` = the handler's task, then `ldx STACK_SAVE_REG / txs`. The handler then runs on its own task's stack, **below** that task's saved frame, with its own ZP, `RAM_BANK_REG` and MMU area.
 3. `jsr` the handler. Handler convention: preserve `RAM_BANK_REG/U`, and return **`C = 1` if the interrupt was claimed**. That stops the chain. Shared sources like the VIA (scheduler tick on T1, serial TX pacing on T2) can have one handler per sub-device.
 4. Switch back: `T` = the interrupted task, restore its SP from `STACK_SAVE_REG`, pull the registers, go through the `W` exit and `rti`.
 5. For logical IRQ 15 (and `BRK`), read `V[4..7]` and dispatch through the software table instead.
 6. Unregistered or unclaimed IRQs increment a spurious counter in the system page and return.
4. **Scheduler hook:** the system-task handler for VIA T1 can set a "reschedule" flag. At step 4, the dispatcher then switches to `NEXT_TASK` instead of the interrupted task (this replaces the dead code in `NMI_HANDLER`).
5. **Cost:** about 60-80 cycles of overhead per IRQ. At 19200 baud a byte arrives about every 1860 cycles, so there's plenty of margin. A driver task's stack needs about 32 bytes of headroom below its saved SP.

API

Call	In	Out
<code>IRQ_REGISTER</code>	<code>.X = IRQ_NUMBER(n)</code> , <code>.A.Y = handler</code> ; the handler runs in the calling task	<code>C = 1</code> if the chain is full
<code>IRQ_UNREGISTER</code>	<code>.X = IRQ#</code> , <code>.A.Y = handler</code>	
<code>SWI_REGISTER</code> / <code>SWI_UNREGISTER</code>	<code>.X = SW#</code> , <code>.A.Y = handler</code> ; runs in the calling task	
<code>SW_INT</code> (fixed)	<code>.A = SW#</code>	saves/restores <code>V</code> , <code>brk</code> followed by a signature byte

When a task completes, the cleanup that runs with `MM_TASK_RESET` also removes that task's handlers. It first calls each driver's `stop` entry, so the device's interrupt is turned off before its handler goes away.

Drivers

- **Driver descriptor** (in ROM, or loaded): `name`, `init`, `stop`, the IRQ#s it uses, and a minimum stack/RAM requirement.

- **Driver tasks** get a new `TASK_STATUS_REG` bit 3 = **Resident**: the task stays allocated but isn't scheduled as a main loop. Its code runs only from IRQs and when it receives messages.
- **Loading**: `DRV_START(descriptor, task)` reserves or uses the given task, runs the driver's `init` in that task (so its state lives in that task's ZP/RAM), and `init` calls `IRQ_REGISTER` with `T_REGISTER`.
- **Serial driver**:
 - RX: its IRQ handler (in the driver task) writes each byte to the **serial-capture task's** message ring, from sender = driver task. The capture task number is a driver variable, changed by a control message (the shell sets it).
 - TX: tasks send bytes by message to the driver task, and the TX-ready IRQ drains them.
 - `WRITE_CHAR` / `READ_CHAR` become message calls. A polled fallback remains for boot/WOZMON before the driver starts.
 - The BIOS `INPUT_BUFFER` at `$0200` and `ZP_READ_PTR` / `ZP_WRITE_PTR` are then retired.
- **Sound driver**: the YM2151 timer IRQ (`SOUND_IRQ_HANDLER`) is registered to the sound task the same way.
- **Forth shell**: words like `drv-load ( desc task -- )`, `drv-stop ( task -- )`, `capture ( task -- )` and `irqs` (list registrations and spurious counts).

Other IO work

- The shared-ring write path is used from IRQ context, so it saves and restores `RAM_BANK_REG/U` and doesn't depend on the MMU area.
- Bulk transfers from drivers (e.g. SD card) use shared buffers passed by shared handle. `SHARED_RING_PUT/GET` for streaming belong here.

Decisions

- The MMU area lives at the top of Task RAM (`$7E00`), and pages are allocated top-down.
- HyForth gets its small-record arena and all large blocks from the MMU.
- Phase 2 runs on the hardware (Forth in task 1, sound and serial in driver tasks). The ACIA's IRQ arrives at `IRQ_NUMBER(1)` as documented (no spurious counts); the dispatcher's poll-all-handlers fallback for unclaimed IRQs stays as a safety net.
- The power-on self test (`POST` in `tests/post.s`, on BIOS page 4) is permanent and will be expanded over time. It runs first at reset, with polled serial output and no IRQs, so it works even when the task/IRQ system doesn't.
- `$7D00-$7DFF` is the task system page. All IRQs, not just software interrupts, run registered handlers in their registered task (IO subsystem phase).
- Handle table: 4-byte entries, with `MMU_TASK_PAGES` = 2 by default (about 100 handles). This can be changed after real use.
- All of a task's memory is reset when the task completes (see `MM_TASK_RESET`).
- Message boxes: one ring for each receiver/sender pair.

HyForth as a shell

A plan for making HyForth a usable shell on HydraFS cards: the cards found and a volume selected at boot, a boot script, a current volume and directory shown in the prompt, the commands to move around and handle files, and running programs: HyForth scripts (`.hys`) and Hydra executables (`.hyx`). The decisions are at the end.

Status: done (build steps 1-6 below). How to use it is in [hyforth.md](#), and writing programs in [programs.md](#). Where it differs from this plan:

- `vol n` and `ls -l` weren't made: `cd /sd/n` goes to card n, and `ls` shows names and sizes.
- `run` tells an executable from a script by its `HYX1` header, not its name.
- The shell waits for a program by making itself the program's parent and pausing (as `TASK_START` does), rather than polling. The program gets the console only if the shell has it.
- A script run by `run` starts with a copy of the shell's stack (it's a `TASK_CLONE`, like a pipeline's stage), so `3 4 add` passes it arguments; executables get none.
- HyForth got `"..."` strings, valid wherever a `q^...^` string is (stack forms, `prompt`, `ctl`, `open ...`), and the parsing forms take a `"quoted name"` with spaces in it.
- The executable loader is on page 7 with the rest of the shell; the build tools are `programs/asm/hyx.cfg` and `hyx.inc` (the header made by `ld65`) and `sim/tools/mkhyx.js` (for raw binaries).

When it was planned: HydraFS was complete (reading, writing, `check`), with HyForth words for files (`ls`, `create`, `mkdir`, `remove`, `rename`) and for the cards themselves (`vols`, `mkfs`, `relabel`, `fsck`, `fsfix`). Every file word took a full path as a `q^...^` string.

1. The current directory, in the IO layer

A **current directory per task**, kept by the IO layer rather than by HyForth, so every program gets it, and the tasks a shell starts (pipeline stages, programs) inherit it:

- **Where:** in the task's IO transfer area, next to its namespace: the namespace goes from 7 entries to 5, which frees 64 bytes, a path of up to 63 characters. It's copied to new tasks as the namespace is (`IO_INHERIT`).
- **IO_OPEN and friends:** a name that doesn't start with `/` is taken as relative to it (`notes.txt`, `games/star.frt`, `../x`). The namespace applies after that, as now.
- **New calls:** `IO_CHDIR` (`.AY` = a path, relative or not: it's checked by opening it as a directory, and stored as a clean absolute path, with `.` and `..` worked out) and `IO_GETCWD` (the path into a buffer).
- **At startup:** the selected volume's root (below), else `/`.
- **Volumes:** a volume is a card, so "the current volume" is just the card in the current directory's path (`/sd/1/...`); `vol 1` is `cd /sd/1`.

(The alternative, keeping the directory in HyForth alone, is simpler, but other programs and pipeline stages wouldn't see it.)

2. Boot

After the drivers start, the boot shell (task 1):

1. **Finds the volumes:** it opens `/sd/0 ... /sd/7`, which starts each card and reads its superblock, and says which hold a HydraFS (`hydrafs 0 2`).
2. **Selects the lowest numbered one:** its root is the shell's current directory, which every task it starts inherits.
3. **Runs** `boot.hys` from that root, if it's there, as `include` does, before the first prompt. Only the boot shell does: a `shell` started later doesn't.

3. The prompt

Set with `prompt ("%v%d> " prompt or q^%v%d> ^ prompt`, the default: `0:/games>` on card 0, `/>` off the cards). In the format:

Field	Is
%v	The volume: 0: when the current directory is on card 0, nothing off the cards
%d	The directory: the path on the card (<code>/games</code>), or the whole path off the cards
%p	The whole path (<code>/sd/0/games</code>)
%l	The card's HydraFS label (<code>GAMES</code> ; nothing off the cards). Added after the plan: read from the card's <code>ctl</code> file, and kept until the prompt moves to another card, or <code>mkfs</code> or <code>relabel</code> runs
%t	The task number
%%	A %

The prompt is printed only when input comes from the console, so a script being read doesn't print prompts.

4. Shell commands

Each command has a **parsing form** for typing, which takes its arguments from the words after it on the line, like a shell (`cd games`), and a **stack form** for definitions and scripts' Forth code, in parentheses, which takes a string: `"..."` or `q^...^` (`"games" (cd)`, `q^games^ (cd)`). A parsing form takes a name with spaces in it in quotes (`cd "my games"`). The file words that take a `"..."` or `q^...^` path today (`ls`, `mkdir`) become (`ls`) and (`mkdir`).

Command	Does
<code>cd dir</code> , <code>cd ..</code> , <code>cd</code>	Change directory (<code>cd</code> alone: the card's root)
<code>pwd</code>	Show the current directory
<code>vol n</code>	Go to card <code>n</code> 's root
<code>ls</code> , <code>ls dir</code>	List a directory (<code>-l</code> : with <code>qid</code> versions and modes, from stat records. As built: <code>-l</code> gives each entry's date and time, from its stamp)
<code>cat file</code>	Show a file
<code>cp from to</code> , <code>mv from to</code>	Copy, move (a move within a card is a rename; across cards, a copy and remove)
<code>rm file</code> , <code>mkdir dir</code> , <code>rmdir dir</code>	Remove, make
<code>vols</code> , <code>mkfs</code> , <code>fscck</code> , <code>fsfix</code> , <code>relabel</code>	The cards (as now)
<code>include file</code>	Run a HyForth script (<code>.hys</code>) in this shell: its definitions stay
<code>run file</code>	Run a program in a task of its own: a script (<code>.hys</code>) or an executable (<code>.hyx</code>); the shell waits for it (<code>Ctrl-C</code> and <code>kill</code> work on it)
<code>name</code>	A word HyForth doesn't know: <code>name.hyx</code> or <code>name.hys</code> , from the current directory, then the selected volume's <code>/bin</code> , is run as <code>run</code> does

5. Running programs

- **Scripts** (`.hys`): `include` points the shell's stdin at the file, reads lines until the end of the file (no prompts, no echo), then puts stdin back; an error stops the script and says which line. `run` does the same in a new shell task, so the script's definitions go away with it.
- **Executables** (`.hyx`): a file with a small header (below), loaded into a new task's RAM and started there, with the shell's `fds` and current directory. A ROM loader does it: the shell opens the file on a spare `fd` and starts a task at the loader, which inherits the `fd`, reads the header and the code into its own RAM, closes the `fd` and jumps to the entry point. The task ends when the program returns (or calls the `exit` call, or is killed), and the shell's prompt comes back.

The executable header (16 bytes, then the code; not relocatable: a program is linked for its load address):

Offset	Size	Field
0	4	Magic: HYX1
4	2	Load address (in task RAM, \$0800-\$7BFF)
6	2	Length of the code
8	2	Entry point
10	2	Flags (0)
12	4	Reserved (0)

Programs call the OS through the \$F8xx thunks, as now. A `sim/tools` script (or a ca65 config) makes the header from a linked binary.

Build order

1. The current directory in the IO layer (`IO_CHDIR`, `IO_GETCWD`, relative names).
2. Boot: finding the volumes, selecting one, `boot.hys`.
3. The prompt, and the commands, with their parsing and stack forms.
4. `include` and `run` for scripts.
5. The executable loader, the header tool, a sample program, and running a program by its name.
6. Docs and regression tests throughout.

Decisions

- Commands have parsing forms for typing and stack forms for definitions.
- The prompt's format can be set (`prompt`), with the volume and directory by default.
- Executables have the header above, and run in a task of their own.
- Scripts have both `include` (in the shell) and `run` (a task of their own).
- `.hys` is a HyForth script, `.hyx` a Hydra executable; the boot script is `boot.hys` in the selected volume's root.

OS ROM reorganisation plan

A plan for tidying the OS ROM: less dead code, clearer ROM page roles, a cleaner source tree and build output. Steps 1-7 are done. (Sizes as of the storage work: page 0 has about 320 bytes free before the thunks, page 1 about 2.3K in gaps and at the end, page 2 about 1.2K, page 3 about 6.9K, pages 4-F nothing used.)

1. Remove dead code (done)

Done: the message rings (msg.s; the system-bank macros are _M_SYS_ENTER / _M_SYS_LEAVE and SYS_BANK in defines.s now, and shared bank IDs \$01-\$08 are free for SH_ALLOC), math.s and WRITE_DEC / WRITE_BYTE_MIN, the unused VIA helpers, COPYTORAM's dots and address printing, DO_WELCOME's vector dump (and the shell's second clear screen, so boot messages stay), the I2C block, the old WDC ACIA alternatives (the WDC 65C51 is back as a build option for the new serial driver: SER_ACIA, TX paced by VIA timer 2), ALTBUF and HyForth's stale address comments. Page 0: about 560 bytes back before the thunks and 260 in BIOS; the OS ZP is 10 bytes smaller.

- **Message rings** (msg.s, 422 bytes of page 0): nothing sends or receives since pipes and /dev/cons replaced them; only MSG_INIT and MSG_RESET_TASK are still called. Also frees shared bank IDs \$01-\$08 (64K of shared RAM), \$8000-\$81FF of bank \$00 and 3 ZP bytes. Keep the bank 0 map macro (renamed).
- **math.s** (about 300 bytes): only WRITE_DEC calls it, and nothing calls WRITE_DEC. ZP_MATH_TEMP goes too.
- **Unused VIA helpers**: the T2 routines, VIA_IS_*.
- **Boot noise**: COPYTORAM's progress dots (thousands of characters: about 2 seconds at every boot and shell restart), and DO_WELCOME's IRQ vector dump.
- **Source clutter**: the I2C .if 0 block in bios.s; the WDC ACIA / software serial timing alternatives in defines.s; stale hardcoded ROM addresses in hyforth.s comments.

2. Page 1's gates (done)

- *Done: the 36 old 15-byte FAR_GATE s in page1.s are FAR_GATE_INLINE s (6 bytes), and the FAR_GATE macro is gone: GATES_P1 went from 776 to 380 bytes.*

3. Page roles and fixed offsets (done)

Done: the self tests are on page 4 (.scope PAGE4, gates in page4.s); page 1 has no fixed offsets (about 2.4K free in one piece) and page 0's BIOS follows the thunks; POST checks PAGE1::forth_main. tools/check_pages.js lists calls to another page that miss a gate (it can't see pointers to ROM data handed across pages: the IO test copies its path names to RAM for that).

Page	Role
0	Kernel: reset and POST core, IRQ dispatch, scheduler, TASK_CALL, MMU and shared memory cores, IRQ handlers, quick-switch code, gates, thunks, COMMON, WOZMON
1	HyForth and the disassembler
2	IO: the IO layer, namespaces, the file servers (serial, sound, pipes, null, zero)
3	Storage: SPI, the SD card, /dev/sd, the HydraFS server (to come: HYDRAFS.md)
4	Diagnostics: POST RAM tests, MMU_TEST, SCHED_TEST, IO_TEST (from pages 1 and 2: about 1.5K)
5+	Later subsystems
Paged ROM	Read-only data: HyForth training scripts, bload libraries, sound patches and tunes, help text, the disassembler's opcode tables

- Only the thunk table (`$F800`), COMMON (`$FD00`), WOZMON (`$FE00`) and the vectors need fixed addresses. Remove page 1's DISASM / DISASM_CODE / FORTH_ROM offsets (about 500 bytes of gaps) and page 0's BIOS offset (it splits page 0's free space in two). POST's P1:4C check reads a fixed `$EA00`: use the symbol.

4. Source tree and build output (done)

Done: bin/ and obj/; folders include/, kernel/, io/, drivers/, tests/, monitor/, tools/; bios.s split into kernel/print.s, drivers/serial.s, drivers/via.s, kernel/vectors.s; defines.s into include/hw.inc, kernel.inc, io.inc, ascii.inc, macros.inc; sound.s has its own segment; the shell is in os_main.s. The old 6502 build (make.bat, os_rom.cfg) is gone. (fs/ comes with the filesystem server.) Later (the code review, [CODE_REVIEW.md](#)): io/ is the IO layer only, the devices' servers are in servers/, HydraFS is in fs/, and the clock chip's driver in drivers/rtc.s.

- **Build output:** the ROM images (os_rom_C02.bin, paged_rom_C02.bin) go in os_rom/bin/ (in source control); everything else the build makes (.o, listing, labels, map) in os_rom/obj/ (not in source control: .gitignore). os_rom/tmp/ goes (removed from source control). makeC02.bat and the linker config name the new places; the sim's default --rom follows.
- **Source folders**, by role, for example:
 - kernel/: boot and POST, tasks and scheduler, IRQs, MMU, shared memory, thunks, gates, COMMON
 - io/: the IO layer, namespaces, the file servers
 - drivers/: serial, sound, VIA, SPI, SD card
 - fs/: the HydraFS server
 - tests/: POST RAM, MMU, scheduler and IO tests
 - monitor/: WOZMON, the disassembler
 - hyforth/ (as now)
 - include/: the split defines.s (below), zero.s
- **Split big files:** bios.s (print routines, serial driver, VIA, dead I2C) into print.s, serial.s, via.s; defines.s (1,000 lines) into hw.inc (VIA, ACIA, YM registers, IO ports), kernel.inc (tasks, IRQs, MMU, errors), io.inc (IO, H9P, namespaces, servers), macros.inc. Give sound.s its own segment (it has none and lands in SHELL); merge the 8-line shell.s into os_main.s.

5. Shared helpers (done)

Done: _M_BANK_ENTER / _M_BANK_LEAVE (_M_SYS_ENTER and _M_IO_MAP_XFER are one-line wrappers), IO_FD_ENTRY (fd * 8) and IO_SRV_COUNT. *Not done:* the quick switches (2-3 instructions each, in different registers: a macro saves nothing), and the bank flips in IO_DEV_FIND and IO_SRV_MAP (not enter / leave pairs).

- One pair of bank mapping macros for the four hand-written variants (_M_SYS_ENTER, _M_IO_MAP_XFER, and inline in NS_PUT_DEV, IO_DEV_FIND, IO_SRV_MAP).
- Macros for the "peek / poke another task's ZP" quick switch (about 8 copies).
- An fd * 8 helper in io.s (about 10 copies), and an IO_SRV_COUNT helper for the servers' "map, set the count, unmap".

6. RAM (done)

Done: the pipe rings come from the pipe task's MMU (PIPE_BUF_PAGE); they used to overlap the MMU's pages from \$0800. *Kept:* the serial rings (\$0200-\$03FF, below the MMU's pages; the IRQ handler's absolute addressing is the fast path), MMU_PAGE_BOTTOM for all tasks, and ZP_D_* in the OS ZP (the disassembler runs in HyForth's task, so it would need its own fixed task ZP anyway).

- The resident tasks' fixed buffers (serial rings \$0200-\$03FF, the pipe table and rings \$0200-\$0AFF) come from the MMU instead (the pipe task's reach above the MMU's bottom page). MMU_PAGE_BOTTOM (\$08: HyForth's buffers) could be per task.

- Move single-purpose OS ZP (the disassembler's `ZP_D_*`) to task ZP.

7. HyForth's core from ROM (done)

Done: HyForth's interpreter and built-in words (headers and code) run from ROM page 1; only its variables (segment `FORTH_DATA`, about 90 bytes) are copied to RAM at `$0800`, and the dictionary starts at `$0B00`. To make room, the code of the bulkier words (the shell's and IO words, tasks, sound, memory records, the stack printers, multiply and divide ...), the error messages, `MALLOC` and the disassembler moved to page A: those words are far words (`def_far : FARWORD` runs their code there; see [rom-layout.md](#)). The debug dump code sits after page 1's thunks. `SYSCALL` jumps through `TEMP1` instead of modifying itself. A pipeline's line takes about 37% fewer cycles (`TASK_CLONE` copies about 7K less), and the shell starts sooner.

- Running HyForth's core from ROM page 1 instead of a 6K RAM copy frees shell RAM and roughly halves `TASK_CLONE`'s time. Needs a RAM trampoline for `SYSCALL` (self-modifying) and changes to the dictionary layout: a project of its own.

Code review: the Hydra-16 software

A review of the whole software tree as of branch `1.8C_0.5` (September 2026): what's good, what's wrong, and what would make it easier to grow. It covers:

- the OS ROM (`os_rom/`, about 32,000 lines of 65C02 assembly);
- the emulator, tests and tools (`sim/`);
- the C library and samples (`programs/`);
- the documentation (`docs/`).

The bugs below were reproduced in the emulator. The other findings come from reading the code, the link map and the build.

Status: fixed on the same branch, except where the table at the end says otherwise ([what was done](#)). The text below is the review as written; its numbers are from before the fixes.

Contents

1. Overall
 2. Bugs
 3. ROM space
 4. Zero page and the namespace
 5. The kernel and the IO layer
 6. The source tree
 7. The build
 8. The emulator
 9. The tests
 10. HyForth
 11. The C library
 12. The documentation
 13. The repository
 14. Recommendations, in order
-

Overall

This is careful, consistent work, and unusually well tested for a hobby OS. What it does well:

- **Every routine says what it takes, gives and preserves** (`IN:`, `OUT:`, `Preserves`, `Modifies`), and the code follows it. Register and flag conventions across gates (`FAR_INLINE`: `.A .X .Y C V` both ways) are stated once and kept.
- **The build checks itself.** Layout `.assert` s (COMMON's copies lining up, the thunk table's place, structure offsets, ZP blocks not overlapping), `check_pages.js` (calls across ROM pages without a gate), `romsum.js` (CRCs for the hardware test), and `sim/regress.js` comparing the C library's addresses with the ROM's.
- **The design is coherent.** Plan 9's ideas (devices as file servers, per-task namespaces, `/env`, exit statuses as notes) are applied consistently. New features (`/rom`, `/dev/snd`, `/dev/time`) slot in as servers without special cases.
- **The tests are real.** 51 regression tests boot the actual ROM images. They cover:
 - seeded or random power-up states, to catch uninitialised RAM;
 - fault injection: stuck IRQs, RAM address lines, a missing chip;
 - both ACIA variants, and SD card images checked by an independent tool.

- **The documentation is thorough**, kept up to date with the code, and has the design history in `docs/plans/`.

The main risks are about growth, not quality:

- **The fixed resources are used up.** Page 0, COMMON, the OS zero page and the per-task namespace are all close to full.
- **The toolchain is Windows-only and manual.**
- **The emulator is one file.**
- **HyForth has a few traps** that will hurt newcomers more than they hurt the author.

Bugs

1. `syscall` can't reach the newer thunks. HyForth's `syscall` (`hyforth/hywords.s`, `SYSCALL`) does `jmp (TEMP1)` while running on BIOS ROM page 1. Page 1's copy of the thunk table (`hyforth/page1.s`) stops at `IO_GETCWD` (`$F8D5`). From `$F8D8` page 1 has its own gates (`GATES_P1`). So calling any thunk added since then runs unrelated page 1 code:

- the semaphores, `TASK_SLEEP`, `TICKS_GET`, `CLOCK_GET`, `TASK_EXITS`, `TASK_JOIN` and `SHELL_CMD`;
- and machine code loaded with `bload` that calls them.

```
/> $F806 $41 0 syscall .          \ WRITE_BYTE: fine
41 00E8
/> $F8EA 0 0 syscall .           \ TICKS_GET, as the API index lists it
!DS PTR ERROR!
```

The docs invite exactly this (`hyforth.md`: "`syscall` ... e.g. a thunk"). Two fixes:

- Recommended: make `syscall` call through page 0 (a far call with `ZP_FAR_PAGE = 0`, which is harmless for RAM addresses). Then every thunk works from HyForth, now and later.
- Or add the missing entries to page 1's copy. This needs room after the table, so `GATES_P1` would have to move.

Also, `syscall` treats `.X = 0` on return as an error (`!SYS ERR!`), and many calls legitimately return `.X = 0`. Returning C (the OS's error convention) would be better.

2. The IO self test assumed free namespace entries. With `/rom` mounted at boot, the IO test (`F88AR`) failed at step `z` (`ERR_IO_NS_FULL`): it binds 4 names and a task has only 5 entries. On this branch `/rom` no longer takes an entry: the IO layer resolves it like `/env`. The test is still fragile. It should unmount between steps, or check how many entries are free first.

3. A mangled line in `sim/regress.js` (around line 67): the closing brace of `zsmSong()` has two old comments fused onto it (`} // programs/c/bin's (make.bat builds them) // Wait n * ~2M cycles before the next key`). Those comments belonged to the `C_SAMPLES` and `W` definitions above. It's harmless, but put them back.

ROM space

Free space per BIOS ROM page, from the link map (`$E000-$FEFF`; `$FF00-$FFF9` is the I/O space on every page):

Page	Role	Free	Where
0	Kernel	40 bytes	31 + 1 + 8
COMMON	(every page)	1 byte	
1	HyForth	284	23 + 5, and 256 unused at \$FE00-\$FEFF
2	IO	971	715 + 256
3	Storage	2,045	
4	Tests, POST	3,332	

Page	Role	Free	Where
5	Far pointers, semaphores, exits	5,985	
6	HydraFS	760	
7	Shell	3,800	
8	Editor	5,221	
9	Client-task servers	4,174	
A	HyForth's far words	1,594	
B	Sound	1,277	
C	Player	6,541	
D	/rom	6,642	
E, F	Empty	7,675 each	

Findings:

- **Every page but 0 has 256 bytes it doesn't use**, at \$FE00-\$FEFF, between COMMON and the I/O space. Page 0 has WOZMON there, and on the others no segment claims it. For page 1 that's ten times its free space: give pages 1-F a HIGH_Pn segment at \$FE00 in os_rom_C02.cfg (HyForth's FORTH_HIGH could spill into it). rom-layout.md's "page 1 has about 40 bytes left" undercounts for this reason.
- **Page 0 and COMMON are the hard limits.**
 - **Move WOZMON off page 0.** It's 248 bytes at \$FE00, used only after bye or a crash. It could run from page 4 (diagnostics) behind a gate. The NMI and reset vectors stay where they are.
 - **Merge the peek routines in COMMON.** PEEK_D_XAM and FP_PEEK_PAGE are the same routine (switch W, read (zp), Y, switch back) with different zero page pointers. One PEEK_PAGE (.X = page, a shared pointer) saves about 15 bytes of COMMON on every page. That's enough for another fast interrupt stub (the VERA's: VIDEO.md).
- **Make the budget visible.** Have the build print this table (it's a few lines of JavaScript over the map, like check_pages.js). Fail, or warn loudly, when page 0 or COMMON drops below a threshold. The free-space numbers written in the docs go stale; one printed by every build doesn't.
- **The paged ROM** (4 MB, 192K used) is the place for everything that isn't code: fonts, help text, more /rom programs and libraries, song banks. The /rom work makes that easy now.

Zero page and the namespace

- **The OS zero page is full**, and the newest servers share scratch by aliasing:
 - the /rom server's ROM_* are other servers' ZP_ENV_*, ZP_TIME_* and ZP_PROC_*;
 - the HydraFS server borrows the SD server's bytes.

The aliasing is correct, because these servers never run at once in one task, but nothing checks that. Make it a declared union: one CLIENT_SCRATCH block of N bytes in zero.s, with each client-task server naming its own layout over it (.struct or TASK_ZP-style macros) and an .assert that each fits. A new server then can't silently use a byte another one is holding across a call.

- **Candidates to move out of the OS zero page:** the disassembler's ZP_D_* (7 bytes) and WOZMON's ZP_WM_* (5). Both run only in HyForth's task (or after bye), so they could live in that task's ZP block. REORG_PLAN.md step 6 kept ZP_D_* because the disassembler needs a fixed address. A fixed address in HyForth's block works just as well.
- **5 namespace entries a task is few.** /sd takes one, and a user's binds and mounts get 4. /rom and /env now cost none, and /dev/vid won't either. Entries are 32 bytes (NS_PREFIX_MAX 13 and NS_TARGET_MAX 15 characters). If more are wanted, the per-task IO block's \$20-\$BF could be relaid: for example, 6 entries of 26 bytes with shorter prefixes, or the current directory moved to the data area's end.

The kernel and the IO layer

- **The scheduler and `TASK_CALL` are well reasoned:** preemptible servers (`TASK_GUEST_OUT_FLAG`), IRQ calls into a busy server, `NO_PREEMPT` around the waiting-flag race in `IO_SERVE`. The comments explain the why, which is what matters most in code like this. Keep it that way.
 - **IRQ handlers run through `TASK_CALL_IRQ`** in their driver's task. That's clean, but slow for anything at a high rate. The VIA, ACIA and YM2151 have fast handlers in `COMMON` for this reason. Write down a rule for drivers: up to a few hundred interrupts a second through the dispatcher; above that, a fast stub (and `COMMON` room: see above).
 - **IO copies every byte twice** (caller $\leftrightarrow$ transfer area $\leftrightarrow$ server), 256 bytes a request, each request a `TASK_CALL`. That's fine for files and the console. Bulk transfers to devices (graphics, PCM) should let the client own the device directly after a claim, as `/dev/snd` already does for channels and as [VIDEO.md](#) plans for the VERA.
 - **IO_OPEN's special prefixes** (`/dev/`, `/env`, `/rom`) are now a small table in `io/io.s` (`S_OWN_PREFIXES`). If more come, make it a list of (prefix, device) pairs in one place, documented in `io.md`, rather than more special cases.
 - **Errors:** the codes are consistent (`kernel.inc`), but the user sees `!IO ERR!` and has to type `ioerr`. A table of short messages (`not found`, `no space`, `busy` ...) in the paged ROM would let HyForth and C's `strerror` say what happened.
-

The source tree

- `os_rom/io/` **holds two things:** the IO layer (`io.s`, `io_p0.s`, `ns.s`, the pipes) and nine file servers (`hfs_*`, `sd_srv`, `ser_srv`, `env_srv`, `proc_srv`, `time_srv`, `rom_srv` ...). Split it into:
 - `io/`: the layer;
 - `servers/`: the devices;
 - `fs/`: HydraFS, its server, writing, check and format, as [REORG_PLAN.md](#) step 4 planned.
- **The thunk table is written twice:** `kernel/thunks.s` (page 0) and `hyforth/page1.s` (page 1, a shorter copy). Only the start and end are `.asserted`. Two entries swapped in one copy would assemble and link, and call the wrong routine. Generate both from one list in an include file (a macro per entry: `THUNK READ_CHAR`), and give page 1 the whole list (see bug 1).
- **Generated files are kept as sources:**
 - `os_rom/songs/test_rom.s` is made by `hysong.js` at every build;
 - `programs/c/bin/*.hyx` and `os_rom/bin/*.bin` are build outputs.

Keeping the ROM images in Git was a deliberate choice (anyone can burn them without a toolchain). But it means every build shows them changed, and branches conflict on them. Better:

- generate into `obj/` and keep only the sources;
 - attach the images to tagged releases (or commit them only at a release);
 - stamp the version and commit into the ROM (below).
- **The `FAR_JMP_GATE` macro** is 15 bytes at each use, against `FAR_GATE_INLINE`'s 6. A `FAR_JUMP_INLINE` like the call's would save space wherever it's used.
 - **Program sources** live in two places: `programs/hello.s` with `hyx.inc` and `make.bat` (assembly), and `programs/c/` (C). Give assembly its own folder (`programs/asm/`) with the same layout as C's (`samples/`, `bin/`), and one build for both.
-

The build

- **It's Windows batch files**, and the two builds find cc65 differently:
 - `os_rom/makeC02.bat` expects `ca65` and `ld65` on the `PATH`;
 - `programs/c/make.bat` uses `CC65_HOME`, defaulting to `C:\source\cc65\win64_snapshot`.

Node.js is already required (`hysong.js`, `mkromfs.js`, `romsum.js`, `check_pages.js`, the tests). So one `build.js` could do the whole sequence on any OS:

1. the song;
2. assemble and link;
3. `/rom`, checksums and the page check;
4. the budget report.

It would find `cc65` through `CC65_HOME` or the `PATH` in both places, and leave the `.bat` files as one-line wrappers. Then Linux and macOS users can build too.

- `check_pages.js` **doesn't stop the build**: `makeC02.bat` runs it last, without checking its exit code. A missing gate is a crash on the hardware, so make it fail the build.
- **Continuous integration**: the whole suite takes about a minute. A GitHub Actions job (build `cc65` from source or fetch a snapshot, build, run `regress.js`) on every push would catch breakage before it reaches a branch.
- **A version stamp**: `HyForth 0.91 05-07-2026` is a hand-edited string (`hyforth/upper.s`). Have the build write the version, date and Git commit into an include file. Then the boot banner, a `ver` word and `/dev/sysname` (Plan 9's) can report them, and a bug report says which ROM it came from.

The emulator

`sim/hydrasim.js` is one file of 829 lines. It holds the CPU, the memory system, every device (ACIA, VIA, SPI and SD, YM2151, DS1747), the command line, the report and the interactive terminal. It's dense but readable, and it's the most valuable tool in the project. For what comes next:

- **Split it into modules**: `cpu65c02.js`, `memory.js` (tasks, banks, the paged ROM's bit swaps), one file per device, `cli.js`. Give each device the same small interface: `read`, `write`, `tick` (cycles), `irq`, `reset`. Then cards plug into slots by number (`--card 0:vera`) as they do on the board.
- **Keep the core free of Node**. No `fs`, `process` or `readline` below the CLI layer, so the same core runs in a browser: a web page with a terminal (`xterm.js`), later the VERA's canvas. That's the easiest way to let people try the Hydra ([NEXT_STEPS.md](#)).
- **The 6502 core's** cycle counts are tested (`cpu-cycles`); its instructions could also be checked against Klaus Dormann's 65C02 functional tests, the standard, which run in seconds.
- **A debugger**:
 - the emulator already has the build's debug info (`--profile` reads `os_rom_C02.dbg`), and `--pc / --watch`;
 - an interactive mode adds breakpoints by label, single steps, and source lines from the listing;
 - that makes ROM work much faster than adding prints.

The tests

- `regress.js` **is one file of 1,100 lines**. Move each area's tests into its own file (`sim/tests/io.js`, `hfs.js`, `sound.js` ...) with the runner separate. It's easier to find a test and to add one.
- **Typing waits by cycle count** (`w(n)`: 2 million cycles before the next key). That's slow (most waits are much longer than needed), and brittle (a slower code path makes a test fail by typing too soon). Let the input wait for text instead, like `expect`: type the next line when the prompt (or a given string) appears. `--mark` already finds text in the output, so the emulator is halfway there. The suite would likely get several times faster.
- **Count and list the tests in one place**. `getting-started.md` says how many there are, and it goes stale with every new test. Have `regress.js --list` produce the documentation's list, or don't put a number in the docs.
- **Gaps**: the tests run one build of the ROM, so its build options go untested:

- the 7.16 MHz build (`CPU_CLOCK_MULT 2`; `hwtest-faults` only runs the normal build at the wrong clock);
- the WDC ACIA build (`SER_ACIA`), whose sending is paced by VIA timer 2: the emulator models the chip (`--acia wdc`), but no test builds the ROM for it.

A test step that builds each variant into its own directory (`--rom`) and boots it would cover them.

HyForth

The shell works well, and its use of the OS (pipes, redirection, programs by name, background tasks) is a highlight. The language has traps for newcomers:

- **A number inside a definition is used while compiling, silently.** `: x 65 . ;` seems to work the first time (`x` prints `0041`: the 65 was left on the stack by the compiler). It fails the second time (`!DS PTR ERROR!`). The docs explain the `lit [ 65 , ]` workaround, but almost every Forth tutorial in the world writes `: x 65 . ;`. Make the interpreter compile a literal when it's compiling and the word is a number. This is the most important usability fix in the review.
- **Output is always 4 hex digits.** `11 .` prints `000B`. Add decimal output (`base`, or `u.` and `.d`), at least as a choice. A beginner's first program prints a number.
- **No line history or editing:** only Backspace. Up/Down for the last lines and Left/Right to edit are what every user will reach for. The console already decodes the arrow keys for conio.
- **Errors are terse** (`!IO ERR!`, then `ioerr` for a number). See the error-message table above.
- `syscall`: see bug 1.

The C library

Good: it covers stdio, files, directories, the environment, processes, sound and the console. It's checked against the ROM by the tests, and documented in a guide of its own. Suggestions:

- **One build** with the ROM's (above), finding cc65 the same way.
- `FILENAME_MAX` is 17 in cc65's headers for this target, while HydraFS names are up to 31 characters and paths 64. `hydra.h` notes it, but a program that sizes buffers with `FILENAME_MAX` will truncate. Provide a `stdio.h` wrapper, or at least say so prominently in `c.md`.
- `MM_ALLOC` from C (paged memory beyond the 32K a task has) is still to do. Larger programs (games, editors) will want it.
- **Next headers:** `vera.h` and a TGI driver (`VIDEO.md`), `i2c.h` and `spi.h` when those devices exist.

The documentation

Thorough and accurate on the whole. Small fixes:

- **The README's repository table** lists `os_rom/`, `sim/`, `board/` and `docs/`, but not `programs/`.
- **Hand-written numbers go stale:** page free space in `rom-layout.md` (and the 256 bytes at `$FE00` it misses), the test count in `getting-started.md`. Generate them, or point to the build's report.
- **For newcomers**, add tutorials before the reference:
 - your first HyForth session;
 - your first C program on a card;
 - your first assembly program;
 - your first driver.

The guides are complete references; a reader who doesn't know what a gate or a task call is needs a gentler road in.

- **The hardware reference's V1 errata** now say the bank bits 6/7 swap is confirmed. The tools depend on it.

The repository

- **No .gitignore at the top.** KiCad's backups (*-backups/), lock files (~*.lck) and fp-info-cache show as untracked under board/. A root .gitignore with those patterns quiets them without touching anything in board/.
- **Untracked os/** at the top level: if it's someone's local work, ignore it locally (.git/info/exclude).
- **Branch naming** (1.8C_0.3 ...) works for one person. With collaborators, a main that always builds and passes, and short-lived feature branches, are easier to follow.

Recommendations, in order

#	What	Why	Size
1	syscall through page 0; one thunk list for both pages	A crash on a documented use	Small
2	HyForth compiles numbers in definitions	The biggest trap for every newcomer	Small to medium
3	A ROM budget report in the build; check_pages fails the build	Page 0 and COMMON are the limits; see them on every build	Small
4	Use the 256 bytes at \$FE00 on pages 1-F; merge the COMMON peeks; WOZMON off page 0	Room for the next features (video, input)	Small to medium
5	build.js for both builds, any OS; CI on GitHub	Lets collaborators build and keeps main green	Medium
6	Split the emulator into modules with a device interface; core without Node	Needed for the VERA and the web emulator	Medium
7	Tests wait for text, not cycles; tests in files by area	Faster, steadier tests	Medium
8	Decimal output, line history and error messages in HyForth	Everyday comfort	Medium
9	A declared client-task ZP scratch union	Protects the servers' shared scratch	Small
10	Split os_rom/io/ into the layer, servers and the filesystem	Easier to find things	Small (moves)
11	Generated files out of Git; release images; a version stamp	Fewer conflicts; traceable ROMs	Small
12	Tutorials	A way in for newcomers	Ongoing

What was done

#	Done	Left
1	syscall is a far word that calls through page 0, and pushes .X (no more silent abort on .X = 0); a new word sys (addr a x y -- a x y p) passes every register and the flags. The thunk table is one list (include/thunks.inc) for both pages, with every address checked; page 1's entries for the calls it has no gate for go on to page 0's (THUNK_TO_P0), so bload ed code can call them. Test: syscall	
2	A number in a definition compiles as lit and the number. Decimal input's range is checked (-32768 was refused; 70000 wrapped silently). Test: forth-numbers	
3	tools/rom_space.js prints the free space on every page at each build, and warns for page 0 and COMMON; check_pages.js fails the build	
4	Pages 0 and 2-F have a HIGH_Pn segment at \$FE00 (kernel/high.s), page 1 FORTH_TOP ; WOZMON is on page 4 (R runs on page 0, reads go through PEEK_D_XAM); COMMON's two peeks are one, PEEK_PAGE ; FAR_JMP_GATE is a 6-byte far call. Page 0: 274 bytes free (from 40), COMMON 21 (from 1)	

#	Done	Left
5	<code>build.js</code> (any OS; cc65 by <code>CC65_HOME</code> , the <code>PATH</code> or the Windows snapshot); the <code>.bat</code> files run it; <code>.github/workflows/build.yml</code> builds cc65 and runs <code>node build.js test</code>	Pushing it to GitHub turns the workflow on
6	<code>sim/lib/</code> : <code>machine.js</code> , <code>cpu65c02.js</code> , a module per device, no Node.js in them; <code>hydrasim.js</code> is the command line. Its output is byte for byte the old emulator's (reports and profiles compared)	The web page itself (NEXT_STEPS.md)
7	The tests are in <code>sim/tests/</code> by area; <code>\p</code> in the input waits for a (settled) prompt, and a run ends 3M cycles after its last command's prompt (<code>--stop-after-input</code> ; <code>fullRun</code> to opt out). The suite takes 36 s (from 52). The two ROM build options are tested too (<code>build.js</code> variants: the CPU at 7.16 MHz, the WDC ACIA), which found that the WDC build no longer linked (page 0 was full: fixed)	14 tests still wait by cycles where a prompt can't be waited for (inside a program, or where time has to pass)
8	Decimal output (<code>decimal</code> , <code>hex</code> , <code>u.</code>); a line editor at the prompt (arrows, Home/End, Delete, history, Ctrl-U); <code>!IO ERR!</code> says why (<code>not found</code>); C's <code>_stroserror</code> too. Tests: <code>line-edit</code> , <code>rom</code>	
9	<code>ZP_CS</code> : the client-task servers' 13 bytes, declared once; <code>CS_FITS</code> checks each server's names for them	
10	<code>io/</code> (the layer), <code>servers/</code> , <code>fs/</code> (HydraFS), <code>drivers/rtc.s</code> ; <code>programs/asm/</code> beside <code>programs/c/</code>	
11	<code>os_rom/VERSION</code> (shown at boot: no date or commit, so a build's images stay the same); the generated <code>test_rom.s</code> is in <code>obj/</code>	Taking the ROM images out of Git, and branch naming: your call (both work as they are)
12	docs/tutorial.md : the first hour, in the emulator	More (a first driver; the VERA, once it's there)

Also: the IO self test needs 2 free namespace entries (not 4); `/rom` takes none (as `/env`); the README lists `programs/` and `build.js`; a root `.gitignore` for KiCad's backups and locks; `FILENAME_MAX`, the IRQ-rate rule for drivers and the names that need no mount are in the docs. Not done: `MM_ALLOC` from C, and a note when a background task ends ([IDEAS.md](#)).

Found on the way, not fixed:

- A key typed while the Hydra boots (before the first prompt) is lost. Piped input to `hydrasim.js -i` loses its first character for it.
- The WDC ACIA build sent slowly at 115200: `words` took about 7.3M cycles (the Rockwell build: 2.5M). Each character's timer 2 interrupt went through the dispatcher and a `TASK_CALL` (about 650 cycles) instead of the fast path the Rockwell build uses at 115200. (Fixed: `SER_T2_FAST` sends for both chips; `words` now takes 2.3M cycles on the WDC build, with no byte written while one is still going, and the `fast-output` and `sound` tests run on it too.)

Hydra-16 documentation

The Hydra-16 is a multitasking 65C02 computer: a W65C02S whose address decoding gives each of 16 tasks its own RAM, zero page, stack and bank selections, running a small preemptive OS with Plan 9-style IO, and HyForth as its shell.

Start here

- **The Hydra-16 in one place:** the master document: every part of the machine and its software, the hardware included, in one page, with links into the documents below. All of the documentation is also one PDF: [hydra-16.pdf](#) (`node sim/tools/mkpdf.js` remakes it, with Microsoft Edge or Chrome).
- **First steps:** a tutorial for the first hour, in the emulator: HyForth, files and a card, a program in C and one in assembly, an LED and a button.
- **Getting started:** build the ROMs, program the chips, connect a terminal, boot, or run it in the emulator.

Using the system

- **HyForth:** the shell and language: words, directories and files, scripts and programs, pipelines, tasks.
- **WOZMON, POST and the self tests:** the monitor and disassembler, reading the power-on self test, the hardware test.

Programming it

The **Programmer's Guide**, by area:

- **ROM layout and the API index:** ROM pages, gates, calling conventions, zero page, error codes, every entry point.
- **Tasks and the scheduler**
- **Interrupts**
- **Memory:** the MMU, shared memory, far pointers.
- **Input and output:** fds, devices, the console, pipes, namespaces.
- **Drivers and file servers**
- **Programs:** Hydra executables (`.hyx`), and building one.
- **The C Programmer's Guide:** writing programs in C with cc65, from the first build to the library, the console, files, processes, assembly, performance and debugging.

The hardware

- **Hardware Reference:** the main board in detail (memory system, address decoding, pseudo-registers, interrupts, clocks, devices, slots, connectors, errata) and the companion cards.

Tools

- **The emulator and tools:** `hydrasim.js` (interactive and scripted), the regression tests, the HydraFS card tool, the `.hyx` header tool.

Plans and design notes

The reasoning behind the design, and what's planned. Parts of them are history (marked done).

- **IO_PLAN.md:** the IO subsystem, scheduler and filesystem steps.
- **MMU_PLAN.md:** the memory manager, IRQ dispatch, drivers, zero-page convention.

- [HYDRAFS.md](#): the SD card filesystem (spec).
- [SHELL.md](#): HyForth as a shell: a current directory, commands, running programs (done).
- [REORG_PLAN.md](#): the ROM reorganisation.
- [SOUND.md](#): the YM2151: its library (done), a song player, a test song that uses the whole chip, and importing music from other machines.
- [NEXT_STEPS.md](#): what's missing to make the Hydra fun and useful for hobbyists and programmers, and the milestones to get there.
- [DISKS.md](#): RAM and ROM disks as HydraFS volumes: `/ram` (each shell's own area of the RAM disk) and `/sram` (the shared RAM disk), with program caches, and `/rom` (the paged ROM as one disk: built).
- [NAMESPACES.md](#): the Plan 9 way: union directories put together with binds and mounts, a default namespace, and `/bin` and `/lib` in place of search paths.
- [PROC.md](#): `/proc`, the tasks as files, with a task's memory (`/proc/N/mem`, `/proc/N/ram`) for its family and task 0, the system task.
- [VIDEO.md](#): the Vera X video card in slot 0 (the VERA: VGA, sprites, PSG and PCM), its keyboard controller, and the software for them.
- [CODE_REVIEW.md](#): a review of the whole software tree, with bugs, ROM and zero page budgets, and recommendations.
- [IDEAS.md](#): ideas for later (wait states for board V2, ...).

Hydra 16

A multitasking 65C02 computer and its operating system.

The Hydra-16's hardware gives each of 16 tasks its own 32K of RAM, zero page and stack included, with its own RAM and ROM bank selections, so a task switch is one register write. On top of that runs a small OS:

- **Scheduling and interrupts:** a preemptive scheduler, and IRQ handlers that run in their driver's own task.
- **Memory:** a memory manager per task, plus shared memory between tasks.
- **IO, Plan 9 style:** devices are file servers; each task has fds, a namespace and pipes.
- **The shell:** HyForth, with `|` pipelines.

The code is built with **cc65** (<https://cc65.github.io/>), and the board is designed in **KiCad 9** (<https://www.kicad.org>).

Quick start

```
node build.js test          build everything (needs Node.js and cc65) and run the regression tests
node sim/hydrasim.js -i    use the Hydra in your terminal, in the emulator (Ctrl-A x quits)
```

Documentation

All documentation is in [docs/](#). [The Hydra-16 in one place](#) covers every part, hardware and software, with links to the rest; it's also one PDF, [docs/hydra-16.pdf](#), with every document in it.

The Hydra-16 in one place	The master document: every part of the system, with links into the others
First steps	A tutorial: the Hydra in the emulator, HyForth, files, a program in C and in assembly
Getting started	Building, programming the chips, the serial terminal, first boot, the emulator
HyForth, WOZMON	Using the system
Programmer's Guide	ROM layout and API, tasks, interrupts, memory, IO, writing drivers
Hardware Reference	The board in detail
Emulator and tools	<code>hydrasim.js</code> , the regression tests, the HydraFS card tool
Plans	Design notes and what's next

Repository

Folder	What
<code>os_rom/</code>	The OS ROM: sources, built images (<code>bin/</code>)
<code>programs/</code>	Programs for the Hydra: the C library and samples (<code>programs/c/</code>), assembly (<code>programs/asm/</code>)
<code>build.js</code>	The build, on any OS: <code>node build.js</code> , <code>node build.js test</code>
<code>sim/</code>	Emulator, regression tests, tools
<code>board/</code>	KiCad schematics and PCBs: main board, memory daughter card, bus breakout card
<code>docs/</code>	Documentation